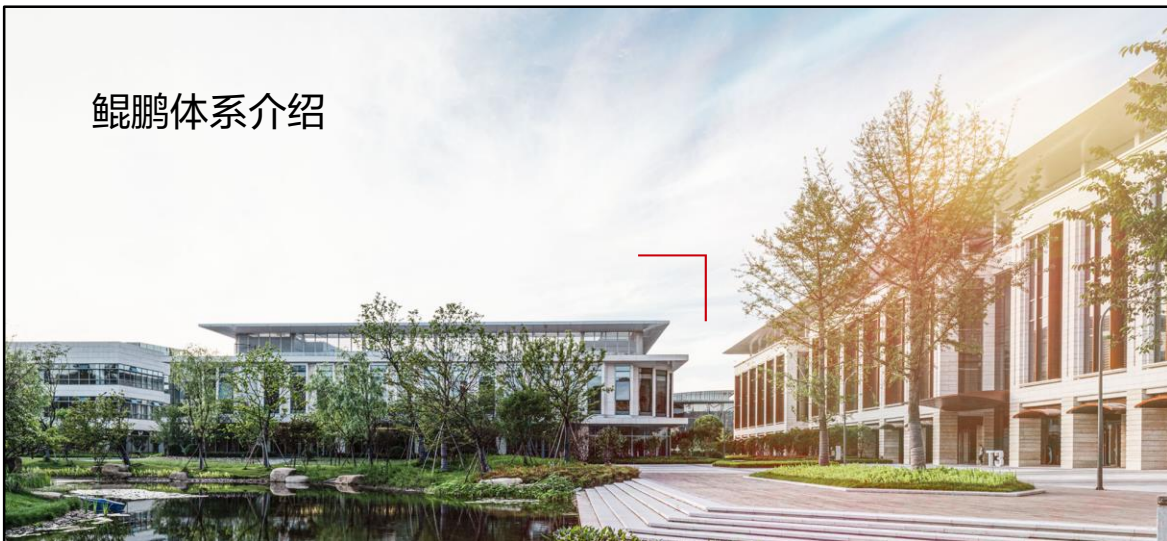


鲲鹏体系介绍



前言

- 本章主要介绍鲲鹏计算产业及鲲鹏生态体系全景，华为鲲鹏处理器架构、技术创新， TaiShan服务器型号、规格及特点， openEuler概念及相关操作， openGauss数据库等内容。

目标

- 学完本课程后，您将能够：
 - 了解计算产业的发展趋势，鲲鹏计算产业以及鲲鹏生态的全景图；
 - 描述华为鲲鹏处理器产品的性能规格、技术创新；
 - 了解TaiShan 200机架服务器和TaiShan 200高密服务器；
 - 了解openEuler操作系统的特点以及openEuler开源社区的作用；
 - 掌握openEuler基础命令操作；
 - 了解openGauss数据库的定位、优势特性及基础操作。

目录

1. 鲲鹏生态体系介绍

- 计算产业发展趋势

- 鲲鹏简介

- 鲲鹏计算产业概述

- 鲲鹏生态体系介绍

2. 华为鲲鹏处理器介绍

3. TaiShan服务器介绍

4. openEuler操作系统介绍

5. openGauss数据库介绍

计算产业概述

- 计算产业是IT技术的基础，是每一次产业变革的驱动力，从云计算、大数据、人工智能到区块链、边缘计算、物联网都离不开强大的计算能力的支持。



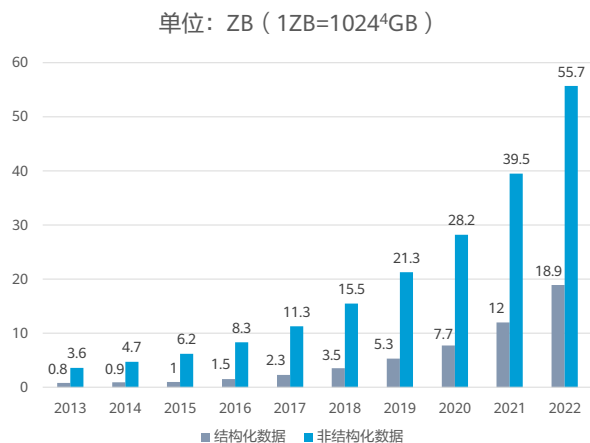
大数据技术对算力的挑战



- 在互联网时代，相比传统的企业电子办公，出现了大量的非结构化数据，对数据的分布式存储和并行技术提出了很高的要求。Google就是传统互联网的典型代表，发表了GFS、MR、BigTable等重要的大数据论文，从而引导了大数据的元年。
- 在移动互联网时代，在互联网上的应用远远比传统互联网（PC访问）丰富，数据的结构也发生了很大的变化，对实时处理的能力和大量并发的要求比以前会有巨大的提高。手机银行就是一个典型的代表，现在的手机银行APP能够提供各种丰富的内容，除了传统的网上交易，还包括资讯，娱乐，购物等。所以对数据处理的能力通常就要求多种技术配合来完成，有批处理，内存计算，交互式查询，流式实时处理等，这样就推动了MR, SPARK, Solr, Storm和YARN的发展。
- 到了即将爆发的物联网时代，将是一个完全的数字化的世界，任何东西都将连接到网络上，而最关键的是要将这些东西关联起来，发现其背后存在的规律，为人类的生活、生产创造更大的价值，这就是人工智能时代的大数据处理，这是大数据发展的第3个阶段。

人工智能对算力的挑战

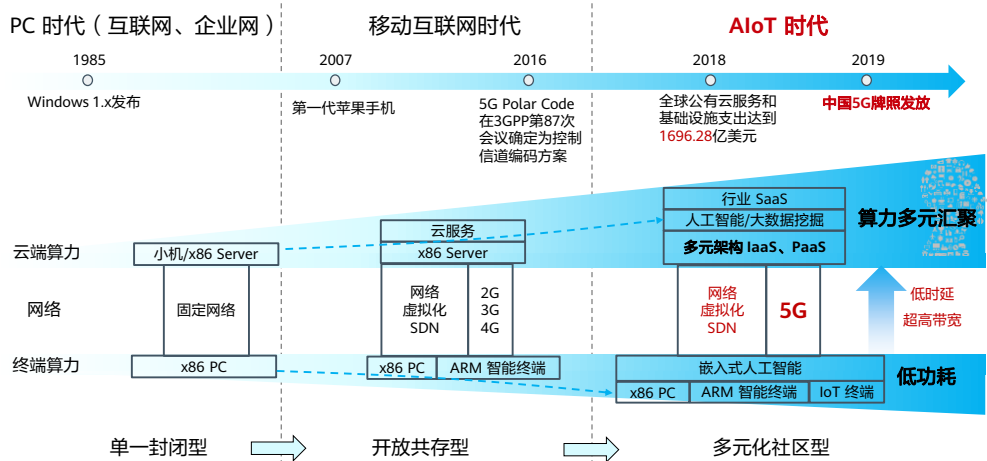
- 数据爆发式增长带来算力需求提升；
- 算力成本不断增加导致企业投入加大；
- 摩尔定律失效，CPU性能提升遭遇瓶颈，传统服务器无法满足算力需求；
- AI计算系统性能与效率亟待提升。



注：数据来源《中国人工智能算力发展评估报告》

- 目前的人工智能，更多的是代表智能的个体，能够通过自身的持续学习能力，智能的完成单点决策。
- AI产业爆发三大条件：算法、算力、数据。
- 机器经验需要由大量的历史数据得出，所以数据收集无处不在，数据的增长会是几何级数的。使用当前集中式的存储和集中式的通信模式，在未来是无论如何都无法通过一个巨型单点支撑如此大的体量，存储和通信能力都是瓶颈，而且效率会非常低下。
- 算力成本也是人工智能行业的一大痛点。现在的人工智能企业的硬件投入巨大。人工智能对计算的需求非常大，因此对高性能计算定制深度学习芯片要求很高，意味着很多企业要花很多钱买算力、建很多计算中心，造成了很大的资源浪费。
- AI计算发展趋势演变过程中面临着巨大的挑战：随着模型所需的精度越高，所需的计算量也会呈现增长趋势。对于未来算法的发展对整个计算需求所造成的挑战会变得更加大，提高整个AI计算系统的性能与效率显得尤为重要。
- 摩尔定律失效，CPU性能提升遭遇瓶颈。Intel宣布正式停用“Tick-Tock”处理器研发模式，未来研发周期将从两年周期向三年期转变。单颗CPU性能的提升在放缓，传统服务器难以满足并行算力需求，服务器CPU出货量增长停滞。
- 如今，AI面临着巨大的计算挑战，提高AI计算系统性能与效率变得尤为重要，需要从系统的角度进行综合考虑。

AIoT时代对算力的挑战



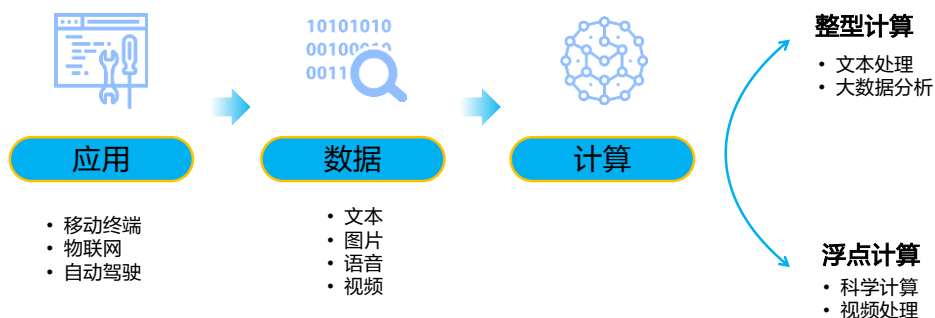
8

Huawei Confidential

HUAWEI

- AIoT (人工智能物联网) = AI (人工智能) + IoT (物联网)。

应用和数据的多样性需要新的计算架构



- 未来信息量巨大，计算无处不在，计算应用的场景多种多样，从扫地机器人到智能手机、智慧家庭、IoT物联网、自动驾驶等。场景的多样性，带来数据的多样性，数字、文本、图片、视频、图像以及结构化数据、非结构化数据等。
- 整型计算胜在文本处理、存储、大数据等；浮点计算胜在科学计算、视频处理等。

新兴计算产业呼唤新的算力

移动智能终端逐渐取代传统PC
2018年移动终端16亿部，PC 2.5亿台

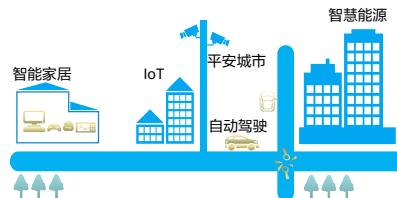


移动终端大规模普及催生多样化应用



新的算力需求

世界正在进入万物互联的时代
2018年全球连接设备数超过230亿



海量数据处理需要大带宽高并发

处理器：x86 -> ARM

部署模式：PC应用 -> 移动应用 -> 移动应用云化

移动终端接入方式：3G -> 4G -> 5G

需要云数据中心侧与端侧同构的算力

需要高性能、高并发、高吞吐的算力

AI算力占数据中心算力80%

边缘侧：数据采集 -> 数据实时智分析

云数据中心侧：承载海量数据的分析、处理和存储

云边协同：中心训练+边缘推理

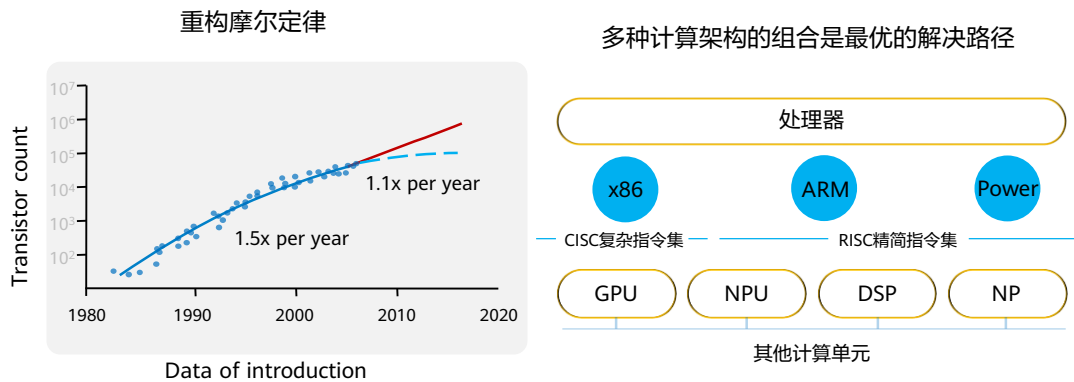
- 当前计算产业呈现两个大的变化趋势：

- 移动智能终端取代传统PC：2018年全球传统PC出货量2.5亿台，连续七年下滑。与之相反，2018年全球移动智能终端出货量突破16亿部。这导致计算正在从x86->ARM；应用正从PC应用->移动应用->移动应用云化。新的算力需求：云数据中心侧与端侧同构的算力。
- 世界正在进入万物互联的时代：2018年全球联接设备的数量已超过230亿；到2025年预计这一数字将突破1000亿，将带来海量的数据。例如：自动驾驶每天产生64TB数据、XX平安城市每天1,000TB数据。

- 对于海量数据的处理的算力需求：

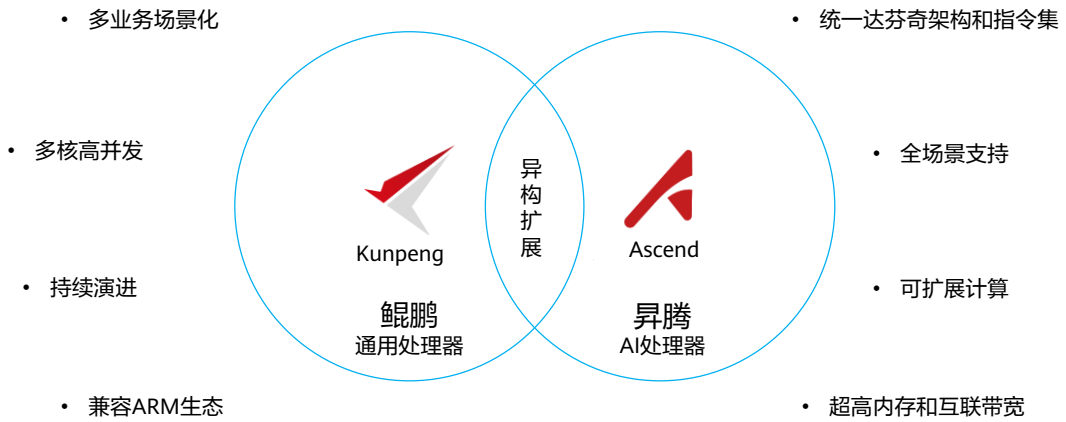
- 边缘侧实时智能处理，需要AI的算力；
- 数据中心侧分析、处理和存储海量的数据，需要高并发、高性能、高吞吐的算力。

多样性计算满足算力需求



- 计算进入多样性时代，将带来大量异构计算的需求，没有一个单一的计算架构能够满足所有场景、所有数据类型的处理。
- 我们看到各种CPU、DSP、GPU、AI芯片、FPGA等同时存在：
 - 多种计算架构共存的异构计算是最优的解决路径。
 - 多种计算架构共存的异构计算，将实现摩尔定律的重构，从而更好满足多业务多样性、数据多样性需求。

鲲鹏 + 昇腾打造核心算力



目录

1. 鲲鹏生态体系介绍

- 计算产业发展趋势
- 鲲鹏简介
- 鲲鹏计算产业概述
- 鲲鹏生态体系介绍

2. 华为鲲鹏处理器介绍

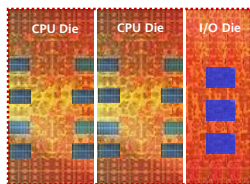
3. TaiShan服务器介绍

4. openEuler操作系统介绍

5. openGauss数据库介绍

鲲鹏简介

鲲鹏是SoC



制程工艺领先：业界领先7nm制程，多Die合封的Chiplet架构

自研多核内核：自研CPU内核算力提升50%，自研片间互联，支持多路互联

率先支持下一代网络和接口：支持8通道内存控制器和100GE端口

鲲鹏是计算平台

处理器->单机->集群，鲲鹏开放硬件平台



鲲鹏开放主板

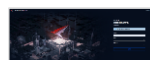


鲲鹏服务器

完备的软件工具链，发挥鲲鹏最佳性能



鲲鹏代码迁移工具



鲲鹏性能分析工具

鲲鹏是生态应用

使能合作伙伴

- 应用
- 中间件
- 数据库
- 操作系统
- 服务器/PC

使能行业应用

- 大数据
- 分布式存储
- 高性能计算
- 原生应用
- 云服务

- Die：在集成电路中，die（芯片）是一小块半导体材料，在其上制造了给定的功能电路。
- SoC：System on Chip的缩写，称为系统级芯片，也有称片上系统，意指它是一个产品，是一个有专用目标的集成电路，其中包含完整系统并有嵌入软件的全部内容。

目录

1. 鲲鹏生态体系介绍

- 计算产业发展趋势
- 鲲鹏简介
- 鲲鹏计算产业概述
- 鲲鹏生态体系介绍

2. 华为鲲鹏处理器介绍

3. TaiShan服务器介绍

4. openEuler操作系统介绍

5. openGauss数据库介绍

超万亿规模的计算产业空间

- 新应用、新技术、新计算架构，百亿级联接、爆炸式数据增长将重塑ICT产业新格局，催生新的计算产业链条，涌现出新的厂家和新的生态体系：
 - 软件：操作系统和虚拟化软件、数据库、中间件、大数据平台、企业应用软件，云服务、数据中心管理服务；
 - 硬件：服务器及部件、企业存储设备；
 - 云服务、数据中心管理服务。



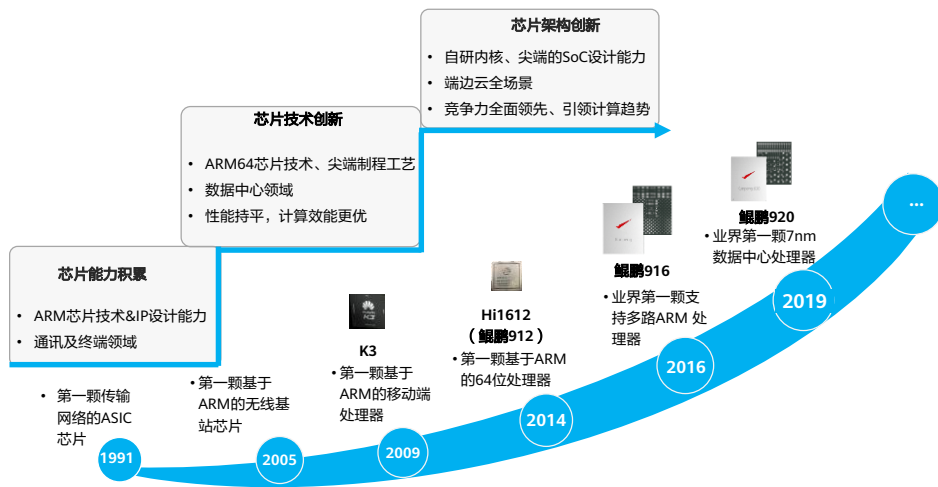
- 哪些是平台（OS+CPU）无关的？如企业应用软件、企业存储、公有云、大数据等。可见计算产业空间大部分都是平台无关的，这也说明了鲲鹏产业的潜在发展空间巨大。

鲲鹏计算产业概述

- 鲲鹏计算产业是基于鲲鹏处理器构建的全栈IT基础设施、行业应用及服务，包括PC、服务器、存储、操作系统、中间件、虚拟化、数据库、云服务、行业应用以及咨询管理服务。



鲲鹏处理器 - 坚持持续创新

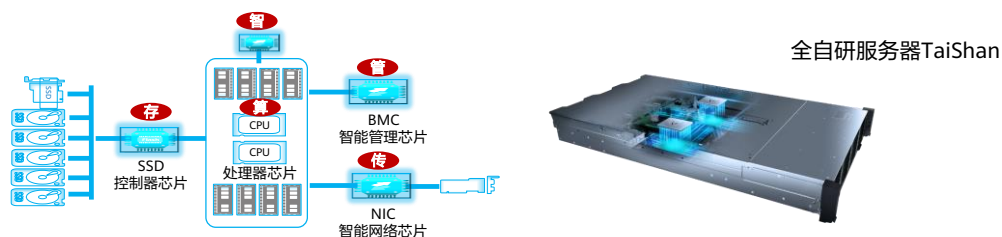


18 Huawei Confidential



- 华为在芯片及处理器领域的探索已经超过了二十年。
- 1991年，研发了第一颗用于传输网络的ASIC芯片。
- 2005年，研发了第一颗基于ARM架构的无线基站芯片。
- 2009年，推出智能手机CPU K3，麒麟芯片的前身，如今麒麟芯片已经把手机带入智慧时代。
- 2016年，推出第一代鲲鹏处理器，面向数据中心应用的鲲鹏916处理器。推出服务器CPU，也就是鲲鹏芯片的前身。
- 2019年，推出鲲鹏920，是业界第一颗7nm数据中心处理器。
- 华为每年将销售收入的10%以上投入研发，其中芯片/SoC（System on a Chip，系统芯片）相关投资超过1/3，使得华为的芯片研发能力实现了从初始技术积累、到技术创新，再到架构创新的飞跃式提升。
- 未来，鲲鹏处理器还将保持长期投入、全面布局、前向兼容、持续演进的产品策略，量产一代、研发一代、规划一代。

全面创“芯” - 构建全自研服务器TaiShan



算



华为鲲鹏 920

ARM处理器芯片

首款7nm ARM服务器处理器，32/48/64核，2.6GHz

存

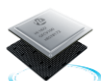


Hi1812

智能SSD控制芯片

PCIe NVMe与SAS融合
智能加速，超强磨损算法

传



Hi1822

智能融合网络芯片

以太网与FC融合，协议加速，
可编程

管



Hi1710

智能管理芯片

内置智能管理引擎
智能故障管理

AI

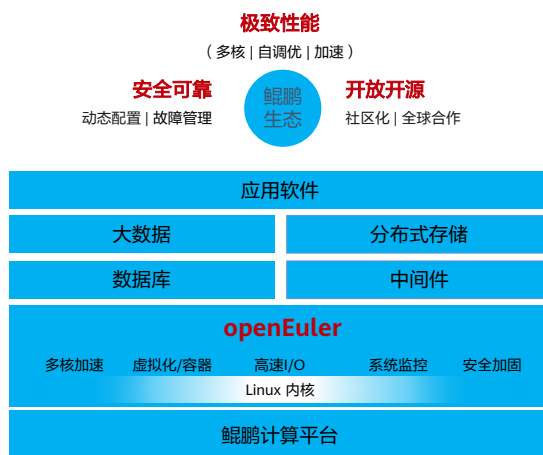


Ascend 310/910

人工智能芯片

达芬奇架构
极致能效与性能

开源开放 - 打造国产操作系统典范



极致性能

- 三级智能调度，多进程并发时延缩短60%；
- 全栈智能自优化（自调节，自感知，自适应）；
- 多核加速、软硬结合虚拟化、微容器。



安全可靠

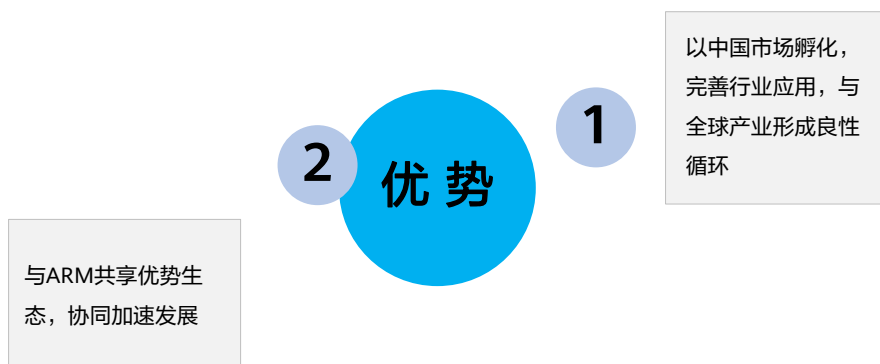
- 动态配置，故障管理，冷热补丁。



开放开源

- 内核贡献全球TOP10，Docker社区贡献排名TOP4；
- 全球合作，开放开源。

鲲鹏计算产业优势



- 在鲲鹏计算产业的框架之下，产业链上下游的厂商可以基于鲲鹏和Arm已有生态，开发出具有差异化优势的产品和解决方案。
- 华为作为鲲鹏计算产业的一员，掌握Arm64处理器核、微架构及芯片设计的关键技术，拥有Armv8永久架构授权，并在此基础上发展了Kunpeng系列处理器。同时华为在芯片领域上的长期投入和持续创新，面向计算、存储、传输、管理和AI打造了全面的具有差异化竞争力的芯片体系，有能力提供支持鲲鹏计算产业持续发展的算力底座需求。
- 鲲鹏计算产业首要任务是基于各行业用户的需求，孵化和完善面向行业的应用，打开产业空间，为所有厂家找到与之相匹配的生产要素和市场机会，通过商业成功促进产业链各环节的持续创新，形成螺旋式上升的发展路径。随着鲲鹏计算产业在中国市场的发展壮大，吸纳更多的海内外企业加入，最终成为具有持续创新能力和全球领先优势的计算产业。

鲲鹏计算产业的典型应用场景

大数据



- ◆ 性能提升30%
- ◆ 加解密引擎
- ◆ 与x86融合部署

分布式存储



- ◆ IOPS性能提升20%
- ◆ 压缩解压引擎
- ◆ 与x86融合部署

数据库



- ◆ RoCE低时延网络
- ◆ 多核调度算法
- ◆ NUMA优化算法

原生应用



- ◆ 原生同构，无性能损耗
- ◆ 企业级可靠方案
- ◆ 百万级应用，成熟生态

云服务



- ◆ 鲲鹏裸金属服务器
- ◆ 鲲鹏弹性云服务器
- ◆ 鲲鹏Kubernetes容器

高性能

多核高并发



高吞吐

原生算力同构

- 鲲鹏计算产业的发展壮大，离不开产业界上下游伙伴的共同努力。在过去几年时间里，华为与各行业ISV共同孵化、完善了五大鲲鹏计算解决方案，在多个行业实现了规模商用。
- 譬如大数据场景，鲲鹏920处理器最高支持64核，多核优势意味着可并行处理更多的数据。实测数据显示，鲲鹏处理器的大数据性能比传统平台提升30%以上。
- 分布式存储场景，鲲鹏处理器通过内置数据压缩引擎，多核并行计算提升分布式存储软件的运行效率，相比传统处理器算法方案减少66%的压缩时间，并实现IOPS性能提升20%以上。
- 在鲲鹏处理器设计之初，华为就充分考虑了如何满足分布式并行计算的需求。在鲲鹏天生多核的基础上，增强了高I/O带宽、高内存带宽、高数据吞吐的设计，更好的满足了IT分布式演进的发展趋势。

目录

1. 鲲鹏生态体系介绍

- 计算产业发展趋势
- 鲲鹏简介
- 鲲鹏计算产业概述
- 鲲鹏生态体系介绍

2. 华为鲲鹏处理器介绍

3. TaiShan服务器介绍

4. openEuler操作系统介绍

5. openGauss数据库介绍

鲲鹏计算产业生态全景

- 华为携手业界技术厂商打造开放包容的鲲鹏计算产业生态，携手伙伴面向行业提供针对性解决方案，通过社区建设以及开发者大赛等活动为开发者提供交流竞赛的平台，积极开展高校合作，进行人才培养。



产业生态 - 鲲鹏计算产业发展总览

- 鲲鹏计算产业目标是建立完善的开发者和产业人才体系，通过产业联盟、开源社区、鲲鹏创新中心旗舰店、行业标准组织一起完善产业链，打通行业全栈，使鲲鹏生态成为开发者和用户的首选。



产业生态 - 携手共建鲲鹏产业云生态

区域科创名片

区域科技城、华为鲲鹏实验室
国家级鲲鹏认证机构、鲲鹏产业基地

产业聚集

国产操作系统、工具链、数据库、中间件、应用软件等

鲲鹏人才聚集

鲲鹏学院、鲲鹏高级人才引入、
开发者社区、技术培训



鲲鹏生态创新中心 信息创新的鲲鹏产业生态聚集圈

一站式鲲鹏开发者社区

线上资源开放

OS开源
数据库开源
鲲鹏单板开放
鲲鹏工具栏

鲲鹏生态实验室

线下运营支撑

鲲鹏创新中心
旗舰店
联合鲲鹏实验室
鲲鹏微认证
鲲鹏培训

政府政策扶持

政策支持

扶持基金
人才激励
党政军市场牵头
国产化标准

鲲鹏伙伴聚集

生态培育

鲲鹏联盟
x86转化
联合解决方案
联合营销

技术生态 - 兼容主流开源组件和国产组件



高校合作 - 携手高校，培养鲲鹏计算人才



2020首批投入**1千万元**，
建设计算机核心课程实践
资源



服务教师和学生**2类人群**，
开展师资培训和创新实践



联合**3类教育组织**，
探索鲲鹏计算产业人才
培养合作模式



依照**4类华为认证标准**，
培养计算产业千军万马

- 每一个新兴产业的发展，都离不开人才的培养。华为面向中国高校推出了促进计算产业人才培养的“1234鲲鹏高校人才计划”：
 - 投入“1”千万：2019年华为将通过教育部产学研合作协同育人项目首批投入一千万软硬件资源，与高校专家共同建设计算机体系结构、操作系统、数据库、人工智能等课程的实验资源；
 - 服务“2”大人群：服务教师和学生两大人群：面对教师，联合开发不少于20门的精品课程和数字教材，并组织鲲鹏生态相关的师资培训和技术研讨；面向学生，推出创新训练营和拔尖人才计划，并例行举办各类创新大赛；
 - 联合“3”类组织：华为将联合教学指导委员会（计算机类专业教执委、软件工程教指委等）、联盟（国家示范性软件学院联盟、信息技术新工科产学研联盟等）、学会（中国计算机学会、中国高等教育学会等）3类教育专家组织，制订关键核心技术领域人才培养方案和专业标准、设计与开发产学研合作课程，建设与推广计算产业开源社区等，探索鲲鹏计算产业人才培养合作的新模式；
 - 培养“4”类人才：基于4类华为认证标准：“鲲鹏应用开发者认证”、“GaussDB数据库认证”、“智能计算认证”、“人工智能认证”，支持中国高校开展鲲鹏生态所需人才的培养，培养计算产业人才。

伙伴生态 - 鲲鹏伙伴计划



- 欧拉扬帆伙伴计划：欧拉扬帆伙伴计划（华为）是华为围绕openEuler推出的一项生态伙伴计划，旨在保障openEuler生态健康有序发展，加速生态伙伴基于openEuler系列产品和部件构建产品与解决方案，使能伙伴生态发展与商业成功。加入我们，您可以获得：
 - openEuler职业认证培训服务及考券。
 - 鲲鹏生态创新中心伙伴资源。
 - 欧拉生态创新中心伙伴资源。
 - 营销材料及展会可使用华为伙伴等级Logo及证书。
 - 在openEuler产业营销活动中，获得参会及品牌露出优先权。
 - 在华为或华为销售伙伴主导的市场机会中，获得项目推荐优先权。
- 鲲鹏展翅伙伴计划：鲲鹏展翅伙伴计划是华为围绕鲲鹏计算平台推出的一项生态伙伴计划，旨在帮助伙伴整合鲲鹏计算产业的全栈技术、品牌资源，构建产品与解决方案竞争力，联接客户与鲲鹏生态系统，创造无限商机。加入该计划，伙伴可以获得：
 - 伙伴技术资料下载。
 - 鲲鹏生态创新中心构建资源。
 - 营销材料及展会可使用华为伙伴等级Logo及证书。
 - 支持伙伴营销活动。
 - 在华为或华为销售伙伴主导的市场机会中，获得项目推荐优先权。
 - 提供等值技术认证支持服务。

社区生态 - 打造一站式云上鲲鹏专区

<https://hikunpeng.com>



鲲鹏社区

社区简介

鲲鹏社区是华为公司围绕鲲鹏技术体系打造的技术支持、知识共享和产业互助平台

社区使命

全面支撑客户、合作伙伴和开发者，围绕鲲鹏技术体系开发有竞争力的计算产品和行业解决方案

社区愿景

共建开放、共赢的鲲鹏生态，使能合作伙伴成功，共同做大鲲鹏产业

- **全套工具链**

- 代码托管 | 流水线编译 | 发布上线 | 维护升级

- **专业移植指导**

- WEB、HPC、数据库、大数据、中间件、开发工具、应用工具、管理与监控、编译工具、OS与驱动

- **开放社区论坛**

- 交流分享 | 学习答疑 | 大咖直播 | 互动收获

- **生态交流孵化平台**

- 线上线下统一的鲲鹏生态交流及商业对接平台，为创业企业提供优势资源的智慧专家团，为社区企业提供最前沿、最深度的创业顾问和教练服务，推动团队快速成长，成为鲲鹏技术应用的先锋和推广者

开发者生态 - 沃土计划2.0



思考题

1. （多选题）以下关于华为鲲鹏计算产业产品描述，正确的是哪些项？（ ）
 - A. 华为鲲鹏处理器
 - B. TaiShan服务器
 - C. 华为云鲲鹏云服务
2. （多选题）以下鲲鹏计算产业，华为提供哪些支持？（ ）
 - A. 工具链
 - B. 云服务
 - C. 社区服务
 - D. 专业服务

- 答案:

- ABC
 - ABCD

目录

1. 鲲鹏生态体系介绍
2. **华为鲲鹏处理器介绍**
 - 处理器架构介绍
 - 华为鲲鹏处理器概述
 - 华为鲲鹏处理器型号及规格
 - 华为鲲鹏处理器技术创新
3. TaiShan服务器介绍
4. openEuler操作系统介绍
5. openGauss数据库介绍

处理器架构介绍

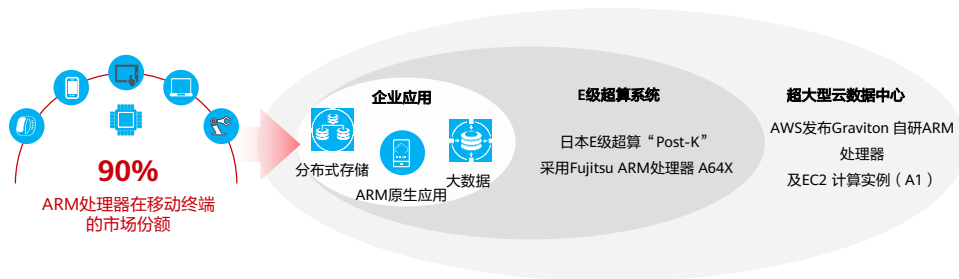
- x86架构是一种CPU架构，标识一套通用的计算机指令集合，采用CISC复杂指令集。(Complex Instruction Set Computer)。
- ARM也是一种CPU架构，有别于Intel、AMD CPU采用的CISC复杂指令集，ARM CPU采用RISC精简指令集(Reduced Instruction Set Computer，精简指令集计算机)。随着物联网、人工智能技术不断的发展与推进，微处理器技术也在不断革新，使得各种新型微处理器的应用也在不断深入，ARM嵌入式技术被广泛地使用。其主要优势有：体积小、功耗低、成本低；指令执行速度更快；很好的兼容8位/16位器件。

	x86	ARM
指令集	CISC	RISC
供应商	主要有Intel和AMD，Intel处于垄断地位	开放的授权策略，众多供应商
产业链	成熟	快速发展中

- 传统的CISC(Complex Instruction Set Computer，复杂指令集计算机)体系由于指令集庞大，指令长度不固定，指令执行周期有长有短，使指令译码和流水线的实现在硬件上非常复杂，给芯片的设计开发和成本的降低带来了极大困难。
- 随着计算机技术的发展需要不断引入新的复杂的指令集，为支持这些新增的指令，计算机的体系结构会越来越复杂。然而，在CISC指令集的各种指令中，其使用频率却相差悬殊，大约有20%的指令会被反复使用，占整个程序代码的80%。而余下的80%的指令却不经常使用，在程序设计中只占20%，显然，这种结构是不太合理的。
- 针对这些明显的弱点，1979年美国加州大学伯克利分校提出了RISC(Reduced Instruction Set Computer，精简指令集计算机)的概念，RISC并非只是简单地减少指令，而是把着眼点放在了如何使计算机的结构更加简单合理地提高运算速度上。
- RISC结构优先选取使用频率最高的简单指令，避免复杂指令；将指令长度固定，指令格式和寻址方式种类减少；以控制逻辑为主，不用或少用微码控制等措施来达到上述目的。
- 将x86和ARM架构相比较，可以看出ARM架构具有更好的并发性能。

ARM架构处理器应用领域

- 目前超过90%的移动终端采用的是ARM架构的处理器。
- 随着IOT、AI和业务云化的发展，ARM在终端的优势地位将会带动其进入数据中心市场，成为下一个快速增长的市场领域。

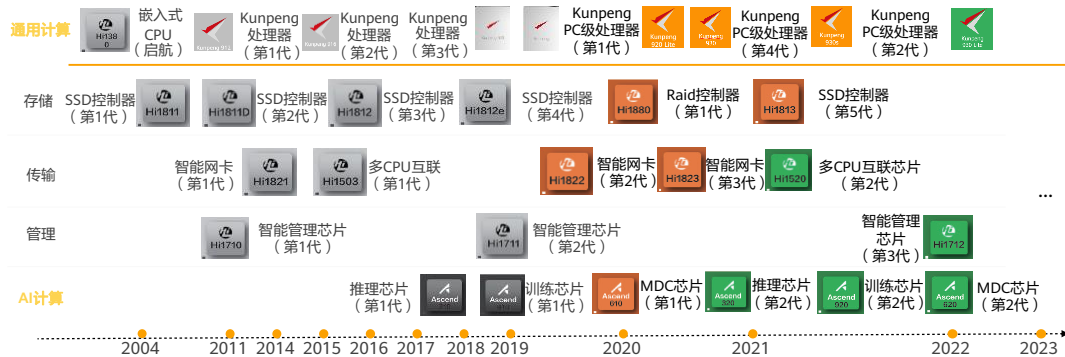


目录

1. 鲲鹏生态体系介绍
2. **华为鲲鹏处理器介绍**
 - 处理器架构介绍
 - **华为鲲鹏处理器概述**
 - 华为鲲鹏处理器型号及规格
 - 华为鲲鹏处理器技术创新
3. TaiShan服务器介绍
4. openEuler操作系统介绍
5. openGauss数据库介绍

华为鲲鹏处理器概述

- 华为鲲鹏处理器是华为自主研发的基于ARM架构的企业级系列处理器产品，包含“算、存、传、管、智”五个产品系统体系。



- 芯片是构建各类计算产品、使能上层软件和应用的基础，也是全产业链可持续创新和发展的驱动力。作为鲲鹏计算产业底座的Kunpeng处理器，华为将持续保持重点投入，秉承量产一代、研发一代、规划一代的演进节奏，落实“长期投入、全面布局，后向兼容和持续演进”的基础战略，通过对产业界提供以Kunpeng系列处理器为核心的芯片族和相应的产品，高效满足市场对新算力的需求。

华为鲲鹏处理器架构特点

- 优点：
 - 采用ARM架构，同样功能性能占用的芯片面积小、功耗低、集成度更高，更多的硬件CPU核具备更好的并发性能。
 - 支持64位指令集，能很好地兼容从IoT、终端到云端的各类应用场景。
 - 大量使用寄存器，大多数数据操作都在寄存器中完成，指令执行速度更快。
 - 采用RISC指令集，指令长度固定，寻址方式灵活简单，执行效率高。
- 不足：
 - 在数据中心领域仍属于新进入者，目前其生态仍处于发展之中。

- 芯片工艺进入28nm以下空间时，靠先进工艺提升性能成本急剧上升，工艺、主频遇到瓶颈后，开始转向增加核数的横向扩展来提升性能。
- 一个ARM核的面积仅为x86核的1/7，同样的芯片尺寸下，ARM的核数是x86的4倍以上。

目录

1. 鲲鹏生态体系介绍
2. **华为鲲鹏处理器介绍**
 - 处理器架构介绍
 - 华为鲲鹏处理器概述
 - **华为鲲鹏处理器型号及规格**
 - 华为鲲鹏处理器技术创新
3. TaiShan服务器介绍
4. openEuler操作系统介绍
5. openGauss数据库介绍

华为鲲鹏通用计算处理器



**鲲鹏912
(Hi1612)**

第一颗基于ARM
的64位CPU

2014



鲲鹏916

业界第一颗支持
多路ARM CPU

2016



鲲鹏920

业界第一颗7nm数据中
心 ARM CPU

2019

- 华为从2004年开始基于ARM技术自研芯片。在通用计算处理器领域，2014年发布Kunpeng 912处理器，2016年发布Kunpeng 916处理器，2019年1月发布Kunpeng 920处理器，Kunpeng 920处理器是业界第一颗采用7nm工艺的数据中心级的ARM架构处理器。

华为鲲鹏920处理器规格



- 支持64核、48核、32核等多种型号
 - 指令集兼容Armv8.2, 最高主频达2.6GHz
 - 每核集成64KB L1 I/D 缓存
 - 每核独享 512KB L2 缓存
 - 平均每核1MB L3 cache
- 8*DDR4控制器, 最高可达2933MT/s
- 集成PCIe/SAS接口
 - 支持PCIe 4.0, 向下兼容PCIe 3.0/2.0/1.0
 - 支持x16, x8, x4, x2, x1 PCIe 4.0, 集成PCIe控制器
 - 支持16*SAS/SATA 3.0控制器
- 支持CCIX接口, 支持加速器的缓存一致性
- 支持2*100G RoCE v2, 支持25GE/50GE/100GE标准NIC
- 支持2P/4P扩展
- 封装大小: 60mm*75mm

华为鲲鹏920处理器特点



高性能

930+, 25% ↑

Estimated SPECint_Rate_base2006 评测跑分

高吞吐

内存带宽: 46% ↑
I/O总带宽: 66% ↑
网络带宽: 4x

高集成

1 颗 = 4颗芯片

高能效

30% ↑

注：和Intel的skylake最高档8180相比



- 和Intel的skylake最高档8180相比，SPECint_Rate_base2006评测跑分，鲲鹏920多达930+，而8180 750+分，跑分高了将近25%，与8180相比，内存带宽，I/O总带宽，能效分别高了46%，66%，30%。

目录

1. 鲲鹏生态体系介绍
2. **华为鲲鹏处理器介绍**
 - 处理器架构介绍
 - 华为鲲鹏处理器概述
 - 华为鲲鹏处理器型号及规格
 - 华为鲲鹏处理器技术创新
3. TaiShan服务器介绍
4. openEuler操作系统介绍
5. openGauss数据库介绍

华为鲲鹏处理器技术创新



内核全自研

- 自研CPU内核，算力提升50%
- 自研片间支持2路/4路互联
- 自研多种硬件协加速引擎



内存/网络接口&IO协议

- 支持8通道DDR4内存控制器
- 支持100G RoCE端口
- 支持PCIe 4.0/CCIX协议



制程工艺领先

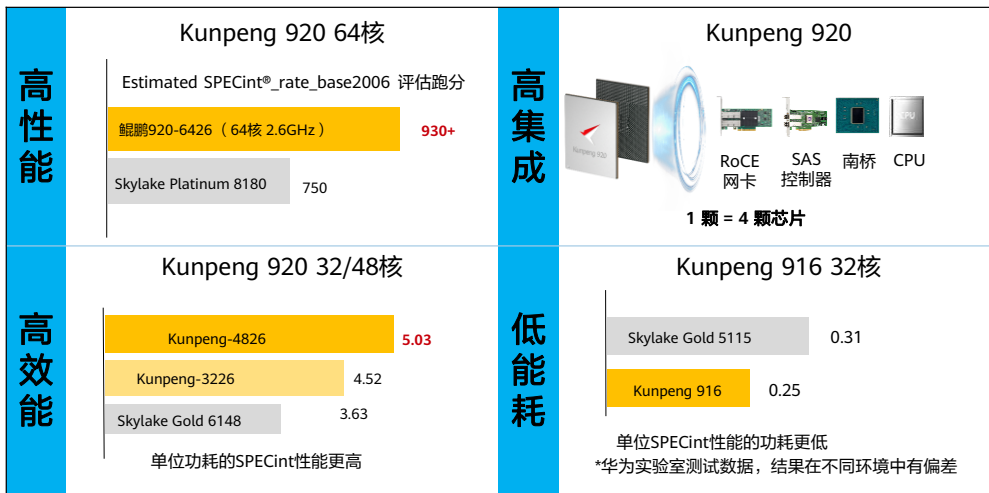
- 业界首款7纳米数据中心ARM CPU
- 采用业界领先的CoWoS封装技术，实现多Die合封



可靠性提升

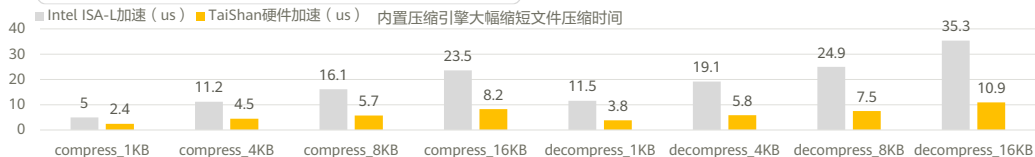
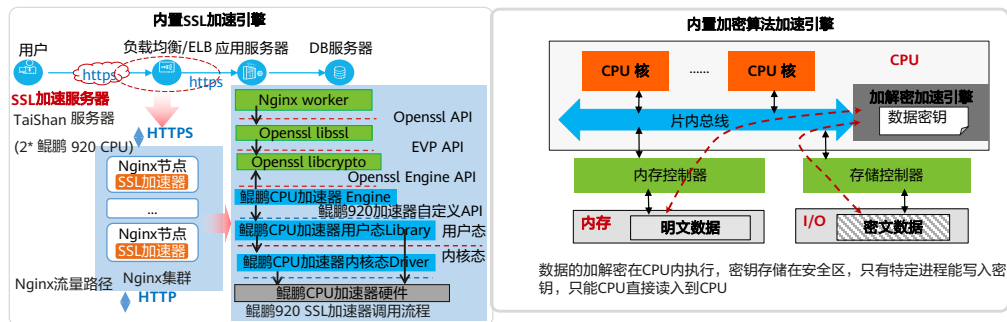
- 支持使用概率高、收益大的RAS特性45条，实现在RAS特性上的增强

内核全自研，性能提升



- 华为鲲鹏芯片的整体性能有很大提升，920 64核芯片SPECint[®]_rate_base2006跑分提升25% 效能提升30%，功耗下降17%，且拥有高集成的特点，一颗芯片集成4颗控制器。
- 用到的创新技术举例：
 - 分支预测算法改进的性能提升；
 - Memory 子系统深度重构，提升内存访问的outstanding深度；
 - 增加发射数到4发射；增加L1 Cache到64KB；每核独享512KB L2 Cache。

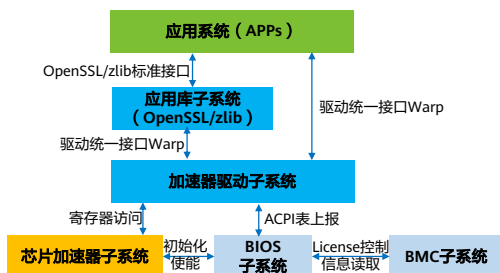
Kunpeng 920内置多种加速引擎



- 加速引擎是TaiShan 200服务器基于Kunpeng 920芯片提供的硬件加速解决方案，包含了对称加密、非对称加密和数字签名、压缩解压缩等算法，用于加速SSL/TLS应用和数据压缩，可以显著降低处理消耗，提高处理器效率。此外，加速引擎对应用层屏蔽了其内部实现细节，用户通过OpenSSL、zlib标准接口即可以实现快速迁移现有业务。

Kunpeng 920加速器简介

Kunpeng 920加速器系统逻辑架构



加速器子系统简介

目前加速引擎主要支持以下算法：

- 摘要算法SM3；
- 对称加密算法SM4，支持CTR/CBC模式；
- 非对称算法RSA，支持异步模型，支持 Key Sizes 1024/2048/3072/4096；
- 压缩解压缩算法，支持zlib/gzip。

安装方式：Kunpeng 920加速器子系统提供RPM安装和源码安装两种方式。

逻辑架构：

- **芯片加速器子系统**：集成在Kunpeng 920芯片中，提供加速器的能力，对上层提供寄存器接口。
- **加速器驱动子系统**：向上层提供各子加速器模块统一的驱动接口。
- **应用库子系统**：应用库子系统包括OpenSSL加速器引擎、zlib替代库等，向上层提供标准接口。
- **应用系统**：用户系统，通过调用应用库子系统或驱动子系统实现加速。
- **BIOS子系统**：单板BIOS软件系统，根据License初始化加速器模块，并上报加速器ACPI表到内核（加速器驱动子系统处理）。ACPI表：高级配置与电源接口（英文：Advanced Configuration and Power Interface，缩写：ACPI）。
- **BMC子系统**：服务器BMC软件系统，管理加速器License。

可靠性提升

- 华为鲲鹏920芯片结合IT产品实际应用需要，新增使用概率高，收益大的RAS特性45条，其中具有代表性的特性如下：

类别	特性
内存	可纠正错误精确统计
	自适应多区域替换（ADDDC）
PCIE	错误在线恢复（OER）
CPU	Core的错误容忍（PDC）
系统	错误恢复
	可纠正错误阈值和计数

- RAS: Reliability—可靠性：指的是系统能够正确产生输出结果达到一个给定时限的能力，这意味着一个具有可靠性的系统必须能够检测某些错误并能够做到自修复，或者对于无法自修复的错误也能够进行隔离并上报更高层次的自恢复机制处理，或者中止受损部分的运行并保障系统其余部分正常运转，更或者能够中止整个系统的工作并报告相应的错误。可靠性通常用FIT(Failure in Time)或MTBF（Mean Time Between Failure）来度量，1FIT = 1 error in 1 Billion Hours，约等于11.4万年MTBF。
- Availability—可用性：指的是系统能够在给定的时间内确保可以运行的能力，即使系统出现一些小的问题也不会影响整个系统的正常运行，在某些情况下甚至可以进行Hot Plug的操作，替换有问题的组件，从而严格的确保系统的宕机时间在一定范围内。可用性通常用一定时间段内的宕机时间（如每年宕机n分钟/小时）或者系统实际运行时间的百分比（如5个9，99.999%）来衡量。
- Serviceability—可服务性：指的是系统能够提供便利的诊断功能，如系统日志，动态检测等手段方便管理人员进行系统诊断和维护操作，从而及早地发现错误并且修复错误。
- 可纠正错误精确统计：内存可纠正错误以rank为单位提供计数及阈值设置，当rank内的可纠正错误达到阈值溢出，触发中断。同时，也需要考虑时间纬度，当给定一段时间内可纠正错误不增加时，计数器自动减1。
- 自适应多区域替换（ADDDC）：支持自适应的虚拟lockstep模式，以bank或者rank为粒度，每个DDR通道支持两个region，提供bank spare copy、DDEC sparing机制。

- 错误在线恢复（OER）：当硬件检测到错误时将相应root port与endpoint之间的link链路断开，接着软件把链路状态写回后，硬件执行链路重新协商。Root port与endpoint之间的链路断开并自恢复的过程，称为OER模式。
- Core的错误容忍（PDC）：Poison Data Containment模式基于数据的实际使用执行错误处理：错误源头和传输过程中，检测到不可纠正错误的模块并不会直接触发严重错误，而是对数据打上“中毒”标记并继续传输；最终使用数据的模块可以执行多样化的处理。
- 错误恢复：大部分的错误，最终能进行recovery的都是OS，但是非执行路径的错误，Firmware有一定的机会做recovery。如果不可纠正错误触发中断，Firmware接管后能够成功recovery，则需要修改MC bank寄存器的错误等级甚至清除错误记录，称之为MCA Recovery 2.0。
- 可纠正错误阈值和计数：当前我们实现Firmware First Handle，可纠正错误如果每次都触发中断，存在风险。比如之前遇到的风暴式可纠正错误，一直触发中断导致系统挂死。所以，我们在硬件上对每个模块实现可纠正错误计数和阈值。

思考题

1. （多选题）以下关于华为鲲鹏920处理器的描述，正确的是哪些项？（ ）
 - A. 采用了7nm的制造工艺
 - B. 支持8通道的DDR4控制器
 - C. 支持PCIe 4.0接口，并兼容PCIe 3.0/2.0/1.0
 - D. 支持多种加速器
2. （多选题）以下关于华为鲲鹏920处理器加速器的描述，正确的是哪些项？（ ）
 - A. SSL加速引擎
 - B. 加解密加速引擎
 - C. 压缩解压缩加速引擎

- 参考答案：

- ABCD
- ABC

思考题

1. （多选题）以下关于华为鲲鹏920处理器的描述，正确的是哪些项？（ ）
 - A. 采用了7nm的制造工艺
 - B. 支持8通道的DDR4控制器
 - C. 支持PCIe 4.0接口，并兼容PCIe 3.0/2.0/1.0
 - D. 支持多种加速器
2. （多选题）以下关于华为鲲鹏920处理器加速器的描述，正确的是哪些项？（ ）
 - A. SSL加速引擎
 - B. 加解密加速引擎
 - C. 压缩解压缩加速引擎

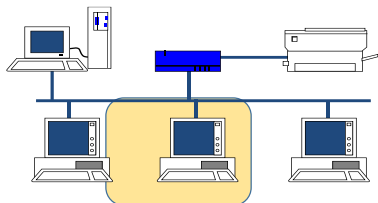
- 参考答案：

- ABCD
- ABC

什么是服务器？

- 服务器就是在网络中为其他客户机提供服务的计算机。

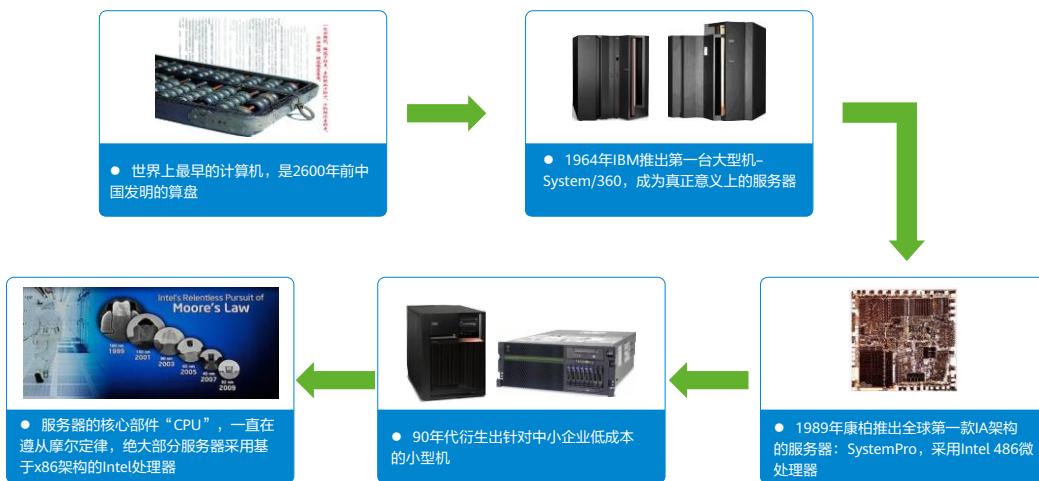
服务器是计算机的一种，它是在网络操作系统的控制下为网络环境里的客户机（如PC）提供共享资源（包括查询、存储、计算等）的高性能计算机，它的高性能主要体现在高速度的CPU运算能力、长时间的可靠运行、强大的I/O外部数据吞吐能力等方面。



服务器是干什么用的？

业务种类	业务名称
核心业务	ERP
	CRM
	数据库
	虚拟化
基础业务	Email及即时通信
	Web服务
	文件及打印
互联网业务	搜索
	内容服务器
	HPC高性能计算
创新业务	Hadoop
	ServerSAN

服务器发展历程



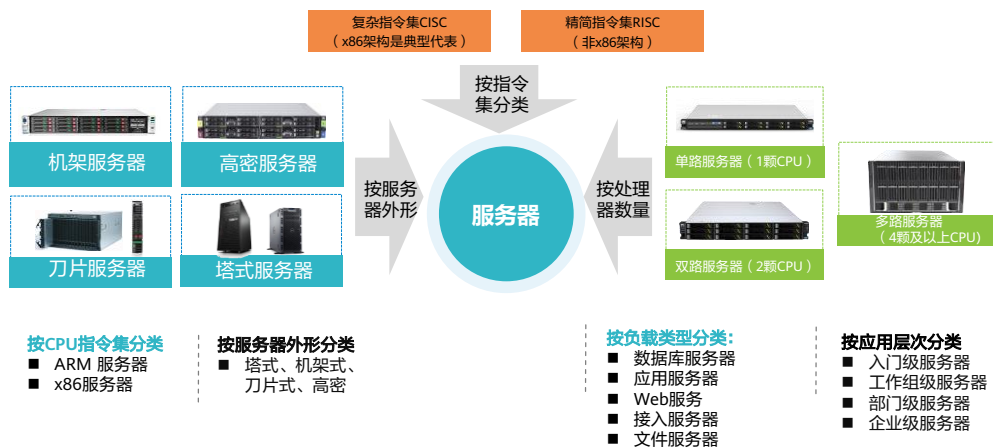
服务器的构成

- 服务器主要由CPU、内存、硬盘三大件组成，配合电源、主板、机箱等基础硬件以提供信息服务。



- 2280爆炸视图：
<https://info.support.huawei.com/computing/server3D/res/server/taishan2280/index.html?lang=cn>。

服务器主流类型



- 机架服务器：发货量最大。
- 刀片服务器：集成度最高。
- 高密服务器：密度最高、省电、节省空间。
- 塔式服务器：一般用于小型办公场所。
- CISC：主要是两家，包括Intel CPU（非安腾系列）、AMD CPU。
- RISC：服务器领域主要是IBM Power系列、Sun Spark系列，消费级的代表是ARM架构的CPU。
 - 单路服务器：低端产品，华为只有一款单路服务器，不是主流。
 - 双路服务器：主力发货机型。

不同服务器的应用场景



- Scale-up 纵向扩展：提升单台服务器的性能，用以完成无法分割的交易型或高计算负载业务，如金融交易、科学研究、气象分析等领域。
- Scale-out 横向扩展：通过分布式架构，将工作任务拆散给多台服务器进行处理，对单台服务器的性能要求不高，但因服务器数量众多所以对空间和能耗敏感，要求省空间、低能耗。
- Hyper-converged 超融合：将计算、存储、网络、管理放到一个箱子中，达到高度融合、简单易用、优化性能的目的。

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. **TaiShan服务器介绍**
 - 服务器通识类知识简介
 - **TaiShan服务器概述**
 - TaiShan 200 机架服务器介绍
 - TaiShan 200 高密服务器介绍
4. openEuler操作系统介绍
5. openGauss数据库介绍

基于华为鲲鹏处理器，构建整机计算能力

高效能计算

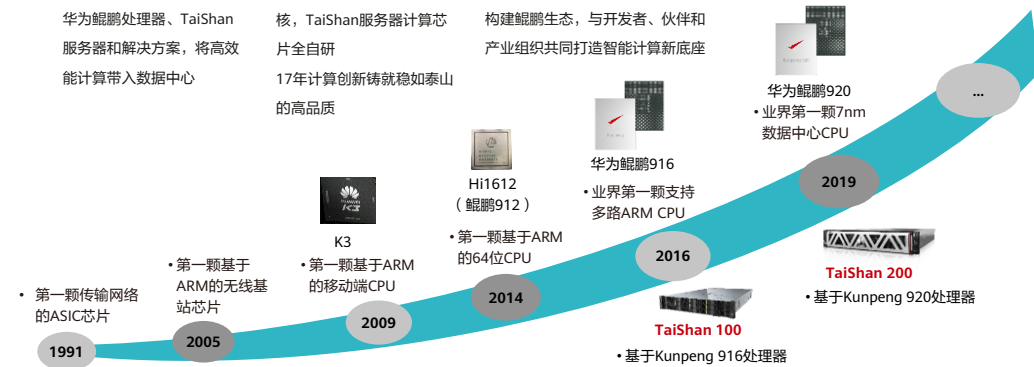
提供兼容ARM架构的高性能
华为鲲鹏处理器、TaiShan
服务器和解决方案，将高效
能计算带入数据中心

安全可靠

华为鲲鹏处理器基于自研内
核，TaiShan服务器计算芯
片全自研
17年计算创新铸就稳如泰山
的高品质

开放生态

开放平台，支持业界主流软硬件；
构建鲲鹏生态，与开发者、伙伴和
产业组织共同打造智能计算新底座

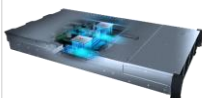


- 2016年推出第一代TaiShan 100服务器基于鲲鹏916处理器，以华为鲲鹏处理器为基础，构建整机计算能力。
- 2019年推出TaiShan 200服务器基于鲲鹏920处理器，目前仍然是市场的主打产品。

TaiShan服务器系列介绍

TaiShan 100服务器

- 基于Kunpeng 916处理器
- 最多16个DDR4内存
- 支持5个PCIe 3.0扩展插槽
- 支持SAS/SATA硬盘和SSD
- 支持板载GE/10GE网络



TaiShan 200服务器

- 基于Kunpeng 920处理器
- 最多32个DDR4内存
- 支持最多8个PCIe 4.0扩展插槽
- 支持NVMe SSD、SAS/SATA硬盘和SSD
- 支持100GE板载网络
- 支持板级和全液冷技术

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. **TaiShan服务器介绍**
 - 服务器通识类知识简介
 - TaiShan服务器概述
 - **TaiShan 200 机架服务器介绍**
 - TaiShan 200 高密服务器介绍
4. openEuler操作系统介绍
5. openGauss数据库介绍

TaiShan 200服务器全景图



61 Huawei Confidential



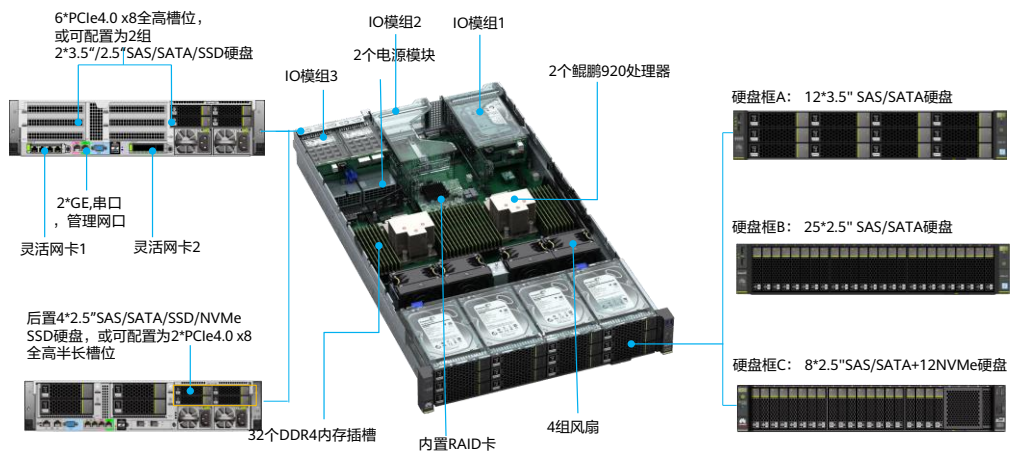
- 基于Kunpeng处理器，华为打造了鲲鹏计算整机产品标杆 – TaiShan服务器。
- TaiShan服务器是华为在计算技术和整机工程方面长期积累的结晶。
- 首先，华为在TaiShan 200服务器上充分应用了散热液冷、高速互联、可靠性设计与质量品控等工程工艺技术，将TaiShan服务器打造成为精品，为Kunpeng处理器应用在数据中心服务器领域树立了一个行业标杆。
- TaiShan服务器目前已经规模商用的有2280均衡型、5280存储型、X6000高密型。

2280均衡型规格与亮点

2280 均衡型	前视图	型号	2280	主要特点
		形态	2U机架服务器	高性能 - 鲲鹏920处理器，性能比肩x86高端型号 8通道内存技术，支持32个DDR4内存插槽，最高内存容量可达4TB - 支持华为Atlas 300 AI加速卡、ES3000 V5 NVMe SSD
		CPU	2个鲲鹏920处理器	
		内存	32个DDR4-2933 DIMM插槽	
		本地存储	16*3.5"SAS/SATA HDD硬盘 27*2.5"SAS/SATA HDD硬盘 16*2.5" NVMe SSD硬盘	
		RAID支持	RAID0/1/5/6/10/50/60等	灵活适配 - 支持多个IO模组，实现丰富的硬盘配置 - 支持板载的灵活网卡，支持GE/10GE/25GE，实现不同网络配置
		PCIe扩展	最多8个PCIe 4.0 x8或3个PCIe 4.0 x16 + 2个PCIe 4.0 x8标准插槽	
		板载网卡	2个板载网络插卡，每个插卡支持4*GE电口或者4*10GE光口或者4*25GE光口	安全可靠 - 采用华为全自研计算芯片 - 整机器件实现全国产化，保障可持续供应
		电源	2个900W或2000W热插拔电源，支持AC 220V或DC 240V，支持1+1冗余	
		风扇	4个热拔插风扇，支持N+1冗余	
		温度范围	5°C~40°C	

- <https://e.huawei.com/cn/products/servers/taishan-server/taishan-2280-v2>。
- 8通道内存技术，比x86 V5 CPU多2个通道，因此可以支持32个DDR4内存插槽，最高内存容量可达4TB。

2280型号内部结构

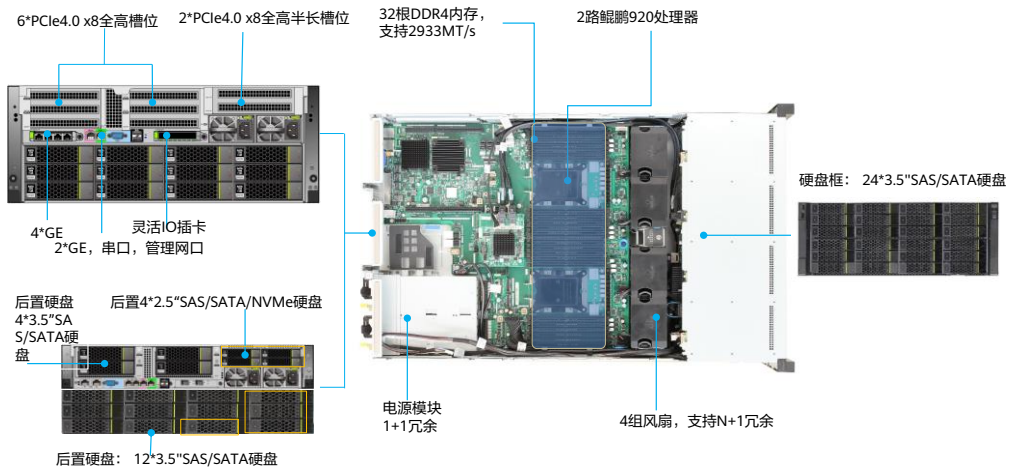


5280存储型规格与亮点

5280 存储型	前视图	型号	5280	主要特点
		形态	4U机架服务器	超大存储 - 支持最多40个3.5"硬盘，本地存储容量可达560TB。 - 鲲鹏920处理器，性能比肩x86高端型号，高效支持数据存储。 8通道内存技术，支持32个DDR4内存插槽，最高内存容量可达4TB。
		CPU	2个鲲鹏920处理器	
		内存	32个DDR4 DIMM插槽，最高2933MT/s	
		本地存储	前端配置24个3.5"SAS/SATA/SSD硬盘，后端可最多配置16个3.5"SAS/SATA/SSD硬盘，以及4个2.5"NVMe SSD硬盘	
	后视图	RAID支持	支持RAID 0, 1, 5, 6, 10, 50, 60 支持超级电容掉电保护	灵活适配 - 支持多个IO模组，实现丰富的硬盘配置。 - 支持板载的灵活网卡，支持GE/10GE/25GE，实现不同网络配置。
		PCIe扩展	最多8个PCIe 4.0 x8或3个PCIe 4.0 x16+2个PCIe x8	
		灵活网卡	4*GE OR 4*10GE OR 4*25GE	
		电源	2个2000W 热插拔电源，支持1+1冗余	
		风扇	4个热插拔风扇，支持N+1冗余	安全可靠 - 采用华为全自研计算芯片。 - 整机器件实现全国产化，保障可持续供应。
		温度范围	5° C~35° C	

- <https://e.huawei.com/cn/products/servers/taishan-server/taishan-5280-v2>。
- 4U跟2U的机型是共主板的，只是硬板背板不同，支持最多40个3.5"硬盘。

5280型号内部结构



- 5280内部结构跟2280大致相同，最主要的差别是多了下面12块硬盘的连接。

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
- 3. TaiShan服务器介绍**
 - 服务器通识类知识简介
 - TaiShan服务器概述
 - TaiShan 200 机架服务器介绍
 - TaiShan 200 高密服务器介绍
4. openEuler操作系统介绍
5. openGauss数据库介绍

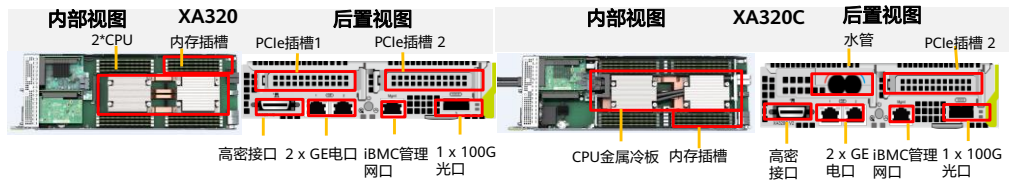
X6000机框规格



产品	高密服务器
形态	2U机框，支持4个XA320/XA320C半宽服务器
电源	2个热插拔3000W电源模块，支持1+1冗余
供电	支持100~240V AC，240V DC
风扇	4个热插拔风扇模组，支持N+1冗余
工作温度	5°C- 35°C
尺寸（宽x深x高）	436mmx818.9mmx86.1mm

- X6000采用机箱与节点分离的模块化设计。机箱为2U标准机箱，满足标准19寸1m机柜安装。

XA320 & XA320C计算节点规格



形态	XA320计算节点	XA320C计算节点	说明
处理器	2*Kunpeng920 (48核2.6G)	2*Kunpeng920 (48核2.6G/64核2.6G)	64核2.6G风冷Q3支持
内存插槽	16个DDR4 2933		目前最大支持32GB，64GB在Q3支持
硬盘数量	支持2~6个2.5"SAS/SATA硬盘		- NVMe在Q3支持 - 风冷目前只支持2个硬盘，48核可以支持6个硬盘 - 风冷64核只能支持2个硬盘
板载网络	2*GE电口+1*100GE光口		
PCIe扩展	支持2个PCIe半高半长的标准扩展插槽	支持1个PCIe半高半长的标准扩展插槽	4*25G标卡（1822）在Q3支持
工作温度	5°C- 35°C		35°C环境温度时每节点最大支持2个硬盘
尺寸（宽x深x高）	177.9mm x 545.5mm x 40.5mm		

- 典型应用场景为HPC高性能计算。
- 48核和64核两款计算节点。

支持3000W电源和液冷散热，释放HPC计算潜力

和英特尔可扩展系列处理器算力对比（2P）

CPU型号	总核数	主频GHz	最大TDP	内存带宽	CFD应用性能*
2*Gold6248	40	2.5	150	192GB/s	1
2*Kunpeng920-4826	96	2.6	150	287GB/s	1.2

X6000超级机柜提升单机柜HPC算力 **20%**

- 相比x86 6248方案，CFD场景单机柜算力提升20%
- 相同空间下可提升用户业务性能20%
- 可缩减用户CFD仿真时间20%，加速业务研发

固定算力高密部署场景

方案	单机柜框数	机柜数	CFD应用性能*	节省空间
2*Kunpeng920-4826@风冷	15	4	240	33%
2*Kunpeng920-4826@全液冷	20	3	240	

X6000超级机柜节省HPC集群机柜数量 **33%**

- 构建固定应用算力HPC集群，X6000风冷散热需要部署4个机柜
- X6000全液冷散热仅需要3个机柜

*使用openFOAM评估，测试数据为华为实验室数据，不同环境可能存在误差。

- 高密设计，比传统机架省空间50%。
- 对比X6000的x86 6248方案，仍然提升单柜算力20%。
- TDP（Thermal Design Power）：散热设计功耗。
- HPC（High Performance Computing）：高性能计算。
- CFD（Computational Fluid Dynamics），即计算流体动力学。

思考题

1. （多选题）以下哪些型号属于TaiShan 200机架服务器？（ ）

- A. 2280
- B. 5280
- C. 2480
- D. RH2288

- 答案：ABC。RH2288属于x86服务器。

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. TaiShan服务器介绍
- 4. openEuler操作系统介绍**
 - Linux操作系统介绍
 - openEuler操作系统介绍
 - 嵌入式OS介绍
 - openEuler基础操作命令
5. openGauss数据库介绍

操作系统介绍

- 操作系统是一个管理计算机硬件与软件资源的程序，同时也是计算机系统的内核与基石。操作系统身负诸如管理与配置内存、决定系统资源供需的优先次序、控制输入与输出设备、操作网络与管理文件系统等基本事务。操作系统也提供一个让使用者与系统交互的操作接口。小到智能穿戴设备、电视盒子，大到电脑、服务器、路由交换设备，都有自己的操作系统。

Android、IOS、鸿蒙、...



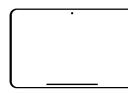
手机

Windows、红旗、
MAC OS...



电脑

Android、IOS、...



平板电脑

Android、鸿蒙、...



智能家居设备

.....

Linux操作系统起源

- 1991年10月5日，Linus Torvalds在comp.os.minix新闻组织上正式对外宣布Linux内核的诞生，并且开源。
- 在开源社区参与者的努力下，1994年3月Linux 1.0内核版本正式发布。
- Linux是一套免费使用和自由传播的类Unix操作系统，是一个基于POSIX和Unix的多用户、多任务、支持多线程和多CPU的操作系统。

- GNU：GNU是一个操作系统，其内容软件完全以GPL方式发布。这个操作系统是GNU计划的主要目标，名称来自GNU's Not Unix!的递归缩写，因为GNU的设计类似Unix，但它不包含具著作权的Unix代码。
 - 1984年由Richard Stallman发起并创建，意在创造一个自由使用、自由学习和修改、自由分发、自由创建衍生版软件的环境。
 - GNU目标是编写大量兼容于Unix系统的自由软件。
- GPL：GNU通用公共许可证简称为GPL，是由自由软件基金会发行的用于计算机软件的协议证书，使用该证书的软件被称为自由软件。
 - 自由软件的通用许可证。
 - 允许用户任意复制、传递、修改及再发布。
 - 基于自由软件修改再次发布的软件，仍需遵循GPL。
- LGPL
 - 相对于GPL较为宽松，允许不公开全部源代码。
 - 为基于Linux平台开发商业软件提供了更多空间。

Linux操作系统组成

- Linux操作系统构成
 - Linux内核
 - 应用程序
- Linux内核项目介绍
 - 主要作者：Linus Torvalds（芬兰）
 - 1991年10月，发布Linux 0.02版（第一个公开版）
 - 1994年3月，Linux 1.0版发布
 - Linux内核的标志——企鹅Tux，取自芬兰的吉祥物

常见的Linux操作系统

Linux操作系统

RedHat	Debian	
CentOS	Ubuntu	
Fedora	deepin	
Kaili Linux	Oracle Linux	
DVL	Arch Linux	
openEuler	中标麒麟	

- Linux开源的缘故，众多公司基于开源源代码做了二次开发，后发布了不同的发行版Linux操作系统，此处仅列举了一部分Linux操作系统。

CentOS简介

- CentOS Linux发行版是一个稳定的，可预测的，可管理的和可复现的平台，源于Red Hat Enterprise Linux（RHEL）依照开放源代码（大部分是GPL开源协议）规定释出的源码所编译而成。
- CentOS在2014年加入红帽后，继续坚持提供免费操作系统的使用和更新。目前CentOS操作系统的使用领域非常广泛，适合服务器使用。
- 2020年12月08日，CentOS 官方宣布CentOS Linux项目停止，因此当前使用CentOS系统的用户面临停服后如何更新、维护、系统迁移等问题。

- <https://www.redhat.com/pt-br/engage/centos>。
- TaiShan服务器支持Cent OS 7.4 for ARM及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

Debian简介

- Debian是迄今为止最遵循GNU规范的Linux系统。提供了接近十万种不同的开源软件支持，认可度和使用率较高，对于各类内核架构支持性良好，稳定性、安全性强，更有免费的技术支持。
- 基于其优秀的稳定性和资源开销小等特点，Debian在虚拟专用服务器中应用比较多。

- Debian社区：<https://www.debian.cn>。

Ubuntu简介

- Ubuntu是Debian的一款衍生版，是一个以桌面应用为主的，开源且免费的操作系统。Ubuntu侧重于在服务器、云计算、甚至一些运行Ubuntu Linux的移动设备上进行系统部署。
- Ubuntu具有庞大的社区力量，用户可以方便地从社区获得帮助。

- Ubuntu社区：<https://ubuntu.com/community>。
- TaiShan服务器支持Ubuntu 16.04.3 LTS ARM及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

deepin简介

- deepin操作系统是由武汉深之度科技有限公司开发的Linux发行版。deepin操作系统拥有自主设计的特色软件：深度软件中心、深度截图、深度音乐播放器和深度影音，全部使用自主的deepinUI。
- deepin操作系统专注于使用者对日常办公、学习、生活和娱乐的操作体验的极致，适合笔记本、桌面计算机和一体机。

- <https://www.deepin.org/index/zh>。
- deepin不完全免费。
- TaiShan服务器支持deepin V15 for ARM及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

中标麒麟简介

- 中标麒麟是中标Linux和银河麒麟合并后发布的一款Linux发行版本，中标麒麟操作系统采用强化的Linux内核，分成桌面版、通用版、高级版和安全版等，满足不同客户的要求，已经广泛地使用在能源、金融、交通、政府、央企等行业领域。

- <https://cs2c.com.cn/scheme/product/1.html>。
- TaiShan服务器支持NeoKylin Server v5.0 U5 for ARM及以上版本，但并不是所有服务器都兼容这些版本，具体兼容信息请在智能计算产品兼容性查询助手上查询。

各行业常用的OS

行业	常用OS
电信运营商	CentOS等
银行、证券	CentOS、中标麒麟、银河麒麟等
互联网	企业定制OS、Debian等
政府、央企	CentOS、中标麒麟、银河麒麟、统信等
电力、能源	CentOS、中标麒麟、银河麒麟、Ubuntu等
交通运输	CentOS、中标麒麟等
安平	CentOS等
教育、医疗	CentOS、银河麒麟等

华为鲲鹏处理器OS兼容性

Kunpeng	常用OS
	deepin
	ubuntu
	银河麒麟
	suse
	Debian
	CentOS
	中标麒麟

- 此处列举了华为鲲鹏处理器兼容的一部分操作系统，目前鲲鹏处理器仅支持Linux类型操作系统，上述列举出了鲲鹏处理器支持的部分操作系统。

TaiShan服务器OS兼容性查询

- 在查询时，可以指定部件类型，这样查询到的OS兼容性信息更加精确。

[illegible]

- 《智能计算产品兼容性查询助手》网站链接如下：
<http://support.huawei.com/online/toolweb/ftca/index?serise=9>。

目录

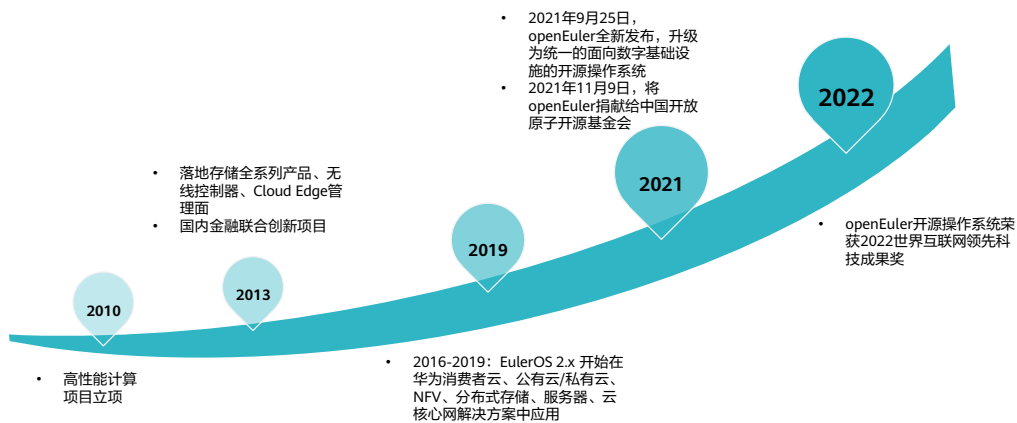
1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. TaiShan服务器介绍
4. **openEuler操作系统介绍**
 - 鲲鹏平台兼容的操作系统介绍
 - openEuler操作系统介绍
 - 嵌入式OS介绍
5. openGauss数据库介绍

openEuler概述

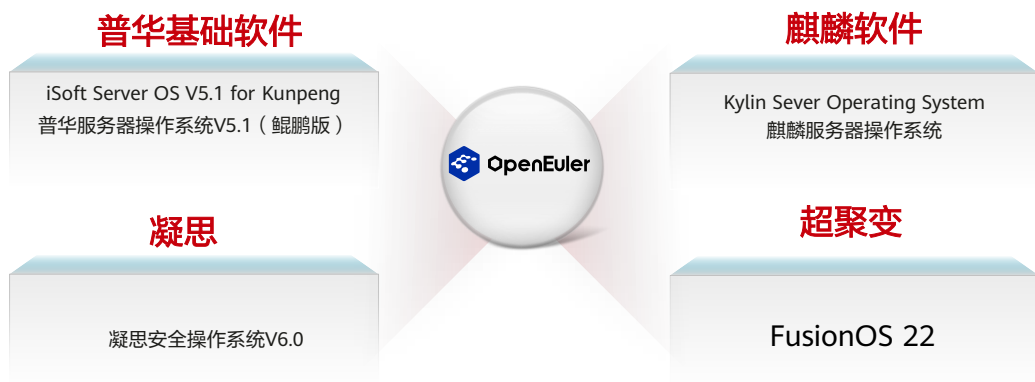
- openEuler脱胎于EulerOS，EulerOS是华为公司自2010年起研发使用的服务器操作系统，Linux发行版之一，名字来源于著名数学家莱昂哈德·欧拉（Leonhard Euler）。
- 2019年9月，EulerOS正式开源，命名为openEuler。
- openEuler是一款开源操作系统。当前openEuler内核源于Linux，支持鲲鹏及其它多种处理器，能够充分释放计算芯片的潜能，是由全球开源贡献者构建的高效、稳定、安全的开源操作系统。openEuler适用于数据库、大数据、云计算、人工智能等应用场景。同时，openEuler也是一个面向全球的操作系统开源社区名称。

- 欧拉操作系统可广泛部署于服务器、云计算、边缘计算、嵌入式等各种形态设备，应用场景覆盖IT（Information Technology）、CT（Communication Technology）和OT（Operational Technology），实现统一操作系统支持多设备，应用一次开发覆盖全场景。

openEuler发展历程

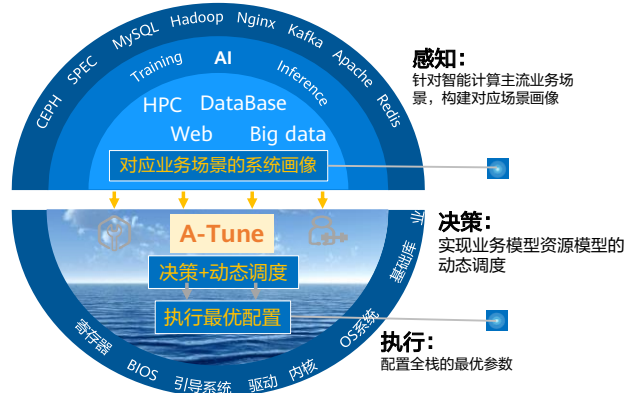


基于openEuler的商用发行版



openEuler技术特性 - A-Tune资源调优自动化

- A-Tune是一种通过非侵入式系统画像的负载感知方法，识别业务并匹配最佳资源模型，实时响应业务特征变化的AI自动调优系统。



- 识别鲲鹏主流行业场景，A-Tune基于系统画像完成业务负载归类，实现资源模型建模和系统自优化，发挥鲲鹏算力。
- 基于机器学习完成系统画像，结合聚类+分类多次迭代实现业务负载的精确归类，涵盖当前ICT业务主流计算场景。
- 基于业务特征完成系统资源建模，合理优化系统资源及参数配置，实现系统能力的价值最大化。
- 7000+系统配置项。

openEuler技术特性 - iSulad容器解决方案（1）

- iSula通用容器引擎（iSulad）是一种新的容器解决方案，具有以下特点：

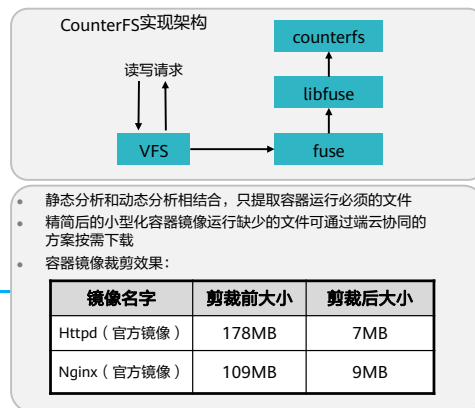
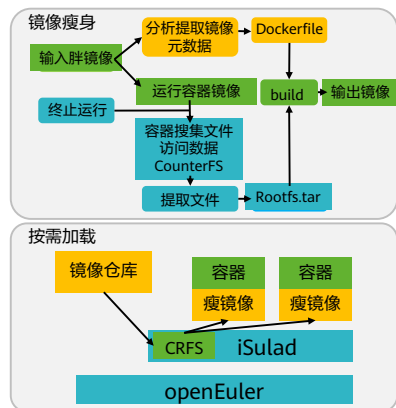
- 快速灵活&轻量级
- 可信&安全启动
- 升级不中断业务
- 增强安全性和调测特性



- iSula通用容器引擎（iSulad）是一种新的容器解决方案，提供统一的架构设计来满足CT和IT领域的不同需求。相比Golang编写的Docker，轻量级容器具有轻、灵、巧、快的特点，不受硬件规格和架构的限制，底噪开销更小，可应用领域更为广泛。openEuler-20.03-LTS中提供容器运行的基础平台iSula。
- 容器根据部署模型可以分成三种模型：应用容器、安全容器、系统容器。
 - 应用容器：应用最广泛的容器形态，容器中仅运行客户定义的应用程序。openEuler-20.03-LTS 集成moby 18.09版本，并在版本基础上进行了bugfix和稳定性增强。
 - 安全容器：本质上是虚拟机，但是接口封装为容器接口。openEuler-20.03-LTS 集成kata-container1.7版本，在社区版本的基础上，进行了稳定性和可靠性加固，增加了异构计算支持，更适于生产环境的Host CGroup配置与 DPDK/SPDK 高性能网络、存储支持。
 - 系统容器：本质上是容器，但是使用方式和虚拟机相同；着重解决虚拟机的厚重问题。iSula容器平台支持创建系统容器，并能支持在系统容器内动态调整设备、运行资源，且提供更优秀的user namespace隔离。

openEuler技术特性 - iSulad容器解决方案

- iSulad容器基础镜像支持按需剪裁，实现极致小型化。



鲲鹏硬件相关的特性 - MPAM

- MPAM (Memory Partitioning and Monitoring) 适用于控制/监控L3 Cache和内存带宽的分配和使用，对标x86高端芯片的RDT特性。
- 在openEuler的21.03版本，也支持通过Arm64架构 Cache QoS 以及内存带宽控制技术进行资源管控。

openEuler其他技术特性概览

系统安装

openEuler-22.03-LTS可以让用户使用kickstart工具进行系统的自动化安装。

场景融合

openEuler-22.03-LTS是欧拉首个支持全场景融合的社区长周期版本，满足服务器、云计算、边缘计算和嵌入式四大场景的多种不同类型设备部署要求和应用场景。

创新特性

openEuler 22.03-LTS合入了openEuler三个创新版中经过商业验证的创新特性，并针对四大场景首次发布新增特性，共新增代码2300 万行。

系统管理

openEuler-22.03-LTS使用systemd进行系统和服务的管理，systemd与SysV和Linux标准的init脚本兼容。



系统安全

openEuler-22.03-LTS提供多重安全手段，包括身份识别与认证、安全协议、强制访问控制、完整性保护、安全审计等安全机制，保障操作系统的安全性，为各类上层应用提供安全基础。

系统分析与调优

openEuler-22.03-LTS支持包含MySQL性能调优和大数据调优，包括如spark调优、hive调优、hbase调优。

编译器

openEuler-22.03-LTS支持使用毕昇编译器。毕昇编译器是华为编译器实验室针对鲲鹏等通用处理器架构场景，打造的一款高性能、高可信及易扩展的编译器工具链，增强和引入了多种编译优化技术，支持C/C++/Fortran等编程语言。

内核

openEuler-22.03-LTS采用kernel版本5.10。Linux Kernel 5.10是一个LTS 版本，提供长期支持并持续更新版本，以保障用户的Linux操作系统安全、可靠。

openEuler开源社区



01

开发者

- openEuler社区采用gitee作为开发系统。
- 开发者可以在开源社区提交bug、参与讨论、贡献代码文案, 也可以为openEuler的技术发展提出宝贵的建议, 共同建设openEuler的繁荣生态。

02

文档

- <https://openeuler.org>提供openEuler的所有文档, 包括发行说明、安装指南、管理员指南, 以及openEuler关键技术描述等文档。任何人都可以在<https://openeuler.org>上查看相关文档, 学习openEuler。

03

下载

- <https://openeuler.org>提供openEuler相关操作系统的下载, 有需求的玩家可以在开源社区下载openEuler相关操作系统。

04

社区会员

- openEuler开源社区欢迎各公司或者组织加入社区, 共同维护和管理社区, 共同打造操作系统生态圈。

- openEuler开源社区网址是: <https://www.openeuler.org/zh/>。

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. TaiShan服务器介绍
4. **openEuler操作系统介绍**
 - 鲲鹏平台兼容的操作系统介绍
 - openEuler操作系统介绍
 - 嵌入式OS介绍
5. openGauss数据库介绍

嵌入式操作系统概述

- 嵌入式操作系统（Embedded Operating System，简称：EOS）是指用于嵌入式系统的操作系统。嵌入式操作系统是一种用途广泛的系统软件，通常包括与硬件相关的底层驱动软件、系统内核、设备驱动接口、通信协议、图形界面、标准化浏览器等。嵌入式操作系统负责嵌入式系统的全部软、硬件资源的分配、任务调度，控制、协调并发活动。它必须体现其所在系统的特征，能够通过装卸某些模块来达到系统所要求的功能。目前在嵌入式领域广泛使用的操作系统有：嵌入式实时操作系统 μ C/OS-II、嵌入式Linux、Windows Embedded、VxWorks等，以及应用在智能手机和平板电脑的Android、iOS等。

嵌入式场景的欧拉OS

- 当前工业、医疗、军工/航空航天等行业，嵌入式OS基本被国外垄断，国内嵌入式软硬件能力已初步具备，但整体呈现“小、散、弱”的特点，技术水平和市场竞争力不足，缺乏完整技术体系。

“十四五”智能制造发展规划

2025年，智能制造装备和工业软件技术水平和市场竞争力显著提升，国内市场满足率分别超过70%和50%。主营业务收入超50亿元的系统解决方案供应商达到10家以上

发改委关基工程（12个领域）

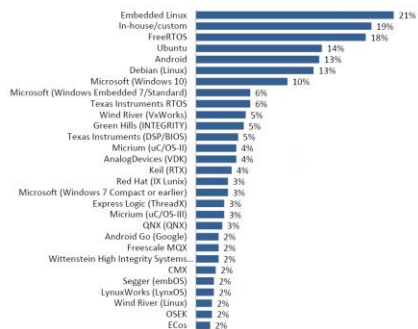
社会保障	国防科技工业	能源
邮政	民航	电信
公路水运	金融	铁路
水利	卫生健康	广电

- FGW办公厅、WXB秘书局【2019】1008号、【2020】409号
- 《关于组织实施关键信息基础设施安全可控应用示范工程的通知》

6大嵌入式行业



全球主要嵌入式操作系统应用情况



来源：EETime调研分析报告

- 全球产业链深度重构，主要嵌入式领域OS被国外垄断，国内力量“小、散、弱”，体系化程度低。

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. TaiShan服务器介绍
4. openEuler操作系统介绍
- 5. openGauss数据库介绍**
 - 数据库概述
 - openGauss数据库介绍
 - openGauss基础操作
 - 应用开发简介

数据库Database

- 数据库是“按照数据结构来组织、存储和管理数据的仓库”，是一个长期存储在计算机内的、有组织的、可共享的、统一管理的大量数据的集合。

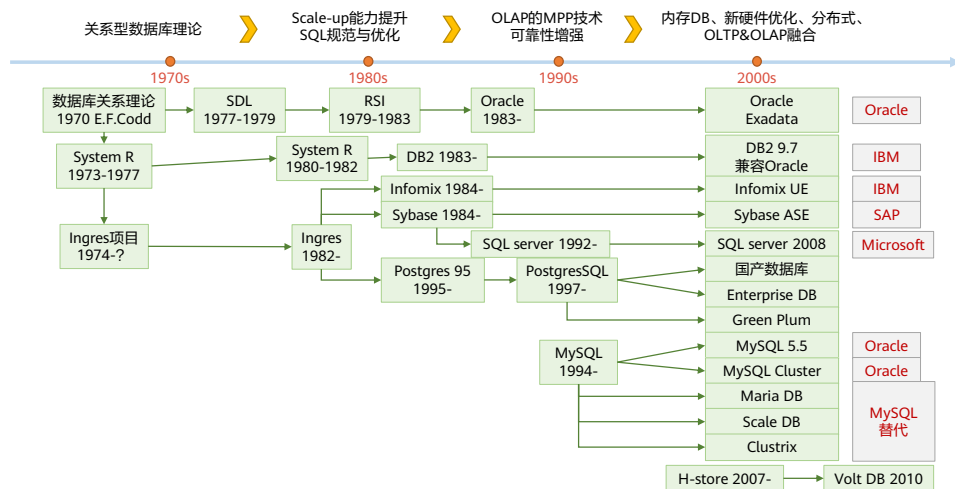


永久存储

有组织

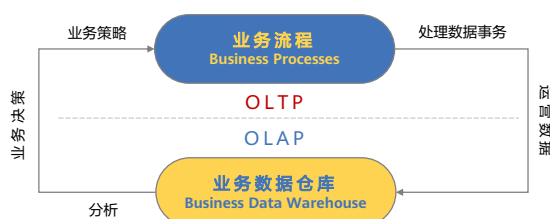
可共享

关系型数据库发展历程



- 关系模型的提出是数据库发展史上具有划时代的重大事件。关系理论研究和关系数据库管理系统研制的巨大成功进一步促进了关系数据库的发展。

关系型数据库主流应用场景



联机事务处理 OLTP On-Line Transaction Processing

- OLTP是传统的关系型数据库的主要应用，事务性非常高的系统，一般都是高可用的在线系统，以小的事务以及小的查询为主，主要是基本的、日常的事务处理，例如银行交易。
- OLTP 系统强调数据库内存效率，评估其系统的时候，一般看其每秒执行的Transaction以及Execute SQL的数量。强调内存各种指标的命中率，强调绑定变量，强调并发操作。

联机分析处理 OLAP On-Line Analytical Processing

- OLAP是数据仓库系统的主要应用，支持复杂的分析操作，侧重决策支持，并且提供直观易懂的查询结果。在这样的系统中，语句的执行量不是考核标准，因为一条语句的执行时间可能会非常长，读取的数据也非常多。
- OLAP 系统则强调数据分析，考核的标准往往是磁盘子系统的吞吐量（带宽），强调SQL执行时长，强调磁盘I/O，强调分区等。

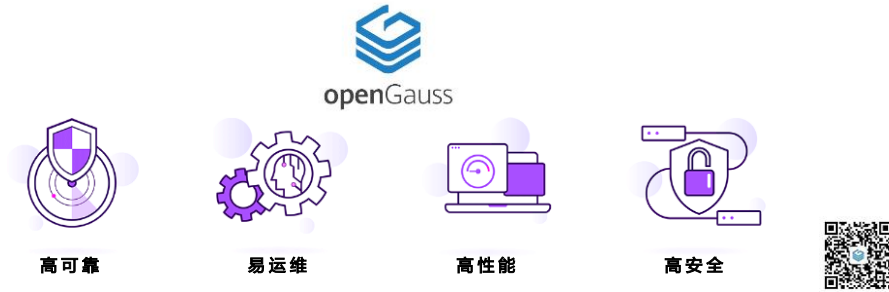
- OLTP是传统关系数据库的主要应用：面向基本的，日常的事务处理，例如银行储蓄业务的存取交易，转账交易等。
 - 特点：（1）大吞吐量：大量的短在线事务（插入、更新、删除），非常快速的查询处理。（2）高并发，（准）实时响应。
 - 典型的OLTP场景：零售系统、金融交易系统、火车机票销售系统、秒杀活动。
- OLAP：联机分析处理的概念最早是E.F.Codd于1993年相对于OLTP系统而提出的。是指对数据的查询和分析操作，通常对大量的历史数据查询和分析。涉及到的历史周期比较长，数据量大，在不同层级上的汇总，聚合操作使得事务处理操作比较复杂。
 - 特点：（1）主要面向侧重于复杂查询，回答一些“战略性”的问题。（2）数据处理方面聚焦于数据的聚合，汇总，分组计算，窗口计算等“分析型”数据加工和操作。（3）从多维度去使用和分析数据。
 - 典型的OLAP场景：报表系统、CRM系统、金融风险预测预警系统、反洗钱系统、数据集市、数据仓库。

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. TaiShan服务器介绍
4. openEuler操作系统介绍
- 5. openGauss数据库介绍**
 - 数据库概述
 - openGauss数据库介绍
 - openGauss基础操作
 - 应用开发简介

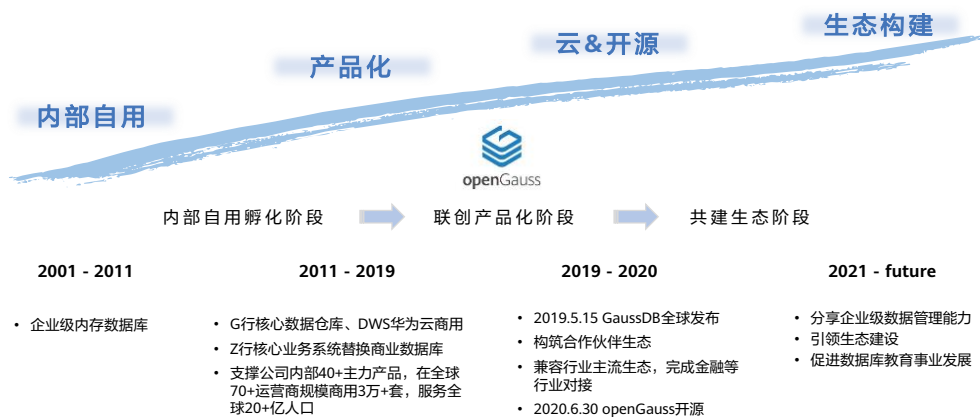
openGauss

- openGauss是一款携手伙伴共同打造全球领先的企业级开源关系型数据库，提供面向多核的极致性能、全链路的业务和数据安全、基于AI的调优和高效运维的能力。openGauss全面友好开放，采用木兰宽松许可证v2发行，深度融合华为在数据库领域多年的研发经验，结合企业级场景需求，持续构建竞争力特性。



- 多种存储模式支持复合业务场景。
- NUMA化数据结构支持高性能。
- 主备模式，CRC校验支持高可用。

openGauss发展历程



高性能

- 提供了面向多核架构的并发控制技术结合鲲鹏硬件优化，在两路鲲鹏下TPCC Benchmark达成性能150万tpmc。
- 针对当前硬件多核的架构趋势，在内核关键结构上采用了NUMA-Aware的数据结构。
- 提供Sql-bypass智能快速引擎技术。

其他性能



高可用

- 支持主备同步、异步以及级联备机多种部署模式。
- 数据页CRC校验，损坏数据页通过备机自动修复。
- 备机并行恢复，10秒内可升主提供服务。



高安全

- 支持全密态计算、访问控制、加密认证、数据库审计、动态数据脱敏等安全特性，提供全方位端到端的数据安全保护。



易运维

- 基于AI的智能参数调优和索引推荐，提供AI自动参数推荐。
- 慢SQL诊断，多维性能自监控视图，实施掌控系统的性能表现。
- 提供在线自学习的SQL时间预测。



全开发

- 采用木兰宽松许可证协议，允许对代码自由修改、使用、引用。
- 数据库内核能力全开放。
- 提供丰富的伙伴认证、培训体系和高校课程。

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. TaiShan服务器介绍
4. openEuler操作系统介绍
- 5. openGauss数据库介绍**
 - 数据库概述
 - openGauss数据库介绍
 - openGauss基础操作
 - 应用开发简介

SQL定义

- 维基百科的定义：
 - SQL（Structured Query Language，结构性查询语言）是一种特定目的编程语言，用于管理关系数据库管理系统，或在关系流数据管理系统中进行流处理。
 - SQL基于关系代数和元组关系演算，包括一个数据定义语言和数据操作语言。SQL的范围包括数据插入、查询、更新和删除，数据库模式创建和修改，以及数据访问控制。

- openGauss支持标准的SQL92/SQL99/SQL2003/SQL2011规范，支持GBK和UTF-8字符集，支持SQL标准函数与分析函数，支持存储过程。
- SQL是用于访问和处理数据库的标准计算机语言。
- SQL提供了各种任务的语句，包括：
 - 查询数据。
 - 在表中插入，更新和删除行。
 - 创建，替换，更改和删除对象。
 - 控制对数据库及其对象的访问。
 - 保证数据库的一致性和完整性。
- SQL语言由用于处理数据库和数据库对象的命令和函数组成。该语言还会强制实施有关数据类型、表达式和文本使用的规则。因此在该章节，除了介绍SQL语法外，还会看到有关数据类型、表达式、函数和操作符等信息。

SQL语法简介

- DDL（Data Definition Language）数据定义语言
 - 用于定义或修改数据库中的对象。如：表、索引、视图、数据库、序列、用户、角色、表空间、存储过程等。
- DML（Data Manipulation Language）数据操纵语言
 - 用于对数据库表中的数据进行操作，如插入，更新和删除。
- DCL（Data Control Language）数据控制语言
 - 用来设置或更改数据库事务、授权操作（用户或角色授权，权限回收，创建角色，删除角色等）、锁表（支持SHARE和EXCLUSIVE两种锁表模式）、停机等。
- DQL（Data Query Language）数据查询语言
 - 用来查询数据库内的数据，如查询数据、合并多个select语句的结果集。

- 常用DDL：
 - 定义数据库：创建数据库（create database），修改数据库属性（alter database）
 - 定义表空间：创建表空间（create tablespace），修改表空间属性（alter tablespace），删除表空间（drop tablespace）
 - 定义表：创建表（create table），修改表属性（alter table），删除表（drop table），删除表中所有数据（truncate table）
 - 定义索引：创建索引（create index），修改索引属性（alter index），删除索引（drop index）
 - 定义角色：创建角色（create role），删除角色（drop role）
 - 定义用户：创建用户（create user），修改用户属性（alter user），删除用户（drop user）
 - 定义视图：创建视图（create view），删除视图（drop view）
 - 定义函数：创建函数（Create function），修改函数属性（alter function），删除函数（drop function）
 - 定义过程语言：创建函数（Create language），修改过程语言（alter language），删除过程语言（drop language）
 - 定义操作符：创建操作符（Create operator），修改操作符（alter operator），删除操作符（drop operator）

- DML:
 - 数据操作：插入数据（insert），更新数据（update）和删除数据（delete）
 - 导入导出：导入（load），导出（dump）
 - 其他：查看执行计划（explain plan）
- DCL:
 - 事务管理：提交事务（commit），回滚事务（rollback）
 - 授权操作：授予权限（grant），回收权限（revoke）
 - 锁表：lock table
 - 停机：shutdown
- DQL:
 - 查询数据（select）
 - 合并多个select语句的结果集

目录

1. 鲲鹏生态体系介绍
2. 华为鲲鹏处理器介绍
3. TaiShan服务器介绍
4. openEuler操作系统介绍
- 5. openGauss数据库介绍**
 - 数据库概述
 - openGauss数据库介绍
 - openGauss基础操作
 - 应用开发简介

ODBC

- 开放数据库连接（Open Database Connectivity，ODBC）是为解决异构数据库间的数据共享而产生的，现已成为WOSA（The Windows Open System Architecture（Windows开放系统体系结构））的主要部分和基于Windows环境的一种数据库访问接口标准。
- ODBC为异构数据库访问提供统一接口，允许应用程序以SQL为数据存取标准，存取不同DBMS管理的数据；使应用程序直接操纵DB中的数据，免除随DB的改变而改变。用ODBC可以访问各类计算机上的DB文件，甚至访问如Excel表和ASCII数据文件这类非数据库对象。

JDBC

- Java数据库连接，（Java Database Connectivity，简称JDBC）是Java语言中用来规范客户端程序如何来访问数据库的应用程序接口，提供了诸如查询和更新数据库中数据的方法。JDBC也是Sun Microsystems的商标。我们通常说的JDBC是面向关系型数据库的。

Psycopg

- psycopg是一个 PostgreSQL数据库的Python语言的接口。
- 它的主要优点是它支持完整的python DBAPI 2.0。它是针对大量多线程的应用程序设计的。
- Psycopg分布包括Zpsycopgda，Zope数据库适配器。

思考题

1. （单选题）下面四组SQL命令，全部属于数据操作语言的命令是？（ ）
- A. CREATE, DROP, UPDATE
 - B. INSERT, UPDATE, DELETE
 - C. INSERT, DROP, ALTER
 - D. UPDATE, DELETE, ALTER

- 答案：C

本章总结

- 本章详细介绍了鲲鹏计算产业以及鲲鹏生态，包括华为鲲鹏处理器，TaiShan服务器，openEuler操作系统、openGauss数据库等内容。
- 通过本章的学习，需了解：
 - 鲲鹏计算产业的组成
 - 鲲鹏生态的概念
 - 华为鲲鹏处理器的型号、技术创新、应用场景
 - TaiShan服务器的规格型号
 - openEuler操作系统
 - openGauss数据库

学习推荐

- openEuler开源社区: <https://openeuler.org>
- 鲲鹏社区: <https://www.hikunpeng.com>
- openGauss开源社区: <https://opengauss.org>

缩略语

- MapReduce：面向大数据并行处理的计算模型、框架和平台。
- SoC：System on Chip的缩写，称为系统级芯片，也有称片上系统，意指它是一个产品，是一个有专用目标的集成电路，其中包含完整系统并有嵌入软件的全部内容。
- MPAM：Memory Partitioning and Monitoring，适用于控制/监控L3 Cache和内存带宽的分配和使用，对标x86高端芯片的RDT特性。
- ODBC：Open Database Connectivity，开发数据库连接，是为解决异构数据库间的数据共享而产生的。
- JDBC：Java Database Connectivity，是Java语言中用来规范客户端程序如何来访问数据库的应用程序接口，提供了诸如查询和更新数据库中数据的方法。

Thank you.

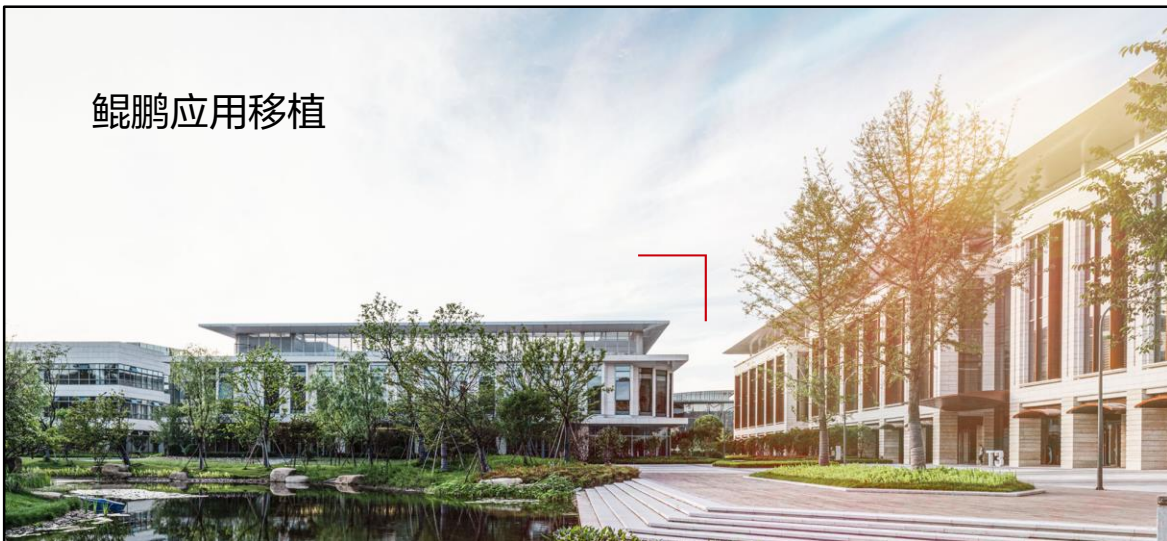
把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。
Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

Copyright©2023 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



鲲鹏应用移植



前言

- 本章将介绍华为鲲鹏平台应用移植的相关知识。包括程序运行原理和软件迁移至鲲鹏计算平台的整个实施过程。从服务器和容器两种应用载体出发，介绍了使用鲲鹏开发套件DevKit完成迁移的具体操作、常见迁移问题及解决思路；另外对鲲鹏应用使能套件BoostKit做了补充介绍。

目标

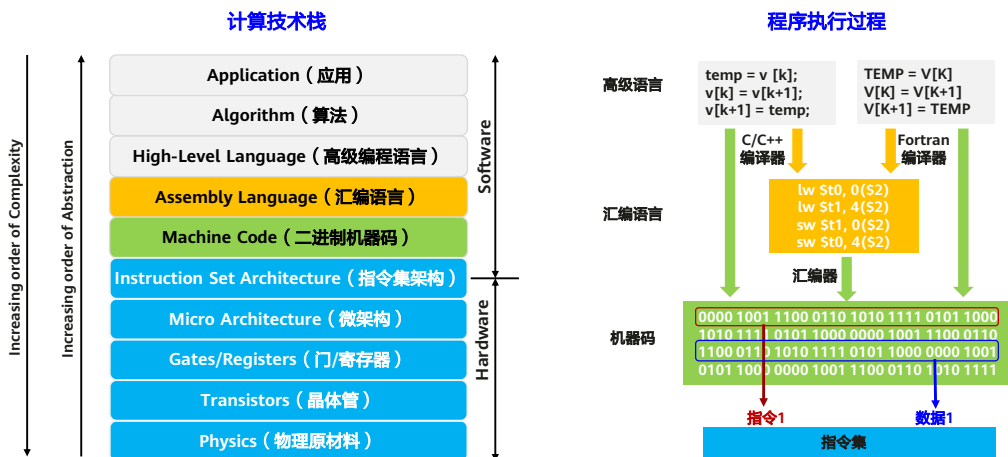
- 学完本课程后，您将能够：
 - 了解程序运行原理；
 - 掌握编译型语言和解释型语言服务器应用迁移方法；
 - 掌握容器应用迁移原理和方法；
 - 了解不同架构软件迁移全流程和常见问题及解决思路；
 - 了解鲲鹏开发套件DevKit和应用使能套件BoostKit的作用及使用方法。

目录

1. 软件迁移原理与迁移过程

- 软件迁移原理
 - 软件迁移过程概述
 - 软件迁移典型案例
- 2. 鲲鹏迁移指导
- 3. 容器迁移指导
- 4. 迁移常见问题及解决思路
- 5. 鲲鹏开发者支持套件

计算技术栈与程序执行过程



- 本节的开头首先抛出来了一个问题，在使用鲲鹏处理器时，为什么要做软件迁移？
- 在讨论这个问题之前，我们首先来看看计算机的技术栈和程序执行的过程。
 - 左边是计算的技术栈，我们可以看到硬件的最底层，是物理材料、晶体管来实现门电路和寄存器，再组成cpu的微架构。cpu的指令集是硬件和软件的接口，应用程序通过指令集中定义的指令驱动硬件完成计算。
 - 应用程序通过一定的软件算法完成业务功能，程序通常使用如C/C++/Java/Go/Python等高级语言开发。高级语言需要编译成汇编语言，再由汇编器按照cpu指令集转换成二进制的机器码。
 - 一个程序在磁盘上存在的形式，是一堆指令和数据所组成二进制机器码，也就是我们通常说的二进制文件。

鲲鹏处理器与x86处理器的指令差异

程序代码 (C/C++) :

```
int main()
{
    int a = 1;
    int b = 2;
    int c = 0;

    c = a + b;

    return c;
}
```

c = a + b;

编译

鲲鹏处理器指令

指令	汇编代码	说明
b9400fe1	ldr x1, [sp,#12]	从内存将变量a的值放入寄存器x1
b9400be0	ldr x0, [sp,#8]	从内存将变量b的值放入寄存器x0
0b000020	add x0, x1, x0	将x1(a)中的值加上x0(b)的值放入x0寄存器
b90007e0	str x0, [sp,#4]	将x0寄存器的值存入内存(变量c)

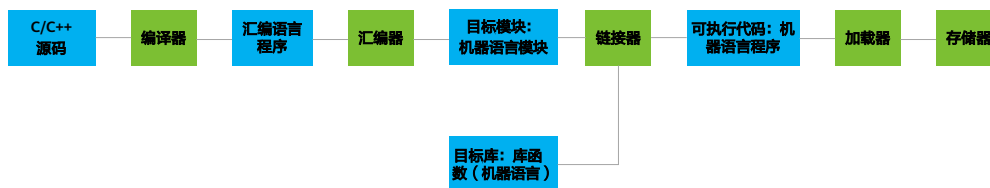
x86处理器指令

指令	汇编代码	说明
8b 55 fc	mov -0x4(%rbp),%edx	从内存将变量a的值放入寄存器edx
8b 45 f8	mov -0x8(%rbp),%eax	从内存将变量b的值放入寄存器eax
01 d0	add %edx,%eax	将edx(a)中的值加上eax(b)的值放入eax寄存器
89 45 f4	mov %eax,-0xc(%rbp)	将eax寄存器的值存入内存(变量c)

- 一行简单的C/C++代码， $c=b+a$ ，在鲲鹏处理器和x86处理两个平台下编译的指令有很大不同。
- 鲲鹏处理器下是由两条ldr指令完成从内存将数据加载到寄存器，一条add指令完成加法计算，最后再使用一条str指令将数据存储到内存中。
- x86处理器下，使用了三条mov指令和一条add指令。两条mov指令将内存中的数据加载到寄存器，一条add指令完成加法，最后再使用一条mov指令将计算后的数据存储到变量c对应的内存。
- 编译器将源码编译成的可执行程序（windows下.exe文件，linux下是二进制的bin文件），这个程序的文件有它自己的结构，但最主要的就是由一堆指令和数据所组成。程序在执行时，cpu将程序中的指令和数据加载到cache缓存中，然后将指令一条条取出，执行。
- 所以不同的cpu所使用的指令集不同，因此同样一个源代码、同样一个功能，在不同的cpu平台下需要执行不同的指令。而这个从源码到指令的翻译工作，就是通常我们所说的编译，由编译器完成。因此当我们将程序移植到不同的cpu平台时，如果cpu的指令集不同，那么就一定需要重新从源码，按照目标平台的指令集，重新翻译一遍。
- Arm指令定长4个字节，x86最长的指令可达15字节。

从源码到可执行程序-编译型语言

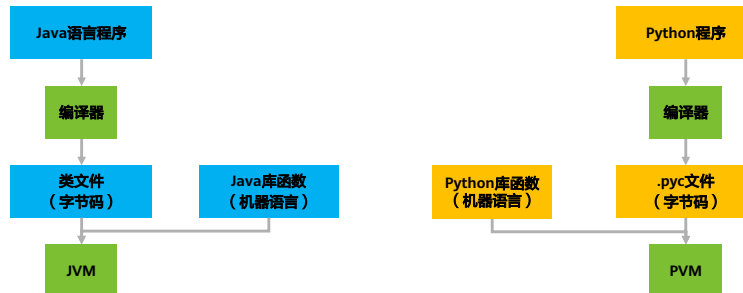
- 编译型语言：典型的如C/C++/Go语言，都属于编译型语言。编译型语言开发的程序在从x86处理器迁移到鲲鹏处理器时，必须经过重新编译才能运行。
- 从源码到程序的过程：源码需要由编译器、汇编器翻译成机器指令，再通过链接器链接库函数生成机器语言程序。机器语言必须与CPU的指令集匹配，在运行时通过加载器加载到内存，由CPU执行指令。



- 编译型语言在执行的时候需要通过编译器得到可执行程序才能加载到内存中进行执行。我们经常会使用`gcc test.c -o test`一步完成，但其实它经历了多个阶段。
- 第一步：预处理，读取源码，对其中的伪指令（以#开头的指令，也就是宏）和特殊符号进行“替代”处理，经过此处理，生成一个没有宏定义、没有条件编译指令、没有特殊符号的输出文件。这个文件的含义同没有经过预处理的源文件是相同的，仍然是C文件。但内容有所不同。可以通过`gcc -E test.c -o test.i`只执行预处理操作得到预编译文件。
- 第二步：编译，编译为汇编代码，可以通过`gcc -S test.i -o test.s`将预编译文件编译输出汇编文件。
- 第三步：汇编，将汇编代码文件通过汇编器汇编成目标文件，可以通过`gcc -c test.s -o test.o`完成。
- 第四步：链接，将目标文件与所需的附加目标文件（静态链接库和动态链接库、C标准输入输出库）连接起来，最终生成可执行文件test：`gcc test.o -o test`。

从源码到可执行程序-解释型语言

- 解释型语言：典型的如Java/Python语言，都属于解释型语言，解释型语言开发的程序在迁移到鲲鹏处理器时，一般不需要重新编译。
- 解释型语言的源代码由编译器生成字节码，然后再由虚拟机解释执行。虚拟机将不同CPU指令集的差异屏蔽，因此解释型语言的可移植性很好。但是如果程序中调用了编译型语言所开发的SO库，那么这些SO库需要重新移植编译。



- 解释型语言在执行时需要先通过初步编译得到字节码文件，然后通过相应语言的虚拟机解释执行。

目录

1. 软件迁移原理与迁移过程

- 软件迁移原理
- 软件迁移过程概述
- 软件迁移典型案例

2. 鲲鹏迁移指导

3. 容器迁移指导

4. 迁移常见问题及解决思路

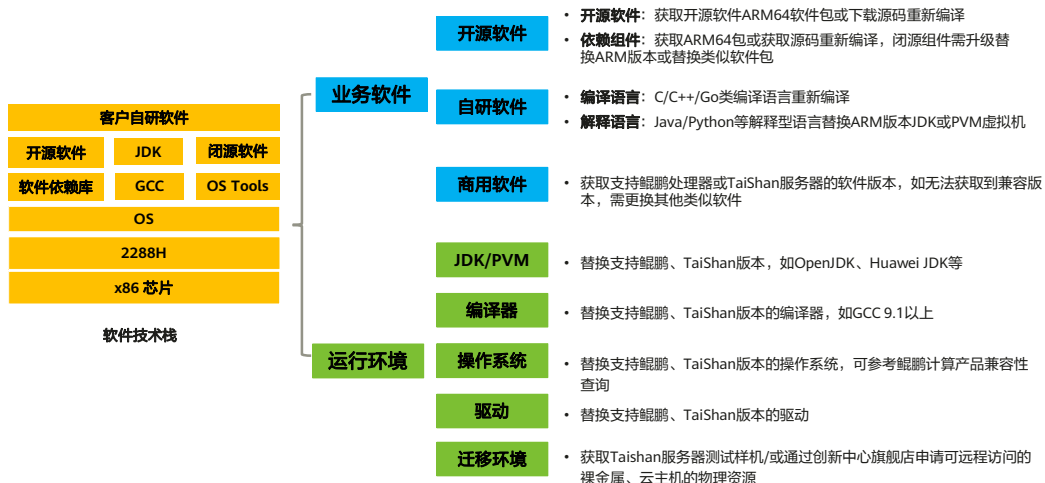
5. 鲲鹏开发者支持套件

迁移过程概述-五个阶段完成软件迁移



- 软件迁移从更大局观的角度可分为五个阶段，第二章我们重点介绍前两个阶段。

阶段一：软件移植-软硬件技术栈整体迁移策略



- 鲲鹏计算产品兼容性查询:
<https://info.support.huawei.com/computing/ftca/kunpeng>。
- 软件兼容性清单: <https://ic-openlabs.huawei.com/client/#/unioncompaty>。
- 鲲鹏社区软件仓库: <https://www.hikunpeng.com/developer/software>。

阶段一：准备迁移环境-基于华为云

通过华为云鲲鹏云服务器搭建开发环境

- 登录华为云账号，进入控制台：<https://www.huaweicloud.com/>
- 选择弹性云服务器，点击购买弹性云服务器
- CPU架构选择鲲鹏计算，按照指导完成鲲鹏弹性云服务器的购买

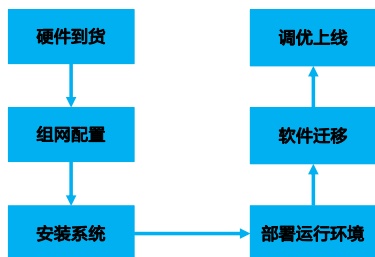


- 购买公有云上的弹性云服务器是性价比最高的获取鲲鹏环境的方式。

阶段一：准备迁移环境-基于本地Taishan服务器

通过购买TaiShan服务器搭建本地环境

- 本地迁移流程



- 服务器要连接公网环境
- 配置物理服务器，组网，需要一定周期
- 设备成本、运维人员成本高

- 弹性云服务器本质上是公有云厂商机房里物理服务器上创建的虚拟机，性能有损耗，如果业务性能要求很高的话，可以选择自己购买物理服务器。

阶段一：准备迁移环境-基于创新中心旗舰店申请远程服务器

通过鲲鹏创新中心旗舰店申请可远程访问的裸金属、云主机的物理资源

通过开放工具化、流程化能力，支持面向全软件栈的鲲鹏生态建设



- 办公环境可以连接公网环境
- 无需配置物理服务器，通过鲲鹏创新中心旗舰店申请远程免费的服务器资源
- 提供认证，生态推广

- 进入鲲鹏创新旗舰店首页：<https://ic-openlabs.huawei.com/client/#/home>，点击远程环境申请。

阶段一：准备迁移环境-基于远程实验室

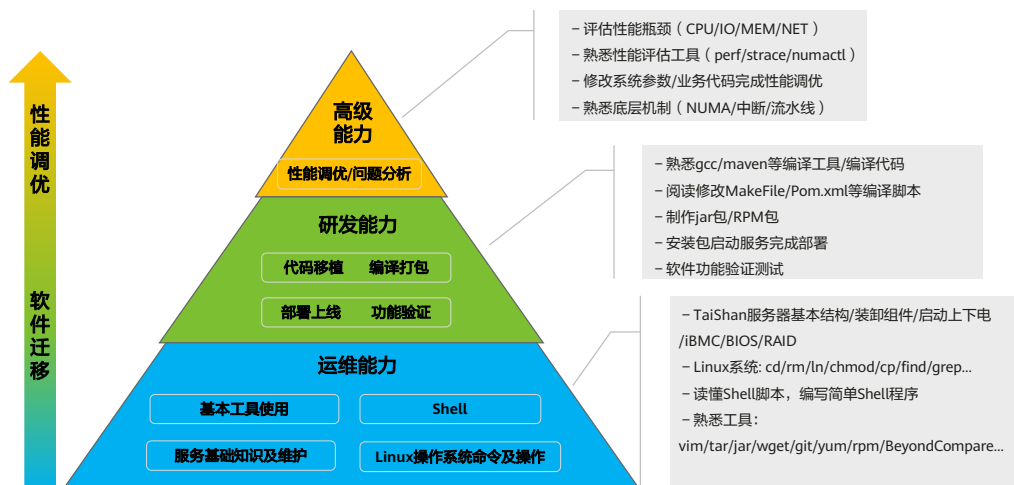
通过鲲鹏社区申请远程实验室资源

- 100+在线鲲鹏虚拟化环境，24小时在线，按时段预约（1天/3天/7天），到期可按需续约。
- 提供鲲鹏远程服务器开发环境，预装“一站式”开发套件，包括鲲鹏代码迁移工具（含鲲鹏原生开发框架和调试器）、鲲鹏编译器、鲲鹏性能分析工具，动态二进制翻译工具（ExaGear）。开发者可远程便捷SSH登录，灵活使用。



- <https://www.hikunpeng.com/developer/cloud-lab?tag=test>。

阶段一：管理工作-软件迁移团队能力模型与要求



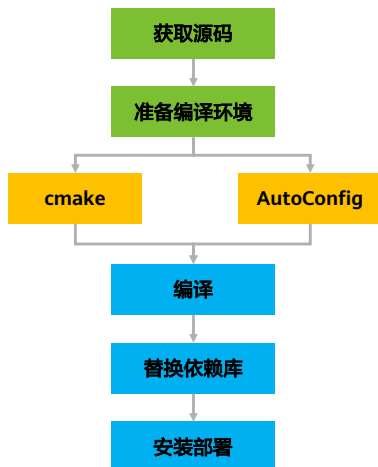
- 迁移团队需要具备的能力是金字塔结构，最基础的是运维能力，比如熟悉服务器操作，Shell脚本和Linux常用命令。另外在编译迁移的时候会涉及到源码的修改和重新编译，所以需要具备一定的开发能力。最顶层是性能调优的能力，因为影响性能的因素有很多，比如应用程序本身，它使用的高级语言，线程并行，锁操作等；应用运行的环境也会影响性能，主要是CPU、内存、磁盘和网络以及JVM等。性能调优工程师不仅需要熟悉业务代码也要熟悉硬件，能够使用工具测试性能并找到瓶颈点调参调优，所以技术要求非常高。

阶段一：管理工作-制定迁移计划



- 在管理方面，组建好迁移团队后就可以制定迁移计划了，比如五个阶段分别有哪些详细的工作内容。

阶段二：软件源码编译移植（C/C++代码）-迁移流程



1、获取源码：

1) 开源：通过github或第三方开源社区获取源码；

2) 自研软件：对接自有代码仓库；

2、准备编译环境：安装编译器gcc等（推荐GCC 7.3.0以上版本）；

3、使用开源软件源码中的cmake或AutoConfig脚本生成Makefile；

4、执行Makefile编译可执行程序；

5、修改源码；

6、替换依赖库：通过开源软件readme文件、编译时的SO库缺失或链接错误，重新编译或替换依赖库；

7、将可执行程序安装部署到生产或测试系统。

- 第二个阶段，执行编译移植。我们先从编译型语言程序比如C/C++代码的迁移看起。

阶段二：软件源码编译移植（C/C++代码）-修改源码

	功能说明	x86代码	鲲鹏代码
数据类型	显示定义char类型 变量为有符号型	<code>char a = 'a'</code>	<code>signed char a = 'a'</code>
Builtin函数	使用编译器自带的 builtin函数	<code>__builtin_ia32_crc32qi (__a, __b);</code>	<code>__builtin_aarch64_crc32b (__a, __b);</code>
	把内存数据预取到 cache中	<code>asm volatile("prefetcht0 %0" :: "m" (*(unsigned long *)x));</code>	<code>#define prefetch(_x) __builtin_prefetch(_x)</code>

- C/C++代码在重新编译前需要修改的源码部分包括数据类型，Builtin函数，还有内联汇编函数等。

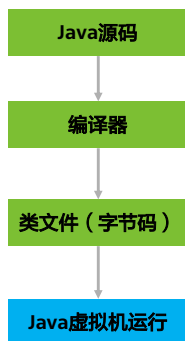
阶段二：软件源码编译移植（C/C++代码）-修改编译选项

	功能说明	x86编译选项	鲲鹏编译选项
64位编译	定义编译生成的应用程序为64位，编译选项不同	-m64	-mabi=lp64
ARM指令集	Makefile中定义指令集类型，由x86修改为ARM	-march=broadwell	-march=armv8-a
编译宏	原有x86版编译宏替换成ARM版	__x86_64__	__ARM_64__

- 除了源码之外，编译选项也需要修改。

阶段二：Java自研代码移植

Java程序编译运行的过程



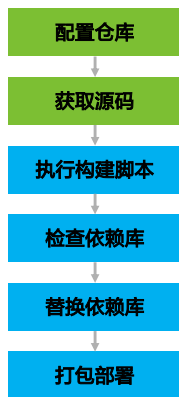
Java程序迁移至鲲鹏平台运行

- 1、安装鲲鹏或ARM版本JDK
- 2、配置JDK环境变量
- 3、编译Java源码生成字节码
- 4、【可选】移植jar包中的依赖库
(替换鲲鹏/ARM版本或获取C/C++源码重新编译)
- 5、【可选】使用新的依赖库重新打jar包
- 6、启动Java程序，调试功能

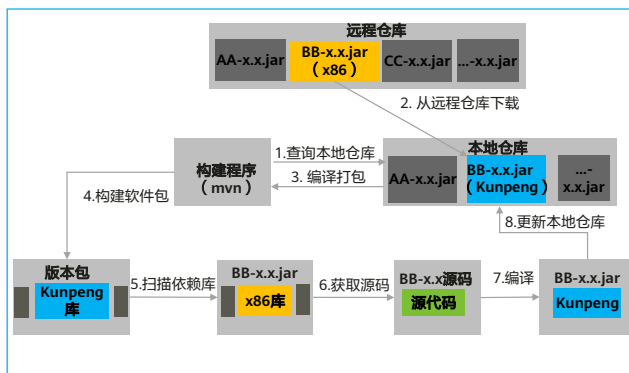
- 接下来介绍解释型语言程序比如Java程序的移植过程，对照Java代码的运行过程来看移植的操作要点。

阶段二：Java开源软件移植（Maven仓库）

开源软件迁移过程（Maven软件仓库）



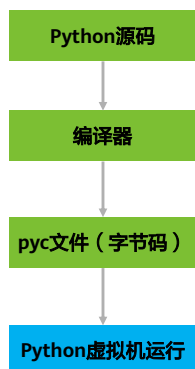
Maven仓库软件构建流程



- Java程序在移植的时候经常会用到maven构建工具来对依赖库进行替换，maven仓库的构建流程见右图。

阶段二：Python自研代码移植

Python程序编译运行的过程

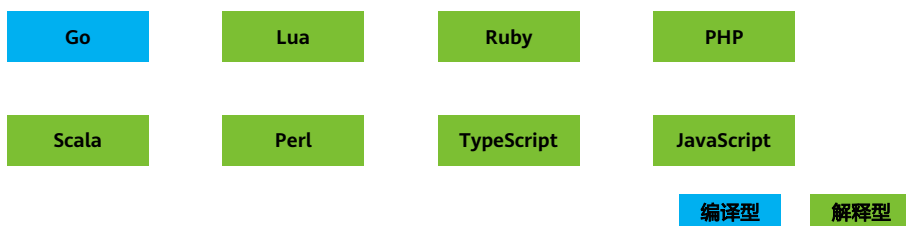


Python程序迁移至鲲鹏平台运行

- 1、使用操作系统自带的Python，或安装兼容鲲鹏/ARM版本的Python软件
- 2、设置Python执行环境变量
- 3、【可选】编译生成pyc文件（字节码）
- 4、【可选】移植外部依赖库
(替换鲲鹏/ARM版本或获取C/C++源码重新编译)
- 5、【可选】更新代码中新的外部依赖库调用代码
- 6、执行Python源码程序

- 另一种常用的解释型语言就是Python程序。

阶段二：其他语种代码移植



编译型：源码需要重新编译

解释型：源码不用编译，安装解释器即可

搭建程序运行环境方法：

- 操作系统自带解释器软件，直接安装即可使用
- 从官网下载支持ARM的解释器软件包，直接解压使用
- 从官网下载x86解释器源码，编译迁移后使用

- 除了常见的C/C++/Java/Python程序外，其他语言代码的移植过程同样需要根据编译型语言和解释型语言程序不同的执行过程进行不同的移植操作。

阶段二：商用闭源软件迁移（商用数据库软件）



- 在迁移的时候如果涉及到商用的闭源软件，比如Oracle商用数据库，一种方法是联系对应厂商询问是否有ARM版本进行替换部署，如果没有的话，就需要考虑替换其他数据库软件，比如国产的GaussDB、达梦等都有ARM版本，或者是选择开源数据库。

目录

1. 软件迁移原理与迁移过程

- 软件迁移原理
- 软件迁移过程概述
- 软件迁移典型案例

2. 鲲鹏迁移指导

3. 容器迁移指导

4. 迁移常见问题及解决思路

5. 鲲鹏开发者支持套件

某行业伙伴快速实现软件迁移的案例

案例：行业伙伴核心应用系统迁移

5人月完成520+源码文件移植，性能提升56%



思考题

1. （多选题）为什么x86架构处理器上的软件在鲲鹏处理器使用时需要移植？
- A. 两种处理器的指令集不同
 - B. 源代码需要按照目标处理器的指令集架构编译或解释成指令才能运行
 - C. 编译选项、编译宏等x86架构和鲲鹏架构表示形式不同
 - D. x86平台上的可执行程序在鲲鹏平台无法直接执行

- ABCD

目录

1. 软件迁移原理与迁移过程
2. **鲲鹏迁移指导**
 - 鲲鹏代码迁移工具介绍
 - 编译型语言应用迁移
 - 解释性语言应用迁移
 - 二进制指令翻译工具ExaGear
3. 容器迁移指导
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

华为鲲鹏代码迁移工具是什么（1）

- 鲲鹏代码迁移工具是华为鲲鹏开发套件DevKit在迁移环节提供的一款可以简化客户应用迁移到基于鲲鹏916/920的服务器过程的工具。工具仅支持x86 Linux软件到Kunpeng Linux上的扫描、分析与迁移，不支持Windows软件代码的扫描、分析与迁移。
- 下载链接：<https://www.hikunpeng.com/developer/devkit/porting-advisor>



- 该工具支持 IDE 插件（VS Code、IntelliJ）和浏览器两种工作模式。

- 前面我们了解了迁移原理和迁移流程，在重新编译前需要对源码修改的部分会感觉难度高又繁琐，为了发展鲲鹏生态，鲲鹏社区有提供相应的工具来简化我们的迁移工作。

华为鲲鹏代码迁移工具是什么（2）

- 浏览器工作模式的软件包既可以部署在鲲鹏服务器上，也可以部署在x86服务器上；详细的工具可支持的运行平台和操作系统对应关系请参见[兼容性查询工具](#)。

鲲鹏代码迁移工具



- 兼容性查询工具：<https://www.hikunpeng.com/zh/developer/developtool>。

鲲鹏代码迁移工具可以做什么

- 代码迁移工具可自动扫描并分析待迁移软件，提供可迁移性评估报告；也可对待迁移软件进行源码分析，准确定位需迁移的代码，并给出友好的迁移指导或一键代码替换；同时支持将x86软件包重构成鲲鹏平台软件包、专项软件一键迁移及其他鲲鹏亲和分析等。
- 当客户有x86平台上源代码的软件要迁移到基于鲲鹏916/920的服务器上时，既可以使用该工具分析可迁移性和迁移投入，也可以使用该工具自动分析出需修改的代码内容，并指导用户如何修改。
- 鲲鹏代码迁移工具既解决了客户软件迁移评估分析过程中人工分析投入大、准确率低、整体效率低下的痛点，通过该工具能够自动分析并输出指导报告；也解决了用户代码兼容性人工排查困难、迁移经验欠缺、反复依赖编译调错定位等痛点。

- 代码迁移工具有三大常用功能：软件迁移评估（分析非源码包并提供迁移报告）、源码迁移（分析源码包并提供移植建议）、软件包重构（将x86软件包重构成鲲鹏软件包）。

鲲鹏代码迁移工具功能特性

功能	描述
软件迁移评估	- 检查用户软件包（RPM、DEB、TAR、ZIP、GZIP文件）中包含的SO（Shared Object）依赖库和可执行文件，并评估SO依赖库和可执行文件的可迁移性。 - 检查用户Java类软件包（JAR、WAR）中包含的SO依赖库和二进制文件，并评估SO依赖库和二进制文件的可迁移性。 - 检查指定的用户软件安装路径下的SO依赖库和可执行文件，并评估SO依赖库和可执行文件的可迁移性。
源码迁移	- 检查用户C/C++/Fortran软件构建工程文件，并指导用户如何迁移该文件。 - 检查用户C/C++/Fortran软件构建工程文件使用的链接库，并提供可迁移性信息。 - 检查用户C/C++/Fortran软件源码，并指导用户如何迁移源文件。其中，Fortran源码支持从Intel Fortran编译器迁移到GCC Fortran编译器，并进行编译器支持特性、语法扩展的检查。 - x86汇编指令转换，分析部分x86汇编指令，并转换成功能对等的鲲鹏汇编指令。
软件包重构	在鲲鹏平台上，分析待迁移软件包构成，重构并生成鲲鹏平台兼容的软件包，或直接提供已迁移了的软件包。
专项软件迁移	在鲲鹏平台上，对部分常用的解决方案专项软件源码，进行自动化迁移修改、编译并构建成鲲鹏平台兼容的软件包。
鲲鹏亲和分析	64位运行模式检查是将原32位平台上的软件迁移到64位平台上，进行迁移检查并给出修改建议。 - 结构体字节对齐检查是在需要考虑字节对齐时，检查源码中结构体类型变量的字节对齐情况。 - 缓存行对齐检查是对C/C++源码中结构体变量进行128字节对齐检查，提升访存性能。 - 内存一致性检查是分析、修复用户软件中的内存一致性问题。

- 除了三大常用功能外，代码迁移工具还支持两个增强功能：专项软件迁移和鲲鹏亲和分析。

其他功能特性

- 工具提供工作空间容量检查，当工作空间容量过低时，向用户提供告警信息。
- 工具向用户提供软件迁移报告，提供迁移工作量评估，支持用户自定义工作量评估标准。
- 用户通过安全传输协议上传软件源码、软件包、二进制文件等资源到工作空间，也可以下载软件迁移报告到本地。
- 工具的安装方式有Web和CLI两种，Web方式下支持Web浏览器访问和CLI命令行方式访问，CLI方式下只支持CLI命令行访问。其中Web浏览器访问支持所有的功能，CLI命令行访问只支持软件迁移评估和源码迁移功能。Web方式下支持多用户并发扫描。

- 使用代码迁移工具时需要将工具包安装在服务器上，然后通过Web访问或者IDE插件使用。如果安装时选择./install cli，那就只能通过命令行方式使用了。

鲲鹏代码迁移工具应用场景

- 软件迁移评估：自动扫描并分析软件包（非源码包）、已安装的软件（仅支持部署在x86服务器上时），提供可迁移性评估报告。
- 源码迁移：当用户有软件要迁移到基于鲲鹏916/920的服务器上时，可先用该工具分析源码并得到迁移修改建议。
- 软件包重构：帮助用户重构适用于鲲鹏平台的软件安装包。
- 专项软件迁移：使用华为提供的软件迁移模板修改、编译并产生指定软件版本的安装包，该软件包适用于鲲鹏平台。
- 鲲鹏亲和分析：支持x86和鲲鹏平台GCC 4.8.5~GCC 10.3.0版本32位应用向64位应用迁移的64位运行模式检查，结构体字节对齐检查、缓存行对齐检查和鲲鹏平台上的弱内存序检查。

特殊场景例外约束

特殊场景	例外约束
构建文件	• cmake解析中的Find功能依赖GCC编译结果，编译选项中有x86特有编译选项如-m64时，如果工具运行环境是鲲鹏环境，cmake无法顺利解析，无法检出相应内容。 • Makefile支持\$(shell shell_command)形式的Shell指令，但不支持直接使用Shell脚本作为构建配置文件。
C/C++代码分析	工具运行环境中缺少用户自定义宏头文件、第三方头文件、某些系统头文件场景下，会影响相关宏的修改点检出，无法给出精确建议。
Intrinsic函数	Intel协处理器相关的130多个Intrinsic函数无法给出准确替换建议。
汇编指令	存在以下场景时，仅能提示用户注意转换代码的使用，不能提示100%精确的修改建议： 1. 当函数存在参数的时候，提示可能存在特殊结构导致函数转换出错； 2. 将栈上的数据入栈传参时，默认该参数为整型。 存在以下场景时无法支持100%准确的自动转换，仅能提示需要手动转换： 1. goto语句、call语句和ret语句； 2. 使用段寄存器或未赋值的物理寄存器； 3. 访问全局符号； 4. 输入输出的变量位宽大于128位； 5. 输入输出的全局变量大于21个； 6. 指令的位宽与变量位宽不一致； 7. 使用Intel和AT&T混合的汇编格式； 8. 汇编存在语法错误。
字节对齐	递归包含头文件场景下，如果用户提供不了一部分依赖包，会造成分析精确度下降。

鲲鹏代码迁移工具-浏览器工作模式

- 浏览器工作模式的软件包安装方式有Web和CLI两种，Web方式下支持Web浏览器访问和CLI命令行方式访问，CLI方式下只支持CLI命令行访问。

```
[root@ecs-hei-forting-serial-2.5.RHEL Linux-Kunpeng]# ./install
usage: install [arguments] install package.

Example:
./install web
./install cmd

Arguments:
[mode]          Only install mode ('web' or 'cmd').
-h or --help    Give this help list.
```

- Web方式：通过浏览器远程使用代码迁移工具各功能，具备图形化界面，操作简单，配置过程清晰可见，可在线浏览分析结果，也可以导出.csv和html两种格式的报告。默认允许10个普通用户同时登录使用，管理员用户不受此限制，单个用户只允许一个活跃会话。
- CLI方式：通过命令行使用代码迁移工具各功能，通过多种代码选项完成各项配置，立即执行分析并打印关键分析结果。详细的分析报告和移植建议保存在.csv文件中，用户可以进入工具安装目录下进行下载查看。

- 代码迁移工具的具体安装过程详见实验手册。

鲲鹏代码迁移工具-浏览器工作模式（Web）

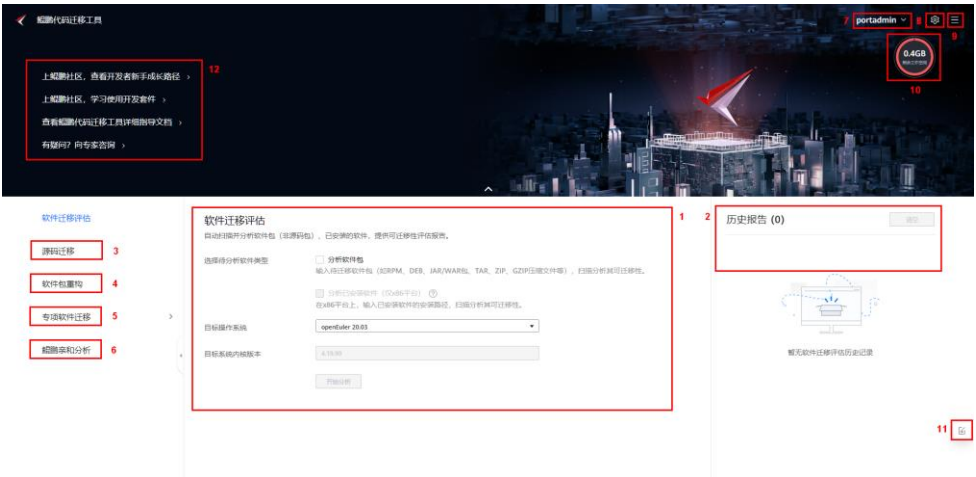
- 登录鲲鹏代码迁移工具Web界面：

- 打开本地PC的浏览器，在地址栏输入“https://部署服务器的ip:端口号”，安装工具默认的HTTPS端口号为8084，请确认防火墙和安全组已开通8084端口；遇到安全告警界面，选择继续前往此网站。（具体安装过程可见实验手册）
- 首次登陆需要设置管理员用户“portadmin”的密码，必须为8-32字符的强口令密码。完成设置后再次输入密码登录工具。



- 鲲鹏代码迁移工具用户的密码有效期为90天，建议在密码有效期到达之前设置新密码。若密码已过期，则需要在登录后先进行密码修改操作。

鲲鹏代码迁移工具-浏览器工作模式（Web）-界面说明（1）



- 登录进入代码迁移工具后界面相关的介绍可见下一页表格。

鲲鹏代码迁移工具-浏览器工作模式（Web）-界面说明（2）

区域	名称	说明
1	软件迁移评估	软件迁移评估入口，用户可在软件迁移评估的创建分析任务区创建新的分析任务。
2	查看历史报告区	展示历史报告。
3	源码迁移	软件迁移入口，用户可在源码迁移的创建分析任务区创建新的分析任务。
4	软件包重构	软件包重构入口，用户可对软件包进行重构分析，构建适用于鲲鹏平台的软件包。
5	专项软件分析	专项软件迁移入口，展示华为积累的基于解决方案分类的软件迁移方法汇总。
6	鲲鹏亲和分析	支持x86和鲲鹏平台GCC 4.8.5~GCC 10.3.0版本32位应用向64位应用的运行模式检查、字节对齐检查、缓存行对齐检查和鲲鹏平台上运行可能存在的内存一致性问题检查。
7	当前用户	展示当前登录用户，并提供修改密码和用户登出的操作入口。
8	配置	提供用户管理，弱口令字典，系统配置，日志，依赖字典，软件迁移模板，扫描参数配置，阈值设置，Web服务端证书，和证书吊销列表功能入口。
9	更多	提供中英文切换，建议反馈，联机帮助，免责声明和鲲鹏代码迁移工具发布信息入口。

- 一般我们直接通过1、3、4区域创建分析任务执行分析，然后查看2区域的报告结果。

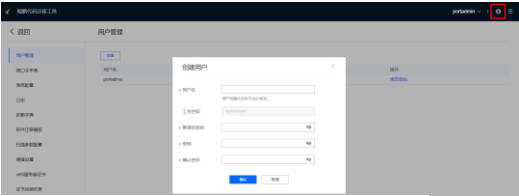
鲲鹏代码迁移工具-浏览器工作模式（Web）-界面说明（3）

区域	名称	说明
10	工作空间/磁盘容量告警区	当工作空间（工具安装目录）剩余容量或磁盘空间剩余容量小于工作空间或磁盘空间的20%时，显示黄色告警；小于5%时，显示红色警告；大于20%时，告警消失。
11	建议反馈	提供进入建议反馈页面的入口。本建议反馈图标在除登录界面以外的界面上都有。
12	案例链接区	提供典型案例链接，并为每个功能提供示例代码。

说明：用户可以同时创建“软件迁移评估”，“源码迁移”，“软件包重构”，“专项软件迁移”，“鲲鹏亲和分析”任务，即支持多任务并行。

鲲鹏代码迁移工具-浏览器工作模式（Web）-使用前配置

- 使用该工具前可根据需要创建普通用户，更改扫描参数设置，历史报告阈值设置等。



The screenshot shows the 'Create User' dialog box in the top-left corner of the web interface. The dialog has fields for 'Username', 'Password', 'Confirm Password', 'Email', and 'Role'. Below the dialog, the 'Threshold Settings' page is visible. It contains two main sections: 'Scan Parameter Configuration' and 'Threshold Settings'.

Scan Parameter Configuration

- 关键字扫描: ☐ 是
- C/C++/Fortran/Go代码迁移工作集评估 (行/人月):
- 汇编代码迁移工作集评估 (行/人月):
- 显示工作集评估结果: ☐ 否
- 用户密码:

Threshold Settings

- 历史报告提示阈值必须小于历史报告最大阈值
- 历史报告提示阈值: (1 ~ 49)
- 历史报告最大阈值: (2 ~ 50)

Buttons: 确认, 取消

鲲鹏代码迁移工具-浏览器工作模式（Web）-帮助文档

- 使用过程中有任何疑问可以在更多-联机帮助获取详细介绍。



鲲鹏代码迁移工具-IDE插件工作模式（VS Code）

- 打开VS Code，在左侧菜单栏中单击  打开鲲鹏代码迁移插件。（插件安装配置过程详见实验手册）
- IDE插件工作模式同样需要后端服务器部署鲲鹏代码迁移工具，可以提前部署完成，在插件界面配置服务器地址；也可以在VS Code插件使用界面一键部署后端代码迁移工具。



- 课程配套的实验手册中就使用的VS Code插件，配置服务器时出现的报错也可以参考实验手册更改IDE设置解决。

鲲鹏代码迁移工具-IDE插件工作模式（IntelliJ）-插件下载

- 登录[鲲鹏代码迁移工具](#)下载页面，单击“IntelliJ”，单击“插件下载”进入下载界面，获取相应版本压缩包。

鲲鹏代码迁移工具

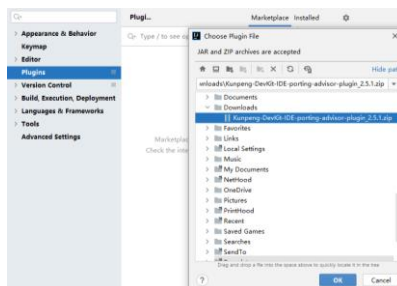
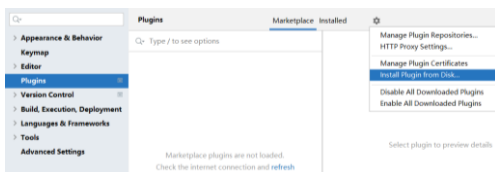
服务端运行平台 x86服务器、基于鲲鹏916的服务器、基于鲲鹏920的服务器
服务端运行操作系统 openEuler 20.03 (LTS)、CentOS 7.6、Ubuntu18.04、麒麟V10、UOS 20等
* 详细的运行平台和操作系统对应关系请参见 [兼容性查询工具](#)



- 代码迁移工具下载：<https://www.hikunpeng.com/developer/devkit/porting-advisor/download?tab=0>。

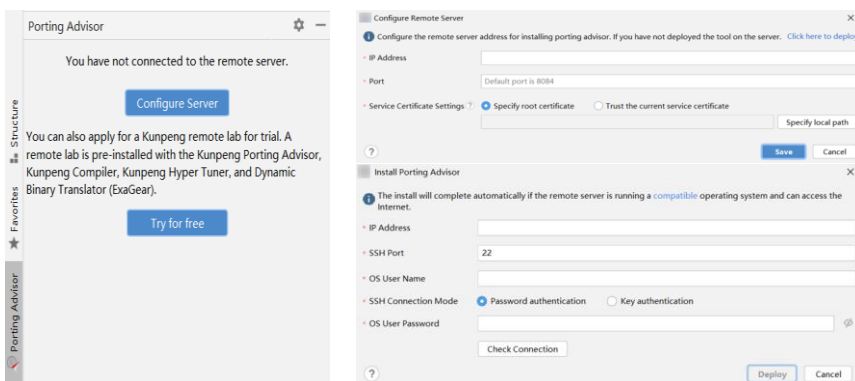
鲲鹏代码迁移工具-IDE插件工作模式（IntelliJ）-插件安装

- 打开IntelliJ，使用快捷键“Ctrl+Alt+S”打开设置，选择“Plugins”，点击设置，选择“Install Plugin from Disk”，找到刚才下载的压缩包，点击OK，按照提示重启IDE完成插件安装。



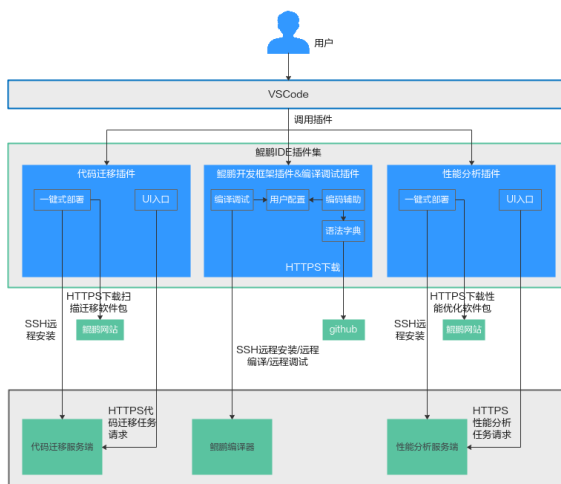
鲲鹏代码迁移工具-IDE插件工作模式（IntelliJ）-配置服务器

- 单击左侧菜单栏，同VS Code步骤，配置远端已经部署代码迁移工具的服务器，或者一键部署远端工具。



鲲鹏代码迁移工具-IDE插件功能特性

- 鲲鹏开发套件工具是一个工具集，由多个插件组成，支持IDE前端界面，支持一键式安装后端，代码编辑体验增强，自动检测安装鲲鹏编译器，编译调试，用例可视化，编码辅助，工程分析扫描。用户可以通过安装Kunpeng DevKit插件直接将四个插件都安装好，也可以单独选择个别插件安装使用，比如单独安装代码迁移插件。

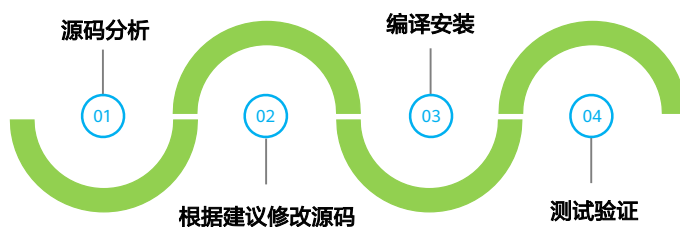


目录

1. 软件迁移原理与迁移过程
2. **鲲鹏迁移指导**
 - 鲲鹏代码迁移工具介绍
 - 编译型语言应用迁移
 - 解释性语言应用迁移
 - 二进制指令翻译工具ExaGear
3. 容器迁移指导
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

C/C++类应用移植

- 基于编译型语言开发的应用程序，例如C/C++语言应用程序，其编译后得到可执行程序，可执行程序执行时依赖的指令是CPU架构相关的。因此，基于x86架构编译的C/C++语言应用程序，无法直接在TaiShan服务器或华为鲲鹏云服务器上运行，需要进行移植编译。
- 应用移植的流程如下：



具体移植案例和过程可参考实验手册。

Fortran类应用移植-案例展示（1）

- Fortran作为编译型语言，迁移过程同样按照源码分析、移植建议、编译安装、测试验证流程。

- 源码包示例：

https://gitlab.com/DL_POLY_Classic/dl_poly/-/archive/RELEASE-1-10/dl_poly-RELEASE-1-10.tar.gz

- 上传源码包至服务器sourcecode目录下；进入解压后的目录，拷贝Makefile，并更改文件属主。
- 执行源码分析扫描过程。扫描根路径选择Makefile文件所在的source路径，类型勾选上Fortran。
- 查看扫描结果。

- `cd dl_poly-RELEASE-1-10/source。`
- `cp ../build/MakePAR ./Makefile。`
- `chown porting:porting Makefile。`

Fortran类应用移植-案例展示（3）

- 按照扫描建议在第59行和65行的FFLAGS后添加“-march=armv8.2-a -mtune=tsv110 -ffree-line-length-none -cpp -std=legacy”。
- 编译安装：
 - cd /opt/portadv/portadmin/sourcecode/dl_poly-RELEASE-1-10/source/
 - make dlpoly
- 设置环境变量：“export PATH=/opt/portadv/portadmin/sourcecode/dl_poly-RELEASE-1-10/execute:\$PATH”。
- 执行以下命令查看成功安装DL_POLY：“which DLPOLY.X”。
- 显示信息如下，则安装成功。

```
[root@localhost source]# export PATH=/opt/portadv/portadmin/sourcecode/dl_poly-RELEASE-1-10/execute:$PATH
[root@localhost source]# which DLPOLY.X
/opt/portadv/portadmin/sourcecode/dl_poly-RELEASE-1-10/execute/DLPOLY.X
```

- 根据提示建议修改源码并编译安装，运行验证。

目录

1. 软件迁移原理与迁移过程
- 2. 鲲鹏迁移指导**
 - 鲲鹏代码迁移工具介绍
 - 编译型语言应用迁移
 - 解释性语言应用迁移
 - 二进制指令翻译工具ExaGear
3. 容器迁移指导
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

Java类应用迁移

- 基于解释型语言开发的应用程序，是与CPU架构不相关的，例如Java、Python，将这类应用程序移植到TaiShan服务器或华为鲲鹏云服务器，无需修改和重新编译，按照与x86一致的方式部署和运行应用程序即可。
- Java应用程序jar包内，可能包含基于C/C++语言开发的SO库文件，这类SO库需要移植编译，使用编译得到的SO库重新打包jar包。移植具体方法与C/C++类应用相同。一般流程为：
 - 上传.jar文件，新建分析软件包任务，分析得到需要迁移的依赖库文件；
 - 配置Maven环境，重新编译打包；
 - 解压进入jar包查看新的SO库验证迁移成功后部署运行。

目录

1. 软件迁移原理与迁移过程
- 2. 鲲鹏迁移指导**
 - 鲲鹏代码迁移工具介绍
 - 编译型语言应用迁移
 - 解释性语言应用迁移
 - 二进制指令翻译工具ExaGear
3. 容器迁移指导
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

华为动态二进制翻译工具ExaGear（1）

- ExaGear是华为自研动态二进制翻译工具，通过在运行时，将x86应用指令翻译为ARM64指令并执行，从而支持Linux x86应用无需重新编译就能运行在ARM64服务器上，帮助客户将Linux x86无源码应用快速迁移到ARM服务器上，且能稳定可靠运行。
- ExaGear利用动态二进制翻译技术，结合动态二进制优化能力，能够稳定支持无源码的x86和ARM32存量业务运行在鲲鹏平台上，并且适于部署的场景广泛，既可以直接部署于操作系统，也可以部署于容器中，甚至是在ExaGear中再部署容器，能够在无源码的情况下屏蔽底层平台差异，低成本解决应用的平滑迁移，释放鲲鹏平台澎湃算力。

- 可通过此链接下载软件包并查看文档：
<https://www.hikunpeng.com/developer/devkit/exagear>。

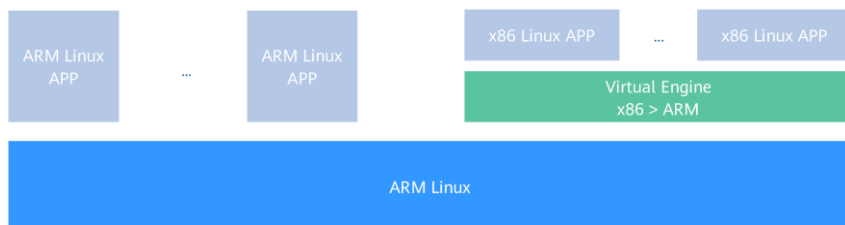
华为动态二进制翻译工具ExaGear（2）

- ExaGear解决了用户将原有平台的软件系统迁移到鲲鹏平台时遇到的痛点问题。在迁移过程中，用户业务按重要性差异，往往可分为三类：无源码业务、有源码的非关键业务和有源码的关键业务。
 - 行业相关的无源码商业软件。这类软件只有二进制、没有源码，在行业内又有相当的影响力。在这样的场景下，需要ExaGear动态二进制翻译技术保障顺利完成迁移。
 - 有源码的非关键业务。使用频率低、对性能不敏感。对于这部分业务，客户就可以利用ExaGear完成动态的二进制翻译，使存量业务应用不需要通过代码移植，就可以直接运行在鲲鹏平台上。这一过程省却了大量的移植或优化源代码所需的人力和时间，而且没有因移植代码而引入额外的稳定性隐患。
 - 性能敏感且有源码的关键业务。建议通过手工的代码移植和性能优化完成迁移，达到最优的性能预期。

- 可通过此链接下载软件包并查看文档：
<https://www.hikunpeng.com/developer/devkit/exagear>。

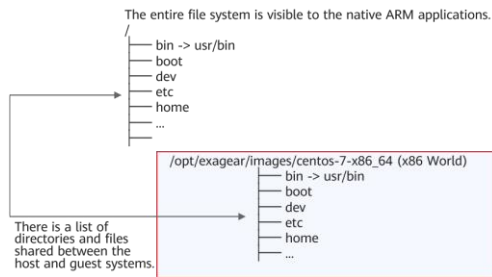
华为动态二进制翻译工具ExaGear-指令翻译引擎

- ExaGear安装完成后，主要有两个组件：指令翻译引擎和x86运行环境。
- ExaGear的指令翻译引擎是一个“中间件”软件解决方案，位于x86应用程序与ARMv8架构服务器之间。x86应用启动时，ExaGear的指令翻译引擎接管x86应用的运行，使用二进制翻译技术将它们转换为兼容ARM的代码，再执行x86应用程序。对最终用户而言，整个过程是简易且透明的。



华为动态二进制翻译工具ExaGear-x86运行环境

- x86运行环境是ExaGear创建的一个包含所有标准库、实用程序、配置文件的x86应用执行环境，这确保了x86应用程序运行所需的基础设施组件的可用性。从技术角度讲，x86运行环境是一个特定的文件目录，包含了x86库、命令、实用程序和其他系统文件。x86应用程序的二进制文件也必须存放于x86运行环境中。



- ExaGear for Server的使用请参考：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/ug-exagear/usermanual/kunpengexagear_06_0007.html。
- ExaGear for Docker的使用请参考：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/ug-exagear/usermanual/kunpengexagear_06_0027.html。

思考题

1. （多选题）使用IDE代码迁移工具插件可以实现以下哪些功能？
- A. 软件迁移评估
 - B. 源码迁移
 - C. 软件包重构
 - D. 鲲鹏亲和分析

- ABCD

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
- 3. 容器迁移指导**
 - 容器相关概念介绍
 - iSula容器引擎介绍
 - 容器迁移背景和原理
 - 容器迁移主要流程
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

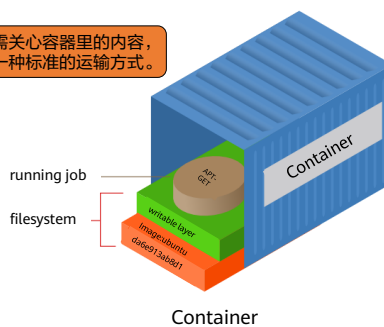
什么是容器

- 容器是一种轻量级、可移植、自包含的软件打包技术，使应用程序可以在几乎任何地方以相同的方式运行。开发人员在自己笔记本上创建并测试好的容器，无需任何修改就能在生产系统的虚拟机、物理服务器或公有云主机上运行。



运输业

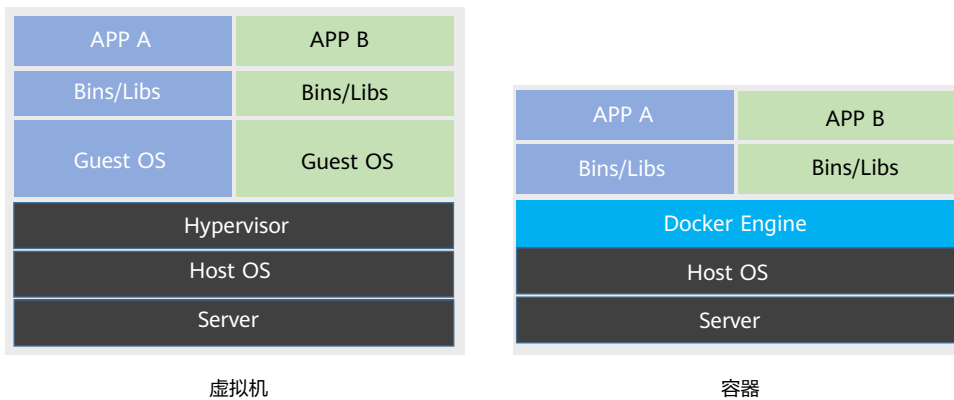
不需关心容器里的内容，
是一种标准的运输方式。



- Container还有另外一个中文意思：集装箱。可以将软件业的打包技术容器和运输业的集装箱做对比分析，集装箱打包的物品对比容器运行业务，集装箱运输工具和方式对比容器运行平台。

容器与虚拟机

- 容器和虚拟机之间的主要区别在于虚拟化层的位置和操作系统资源的使用方式。



- 容器和虚拟机从该架构图来看最明显的区别就是每个虚拟机有自己的Guest OS，而所有容器共享宿主机OS内核。

容器与虚拟机参数对比

参数	容器 (Docker)	对比	虚拟机
快速创建, 删除	启动应用	>	启动Guest OS+启动应用
交互, 部署	容器镜像	=	虚拟机镜像
密度	单Node 100~1000	>	单Node 10~100
更新管理	迭代式更新, 通过修改Dockerfile, 对增量内容进行分发, 存储, 节点启动	>	向虚拟机推送安装, 升级应用软件补丁包
启动时间	秒级启动	>	分钟级启动
轻量级	镜像大小通常以M为单位	>	镜像大小通常以G为单位
性能	Docker共享宿主机内核, 系统级虚拟化, 占用资源少, 没有Hypervisor层开销, 性能基本接近物理机	>	虚拟机需要Hypervisor层支持, 虚拟机具有完整的GuestOS, 虚拟化开销大, 因而性能通常没有容器性能好
安全性	Docker具有宿主机root权限, 有一定安全隐患	<	资源隔离, 相对安全
高可用性	通过业务本身的高可用性来保证	<	武器库丰富: 快照, 克隆, HA, 动态迁移, 异地容灾, 异地双活
使用要求	共享宿主机内核, 不用考虑CPU是否支持虚拟化技术	>	基于硬件的完全虚拟化, 需要硬件CPU虚拟化技术支持

- 可以重点关注密度、启动速度、轻量级和安全性、性能几个方面。

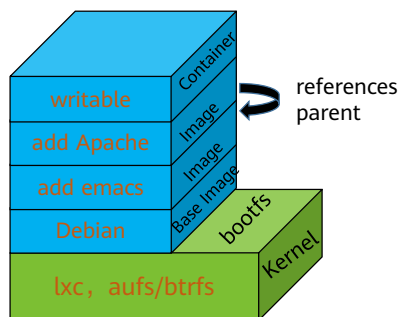
什么是Docker

- Docker是最受大众关注的容器技术，并且现在“几乎”成为事实上的容器标准。
- 简单的应用场景
 - Web应用的自动化打包和发布。
 - 自动化测试和持续集成、发布。

Docker容器与镜像

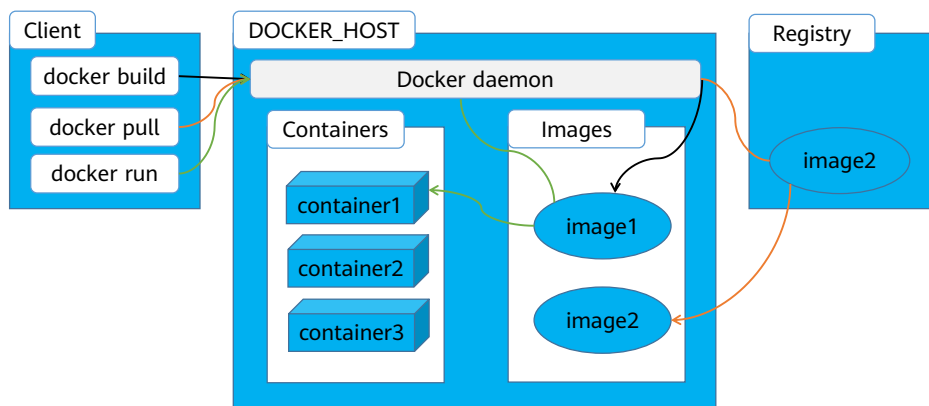
- Docker容器通过Docker镜像来创建。
- 容器与镜像的关系类似于面向对象编程中的对象与类。

Docker	面向对象
容器	对象
镜像	类



- 镜像是多层的文件系统，当镜像运行成一个容器时，就多了一层可写的容器层。Copy on write的特性保证了镜像的共享型，新增的内容只在容器层，需要修改的内容会复制一份在容器层里，不会影响底层镜像文件，同样，需要删除文件时，并不会真的删除底层镜像内容，而是在容器层新增一个对应的whiteout文件，表示该内容已被擦除。

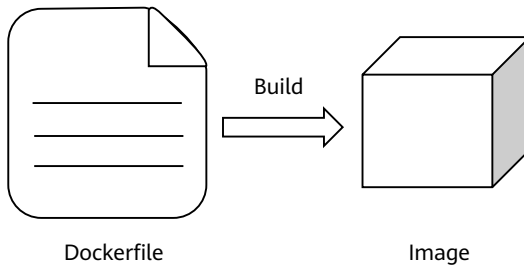
Docker架构



- Docker架构分为三部分：客户端、引擎服务、镜像仓库。使用者在客户端输入命令，Docker daemon执行命令并根据命令和仓库交互。比如执行docker run的命令运行镜像创建容器。
- 镜像可以通过docker pull命令从本地仓库或者远程仓库拉取，也可以通过docker build命令指定Dockerfile脚本文件构建，还可以通过docker commit命令将容器打包成镜像。

Dockerfile

- Dockerfile是一个文本格式的配置文件，包含创建镜像所需要的全部指令。基于Dockerfile中的指令，用户可以快速创建自定义的镜像。



- Dockerfile配置文件说明了镜像的每一层内容，通过docker build命令可以基于Dockerfile文件构建镜像。
- Dockerfile文件的第一行指定镜像的第一层：base image，一般是FROM某一个基础操作系统镜像或者scratch空镜像。

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
- 3. 容器迁移指导**
 - 容器相关概念介绍
 - iSula容器引擎介绍
 - 容器迁移背景和原理
 - 容器迁移主要流程
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

什么是iSula

- 在居住在中南美洲亚马逊丛林的巴西原住民眼里，iSula是一种非常强大的蚂蚁，被它咬一口，犹如被子弹打到那般疼痛，学术上称为“子弹蚁”。华为容器技术方案品牌因其“小个头、大能量”的含义而取名。
- iSula包含全量的容器软件栈，包括引擎、网络、存储、工具集与容器OS；iSulad作为其中轻量化的容器引擎与子弹蚂蚁“小个头、大能量”的形象不谋而合。



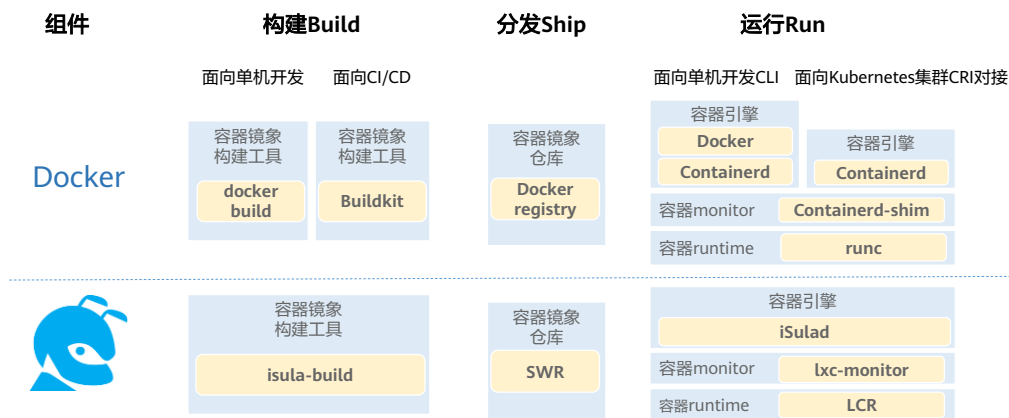
- iSula是除了Docker之外的容器技术，openEuler操作系统同时支持Docker和iSula。

iSula容器应用场景

- openEuler软件包中提供容器运行的基础平台iSula。iSula基础容器平台同时提供Docker engine与轻量化容器引擎iSulad，用户可根据需要自主选择。
- iSula通用容器引擎相比Docker，是一种新的容器解决方案，提供统一的架构设计来满足CT和IT领域的不同需求；相比Golang编写的Docker，iSula轻量级容器使用C/C++实现，具有轻、灵、巧、快的特点，不受硬件规格和架构的限制，底层开销更小，可应用领域更为广泛。
- 同时根据不同使用场景，提供多种容器形态，包括：
 - 适合大部分通用场景的普通容器。
 - 适合强隔离与多租户场景的安全容器。
 - 适合使用systemd管理容器内业务场景的系统容器。

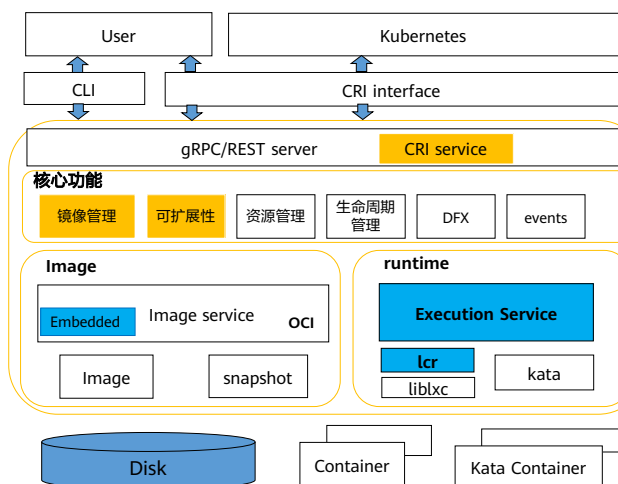
isula-build和iSulad

- 构建侧（isula-build）与运行侧（iSulad）分离



- iSulad是一个基于OCI标准的容器运行引擎，强调简单性、健壮性和轻量化。
- 作为守护进程，iSulad提供容器生命周期管理相关服务：包括镜像的传输和存储、容器执行和监控管理、容器资源管理以及网络等。iSulad对外提供与docker类似的CLI命令行接口，可使用该命令行进行容器管理；并且提供符合CRI接口标准的gRPC API，可供kubernetes按照CRI接口协议调用。

iSulad架构



- 使用C/C++语言编写
- 提供CRI接口
- 支持OCI镜像，支持平滑替换 Docker
- 支持应用容器、系统容器、安全容器等多种容器形态
- 提供便捷使用的命令行
- 提供插件化架构，可根据用户需求开发定制化插件

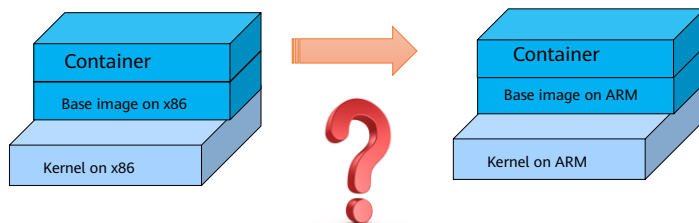
- iSulad是一个新的通用容器引擎，提供统一的架构设计来满足CT和IT领域的不同需求。它具有轻、快、易、灵的特点，这与子弹蚂蚁"小个头、大能量"的形象不谋而合。
- iSulad的特点如下：
 - 轻量语言：C/C++，Rust on the way
 - 北向接口：提供CRI接口，支持对接Kubernetes;同时提供便捷使用的命令行
 - 南向接口：支持OCI runtime和镜像规范，支持平滑替换
 - 容器形态：支持系统容器、虚拟机容器等多种容器形态
 - 扩展能力：提供插件化架构，可根据用户需要开发定制化插件
- 以上的特点使得iSulad可以不受硬件规格和架构的限制，更小的底噪开销也使得它的可应用领域更为广泛。

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
- 3. 容器迁移指导**
 - 容器相关概念介绍
 - iSula容器引擎介绍
 - 容器迁移背景和原理
 - 容器迁移主要流程
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

容器迁移的背景

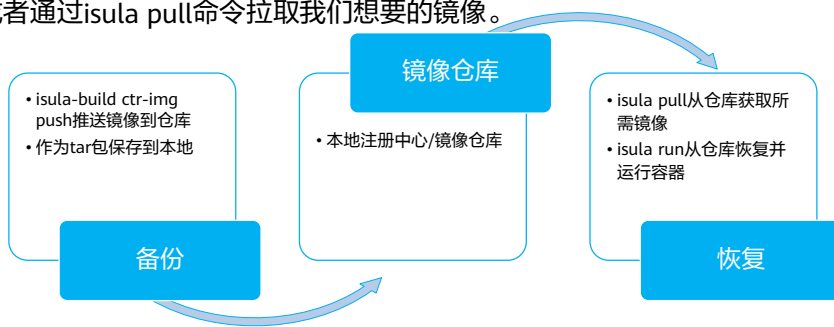
- 随着容器技术应用领域的不断拓展，在不同计算平台上容器也能无差异发挥价值。由此，提出了容器迁移策略。



- 业务的运行载体可以是裸机也可以是虚拟机或容器，那容器应用在跨平台的时候该如何迁移呢？

同一架构下的容器迁移

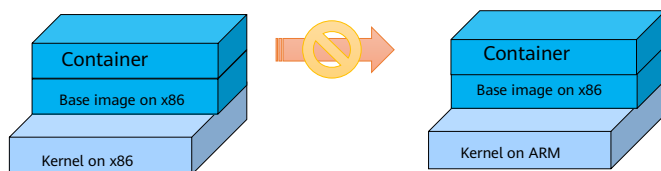
- 我们可以将任何一个容器从一台机器迁移到另一台机器。在迁移过程中，首先要把容器备份为镜像快照，然后将该镜像推送到仓库或者保存到本地。如果我们将镜像推送到了仓库，那就可以简单地从任何我们想要的机器上使用isula run命令来恢复并运行该容器，或者通过isula pull命令拉取我们想要的镜像。



- 如果是相同架构，可以简单地通过容器打包成的镜像平滑迁移，直接在新平台上获取镜像，运行镜像，就可以运行相同应用。

不同架构下的容器兼容性

- 跨平台的容器无法运行，会出现格式错误。
 - x86平台获取的镜像是适用于x86平台，当迁移到鲲鹏平台，容器无法执行。
 - 在基于ARM的平台中，isula pull方式或者Dockerfile方式获取或者构建的镜像均为基于ARM平台的，同样也无法在x86上运行。
- 鲲鹏平台容器需要重新构建。



- 不同架构下，从仓库获取的x86容器镜像在ARM平台上无法成功运行，就需要重新构建新架构平台的镜像和容器。

目录

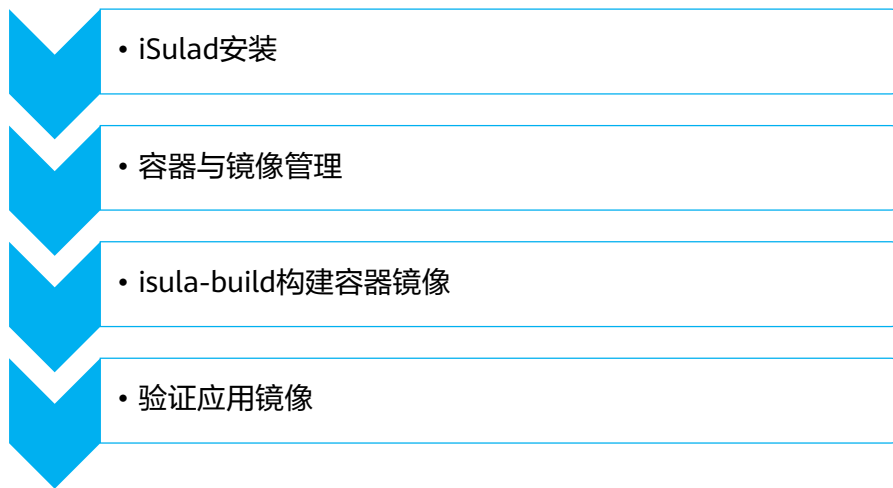
1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
- 3. 容器迁移指导**
 - 容器相关概念介绍
 - iSula容器引擎介绍
 - 容器迁移背景和原理
 - 容器迁移主要流程
4. 迁移常见问题及解决思路
5. 鲲鹏开发者支持套件

鲲鹏平台构建Redis容器应用

- 由于跨计算架构容器无法迁移，需重新构建，鲲鹏平台上我们通过编写Dockerfile构建新的Redis镜像并运行。
- Dockerfile文件内容：
 - FROM hub.oepkgs.net/openeuler/openeuler:20.09
 - WORKDIR /home
 - RUN yum -y install wget gcc make libgcc gcc-c++ glibc-devel tar --nogpgcheck && \
 - wget https://obs-mirror-ftp4.obs.cn-north-4.myhuaweicloud.com/database/redis-4.0.3-aarch64.tar.gz && \
 - tar -zxvf redis-4.0.3-aarch64.tar.gz && mv redis-4.0.3/ redis && rm -f redis-4.0.3-aarch64.tar.gz
 - WORKDIR /home/redis
 - RUN make && make install
 - VOLUME /data
 - EXPOSE 6379
 - CMD ["redis-server"]

- 比如要实现Redis容器应用的迁移，直接获取x86的Redis容器镜像无法在鲲鹏平台使用，我们可以通过编写Dockerfile配置文件构建新的Redis镜像并运行的方式获得鲲鹏平台的Redis容器应用。
- Dockerfile的第一行是基于openeuler:20.09的基础镜像，RUN指令对应的是安装依赖环境、获取Redis源码包、解压并重命名、编译和安装的操作。

鲲鹏平台容器配置步骤（iSula）



- 在鲲鹏平台使用iSula运行Redis容器应用的配置步骤：安装iSulad容器引擎和构建工具，制作Dockerfile文件，构建镜像，运行容器并验证Redis。
- 具体操作过程详见实验手册。

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
3. 容器迁移指导
- 4. 迁移常见问题及解决思路**
 - 常见编译问题
 - 常见功能问题
 - 常见工具问题
5. 鲲鹏开发者支持套件

C/C++语言char数据类型默认符号不一致问题（1）

问题现象

C/C++代码在编译时遇到如下提示：

告警信息：warning: comparison is always false due to limited range of data type。

原因分析

char变量在不同CPU架构下默认符号不一致，在x86架构下为signed char，在ARM64平台为unsigned char，移植时需要指定char变量为signed char。

解决方案

1. 在编译选项中加入"-fsigned-char"选项，指定ARM64平台下的char为有符号数；
2. 将char类型直接声明为有符号char类型：signed char。

C/C++语言char数据类型默认符号不一致问题（2）

```
#include <stdio.h>
int main()
{
    char ch=-1;
    printf("ch=%d\n",ch);
    return 0;
}
```



鲲鹏弹性云服务器

```
[root@ecs-kunpeng ~]# ./charTest
ch=255
```

x86弹性云服务器

```
[root@ecs-x86 ~]# ./charTest
ch=-1
```

可以看到：相同的代码，鲲鹏下char默认为unsigned char类型，所以赋值为-1的时候，输出的为-1对256取模的结果即255，x86中的char默认为signed char类型，输出为-1。

C/C++语言中调用汇编指令编译错误

问题现象

C/C++代码在编译时遇到如下提示：

错误信息：error: impossible constraint in 'asm' __asm__ __volatile__

原因分析

代码中使用汇编指令，而汇编指令与cpu指令集强相关。在x86架构CPU中的汇编指令需要修改为鲲鹏处理器平台的指令才能编译通过，实现功能替换。

解决方案

1. 本例中的代码调用了x86平台的汇编指令，修改为鲲鹏处理器对应的指令即可；
2. 部分功能可以修改为使用编译器自带的builtin函数，在基本不降低性能的前提下，提升代码的可移植性。

编译错误：无法识别-m64编译选项

问题现象

C/C++代码在编译时遇到如下提示：

错误信息：gcc: error: unrecognized command line option '-m64'

原因分析

-m64是x86 64位应用编译选项，m64选项设置int为32bits及long、指针为64 bits，为AMD的x86 64架构生成代码。在鲲鹏处理器平台无法支持。

解决方案

将鲲鹏处理器平台对应的编译选项设置为-mabi=lp64，重新编译即可。

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
3. 容器迁移指导
- 4. 迁移常见问题及解决思路**
 - 常见编译问题
 - 常见功能问题
 - 常见工具问题
5. 鲲鹏开发者支持套件

超出整型取值范围时浮点型转整型与x86不一致

问题现象

C/C++双精度浮点型转整型数据时，如果超出了整型的取值范围，鲲鹏处理器的表现与x86平台的表现不同。测试代码如下：

```
long aa = (long)0x7FFFFFFFFFFFFFFF;  
long bb;  
bb = (long)(aa*(double)10);  
// x86: aa=9223372036854775807, bb=-  
9223372036854775808  
// Kunpeng: aa=9223372036854775807,  
bb=9223372036854775807
```

原因分析

x86（指令集）中的浮点到整型的转换指令，定义了一个 indefinite integer value——“不确定数值”（64bit: 0x8000000000000000），大多数情况下x86平台确实都在遵循这个原则，但是在从double向无符号整型转换时，又出现了不同的结果。鲲鹏的处理则非常清晰和简单，在上溢出或下溢出时，保留整型能表示的最大值或最小值。

C/C++语言double类型超出整型取值范围向整型转换参照表

CPU	double值	转为long变量保留值	CPU	double值	转为unsigned long变量值
x86	正值超出long范围	0x8000000000000000	x86	正值超出long范围	0x0000000000000000
x86	负值超出long范围	0x8000000000000000	x86	负值超出long范围	0x8000000000000000
鲲鹏	正值超出long范围	0x7FFFFFFFFFFFFFFF	鲲鹏	正值超出long范围	0x7FFFFFFFFFFFFFFF
鲲鹏	负值超出long范围	0x8000000000000000	鲲鹏	负值超出long范围	0x0000000000000000

CPU	double值	转为int变量保留值	CPU	double值	转为unsigned int变量值
x86	正值超出int范围	0x80000000	x86	正值超出unsigned int范围	0x80000000
x86	负值超出int范围	0x80000000	x86	负值超出unsigned int范围	0x80000000
鲲鹏	正值超出int范围	0x7FFFFFFF	鲲鹏	正值超出unsigned int范围	0x7FFFFFFF
鲲鹏	负值超出int范围	0x80000000	鲲鹏	负值超出unsigned int范围	0x80000000

对结构体中的变量进行原子操作时程序异常

问题现象

程序调用原子操作函数对结构体中的变量进行原子操作，程序coredump，堆栈如图：

原因分析

鲲鹏处理器对变量的原子操作、锁操作等用到了ldaxr、stlaxr等指令，这些指令要求变量地址必须按变量长度对齐，否则执行指令会触发异常，导致程序coredump。

一般是因为代码中对结构体进行强制字节对齐，导致变量地址不在对齐位置上，对这些变量进行原子操作、锁操作等会触发问题。

解决方案

代码中搜索“#pragma pack”关键字（该宏改变了编译器默认的对齐方式），找到使用了字节对齐的结构体，如果结构体中变量会被作为原子操作、自旋锁、互斥锁、信号量、读写锁的输入参数，则需要修改代码保证这些变量按变量长度对齐。

```
Program received signal SIGBUS, Bus error.
0x00000000040083c in main () at /root/test/src/main.c:19
19  __sync_add_and_fetch(&a.count, step);
(gdb) disassemble
Dump of assembler code for function main:
0x000000000400824 <+0>:  sub    sp, sp, #0x10
0x000000000400828 <+4>:  mov     x0, #0x1                                // #1
0x00000000040082c <+8>:  str     x0, [sp, #8]
0x000000000400830 <+12>: adrp    x0, 0x420000
<_libc_start_main@got.plt>
0x000000000400834 <+16>: add     x0, x0, #0x31 //将变量的地址放入x0寄存器
=> 0x000000000400838 <+20>: ldr     x1, [sp, #8] //指定ldxr取数据的长度（此处为8字节）
=> 0x00000000040083c <+24>: ldaxr   x2, [x0] //ldxr从x0寄存器指向的内存地址中取值
0x000000000400840 <+28>: add     x2, x2, x1
0x000000000400844 <+32>: stlaxr  w3, x2, [x0]
0x000000000400848 <+36>: cbnz    w3, 0x40083c <main+24>
0x00000000040084c <+40>: dmb     ish
0x000000000400850 <+44>: mov     w0, #0x0                                // #0
0x000000000400854 <+48>: add     sp, sp, #0x10
0x000000000400858 <+52>: ret
End of assembler dump.
(gdb) p /x $x0
$4 = 0x420039 // x0寄存器存放的变量地址不在8字节地址对齐处
```

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
3. 容器迁移指导
- 4. 迁移常见问题及解决思路**
 - 常见编译问题
 - 常见功能问题
 - 常见工具问题
5. 鲲鹏开发者支持套件

鲲鹏代码迁移工具源码路径错误

问题现象	原因分析						
使用代码迁移工具执行分析任务时，点击空白区域，找不到待分析的源码文件： 分析源码 检查分析C/C++/Fortran/Go/解释型语言/汇编等源码文件，定位出需迁移代码并给出迁移指导，支持迁移编辑及一键代码替换功能。 * 源码文件存放路径 /opt/portadv/portadmin/sourcecode/ 需要填写相对路径，可以通过以下两种方式实现： 1. 单击“上传”按钮上传压缩包（上传过程中自动解压）或文件夹； 2. 先将源码文件手动上传到服务器上本工具的指定路径下（/opt/portadv/portadmin/sourcecode/），给porting用户开通读写和执行权限，再单击此输入框选择下拉框中对应的源码路径即可。 上传 ▾	源代码需要放置到代码迁移工具的安装目录下对应分析任务的子目录。 解决方案 在服务器登录界面，将源码包放置Web界面显示的路径下，再次回到Web工具界面点击空白处，选择需要分析的源码包。 分析源码 检查分析C/C++/Fortran/Go/解释型语言/汇编等源码文件，定位出需迁移代码并给出迁移指导，支持迁移编辑及一键代码替换功能。 * 源码文件存放路径 /opt/portadv/portadmin/sourcecode/ 上传 ▾ * 源码类型 <table border="1"><tbody><tr><td>PortingTest</td><td>🗑</td></tr><tr><td>PortingTest.zip</td><td>🗑</td></tr><tr><td>megahit</td><td>🗑</td></tr></tbody></table> 目标操作系统	PortingTest	🗑	PortingTest.zip	🗑	megahit	🗑
PortingTest	🗑						
PortingTest.zip	🗑						
megahit	🗑						

- 代码迁移工具不同的分析任务需要将待分析软件包放置不同的服务器路径下。
- 软件迁移评估需要在package目录下，源码分析需要在sourcecode目录下，软件包重构需要在packagerebuild目录下。

maven软件仓库编译错误: 无法找到依赖库

问题现象

maven编译报错:

Failed to execute goal on project hadoop-auth: Could not resolve dependencies for project

org.apache.hadoop:hadoop-auth:jar:2.7.1.2.3.4.7-4:

The following artifacts could not be resolved:

org.mortbay.jetty:jetty-util:jar:6.1.26.hwx,

org.mortbay.jetty:jetty:jar:6.1.26.hwx,

org.apache.zookeeper:zookeeper:jar:3.4.6.2.3.4.7-4:

Failure to find **org.mortbay.jetty:jetty-**

util:jar:6.1.26.hwx in

<https://repository.apache.org/content/repositories/snapshots>

原因分析

从远程仓库下载jetty-util-6.1.26.hwx.jar包失败，原因是远程仓库中没有对应的jar。

解决方案

修改maven安装路径下的conf/settings.xml，在mirror标签中设置maven远程仓库路径，比如华为云：

```
<mirror>
<id>mirror</id>
<mirrorOf>*</mirrorOf>
<name>huaweicloud</name>
<url>https://repo.huaweicloud.com/repository/maven/</url>
</mirror>
```

思考题

1. （多选题）在向鲲鹏处理器迁移软件时，以下哪些是可能导致编译错误或告警的原因？
- A. 编译选项不同
 - B. 数据类型不同
 - C. 汇编指令
 - D. maven版本过低

- ABCD

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
3. 容器迁移指导
4. 迁移常见问题及解决思路
- 5. 鲲鹏开发者支持套件**
 - 鲲鹏开发套件DevKit
 - 鲲鹏应用使能套件BoostKit

鲲鹏开发套件DevKit（1）

- <https://www.hikunpeng.com/developer/devkit>



- 在鲲鹏社区首页找到开发者，选择技术主题：鲲鹏开发套件，可以找到这一系列简化我们做原生开发或者x86向鲲鹏迁移、迁移后测试调优操作的工具。

鲲鹏开发套件DevKit（2）

- 鲲鹏开发套件面向全研发作业流程，提供涵盖代码迁移、开发调试、编译、测试、调优及诊断各环节的开发使能工具，提升应用迁移和调优效率，加速原生开发，方便开发者快速开发出鲲鹏亲和的高性能软件。
- 除了前面介绍过的代码迁移工具和动态二进制翻译工具外，鲲鹏开发套件还包括鲲鹏开发框架插件、鲲鹏编译调试插件、毕昇编译器、毕昇JDK、GCC for openEuler、云测服务、鲲鹏性能分析工具。

鲲鹏开发框架插件（1）

- 鲲鹏开发框架插件是一款使用户在鲲鹏服务器上进行开发的工具。鲲鹏开发框架充分利用鲲鹏平台的各类型算力及性能更优的第三方组件，提供鲲鹏工程向导、启发式编程、代码亲和检查等能力，一键引入鲲鹏加速库、快速构建鲲鹏应用软件框架，帮助开发者更便捷地开发鲲鹏应用，使能开发者高效创新。
 - 鲲鹏工程向导：通过工程向导便捷地生成C/C++新工程：基于cmake/make规范，根据用户选择下载鲲鹏加速库，生成相应demo code和编译所需的基础构建脚本，构建鲲鹏版“hello world”可编译工程。通过鲲鹏工程向导能自动化新建安全计算应用，支持部署SDK、检查编译环境、操作系统版本等。能基于高性能通信库和数学库创建高性能计算应用，用户能够通过扩展工程样例，提升开发效率。

- <https://www.hikunpeng.com/developer/devkit/develop-frame>。

鲲鹏开发框架插件（2）

- 安全计算应用：鲲鹏开发框架能自动化新建安全计算应用，支持部署SDK、检查编译环境和操作系统版本等，并且提供多种安全计算工程。
- 高性能计算应用：支持部署高性能计算SDK、检查编译环境和操作系统版本等，并支持创建高性能通信库（Hyper MPI）工程和数学库（KML）工程。
- 启发式编程：在用户编码时针对鲲鹏加速库等函数进行智能联想补全，展示函数的详细信息，高亮显示智能补全的函数。同时还提供了函数搜索跳转功能，帮助用户更便捷地使用鲲鹏加速库等函数，提高开发效率，提升软件性能。
- 鲲鹏亲和分析：扫描用户选中的代码文件，识别出可以用鲲鹏加速库替换的函数和汇编指令，并生成相应的可视化报告。

鲲鹏编译调试插件

- 鲲鹏编译调试插件一键安装即插即用，支持CPU应用和GPU应用的多种调试方式，以及鲲鹏平台远程调试功能。用户可以在操作中设置断点、查看线程、函数堆栈、寄存器信息或变量信息，支持汇编指令的断点执行和单步调试；提供图形化界面，大幅提升调试效率。
 - 支持用户配置代码编译和调试任务。
 - 支持鲲鹏平台调试NVIDIA CUDA程序，通过统一的调试界面使用CUDA-GDB调试GPU应用。

- <https://www.hikunpeng.com/developer/devkit/develop-frame/complier-debugger>。

鲲鹏编译解释工具（1）

- 毕昇编译器基于开源LLVM开发，并进行了优化和改进，同时支持Fortran语言前端，是针对鲲鹏平台的高性能编译器。除LLVM通用功能和优化外，对中端及后端的关键技术点进行了深度优化，并集成Auto-tuner特性支持编译器自动调优。
- 支持鲲鹏微架构芯片及指令优化。
- 通过软硬协同提供相比开源LLVM更高的性能。
- 集成Auto-tuner特性支持编译器自动调优。

- 毕昇编译器：<https://www.hikunpeng.com/developer/devkit/compiler/bisheng>。

鲲鹏编译解释工具（2）

- 毕昇JDK基于OpenJDK开发，是一个高性能、可用于生产环境的OpenJDK发行版。毕昇JDK运行在华为内部多个产品上，积累了大量使用场景和Java开发者反馈的问题和诉求，解决了业务实际运行中遇到的多个问题，并在ARM架构上进行了性能优化。毕昇JDK运行在大数据等场景下可以获得更好的性能。毕昇JDK 8与Java SE标准兼容，目前支持Linux/aarch64和Linux/x86_64平台。毕昇JDK同时是OpenJDK的下游，会持续稳定为OpenJDK社区做出贡献。
- 针对ARM架构和大数据场景优化，可以获得更好的性能。

- 毕昇JDK：<https://www.hikunpeng.com/developer/devkit/compiler/jdk>。

鲲鹏编译解释工具（3）

- GCC for openEuler是基于开源GCC开发的编译器工具链（包含编译器，汇编器，链接器），在openEuler社区开源发布，并通过鲲鹏社区免费提供二进制包，支持aarch64处理器架构。
- 支持鲲鹏微架构芯片及指令优化。
- 通过软硬协同提供相较开源GCC更高的性能。
- 高性能计算典型应用性能深度优化。

- GCC for openEuler: <https://www.hikunpeng.com/developer/devkit/compiler/gcc>。

鲲鹏云测服务

- 云测服务面向开发者提供基于鲲鹏平台的兼容性测试、可靠性测试、安全测试、功能测试、性能测试服务功能，帮助您快速识别和定位应用程序在运行阶段的问题。

主要功能	功能描述
兼容性测试	通过待测试应用软件在鲲鹏环境启动前后资源波动异常检测、验证应用软件启动和停止，自动检测应用软件在鲲鹏平台上的可运行性、兼容性问题。
可靠性测试	通过待测试应用软件在稳定运行期间的系统资源内存的波动异常检测、在异常终止测试场景检测应用运行，自动评估应用软件在鲲鹏平台上的稳定性和可靠性。
安全测试	通过待测试应用软件在运行情况下的端口扫描、病毒扫描和漏洞扫描，自动发现应用软件可能存在的安全风险。
功能测试	基于pytest和shUnit2全功能测试框架，对应用软件进行自动化功能测试。该功能用户需要编写和上传测试用例脚本。
性能测试	集成Apache JMeter等的性能测试工具，对待测试应用软件进行性能测试，并给出性能分析结果。该功能用户需要编写和上传测试用例脚本。

- <https://www.hikunpeng.com/developer/devkit/cloudTest>。

鲲鹏性能分析工具（1）

- 性能分析工具支持鲲鹏平台上的系统性能分析、Java性能分析和系统诊断，提供系统全景及常见应用场景下的性能采集和分析功能，并基于调优专家系统给出优化建议。同时提供调优助手，指导用户快速调优系统性能。
- 鲲鹏性能分析工具支持IDE插件（VS Code、IntelliJ）和浏览器两种工作模式，分别同性能分析Server一起完成性能分析和优化等任务。

- <https://www.hikunpeng.com/developer/devkit/hyper-tuner>。

鲲鹏性能分析工具（2）

- 性能分析工具包含以下子工具：
 - 调优助手：通过系统化组织和分析性能指标、热点函数、系统配置等信息，形成系统资源消耗链条，引导用户分析性能瓶颈，并给出优化建议和操作指导，实现快速调优。
 - 系统性能分析：基于鲲鹏服务器的性能分析工具，针对多个专项分析系统性能状况，识别系统性能瓶颈，并给出针对性的多维度优化建议。
 - Java性能分析：针对基于鲲鹏的服务器上运行的Java程序的性能分析和优化工具，图形化显示Java程序的堆、线程、锁、垃圾回收等信息，帮助用户采取针对性优化。
 - 系统诊断：能够快速定位内存、网络、存储等部件的异常，提供内存泄漏诊断、内存消耗信息分析展示、OOM诊断能力、网络IO诊断能力、存储IO诊断能力，帮助用户识别出源代码中内存使用的问题点。

目录

1. 软件迁移原理与迁移过程
2. 鲲鹏迁移指导
3. 容器迁移指导
4. 迁移常见问题及解决思路
- 5. 鲲鹏开发者支持套件**
 - 鲲鹏开发套件DevKit
 - 鲲鹏应用使能套件BoostKit

鲲鹏应用使能套件BoostKit（1）

- 鲲鹏应用使能套件帮助我们提升业务应用在鲲鹏平台的性能表现。



- <https://www.hikunpeng.com/developer/boostkit>。

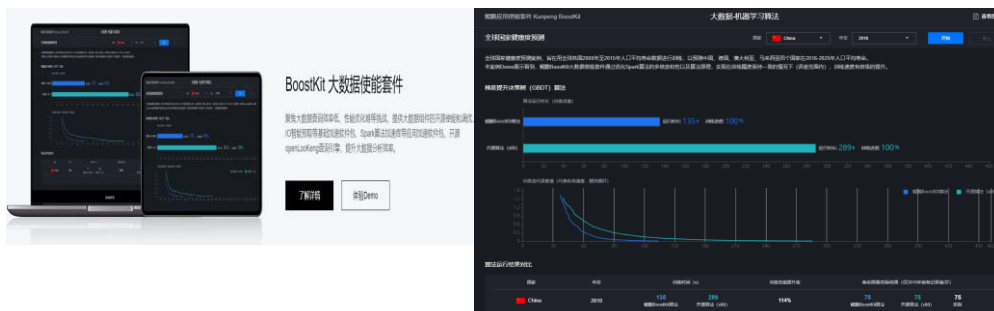
鲲鹏应用使能套件BoostKit（2）

- 鲲鹏应用使能套件：共享8大场景最佳实践，从硬件、基础软件到应用开展全栈优化（包括算法优化、软硬协同、原生架构独创技术等），提供高性能开源组件、基础加速软件包、应用加速软件包，使能应用极致性能。
 - 高性能开源组件：鲲鹏BoostKit提供海量的高性能开源组件，使能主流开源软件支持鲲鹏高性能。
 - 基础加速软件包：鲲鹏BoostKit加速库提供基于ARM指令深度优化和基于鲲鹏KAE（鲲鹏硬件加速引擎）开发的加速库，覆盖系统库、压缩、加解密、媒体、数学库、存储、网络等7类加速库，为大数据加解密、分布式存储压缩、视频转码等应用场景提供高性能加速。
 - 应用加速软件包：使能数据处理极致性能、数据访问极致高效和云手机极致体验。

- 鲲鹏 BoostKit 大数据使能套件：
<https://www.hikunpeng.com/zh/developer/boostkit/big-data>。
- 鲲鹏 BoostKit 分布式存储使能套件：
<https://www.hikunpeng.com/zh/developer/boostkit/sds>。
- 鲲鹏 BoostKit 数据库使能套件：
<https://www.hikunpeng.com/zh/developer/boostkit/database>。
- 鲲鹏 BoostKit 虚拟化使能套件：
<https://www.hikunpeng.com/zh/developer/boostkit/virtualization>。
- 鲲鹏 BoostKit ARM 原生使能套件：
<https://www.hikunpeng.com/zh/developer/boostkit/arm-native>。
- 鲲鹏 BoostKit Web 使能套件：
<https://www.hikunpeng.com/zh/developer/boostkit/web>。
- 鲲鹏 BoostKit CDN 使能套件：
<https://www.hikunpeng.com/zh/developer/boostkit/cdn>。
- 鲲鹏 HPC：<https://www.hikunpeng.com/zh/developer/hpc>。

场景化应用使能套件Demo体验

- 例如大数据使能套件Demo：
 - <https://www.hikunpeng.com/zh/developer/boostkit/demo/bigdata>。



- <https://www.hikunpeng.com/developer/boostkit/scene>：通过该链接直达应用使能套件8大场景的实践案例，点击“体验Demo”直观化查看前后对比。

思考题

1. （多选题）鲲鹏开发套件（DevKit）提供了哪些工具？
- A. 鲲鹏代码迁移工具
 - B. 毕昇编译器
 - C. 毕昇JDK
 - D. 鲲鹏性能分析工具

- ABCD

本章总结

- 本章为大家介绍了鲲鹏应用移植原理及操作流程，借助代码迁移工具实现编译型语言应用和解释型语言应用基于服务器平台和容器平台的移植，并针对移植过程中常见问题做了说明。
- 同时为大家介绍了鲲鹏开发者生态工具套件DevKit和BoostKit，帮助大家更好地完成迁移及性能提升。

学习推荐

- 《 TaiShan代码移植指导 》
 - <https://bbs.huaweicloud.com/forum/thread-22606-1-1.html>
- 鲲鹏社区
 - <https://hikunpeng.com>
- 《 计算机组成与设计（ ARM版 ） 》

缩略语

- ARM: Advanced RISC Machines, 过去也称作Acorn RISC Machines, 通常指使用精简指令集RISC的处理器架构。
- RPM: Red Hat Package Manager, Red Hat贡献出来的软件包管理工具; 它是一种用于互联网下载包的打包及安装工具, 它包含在某些Linux发行版中。
- DEB: GNU计划, 有译为“革奴计划”, 是由理查德·斯托曼在1983年9月27日公开发起的自由软件集体协作计划。它的目标是创建一套完全自由的操作系统GNU。

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。
Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

Copyright©2023 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



应用性能测试与调优



前言

- 软件迁移到新的平台之后，如果要软件更好地适配新的系统，并发挥出系统最大的性能，需要对软件进行性能测试和调优。本章节就介绍了软件性能测试和调优的常见思路和方法。

目标

- 学完本课程后，您将能够：
 - 了解软件工程中性能测试的必要性；
 - 了解常见的性能测试方法论；
 - 熟悉常见测试方法；
 - 熟悉Linux下用于性能监控的工具；
 - 熟悉常用软件的一些性能测试方法及测试工具使用；
 - 了解鲲鹏系统性能分析工具的架构、功能特性和使用方法；
 - 了解鲲鹏Java性能分析工具的架构、功能特性和使用方法。

目录

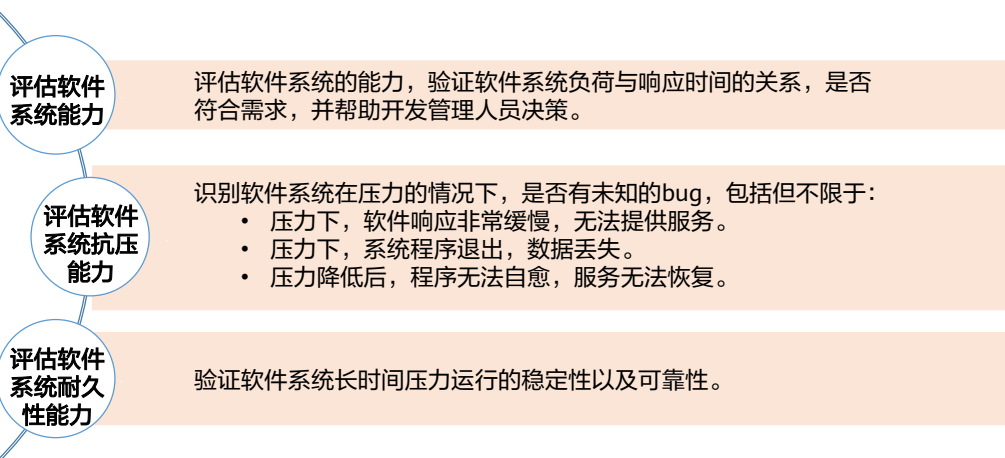
1. 软件性能测试

- 性能测试方法与工具
 - 常用软件性能测试案例

2. 鲲鹏通用性能调优方法

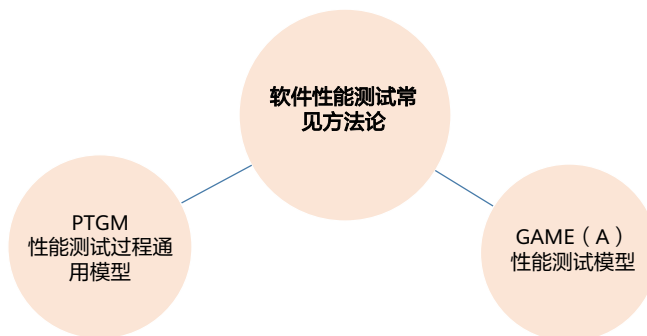
3. 鲲鹏性能分析工具

软件性能测试目的



性能测试方法论

- 对软件系统进行性能测试，有两种常见的测试方法论：



PTGM性能测试过程模型

- PTGM（Performance Testing General Model）性能测试过程模型，分为以下五个阶段：



- PTGM是经典的性能测试方法论之一，旨在梳理性能测试的流程和相关环节使用到的测试工具，是系统性的测试方法，后续介绍到的具体测试场景和测试工具，逻辑上都遵循PTGM的测试方法，是经典单体应用开发时代常用的测试方法，体现在：流程明确，重视测试文档。

性能测试GAME（A）模型

- 性能测试GAME（A）模型包含以下五个重要项目：
 - 目标（Goal）：确定本次测试目标，选择测试设计策略
 - 分析（Analysis）：分析性能需求，分析系统架构
 - 度量（Metrics）：场景的定义、事务的定义、虚拟用户的pass/fail的标准
 - 执行（Execution）：环境、数据、脚本的准备，场景及监控器的运行，生成报告
 - 调整（Adjust）：应用程序的修改，中间件的调优

- GAME 测试模型，较前述PTGM模型而言，是重视测试效果和执行的测试方法论，系统引入了测试结果的反馈机制和再调整方法，适合快速迭代的应用场景。GAME/PTGM作为两个经典测试方法论，没有优劣之分，是与应用开发和使用场景相适应的测试理论。

常见性能测试方法

- 以下是七种常见的性能测试方法：

测试方法	说明
负载测试	Load Testing是指模拟真实的用户行为，通过不断加压直到性能出现瓶颈或资源达到饱和。负载测试是最经常进行的性能测试，用于测量系统的容量，发现系统瓶颈并配合性能调优。有时候也称为可量性测试 Scalability Testing。
压力测试	Stress Testing是指测试系统在一定的饱和状态下系统的处理能力。负载测试的不断加压到一定阶段即是压力测试，两者没有明确的界限。压力测试通常设定到CPU使用率达到75%以上，内存使用率达到 70%以上，用于测试系统在压力环境下的稳定性。此处是指过载情况下的稳定性，略微不同于7*24长时间运行的稳定性。
基准测试	Benchmark Testing指通过设计科学的测试方法、测试工具和测试系统，实现对一类测试对象的某项性能指标进行定量的和可对比的测试。
可靠性测试	Reliability Testing是指加载一定的业务压力，同时让此压力持续运行一段时间，测试系统是否可以稳定运行. 可以理解为压力测试关注的是过载压力，可靠性测试关注的是持续时间。
并发测试	Concurrency Testing是模拟用户并发访问同一应用的测试，用于发现并发问题诸如内存泄漏，线程锁，资源争用，数据库死锁。
配置测试	Configuration Testing验证各种配置对系统性能的影响，用于性能调优和规划能力。
可恢复测试	Failover Testing针对有冗余备份和负载均衡的系统，检验系统局部故障时用户所受到的影响。



- 表格中列出的常见性能测试方法，在实际应用中不一定全部都会测试，而是根据具体场景和要求，针对性的选定测试方法。

LoadRunner性能测试过程

- LoadRunner性能测试过程，分为以下六个步骤：
 - 性能需求测试设计：针对需求，划定测试范围，测试内容。
 - 计划测试：定义明确的测试计划，制定LoadRunner场景，完成负载测试目标。
 - 创建VU脚本：确定每个VU执行的操作，确定同时多少个VU执行，选择具体的事务度量。
 - 创建测试场景：使用LoadRunner Controller创建场景。
 - 运行测试场景：执行任务来模拟服务器上的用户负载。
 - 测试结果分析：使用LoadRunner的图和报告来分析应用程序的性能。

- VU脚本：LoadRunner中用户根据测试场景和测试目的，生成的测试流程和参数脚本。
- LoadRunner工具是HP公司开发的测试软件，在业界广泛使用，目前在web领域使用的比较多，其中重要的一层概念是VU，即测试过程中的测试流程和参数脚本，与前面介绍的PTGM和GAME比较来看，LoadRunner属于具体的测试软件，把方法论融入到具体实践中。关于LoadRunner，有大量书籍系统介绍这一测试工具，本节作引导介绍，不做具体演示。

性能指标与部分应用关系

	门户网站	办公OA系统	数据交换平台	邮件系统	交易平台	电子政务系统
最大用户并发数				√	√	
吞吐量	√		√	√	√	
平均事务响应时间	√	√	√	√	√	√
事务通过率		√	√	√	√	√
CPU性能指标	√	√	√	√	√	√
内存性能指标	√	√	√	√	√	√
磁盘I/O性能指标	√	√	√	√	√	√
中间件性能指标			√	√	√	
数据库性能指标	√	√	√	√	√	√
网络性能			√	√	√	
其他性能	√	√	√	√	√	√



- 通过表格中的信息，可以了解到：常用信息系统的主要性能指标，是对前面系统测试方法的补充说明，通过生活中能接触到的，比如OA系统/交易平台等信息化软件，让抽象性能指标和实际应用建立联系。

目录

1. 软件性能测试

- 性能测试方法与工具
- 常用软件性能测试案例

2. 鲲鹏通用性能调优方法

3. 鲲鹏性能分析工具

性能测试常用工具



- 在软件测试领域，技术人员常接触各种具体测试工具，通过测试工具热点词云，可以定性了解哪些是常用热门测试工具。
- 例如：LoadRunner、JMeter、Wrk等。

性能测试工具介绍（1）

- **LoadRunner**是由HP开发的Web服务器性能测试工具，体系庞大，功能丰富，可以提供各种传输协议以分析服务器性能。
- **Jmeter**是java平台的性能开源测试工具，功能强大，方便易用。
- **ab**是Apache提供的一款测试工具，具有简单易上手的特点，在测试Web服务时非常实用。ab可以在Windows系统中使用，也可以在Linux系统中使用。
- **WebBench**由Lionbridge公司开发，主要测试每秒钟请求数和每秒钟数据传输量，同时支持静态、动态、SSL，部署简单，静动态均可测试。适用于小型网站压力测试（单例最多可模拟3万并发）。
- **MaxQ**是一个Web功能测试工具。它包括一个记录测试脚本的HTTP代理，一个用于重放测试的命令行实用程序。

性能测试工具介绍（2）

- **MySQLslap**是MySQL自带的基准测试工具，优点：查询数据，语法简单，灵活容易使用。MySQLslap为MySQL性能优化前后提供了直观的验证依据，建议系统运维和DBA人员应该了解常见压力测试工具，才能处理线上数据库支撑的用户流量上限及其抗压性等问题。
- **Sysbench**是一个开源、模块化、跨平台多线程性能测试工具，可以用来进行CPU、内存、磁盘I/O、线程、数据库的性能测试。目前支持的数据库有MySQL、Oracle和PostgreSQL。
- **Pgbench**是基于tpc-b模型的PostgreSQL测试工具。它属于开源软件，主要是对PostgreSQL进行压力测试的一款简单程序，SQL命令可以在一个连接中顺序地执行，通常会开多个数据库Session，并且在测试最后形成测试报告，得出每秒平均事务数，pgbench可以测试select、update、insert、delete命令，用户可以编写自己的脚本进行测试。

常用软件及其测试工具举例

测试场景	性能测试对象	测试工具
中间件	Nginx	wrk
	Tomcat	wrk
	Redis	redis-benchmark
数据库	MySQL	sysbench
	memcached	memaslap
	PostgreSQL	pgbench

wrk测试Nginx性能

- 测试命令：

```
wrk -t12 -c100 -d30s http://127.0.0.1
```

- 说明：

- 模拟12个线程、100次连接、压力测试持续30秒，测试“http://127.0.0.1”页面的性能，其中http://127.0.0.1为本地nginx访问链接，wrk对本地安装的nginx进行性能测试。

- 返回结果：

- 见右图

- 结果说明：

- Avg Stdev Max +/- Stdev
- （平均值）（标准差）（最大值）（正负一个标准差所占比例）
- Latency: 延迟
- Req/Sec: 处理中的请求数
- 1011584 requests in 30.04s, 819.97MB read（30.04秒内共处理完成了1011584个请求，读取了819.97MB数据）

```
[root@ecs-24f5 ~]# wrk -t12 -c100 -d30s http://127.0.0.1
Running 30s test @ http://127.0.0.1
 12 threads and 100 connections
Thread Stats   Avg      Stdev   Max    +/- Stdev
Latency       2.94ms   1.45ms  61.45ms  91.18%
Req/Sec       2.82k    169.14   5.32k    79.97%
1011584 requests in 30.04s, 819.97MB read
Requests/sec: 33676.74
Transfer/sec: 27.30MB
```



- 本测试案例旨在说明测试工具的使用方法，及测试结果的概念解读，因不存在具体业务的对比数据，测试结果也就不存在实际意义。

wrk测试Tomcat性能

- 测试命令：

内网测试：wrk -t12 -c100 -d30s http://127.0.0.1:8080

外网测试：wrk -t12 -c100 -d30s http://云服务器公网IP地址:8080
- 说明：
 - 模拟12个线程、100次连接、压力测试持续30秒，测试“http://127.0.0.1:8080”页面的性能，其中8080为tomcat端口，wrk对本地安装的tomcat进行性能测试。
- 返回结果：（见右图）
- 结果说明：
 - Avg Stdev Max +/- Stdev
 - （平均值）（标准差）（最大值）（正负一个标准差所占比例）
 - Latency: 延迟
 - 514174 requests in 30.09s, 5.43GB read（30.09秒内共处理完成了514174个请求，读取了5.43GB数据）
 - Requests/sec: 17086.53（平均每秒处理完成17086.53个请求）

```
[root@ecs-24f5 ~]# wrk -t12 -c100 -d30s http://127.0.0.1:8080
Running 30s test @ http://127.0.0.1:8080
12 threads and 100 connections
Thread Stats   Avg      Stdev   Max    +/- Stdev
  Latency    10.10ms  19.58ms 430.36ms  94.10%
  Req/Sec    1.44k    571.09   3.77k    71.12%
514174 requests in 30.09s, 5.43GB read
Requests/sec: 17086.53
Transfer/sec: 184.72MB
```

MySQL性能测试

- MySQL是一个关系型数据库管理系统。
- sysbench是一款开源多线程性能测试工具，可以执行CPU、内存、线程、IO、数据库等方面性能测试。
- 对MySQL的基准测试，有如下两种思路：
 - 针对整个系统的基准测试：通过http请求进行测试，如通过浏览器、APP或postman等测试工具。该方案的优点是能够更好的针对整个系统，测试结果更加准确；缺点是设计复杂实现困难。
 - 只针对MySQL的基准测试：优点和缺点与针对整个系统的测试恰好相反。
- 在针对MySQL进行基准测试时，一般使用专门的工具进行，例如MySQLslap、sysbench等。其中，sysbench比MySQLslap更通用、更强大，且更适合InnoDB（因为模拟了许多InnoDB的IO特性），下面介绍使用sysbench进行基准测试的方法。

- MySQL是一个关系型数据库管理系统，由瑞典MySQL AB 公司开发，属于 Oracle 旗下产品。MySQL 是最流行的关系型数据库管理系统之一，在 WEB 应用方面，MySQL是最好的 RDBMS (Relational Database Management System，关系数据库管理系统) 应用软件之一。
- MySQL的指标是非常多，重点关注：
 1. QPS (queries per seconds): 每秒钟查询数量。
 2. TPS (Transaction per seconds): 每秒的事务数。
 3. 线程连接数。
 4. Query Cache 查询缓存。

MySQL性能测试准备

- 步骤一：登录MySQL

```
mysql -uroot -p123456
```

- 步骤二：创建sysbench测试使用的数据库“dbtest”

```
create database dbtest;  
show databases;
```

```
mysql> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| dbtest |  
| mysql |  
| performance_schema |  
| sys |  
+-----+  
5 rows in set (0.00 sec)
```


MySQL性能测试：准备数据

```
sysbench /usr/local/share/sysbench/oltp_read_write.lua --mysql-host=127.0.0.1 --mysql-port=3306 --mysql-user=root --mysql-password=123456 --mysql-db=dbtest --db-driver=mysql --tables=1 --table-size=10000 --report-interval=30 --threads=1 --time=30 prepare
```

```
[root@ecs-24f5 ~]# sysbench /usr/local/share/sysbench/oltp_read_write.lua --mysql-host=127.0.0.1 --mysql-port=3306 --mysql-user=root --mysql-password=123456 --mysql-db=dbtest --db-driver=mysql --tables=1 --table-size=10000 --report-interval=30 --threads=1 --time=30 prepare
sysbench 1.0.19 (using bundled LuaJIT 2.1.0-beta2)

Creating table 'sbtest1'...
Inserting 10000 records into 'sbtest1'
Creating a secondary index on 'sbtest1'...
```

MySQL性能测试：执行测试（1）

```
sysbench /usr/local/share/sysbench/oltp_read_write.lua --mysql-host=127.0.0.1 --mysql-port=3306 --mysql-user=root --mysql-password=123456 --mysql-db=dbtest --db-driver=mysql --tables=1 --table-size=10000 --report-interval=30 --threads=1 --time=30 run
```

- 其中，对于比较重要的信息包括：
 - queries：查询总数即qps
 - transactions：事务总数即tps
 - Latency-95th percentile：前95%的请求的最大响应时间，本例中是7.7毫秒

MySQL性能测试：执行测试（2）

```
[root@ecs-24f5 ~]# sysbench /usr/local/share/sysbench/oltp_read_write.lua --mysql-host=127.0.0.1
--mysql-port=3306 --mysql-user=root --mysql-password=123456 --mysql-db=dbtest --db-
river=mysql --tables=1 --table-size=10000 --report-interval=30 --threads=1 --time=30 run
sysbench 1.0.19 (using bundled LuaJIT 2.1.0-beta2)

Running the test with following options:
Number of threads: 1
Report intermediate results every 30 second(s)
Initializing random number generator from current time

Initializing worker threads...

Threads started!

SQL statistics:
  queries performed:
    read:                90510
    write:               25860
    other:               12930
    total:              129300
```

MySQL性能测试：执行测试（3）

```
transactions:          6465  (215.47 per sec.)
queries:               129300 (4309.48 per sec.)
ignored errors:        0      (0.00 per sec.)
reconnects:            0      (0.00 per sec.)

General statistics:
  total time:           30.0026s
  total number of events: 6465

Latency (ms):
  min:                  2.98
  avg:                  4.64
  max:                  31.39
  95th percentile:     7.70
  sum:                  29989.54

Threads fairness:
  events (avg/stddev):  6465.0000/0.00
  execution time (avg/stddev): 29.9895/0.00
```

MySQL性能测试：清理数据

- 执行完测试后，清理数据，否则后面的测试会受到影响。

```
sysbench /usr/local/share/sysbench/oltp_read_write.lua --mysql-host=127.0.0.1 --mysql-port=3306 --mysql-user=root --mysql-password=123456 --mysql-db=dbtest --db-driver=mysql --tables=1 --table-size=10000 --report-interval=30 --threads=1 --time=30 cleanup
```

```
[root@ecs-24f5 ~]# sysbench /usr/local/share/sysbench/oltp_read_write.lua --mysql-host=127.0.0.1 --mysql-port=3306 --mysql-user=root --mysql-password=123456 --mysql-db=dbtest --db-driver=mysql --tables=1 --table-size=10000 --report-interval=30 --threads=1 --time=30 cleanup
sysbench 1.0.19 (using bundled LuaJIT 2.1.0-beta2)

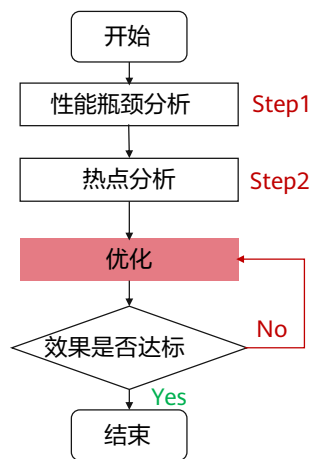
Dropping table 'sptest1'...
```

目录

1. 软件性能测试
2. **鲲鹏通用性能调优方法**
 - CPU和内存子系统性能调优
 - 网络子系统性能调优
 - 磁盘IO子系统调优
 - 应用程序调优
3. 鲲鹏性能分析工具

性能调优一般过程

- 调优的目的是缩小系统性能参数与目标值之间的差距，通常要经历以下过程



Step1: 从物理资源使用情况入手

CPU	top/vmstat
Memory	top/vmstat/free
IO	top/vmstat/iostat
Network	iftop/netstat

Step2: 进一步热点分析和问题定位

CPU	perf/gprof
Memory	valgrind
IO	iostat/iotop
Network	netstat/netperf

- 软件在基础设施（服务器、虚拟机、容器等）上运行，难免会因为软、硬件故障出现性能下降、运行异常等问题。
- 性能调优的过程如同去做体检，并不是所有系统都需要严格的性能调优，但一般调优过程分为两步：
 - 性能瓶颈分析，类似于全身体检，定位影响软件性能的主要指标异常，为后续的深度调优做基础；
 - 热点分析，在发现性能瓶颈后，需要对瓶颈点做细致分析，定点排查，找出影响软件性能的关键因素，为后续调优提供做准备。
- 不同阶段，使用的Linux命令是不同的，PPT中罗列的若干命令，功能强大，值得深入学习。

CPU和内存子系统性能调优概述

- 性能优化思路如下：
 - 如果CPU利用率不高，说明资源没有充分利用，可以通过工具（如perf）查看应用程序阻塞在哪里（一般为磁盘），网络或应用程序业务逻辑是否有休眠或信号等待，这些优化措施在其他章节描述。
 - 如果CPU利用率高，可以选择更好的硬件，优化硬件的配置参数来适配业务场景，或者通过优化软件来降低CPU占用率。
- 根据CPU的能力配置合适的内存条，建议内存满通道配置，发挥内存最大带宽：一颗鲲鹏920处理器的内存通道数为8，两颗鲲鹏920处理器的内存通道数为16；建议选择高频率的内存条，提升内存带宽：鲲鹏920在1DPC配置时，支持的内存最高频率为2933MHz。

常用性能监测工具 - top

- top是最常用的Linux性能监测工具之一。通过top工具可以监视进程和系统整体性能。

```
[root@localhost ~]#  
[root@localhost ~]# top  
top - 19:23:49 up 3 days, 5:24, 4 users, load average: 3.17, 2.63, 2.80  
Tasks: 704 total, 1 running, 338 sleeping, 0 stopped, 0 zombie  
%Cpu(s): 2.8 us, 0.6 sy, 0.0 ni, 96.1 id, 0.5 wa, 0.0 hi, 0.0 si, 0.0 st  
KiB Mem : 26727008+total, 23552249+free, 25547456 used, 6200128 buff/cache  
KiB Swap: 4194240 total, 4026176 free, 168064 used, 21798764+avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
22083	root	20	0	161.9g	7.1g	86208	S	132.0	2.8	215:35.41	impalad
21347	root	20	0	8871552	5.9g	38272	S	88.1	2.3	65:10.36	kudu-tserver
27953	root	20	0	118400	8448	3712	R	1.3	0.0	0:00.09	top
22761	root	20	0	118400	8448	3712	S	0.7	0.0	1:09.88	top
5709	root	20	0	433536	20480	10112	S	0.3	0.0	3:58.31	NetworkManager
11035	root	20	0	3344832	769984	35840	S	0.3	0.3	14:21.12	java
11202	root	20	0	3323072	818240	35840	S	0.3	0.3	15:34.67	java
12948	root	20	0	413120	46272	27584	S	0.3	0.0	3:07.55	statestored
12949	root	20	0	33.6g	1.7g	67776	S	0.3	0.7	11:26.51	catalogd
21248	root	20	0	820800	62976	36096	S	0.3	0.0	0:20.40	kudu-master
1	root	20	0	164480	16512	5824	S	0.0	0.0	0:17.23	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.38	kthreadd
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0:0H
5	root	20	0	0	0	0	I	0.0	0.0	0:00.03	kworker/u128:0
7	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	mm_percpu_wq

- CPU项：显示当前总的CPU时间使用分布。
- us表示用户态程序占用的CPU时间百分比。
- sy表示内核态程序所占用的CPU时间百分比。
- wa表示等待IO等待占用的CPU时间百分比。
- hi表示硬中断所占用的CPU时间百分比。
- si表示软中断所占用的CPU时间百分比。
- 通过这些参数可以分析CPU时间的分布，是否有较多的IO等待。在执行完调优步骤后，也可以对CPU使用时间进行前后对比。如果在运行相同程序、业务情况下CPU使用时间降低，说明性能有提升。
- KiB Mem：表示服务器的总内存大小以及使用情况。
- KiB Swap：表示当前所使用的Swap空间的大小。Swap空间即当内存不足的时候，把一部分硬盘空间虚拟成内存使用。如果当前所使用的Swap空间大于0，可以考虑优化应用的内存占用或增加物理内存。

常用性能监测工具 - perf

- perf工具是非常强大的Linux性能分析工具，可以通过该工具获得进程内的调用情况、资源消耗情况并查找分析热点函数。

```
Samples: 14K of event 'cycles:ppp', Event count (approx.): 4104207681
```

Overhead	Shared Object	Symbol
6.41%	[kernel]	[k] arch_cpu_idle
2.38%	[kernel]	[k] finish_task_switch
2.07%	perf	[.] __symbols_insert
1.91%	[kernel]	[k] module_get_kallsym
1.53%	libpython2.7.so.1.0	[.] PyEval_EvalFrameEx
1.50%	libc-2.17.so	[.] strlen
1.48%	libc-2.17.so	[.] strchr
1.42%	perf	[.] rb_next
1.33%	[kernel]	[k] __ll_sc_atomic_sub_return_release
1.23%	[kernel]	[k] copy_page
1.22%	[kernel]	[k] __ll_sc_cmpxchg_case_acq_4
1.20%	libc-2.17.so	[.] __libc_calloc
1.17%	[kernel]	[k] el0_svc_naked
1.11%	[kernel]	[k] __flush_cache_user_range
1.00%	libc-2.17.so	[.] __int_malloc
0.95%	[kernel]	[k] clear_page
0.90%	libc-2.17.so	[.] __int_free

- 通过热点函数，可以找到消耗资源较多的行为，从而有针对性的进行优化。

- 通过perf top命令查找热点函数。
- 该命令统计各个函数在某个性能事件上的热度，默认显示CPU占用率，可以通过“-e”监控其他事件。
- Overhead表示当前事件在全部事件中占的比例。
- Shared Object表示当前事件生产者，如kernel、perf命令、C语言库函数等。
- Symbol则表示热点事件对应的函数名称。

常用性能监测工具 - strace (1)

- strace是Linux环境下的程序调试工具，用来跟踪应用程序的系统调用情况。strace命令执行的结果就是按照调用顺序打印出所有的系统调用，包括函数名、参数列表以及返回值等。
- 命令格式：strace [参数]
- 常用参数如下：
 - -T 显示每一调用所耗的时间。
 - -tt 在输出中的每一行前加上时间信息，微秒级。
 - -p 跟踪指定的线程ID。

常用性能监测工具 - strace (2)

[illegible]

- 通过strace命令 调取 执行 `cat /etc/passwd` 这个命令过程的系统响应。有分配内存空间、打开文件、读取文件等诸多详细步骤。
- 可以使用strace对应用的系统调用和信号传递的跟踪结果来对应用进行分析，以达到解决问题或者是了解应用工作过程的目的。
- strace的最简单的用法就是执行一个指定的命令，在指定的命令结束之后它也就退出了。
- 在命令执行的过程中，strace会记录和解析命令进程的所有系统调用以及这个进程所接收到的所有的信号值。

常用性能监测工具 - numactl

- numactl工具可用于查看当前服务器的NUMA节点配置、状态，可通过该工具将进程绑定到指定CPU core，由指定CPU core来运行对应进程。

```
[root@localhost ~]# numactl -H
available: 4 nodes (0-3)
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
node 0 size: 64794 MB
node 0 free: 55404 MB
node 1 cpus: 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
node 1 size: 65404 MB
node 1 free: 58642 MB
node 2 cpus: 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
node 2 size: 65404 MB
node 2 free: 61181 MB
node 3 cpus: 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
node 3 size: 65402 MB
node 3 free: 55592 MB
node distances:
node  0  1  2  3
0:  10  15  20  20
1:  15  10  20  20
2:  20  20  10  15
3:  20  20  15  10
```

```
[root@localhost test]# numastat

```

	node0	node1	node2	node3
numa_hit	68641597	59339134	50159172	55076006
numa_miss	1	1005288	188567	2132
numa_foreign	1171587	2133	0	22268
interleave_hit	1476	1463	1477	1410
local_node	68640960	59318730	50157010	55073766
other_node	738	1019692	180729	4372

- NUMA (Non-Uniform Memory Access) 是一种分布式存储器访问方式。
- 在多核处理器并行中广泛应用的今天，当业务需要处理大量并发任务时，可参考调优选项进行配置。
- 从numactl执行结果可以看到，示例服务器共划分为4个NUMA节点。每个节点包含16个CPU core，每个节点的内存大小约为64GB。
- 同时，该命令还给出了不同节点间的距离，距离越远，跨NUMA内存访问的延时越大。应用程序运行时应减少跨NUMA访问内存。
- 可以通过numastat命令观察各个NUMA节点的状态。
- numa_hit表示节点内CPU核访问本地内存的次数。
- numa_miss表示节点内核访问其他节点内存的次数。跨节点的内存访问会存在高延迟从而降低性能，因此，numa_miss的值应当越低越好，如果过高，则应当考虑绑核。

通用优化参数

优化项	解释	默认值	生效范围	鲲鹏920
优化应用程序的NUMA配置	在NUMA架构下，CPU core访问临近的内存时访问延迟更低。将应用程序绑在一个NUMA节点，可减少因访问远端内存带来的性能下降。	默认不绑定核	立即生效	具备
修改CPU预取开关	内存预取在数据集中场景下可以提前将要访问的数据读到CPU cache 中，提升性能；若数据不集中，导致预取命中率低，则浪费内存带宽。	on	重启生效	具备
调整定时器机制	nohz机制可减少不必要的时钟中断，减少CPU调度开销。	不同OS默认配置不同 Euler: nohz=off	重启生效	具备
调整内存的页大小	内存的页大小越大，TLB中每行管理的内存越多，TLB命中率就越高，从而减少内存访问次数。	不同OS默认配置不同： 4KB或64KB	重新编译内核、更新内核后生效	具备
优化应用程序的线程并发数	适当调整应用的线程并发数，使得充分利用多核能力和资源争抢之间达到平衡。	由应用本身决定	立即生效或重启生效（由应用决定）	具备



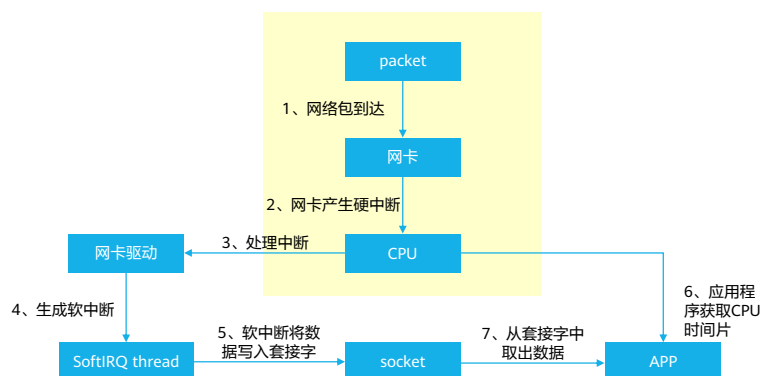
- 面向计算场景调优的通用优化参数，重点要说明的是在鲲鹏多核处理器场景下的NUMA和内存页大小的选择。
- 只有在了解NUMA工作原理和内存页大小调整后的系统工作方式后，才能进行测试调优，不建议非专业人士调整系统预设的配置参数。

目录

1. 软件性能测试
- 2. 鲲鹏通用性能调优方法**
 - CPU和内存子系统性能调优
 - **网络子系统性能调优**
 - 磁盘IO子系统调优
 - 应用程序调优
3. 鲲鹏性能分析工具

网络子系统调优概述

- 网络子系统处理过程，围绕优化网卡性能和利用网卡分担CPU的压力来提升性能。



- 在高并发的业务场景下，推荐使用两块网卡，减少跨片内存访问的次数。将两块网卡分别绑定在服务器的不同CPU上，每个CPU只处理对应的网卡数据。
- 高并发场景还可以为网卡选择x16的PCIE卡。
- 高并发场景是指：在同时或极短时间内，有大量的请求到达服务端，每个请求都需要服务端耗费资源进行处理，并做出相应的反馈。

常用性能监测工具 - ethtool (1)

- ethtool是一个Linux下功能强大的网络管理工具，目前几乎所有的网卡驱动程序都有对ethtool的支持，可以用于网卡状态/驱动版本信息查询、收发数据信息查询及能力配置以及网卡工作模式/链路速度等查询配置。
- 命令格式：ethtool [参数]
 - ethX 查询ethx网口基本设置，其中x是对应网卡的编号，如eth0、eth1等。
 - -k 查询网卡的Offload信息。
 - -K 修改网卡的Offload信息。
 - -c 查询网卡聚合信息。
 - -C 修改网卡聚合信息。

常用性能监测工具 - ethtool (2)

`ethtool -s eth0 speed 100 duplex full autoneg off` 修改网关速率、双工和协商模式

```
[root@ecs-1911 ~]# ethtool eth0
Settings for eth0:
    Supported ports: [ ]
    Supported link modes:   Not reported
    Supported pause frame use: No
    Supports auto-negotiation: No
    Supported FEC modes: Not reported
    Advertised link modes:  Not reported
    Advertised pause frame use: No
    Advertised auto-negotiation: No
    Advertised FEC modes: Not reported
    Speed: Unknown!
    Duplex: Unknown! (255)
    Port: Other
    PHYAD: 0
    Transceiver: internal
    Auto-negotiation: off
    Link detected: yes
```

```
[root@ecs-1911 ~]# ethtool -s eth0 speed 100 duplex full autoneg
[root@ecs-1911 ~]# ethtool eth0
Settings for eth0:
    Supported ports: [ ]
    Supported link modes:   Not reported
    Supported pause frame use: No
    Supports auto-negotiation: No
    Supported FEC modes: Not reported
    Advertised link modes:  Not reported
    Advertised pause frame use: No
    Advertised auto-negotiation: No
    Advertised FEC modes: Not reported
    Speed: 100Mb/s
    Duplex: Full
    Port: Other
    PHYAD: 0
    Transceiver: internal
    Auto-negotiation: off
    Link detected: yes
```

- 命令测试环境为云主机。

通用调优参数

优化项	解释	默认值	生效范围	鲲鹏920
调整TLP（Transaction Layer Packet）的最大有效负载	调整PCIE总线每次数据传输的最大值	128B	重启生效	具备
设置网卡队列数	调整网卡队列数量	不同操作系统&不同网卡	立即生效	具备
将每个网卡中断分别绑定到距离最近的核上	减少跨NUMA访问内存	Irqlbalance	立即生效	具备
聚合中断	调整合适的参数以减少中断处理次数	不同操作系统&不同网卡	立即生效	具备
开启TCP分段Offload（卸载）	将TCP的分片处理交给网卡处理	关闭	立即生效	具备



- 网络场景通用调优参数，不建议非专业人士调整系统预设的配置参数。
- TLP（Transaction Layer Packet）是网络传输中第三层事务层封装的数据包，TLP的有效负载决定数据传输的最大值。
- 网卡多队列技术是指将各个队列通过中断绑定到不同的核上，从而解决网络I/O带宽升高时单核CPU的处理瓶颈，提升网络PPS和带宽性能。经过测试，在相同的网络PPS和网络带宽的条件下，与1个队列相比，2个队列最多可提升性能达50%到100%，4个队列的性能提升更大。
- TCP分段：tcp会将应用层交付下来的数据分为tcp认为最适合发送的数据块，单位为字节，如果tcp接收到的数据包超过mss就会进行分段。

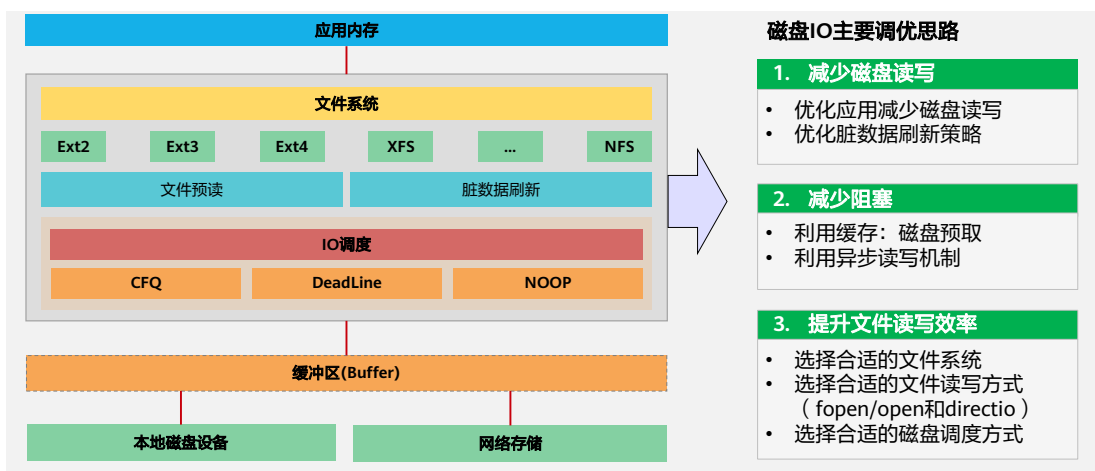
目录

1. 软件性能测试
2. **鲲鹏通用性能调优方法**
 - CPU和内存子系统性能调优
 - 网络子系统性能调优
 - **磁盘IO子系统调优**
 - 应用程序调优
3. 鲲鹏性能分析工具

磁盘IO子系统调优概述

- CPU的Cache、内存和磁盘之间的访问速度差异很大，当CPU计算所需要的数据并没有及时加载到内存或Cache中时，CPU将会浪费很多时间等待磁盘的读取。
- 计算机系统通过cache、RAM、固态硬盘、磁盘等多级存储结构，并配合多种调度算法，来消除或缓解这种速度不对等的影响。
- 缓存空间总是有限的，可以利用局部性原理，尽可能地将热点数据提前从磁盘中读取出来，降低CPU等待磁盘的时间浪费。因此部分优化手段其实是围绕着如何更充分地利用Cache获得更好的IO性能。

磁盘IO子系统调优策略



- 持久化数据通常存放在本地磁盘设备或远程网络存储，服务器或计算单元在调用数据的过程中需要经过多个环节，其中包括：
 - 缓冲区(Buffer)。
 - 文件系统 (Filesystem) 。
 - 应用内存 (Memory) 。
- IO子系统的优化，可以上3个角度出发，需根据数据的特征，有差异的设计数据访问策略，从而提高系统的访问效率，包括：
 - 减少磁盘读写。
 - 减少阻塞。
 - 提升文件读写效率。

常用性能检测工具 - iostat

- iostat是调查磁盘IO问题使用最频繁的工具。它汇总了所有在线磁盘统计信息，为负载特征归纳，使用率和饱和度提供了指标。它可以由任何用户执行，统计信息直接来源于内核，因此这个工具的开销基本可以忽略不计。
- 命令格式：直接使用命令+参数，例如

```
iostat -d -k -x 1 100
```

Device:	rrqm/s	wrqm/s	r/s	w/s	rkB/s	wkB/s	avgrq-sz	avgqu-sz	await	svctm	%util
sda	0.02	7.25	0.04	1.90	0.74	35.47	37.15	0.04	19.13	5.58	1.09
dm-0	0.00	0.00	0.04	3.05	0.28	12.18	8.07	0.65	209.01	1.11	0.34
dm-1	0.00	0.00	0.02	5.82	0.46	23.26	8.13	0.43	74.33	1.30	0.76
dm-2	0.00	0.00	0.00	0.01	0.00	0.02	8.00	0.00	5.41	3.28	0.00

- 重要参数详解：
- rrqm/s和wrqm/s，每秒合并后的读或写的次数（合并请求后，可以增加对磁盘的批处理，对HDD还可以减少寻址时间）。如果值在统计周期内为非零，也可以看出数据的读或写操作是连续的，反之则是随机的。
- 如果%util接近100%（即使用率为百分百），说明产生的I/O请求太多，I/O系统已经满负荷，相应的await也会增加，CPU的wait时间百分比也会增加（通过TOP命令查看），这时很明显就是磁盘成了瓶颈，拖累整个系统。这时可以考虑更换更高性能的磁盘，或优化软件以减少对磁盘的依赖。
- await（读写请求的平均等待时长）需要结合svctm参考。svctm和磁盘性能直接有关，它是磁盘内部处理的时长。await的大小一般取决于svctm以及I/O队列的长度和。svctm一般会小于await，如果svctm比较接近await，说明I/O几乎没有等待时间（处理时间也会被算作等待的一部分时间）；如果wait大于svctm，差的过高的话一定是磁盘本身IO的问题；这时可以考虑更换更快的磁盘，或优化应用。
- 队列长度（avgqu-sz）也可作为衡量系统I/O负荷的指标，但是要多统计一段时间后查看，因为有时候只是一个峰值过高。

通用调优参数

优化项	解释	默认值	生效范围	鲲鹏920
脏数据缓存到期时间	调整脏数据缓存到期时间，分散磁盘的压力	3000（单位为1/100秒）	立即生效	具备
脏页面占用总内存最大的比例	调整脏页面占用总内存最大的比例（以memfree+Cached-Mapped为基准），增加PageCache命中率	10%	立即生效	具备
脏页面缓存占用总内存最大的比例	调整脏页面占用总内存最大的比例，避免磁盘写操作变为O_DIRECT同步，导致缓冲机制失效	40%	立即生效	具备
调整磁盘文件预读参数	根据局部性原理，在读取磁盘数据时，额外地多读一定量的数据缓存到内存	128KB	立即生效	具备
磁盘IO调度方式	根据业务处理数据的特点，选择合适的IO调度器	cfq	立即生效	具备
文件系统	选用性能更好的文件系统以及文件系统相关的选项	N/A	立即生效	具备

- 不建议非专业人士调整系统预设的配置参数。
- 脏数据（Dirty Read）是指源系统中的数据不在给定的范围内或对于实际业务毫无意义，或是数据格式非法，以及在源系统中存在不规范的编码和含糊的业务逻辑。
- 关于局部性原理：CPU访问存储器时，无论是存取指令还是存取数据，所访问的存储单元都趋于聚集在一个较小的连续区域中，下属三个子概念：
 1. 时间局部性：如果一个信息项正在被访问，那么在近期它很可能还会被再次访问。
 2. 空间局部性：在最近的将来将用到的信息很可能与现在正在使用的信息在空间地址上是临近的。
 3. 顺序局部性：在典型程序中，除转移类指令外，大部分指令是顺序进行的。顺序执行和非顺序执行的比例大致是5:1。此外，对大型数组访问也是顺序的。
- IO调度器（IO Scheduler）是操作系统用来决定块设备上IO操作提交顺序的方法。存在的目的有两个，一是提高IO吞吐量，二是降低IO响应时间。然而IO吞吐量和IO响应时间往往是矛盾的，为了尽量平衡这两者，IO调度器提供了多种调度算法来适应不同的IO请求场景。Linux 2.6内核提供的几种IO调度算法：
 1. NOOP(No Operation)算法的全写为No Operation。该算法实现了最简单的FIFO队列，所有IO请求大致按照先后来后到的顺序进行操作。
 2. CFQ(Completely Fair Queuing)把进程当成了基本的调度单位，也就是说各个请求现在都是属于进程的，CFQ调度的是进程。
 3. DEADLINE在CFQ的基础上，解决了IO请求饿死的极端情况。
 4. ANTICIPATORY的在DEADLINE的基础上，为每个读IO都设置了6ms的等待时间窗口。如

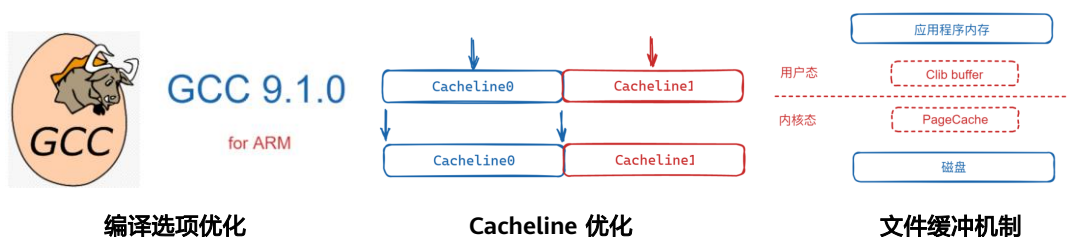
果在这6ms内OS收到了相邻位置的读IO请求，就可以立即满足。

目录

1. 软件性能测试
- 2. 鲲鹏通用性能调优方法**
 - CPU和内存子系统性能调优
 - 网络子系统性能调优
 - 磁盘IO子系统调优
 - 应用程序调优
3. 鲲鹏性能分析工具

应用程序调优

- 应用程序部署到鲲鹏服务器上以后，需要结合芯片和服务器特点优化代码性能，使硬件能力得到充分发挥。本节列举几个典型场景，涉及编译器配置、Cacheline优化、文件缓存机制、Neon指令加速。



- 前面介绍的都是面向系统配置的性能调优思路及通用优化参数。
- 从本节开始，将围绕上层应用程序，介绍在鲲鹏处理器场景下的几个常见调优场景或鲲鹏处理器特色的调优选项，分别是：
 - 基于鲲鹏GCC的编译选项优化。
 - 基于鲲鹏处理器指令定长、多寄存器的Cacheline 优化。
 - 针对特定应用场景的文件缓存机制。

编译选项优化

选项-mcmodel=medium、-mlarge-data-threshold=n

此选项使能了32bit之外的动态取址操作。在使用-mcmodel=medium时，对于符号size大于aarch64_data_threshold（默认值 $2^{16} = 65536$ ，客户可以使用-mlarge-data-threshold=n选项指定大符号的阈值为n）的符号使用**通过mov序列来获取PC值的offset，再与PC值相加**的方式实现64bit的相对PC寻址，在地址无关选项打开时，可以实现64bit相对PC寻址，获取GOT表入口，并且通过mov序列+LDR方式获取符号。

- 编译型语言（C/C++/Go/Fortan），从高级语言转化为机器可执行的二进制代码前，为提高代码的执行效率，需要进行多个层次优化，面向不同使用场景，对应有不同的编译选项。
- 扩充64位取值的编译选项，在鲲鹏处理器上可使能动态访问64位的取值范围，在科学计算场景，更高取值范围，能让程序的运行精度得到提高，此编译选项作为鲲鹏处理器的特性。

案例说明

如下图1，假设foovar的符号距离可将寻址指令的距离大于4GB，mcmodel=small使用ADRP+ADD指令进行符号拿取，而foovar在链接时计算距离的方式是如失败案例中的.bss+size方式，在链接时会报relocation truncated to fit错误。在此，使用图2中的mov序列+PC寻址方式寻址范围扩大至64位，解决由于地址溢出导致的报错。

图1

```
main:
    stp     x29, x30, [sp, -32]!
    add     x29, sp, 0
    adrp    x0, foovar
    add     x0, x0, :lo12:foovar
    str     x0, [x29, 24]
    ldr     x0, [x29, 24]
    ldr     w1, [x0]
    adrp    x0, .LC0
    add     x0, x0, :lo12:LC0
    mov     w2, w1
    ldr     x1, [x29, 24]
    bl      printf
    ldp     x29, x30, [sp], 32
    ret
```

图2

```
main:
    stp     x29, x30, [sp, -32]!
    add     x29, sp, 0
    movz    x0, :prel_g3:foovar
    movk    x0, :prel_g2_nc:foovar
    movk    x0, :prel_g1_nc:foovar
    movk    x0, :prel_g0_nc:foovar
    adrp    x8, .
    sub     x8, x8, 0x4
    add     x0, x0, x8
    str     x0, [x29, 24]
    ldr     x0, [x29, 24]
    ldr     w1, [x0]
    adrp    x0, .LC0
    add     x0, x0, :lo12:LC0
    mov     w2, w1
    ldr     x1, [x29, 24]
    bl      printf
    ldp     x29, x30, [sp], 32
    ret
```

- 图1是源码未使用-mcmodel=medium、-mlarge-data-threshold=n编译选项优化的汇编代码。
- 图2是使用编译选项后得到的汇编代码。
- 两者的区别在于取值时使用的汇编指令不同，图2汇编代码经编译选项优化，能成功完成64位的寻址空间，解决由于地址溢出导致的报错。

结果验证

hello.f90 用例（如右所示）

编译命令：

```
gfortran -mcmodel=medium hello.f90 -o hello.out -mlarge-data-threshold=1
```

```
[root@XA320V2-108 15:50:43 test]# gfortran -mcmodel=medium hello.f90 -o hello.out -mlarge-data-threshold=1
[root@XA320V2-108 15:50:49 test]# gfortran hello.f90 -o hello.out
/tmp/cc48pJBo.o: in function 'foo_1':
hello.f90:(.text+0x07c): relocation truncated to fit: R_AARCH64_ADR_PREL_PG_HI21 against symbol 'baz_' defined
in COMMON section in /tmp/cc48pJBo.o
hello.f90:(.text+0x8B8): relocation truncated to fit: R_AARCH64_ADR_PREL_PG_HI21 against symbol '.bss'
hello.f90:(.text+0xa48): relocation truncated to fit: R_AARCH64_ADR_PREL_PG_HI21 against symbol 'baz_' defined
in COMMON section in /tmp/cc48pJBo.o
/tmp/cc48pJBo.o: in function 'MAIN_1':
hello.f90:(.text+0xc44): relocation truncated to fit: R_AARCH64_ADR_PREL_PG_HI21 against symbol 'baz_' defined
in COMMON section in /tmp/cc48pJBo.o
hello.f90:(.text+0x108): relocation truncated to fit: R_AARCH64_ADR_PREL_PG_HI21 against symbol 'baz_' defined
in COMMON section in /tmp/cc48pJBo.o
collect2: error: ld returned 1 exit status
```

可见如果不加-mcmodel=medium，bar1符号就无法寻址，会报错。

此例中有size=85899*1000*10的大符号bar1，其阈值大于-mlarge-data-threshold=1选项指定的阈值，因此实现了64bit的相对PC寻址。

```
program main
common/baz/a, b, c
real a, b, c
b = 0.0
call foo()
print*, b
end

subroutine foo()
common/baz/a, b, c
real a, b, c

integer, parameter :: nx = 85899
integer, parameter :: ny = 1000
integer, parameter :: nz = 10
integer, parameter :: nf = 2000
real :: bar1(nf, nx*ny*nz)
real :: bar1(nf, nx*ny*nz)
!real, allocatable, dimension(:, :) :: bar
!allocate(bar(nf, nx*ny*nz))

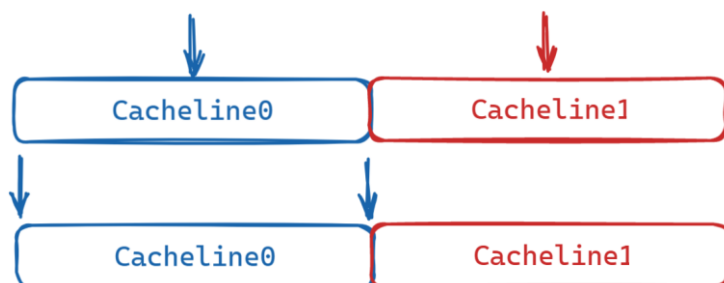
bar = 1.0
b = bar(12, 320*138*420)
b = bar1(12, 321*138*420)

return
end
```

- 左图是测试案例的执行过程，右图是测试案例hello.f90源码，f90是Fortan语言文件的后缀。

Cacheline优化

- CPU标识Cache中的数据是否为有效数据不是以内存位宽为单位，而是以Cacheline为单位。这个机制可能会导致伪共享（false sharing）现象，从而使得CPU的Cache命中率变低。出现伪共享的常见原因是高频访问的数据未按照Cacheline大小对齐。



- Cacheline 对齐是鲲鹏处理器在内存使用效率上做出的改进，配合openEuler操作系统新内核的设计与使用，在虚拟化/java内存消耗性应用场景，能优化系统程序的执行效率。
- 大部分处理器中Cache到主存是以Cacheline（一般为64Byte，也有地方称Cache块，本文统一使用Cacheline）传输的，CPU从内存加载数据是一次一个Cacheline，CPU往内存写数据也是一次一个Cacheline。假设处理器首次访问数据对象A，其大小刚好为64Byte，如果数据对象A首地址并没有进行对齐，即数据对象A占用两个不同Cacheline的一部分，此时处理器访问该数据对象时需要两次内存访问，效率低。但是如果数据对象A进行了内存对齐，即刚好在一个Cacheline中，那么处理器访问该数据时只需要一次内存访问，效率会高很多。编译器可以通过合理安排数据对象，避免不必要地将它们跨越在多个Cacheline中，尽量使得同一对象集中在一个Cache中，进而有效地使用Cache来提高程序的性能。通过顺序分配对象，即如果下一个对象不能放入当前Cacheline的剩余部分，则跳过这些剩余的部分，从下一个Cacheline的开始处分配对象，或者将大小（size）相同的对象分配在同一个存储区，所有对象都对齐在size的倍数边界上等方式达到上述目的。

案例说明

- Chacha20 加密算法每次加密64byte字节数据，没有考虑测试数据Cacheline对齐，生成随机测试数组的代码如下：

```
char *t_make_char(unsigned int len)
{
    // 最大4294M，即4.2G，不做入参判断
    char *arr = (char *)malloc(len);
    if (arr == NULL) {
        printf("%s make char malloc
failed!\n", alarm_string);
        return NULL;
    }
    t_srand();
    for (unsigned int i = 0; i < len; i++) {
        arr[i] = rand() & 0xff;
    }
    return arr;
}
```

Cacheline 不对齐：

```
[root@hadoop-master chacha_c]# ./chachac 10000000
C api test: 0x40003c808010
time used: 35792 us
```

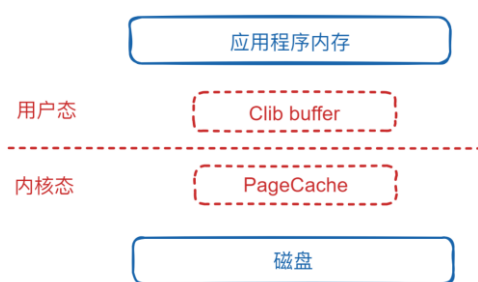
Cacheline 对齐后，用时减少30%

```
[root@hadoop-master chacha_c]# ./chachac 10000000
C api test: 0x40002fee4040
time used: 25674 us
```

- 左图是测试案例源码，右图比较了Cacheline优化前后的执行效率。
- 案例演示的是代码层面的改写，需要底层处理器支持这样的功能，才能做到效率优化，即只有在鲲鹏处理器上，才能得到如上效果。
- Cacheline对齐，需要借助编译器的特定编译选项，在深度适配鲲鹏处理器的毕昇编译器上，有针对Cacheline自动对齐的优化。

文件缓存机制选择

- 内存访问速度要高于磁盘，应用程序在读写磁盘时，通常会经过一些缓存，以减少对磁盘的直接访问。



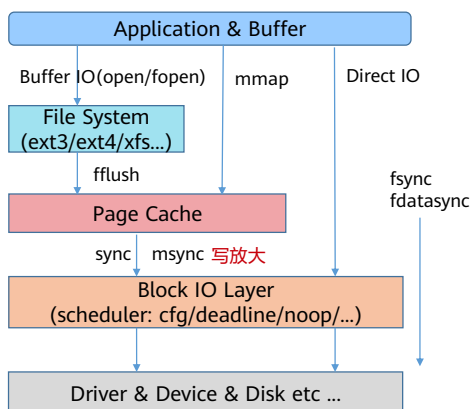
Clib buffer：用户态的一种数据缓冲机制，开启时，数据从应用程序buffer拷贝至clib buffer后，不会立即将数据同步到内核，而是缓冲到一定规模或者主动触发，才会同步到内核；查询数据时，优先从clib buffer查询数据，从而减少用户态和内核态的切换。

PageCache：内核态的一种文件缓存机制，用户在读写文件时，先操作PageCache，内核根据调度机制或者被应用程序主动触发时，会将数据同步到磁盘，从而减少磁盘访问。

- 存储层优化需要着重考虑文件系统的运行机制，数据在文件系统的存储过程有多种缓存机制可供选择，需要根据数据存储过程的实际情况，选择适合的文件缓存机制。

场景选择

- 应用程序根据自己的业务特点选择合适的文件读写方式。



内存拷贝次数: $\text{fopen} > \text{open} > \text{direct io} \approx \text{mmap}$

系统调用次数: $\text{open} = \text{direct io} > \text{fopen} > \text{mmap}$

fread/fwrite函数使用了clib buffer缓存机制, read/write没有, 因此fread/fwrite比read/write多一层内存拷贝, 即从应用程序buffer至clib buffer的拷贝, 但是fread/fwrite比read/write有更少的系统调用。对于每次读写字节数较大的操作, 内存拷贝比系统调用占用更多资源, 可以使用read/write来减少内存拷贝; 对于每次读写字节数较少的操作, 系统调用比内存拷贝占用更多资源, 建议使用fread/fwrite来减少系统调用次数。

O_DIRECT模式没有使用PageCache, 少了一层内存拷贝, 但是因为没有缓冲导致每次都是从磁盘里面读取数据。适用场景为: 应用程序有自己的缓冲机制; 数据读写一次后, 后面不再从磁盘读这个数据。

- 重点解读数据从应用到落盘的几个过程, 过程中在与不同存储层打交道时, 会使用不同的写入/同步机制。
- 不同业务场景在内存拷贝和系统调用上存在差异, 要针对性的指定不同机制, 来优化文件系统的整体效率。

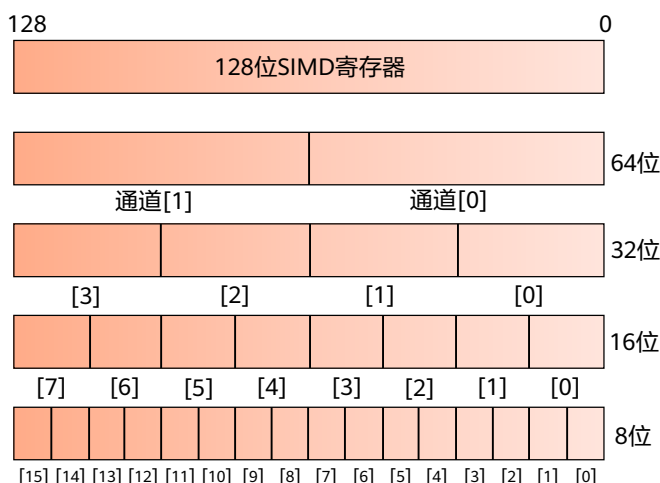
Neon - 指令加速

原理知识:

- SIMD(Single Instruction Multi Data) 允许一条指令产生多个可以并行执行的操作;
- Kunpeng 920支持Armv8.2A指令集, 有32个128bits的向量寄存器。

使用方法:

- 编译器自动向量化优化, 加入-ftree-vectorize;
- 使用Neon Intrinsics改写循环体;
- 使用OpenMP的#pragma omp simd。



- SIMD, 即 single instruction multiple data, 单指令流多数据流, 也就是说一次运算指令可以执行多个数据流, 从而提高程序的运算速度, 实质是通过 数据并行 来提高执行效率。
- ARM NEON 是 ARM 平台下的 SIMD 指令集, 利用好这些指令可以使程序获得很大的速度提升。不过对很多人来说, 直接利用汇编指令优化代码难度较大, 这时就可以利用 ARM NEON intrinsic 指令, 它是底层汇编指令的封装, 不需要用户考虑底层寄存器的分配, 但同时又可以达到原始汇编指令的性能。

案例说明

- 使用Neon指令改写后的循环体，执行效率更高。

正常循环体

```
#include <stdio.h>
#include <math.h>
#include <string.h>

void vadd(int length, float* src)
{
    for (int i = 0; i < length; i++) {
        src[i] += src[i] * src[i];
    }
}

int main(int argc, char **argv)
{
    float *src = (float *)malloc(100000000 * sizeof(float));
    for (int i = 0; i < 100000000; i++) {
        src[i] = rand() * 100;
    }
    vadd(100000000, src);
}
```

Performance counter stats for './vadd':

2,299.48 msec	task-clock:u	#	0.999 CPUs utilized
0	context-switches:u	#	0.000 K/sec
0	cpu-migrations:u	#	0.000 K/sec
359	page-faults:u	#	0.169 K/sec
5,761,850,685	cycles:u	#	2.506 GHz
7,690,540,540	instructions:u	#	1.33 insn per cycle
<not supported>	branches:u		
3,334,224	branch-misses:u		

2.301245250 seconds time elapsed

2.244850000 seconds user
0.047914000 seconds sys

改写后循环体

```
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <arm_neon.h>

static inline void vadd(int length, float* src)
{
    float32x4_t vsrc;
    float32x4_t vtemp;
    for (int i = 0; i < length / 4; i++) {
        vsrc = vldq_f32(src + i * 4);
        vtemp = vmlaq_f32(vsrc, vsrc, vsrc);
        vstq_f32(src + i * 4, vtemp);
    }
}

int main(int argc, char **argv)
{
    float *src = (float *)malloc(100000000 * sizeof(float));
    for (int i = 0; i < 100000000; i++) {
        src[i] = rand() * 100;
    }
    vadd(100000000, src);
}
```

Performance counter stats for './vadd_inline':

2,130.46 msec	task-clock:u	#	1.000 CPUs utilized
0	context-switches:u	#	0.000 K/sec
0	cpu-migrations:u	#	0.000 K/sec
510	page-faults:u	#	0.239 K/sec
5,366,440,557	cycles:u	#	2.519 GHz
7,690,540,553	instructions:u	#	1.43 insn per cycle
<not supported>	branches:u		
3,232,127	branch-misses:u		

2.131350040 seconds time elapsed

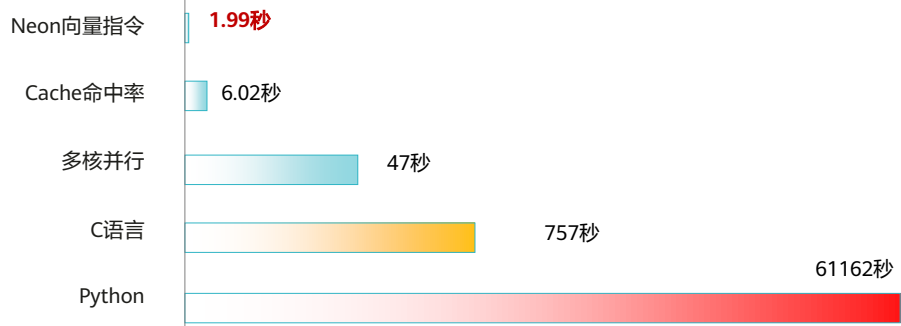
2.049101000 seconds user
0.080029000 seconds sys



- 左图是未调用neon函数实现的代码结构，因循环体中有大量数据计算，使用常规结构来完成计算，用时偏高，总用时2.30s。
- 右图是调用neon函数实现的代码结构，将常规循环体结构改写为向量计算，从而加速计算过程的数据调用和存取，总用时2.13s，相比效率提升7.4%。

Neon指令加速

4800*4800 矩阵乘法加速效果实测结果



- 在矩阵乘法等计算密集型的场景，多核处理器的Neon指令具备明显执行效率上的优势。

目录

1. 软件性能测试
2. 鲲鹏通用性能调优方法
- 3. 鲲鹏性能分析工具**
 - 工具介绍
 - 调优助手
 - 系统性能分析
 - Java性能分析
 - 系统诊断

鲲鹏性能分析工具介绍

- 挑战: 软件运行时遇到性能瓶颈，传统手动调优方式，性能分析和调优手段单一，定位困难，对人员技能要求高，效率和准确率低下。
- 方案: 鲲鹏提供**系统性能分析工具**和**JAVA性能分析工具**，能够在软件运行状态下，自动采集系统数据，分析出系统性能指标，定位到瓶颈点及热点函数，给出调优建议，从而达到软件和鲲鹏平台融合的最佳性能。

鲲鹏性能分析工具

性能分析工具支持鲲鹏平台上的系统性能分析、Java性能分析和系统诊断，提供系统全景及常见应用场景下的性能采集和分析功能，并基于调优专家系统给出优化建议。同时提供调优助手，指导用户快速调优系统性能。点击查看 [鲲鹏性能分析工具最新动态](#)，性能分析工具包含以下四个子工具：



调优助手

调优助手通过系统化组织和分析性能指标、热点函数、系统配置等信息，形成系统资源消耗链条，引导用户根据优化路径分析性能瓶颈，并针对每条优化路径给出优化建议和操作指导，以此实现快速调优



系统性能分析

基于鲲鹏微架构、硬件、操作系统、应用进程/线程、函数等各层次的性能数据，针对多个专项分析系统性能状况，识别系统性能瓶颈，并给出针对性的多维度优化建议



Java性能分析

针对基于鲲鹏的服务器上运行的Java程序的性能分析和优化工具，能图形化显示Java程序的堆、线程、锁、垃圾回收等信息，收集热点函数、定位程序瓶颈点，帮助用户采取针对性优化



系统诊断

能够快速定位内存、网络、存储等部件的异常，提供内存泄漏诊断、内存消耗信息分析展示、OOM诊断能力、网络IO诊断能力、存储IO诊断能力，帮助用户识别出源代码中内存使用的问题点

- 鲲鹏性能分析工具是鲲鹏平台系统分析和性能调优的集成工具，官方网站：
- <https://www.hikunpeng.com/zh/developer/devkit/hyper-tuner>。

部署与使用环境

- 基于鲲鹏920的服务器
- 虚拟机
- 容器



- 部署可以基于本地服务器，也可基于云化基础设施，比如虚拟机、容器。

性能分析工具两种使用方式



打开本地PC机的浏览器，
在地址栏输入 **https://部署服务器的公网IP:端口号**

web

插件

性能分析工具

ECS

在VSCode中安装插件，
并配置访问服务器IP

- Web和插件端两种访问，本质没有什么区别，都需要安装工具端，为开发人员方便考虑，在Vscode中集成了相应的插件，本质也是调用服务端的能力。
- 在浏览器地址栏输入https://部署服务器的弹性公网:端口号。
- （例如：https://10.116.239.242:8086）默认用户名为tunadmin，首次登陆需设定密码。

目录

1. 软件性能测试
2. 鲲鹏通用性能调优方法
- 3. 鲲鹏性能分析工具**
 - 工具介绍
 - 调优助手
 - 系统性能分析
 - Java性能分析
 - 系统诊断

调优助手简介

- 性能调优分为硬件调优和软件调优两部分，调优前需了解硬件（服务器/虚拟机）基本情况，明确性能瓶颈后，再进一步系统分析调优项。
- 调优助手，能系统化组织性能指标，引导用户分析性能瓶颈，实现快速调优。



- 调优助手是针对基于鲲鹏的服务器的调优工具，能系统化组织性能指标，引导用户分析性能瓶颈，实现快速调优。
- 调优助手的主要价值点：在软件调优之前，帮助工程师了解系统硬件基本情况，如CPU架构属性、内存使用、存储状态、网络连接、OS配置等偏硬件层面的配置。类似于物理服务器的IBMC界面。

功能特性

功能	描述
系统配置分析	通过任务分析，快速了解主机操作系统、内存、磁盘占用、NUMA节点等常用调优对象 硬件信息 ，并根据业务类型，给出基于SMMU、内存脏数据参数等调优建议。
热点函数分析	通过任务分析，快速了解主机造成资源消耗项的热点函数，点击热点函数可追踪函数调用栈，类似perf，给出加速库函数、常用热点函数等调优建议。
系统性能分析	通过任务分析，快速了解主机CPU、内存、存储、网络四大系统资源占用情况，方便定位资源瓶颈，给出网络IO、调度开销、内存SWAP等调优建议。
进程/线程分析	通过任务分析，快速了解主机所有进程的CPU、内存、磁盘IO资源消耗占比，给出SWAP优化、热点函数分析等调优建议。

- 系统硬件侧的调优有以上几个方面，不建议非专业人士调整系统预设的配置参数。
- SMMU (system mmu),是I/O device与总线之间的地址转换桥，它与mmu的功能类似，可以实现地址转换，内存属性转换，权限检查等功能，SMMU可以为ARM架构下实现虚拟化扩展提供支持。
- Non-Uniform Memory Access(NUMA)，CPU在访问靠近它的本地内存的时候就比较快，访问其他CPU的内存或者全局内存的时候就比较慢。
- 热点函数：程序在操作系统运行过程中，被频繁调用或占用较多物理资源的函数。
- SWAP：在物理内存（RAM）被充满时被使用。如果系统需要更多的内存资源，而物理内存已经充满，内存中不活跃的页就会被移到交换空间去。虽然交换空间可以为带有少量内存的机器提供帮助，但是这种方法不应该被当做是对内存的取代。交换空间位于硬盘驱动器上，它比进入物理内存要慢。

应用场景 - 调优助手

- 细致调优前，可基于调优助手快速了解鲲鹏主机基本情况，方便定位资源瓶颈点，为后续细致调优做准备。
- 调优助手可基于**系统分析**和**应用分析**两种对象，来了解主机基本情况。
- 系统分析：采集整个服务器的数据，无需关注系统中有哪些类型的应用在运行，采样时长由配置参数控制，适用于多业务混合运行和有子进程的场景。
- 应用分析：采集指定应用或进程的数据进行分析。
 - Launch Application：采集的同时启动Application，采样时长受Application的运行时长控制（适用于Application运行时长较短的场景）。该场景分析涵盖Application及其子进程。
 - Attach to Process：采集时关联服务器中已存在的PID，需要配置采样时长参数（适用于应用运行时间较长的场景）。该场景分析仅涵盖参数中的PID，不涵盖PID对应的子进程。

创建分析任务

- 调优助手页签，点击新建分析任务。



- 可通过鲲鹏开发者官网实验环境，熟悉工具使用方法，链接：
- <https://www.hikunpeng.com/zh/learn/experiments/detail/T221121003>。
- 采样时长：设置记录的时间。默认为15秒，取值范围2~300秒。【主参数】。
- 采集文件大小：设置采集文件的大小。默认为100，取值范围1~100。
- 根据实际情况选择业务类型。可选类型包括：CPU密集型、网络IO密集型和存储IO密集型。可以选择1~3个业务类型。选中后，选项的文字颜色变为蓝色。默认三个选项均被选中。

查看分析结果 - 系统配置分析

- 给出常见基于系统配置的调优建议（如物理机建议关闭SMMU，内存脏数据参数设置等）。
- 系统配置详细数据中，可查询主机 CPU、内存、存储、网络、OS配置的硬件容量和参数。



- 左图点击1，可显示系统配置详细数据，如胶片右图所示；点击2，可显示调优项的具体操作命令（胶片中未展示）。
- 细节参数，详见：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpengfeat_06_0141.html。

查看分析结果 - 热点函数分析

- 给出主机当前热点函数调优建议和通用加速库函数配置说明。
- 在热点函数详细数据中，可查看主机所有函数信息，点击具体函数可查看调用栈。



- 操作如系统配置分析中所述。
- 细节参数详见：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpengfeat_06_0142.html。

查看分析结果 - 系统性能分析

- 给出主机当前系统性能各指标，指标数值超过30%，默认被纳入待调优范围。
- 在系统性能详细数据中，可查看主机CPU、内存、存储、网络的资源利用率。



- 操作如系统配置分析中所述。
- CPU指标，可根据业务类型，切换不同指标。
- 细节参数详见：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpengfeat_06_0143.html。

查看分析结果 - 进程/线程性能分析

- 在进程/线程性能详细数据中可查询每个进程的在系统中的资源占用情况。

tuningAssistant_2022... × |

进程/线程性能详细数据

 ×

工程名称 project_20220926_131534 任务名称 tuningAssistant_20220926_131541 节点名称 节点IP

所有进程

PID/TID	CPU							Memory				
	%...	⑦	%...	⑦	%...	⑦	min...	ma...	VSZ...	RSS...	%M...	
▶ PID1	0.33		0.33		0.06		0.66	0.0	0.0	176832	17536	0.03
▶ PID2	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0
▶ PID3	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0
▶ PID4	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0
▶ PID6	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0
▶ PID8	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0
▶ PID9	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0
▶ PID10	0.0		0.0		0.27		0.0	0.0	0	0	0	0.0
▶ PID11	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0
▶ PID12	0.0		0.0		0.0		0.0	0.0	0	0	0	0.0



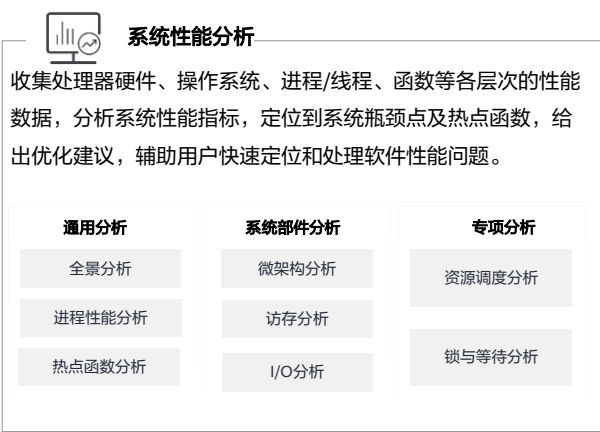
- 操作如系统配置分析中所述。
- 截图为测试案例，因系统中未运行业务软件，故整体资源占用均较低。
- 细节参数详见：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpengfeat_06_0144.html。

目录

1. 软件性能测试
2. 鲲鹏通用性能调优方法
- 3. 鲲鹏性能分析工具**
 - 工具介绍
 - 调优助手
 - 系统性能分析
 - Java性能分析
 - 系统诊断

系统性能分析工具简介

- 系统性能分析工具是华为鲲鹏性能分析工具的子工具。



- 系统性能分析基于鲲鹏微架构、硬件、操作系统、应用进程/线程、函数等各层次的性能数据，针对多场景分析系统性能状况，定位出系统性能瓶颈点及热点函数等问题，并给出针对性的多维度优化建议。

功能特性（1）

功能	描述
OpenMP/MPI分析	HPC分析通过采集系统的PMU事件并配合采集面向OpenMP和MPI应用的关键指标，帮助用户精准获得Parallel region及Barrier-to-Barrier的串行及并行时间，校准的2层微架构指标，指令分布及L3的利用率和内存带宽等信息。
全景分析	通过采集系统软硬件配置信息，以及系统CPU、内存、存储IO、网络IO资源的运行情况，获得对应的使用率、饱和度和错误次数等指标，以此识别系统性能瓶颈。针对部分系统指标项，根据当前已有的基准值和优化经验提供优化建议。针对大数据场景、数据库场景和分布式存储场景的硬件配置、系统配置和组件配置进行检查并显示不是最优的配置项，同时分析给出典型硬件配置及软件版本信息。
微架构分析	基于ARM PMU（Performance Monitor Unit）事件，获得指令在CPU流水线上的运行情况，可以帮助用户快速定位当前应用在CPU上的性能瓶颈，用户可以有针对性地修改自己的程序，以充分利用当前的硬件资源。
访存分析	基于CPU访问缓存和内存事件，分析访存过程中可能的性能瓶颈，给出造成这些性能问题的可能原因及优化建议。
I/O分析	分析存储IO性能。以存储块设备为分析对象，分析得出块设备的I/O操作次数、I/O数据大小、I/O队列深度、I/O操作时延等性能数据，并关联到造成这些I/O性能数据的具体I/O操作事件、进程/线程、调用栈、应用层I/O APIs等信息。根据I/O性能数据分析给出进一步优化建议。



- OpenMP：Open Multi-Processing 开源并行处理机制。用于共享内存并行系统的多处理器程序设计的一套指导性编译处理方案，常用于科学计算（HPC）场景。

功能特性（2）

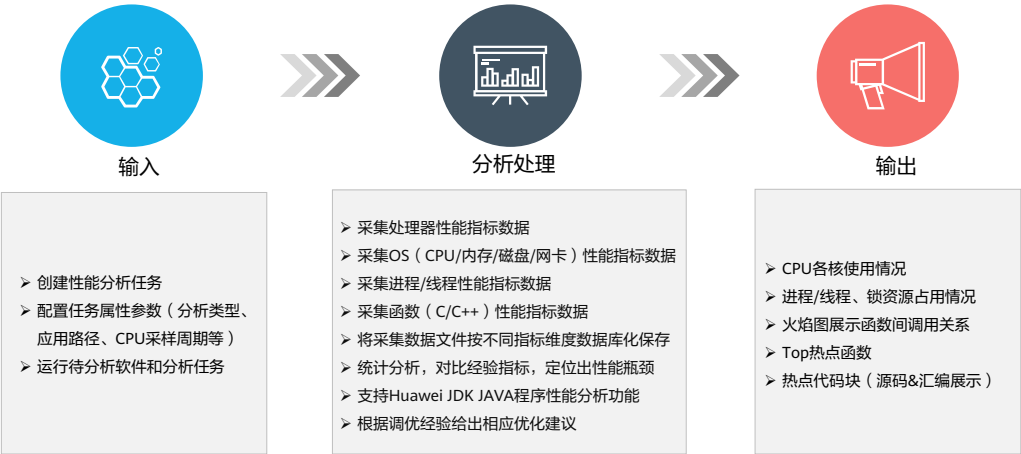
功能	描述
进程性能分析	采集进程/线程对CPU、内存、存储IO等资源的消耗情况，获得对应的使用率、饱和度、错误次数等指标，以此识别进程/线程性能瓶颈。针对部分指标项，根据当前已有的基准值和优化经验提供优化建议。针对单个进程，还支持分析它的系统调用情况。
资源调度分析	基于CPU调度事件分析系统资源调度情况，主要包括： 分析CPU核在各个时间点的运行状态，如：Idle、Running，以及各种状态的时长比例。 分析进程/线程在各个时间点的运行状态，如：Wait、Schedule和Running，以及各种状态的时长比例。 分析进程/线程切换情况，包括：切换次数、平均调度延迟时间、最小调度延迟时间和最大延迟时间点。 分析各个进程/线程在不同NUMA节点之间的切换次数。如果切换次数大于基准值，能给出绑核优化建议。
热点函数分析	分析C/C++程序代码，找出性能瓶颈点，获得对应的热点函数及其源码和汇编指令；支持通过火焰图展示函数的调用关系，给出优化路径。
锁与等待分析	分析glibc和开源软件（如MySQL、Open MP）的锁与等待函数（包括sleep、usleep、mutex、cond、spinlock、rwlock、semaphore等），关联到其归属的进程和调用点，并根据当前已有的优化经验给出优化建议。

- 火焰图：性能分析中常用到的工具，能较清晰的展示出当前系统热点函数和调用关系，帮助定位复杂系统的资源瓶颈。

应用场景

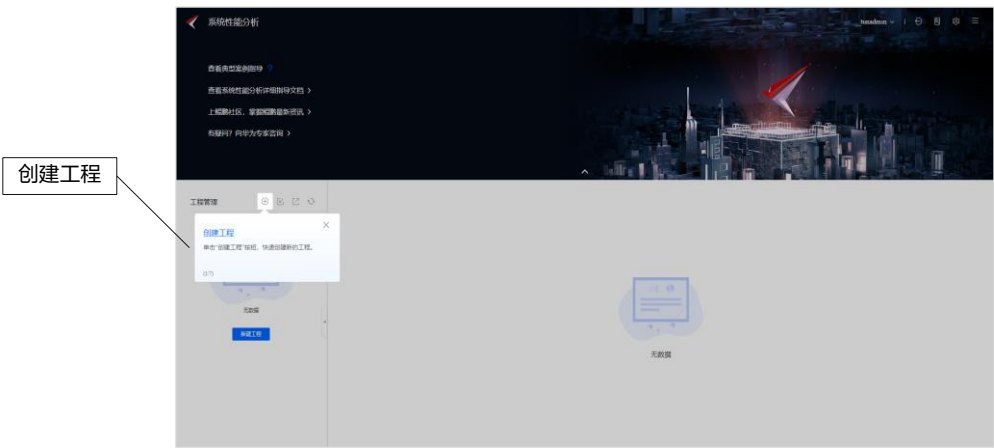
- 客户软件在基于鲲鹏的服务器上运行遇到性能问题时，可用系统性能分析来定位调优项。
- 系统性能分析工具将采集系统如下数据：
 - 系统软硬件配置和运行信息，例如：CPU类型、内存部署槽位、Kernel版本、内核参数、文件系统、系统运行日志参数等。
 - 系统的CPU、内存、存储IO、磁盘IO等性能指标；处理器PMU、SPE的性能数据。
 - 处理器访问Cache/内存的次数、带宽、吞吐率等。
 - 系统内核进行CPU资源调度、IO操作等数据。
 - 进程/线程的CPU、内存、存储IO、上下文切换、系统调用等数据。
 - 进程命令行信息，包括：进程名、进程参数。
 - 系统的热点函数及其调用栈；热点函数归属的程序/动态库（包含绝对路径）；热点函数的汇编指令和热点指令；热点函数所对应的源代码（需要用户自行提供）。

系统性能分析工具业务流程



- 可通过鲲鹏开发者官网实验环境，熟悉工具使用方法，链接：
- <https://www.hikunpeng.com/zh/learn/experiments/detail/T221121003>。

步骤1 全景分析任务 - 创建工程



步骤2 新建系统性能工程

- Step1: 新建工程名称
- Step2: 基于业务需要选择分析场景
选择HPC场景
- Step3: 管理节点

新建系统性能工程 ×

1

* 工程名称 project_20220927_113258

* 选择场景

2

通用场景 大数据 分布式存储 HPC 数据库

采集系统的PMU事件并结合OpenMP和MPI应用的关键指标，多维度数据关联呈现。

* 选择节点

3

选择位于MPI集群中的节点，最多支持10个不同集群同时采集。 管理节点

已选: 1 全部: 1

节点名称	节点状态
☒ [节点名称]	● 在线

确认 取消

- 管理节点，可添加多个节点信息，用于分析集群内的系统资源情况。
- HPC案例分析背景：风雷是流体力学领域重要国产软件（对标OpenFOAM），希望联合鲲鹏进行深入合作。在迁移调优后发现使用**MPICH**性能比**HMPI**性能高17%（快133s）。

案例：创建OpenMP/MPI分析任务

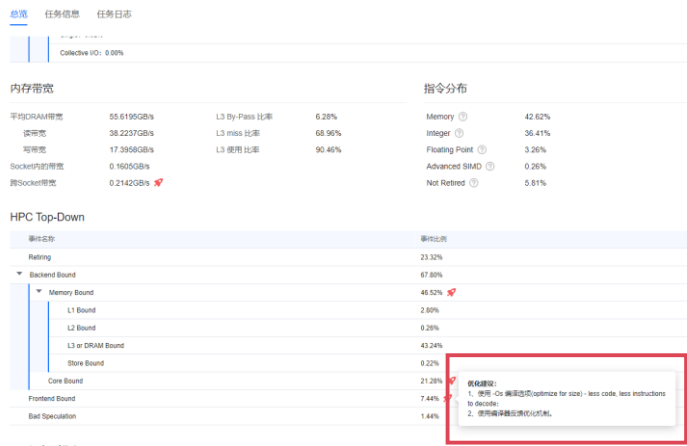
- Step1：分析对象-系统
- Step2：分析类型-OpenMP/MPI
- Step3：采集模式-OpenMP
- Step4：采样模式-Summary



- OpenMP/MPI分析只支持物理服务器上使用，且OpenMP/MPI任务中Top-Down和DRAM带宽数据需要操作系统内核4.19及以上或patched openEuler 4.14内核及以上。
- Ubuntu系统中OpenMP/MPI任务数据采集可能会失败，详情请参考Ubuntu系统OpenMP/MPI任务数据采集失败；其中Ubuntu 20.04.1暂不支持OpenMP/MPI任务。

步骤3 查看分析结果

- 分析结果显示为**后端依赖型**应用场景，该场景下程序指令的执行效率直接影响整体性能。
- 建议使用-Os编译选项优化指令数量，减少译码。



- Top-Down 是一个兼具通用型和可操作型的应用程序评价系统，旨在分析每一个cpu微指令对不同系统微资源的利用依赖度。

编译器优化选项

- GCC提供大量优化等级，用来对编译时间、目标文件大小、执行效率三个维度进行不同取舍和平衡
 - -O0，最少优化。（默认编译选项）（最大程度配合产生代码调试信息，可以在任何代码行打断点）
 - -O或-O1，有限优化。（编译时占用稍微多的时间和相当大的内存，减少代码生成尺寸、缩短执行时间）
 - -O2，高度优化。（在-O1的基础上，尝试更多的寄存器级的优化以及指令级的优化）（调试信息不友好，有可能会修改代码和函数调用执行流程，自动对函数进行内联）
 - -Os主要对指令尺寸进行优化。打开大部分O2优化中不会增加程序大小的优化选项，并对程序代码大小做更深层优化。

- 编译器的结构包括词法分析器(Lexical Analyzer)，语法分析器(Syntax Parser)，语言分析器(Semantic Analyzer)，中间代码生成器(Intermediate Code Generator)，代码优化器(CodeOptimizer)，符号表(Symbol Table)，错误处理器(Error Handler)。词法分析器直到中间代码生成器又称为前端(FrontEnd)，代码优化器到代码生成器又称后端(Back End)。
- 编译器的优化步骤在整个编译器中是最重要的，也是最难的。它重要是因为一个编译器的好坏主要就是看这个编译器优化的效果是否良好。如果一个编译器的优化效果很差，那么由该编译器编译出与用机器语言编写的程序对系统资源的开销是相当大的，而程序设计语言的设计者往往希望编译器能够编译出与用机器语言编写的程序效率相当的程序；它难是因为优化中的众多问题都没有定型的好算法。有些优化问题的求解甚至是不可计算的。现代系统结构中流水线，超标量以及多核处理器的出现无疑给编译器的设计实现者加重了任务。
- 普通优化一般都会经过以下几个过程：常量转换，将源文件中的常量变量及常量表达式都用其真实值来代替，这将可以节省一定的时间和空间；公共子表达式求值，将多次出现的子表达式预先求值，并存入临时变量，这样可以避免重复求值，但必须保证子表达式的值在任何地方都不会改变；冗余代码的删除，将那些并不会用到的代码删除；优化存储器，将频繁使用的临时变量放入寄存器中；表达式求值优化，改变表达式求值顺序，有时甚至能达到优化目的；函数过程内联展开，将自程序代码内嵌到调用代码中，这样可以避免函数调用的开销。普通优化还有很多，此处只是试举几例。

分析结果2 - 热点函数异常

- 分析结果显示：函数LUSGS内的Allreduce操作耗时异常。

MPI运行时指标

MPI Rank	function	Caller location	Wait Rate(%)	Data Size(bytes)	Avg Time(ms)	Call Count
83	MPI_Waitall	PHSPACE: Controller: Commun.....	20.31	0	699.1473	400
15	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	20.88	824568	249.1610	799
9	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	20.49	824568	244.4091	799
8	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	20.41	824568	243.5117	799
26	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	19.91	824568	237.4247	799
29	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	19.87	824568	236.9210	799
14	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	19.73	824568	235.4527	799
30	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	19.63	824568	234.0207	799
24	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	19.56	824568	233.2797	799
27	MPI_Allreduce	PHSPACE: Controller: LUSGS@l...	19.55	824568	233.1226	799

10 总条数: 4,097 1 2 3 4 5 ... 410 1 跳转



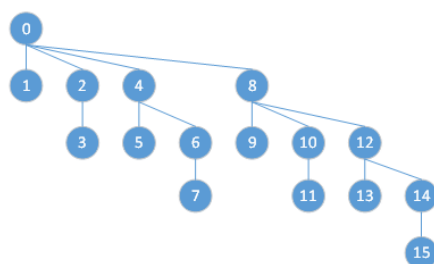
- LUSGS函数是非结构网格的求解方法，主要应用于科学计算场景。
- 分析结果中Allreduce操作的平均时间均在230ms以上，结果异常。

LUSGS 函数分析（1）

- 函数PHSPACE::Controller::LUSGS中，CommunicationDQ函数在每个进程中的执行时间相差很大（不同步），导致AllReduce操作时延异常大。
- CommunicationDQ函数有5个Step：
 - Step 0: Compressing data firstly.
 - Step 1: Communication
 - Step2: MPI waiting for Non-blocking communications
 - Step3: Translate the data container.
 - Step4: Free the buffers

LUSGS 函数分析（2）

- HMPI算法将AllReduce操作分为多个Step，每个Step都要等到上一个Step执行完成。
- 通过在每个Step间加上一个MPI_Barrier操作，可以使软件用
HMPI执行的时间减少132s。



Binomial tree over 16 process

Step 1: 0→8

Step 2: 0→4, 8→12

Step 3: 0→2, 4→6, 8→10, 12→14

Step 4: 0→1, 2→3, 4→5, 6→7, 10→11, 12→13, 14→15

目录

1. 软件性能测试
2. 鲲鹏通用性能调优方法
- 3. 鲲鹏性能分析工具**
 - 工具介绍
 - 调优助手
 - 系统性能分析
 - Java性能分析
 - 系统诊断

Java性能分析工具简介

- Java性能分析工具是华为鲲鹏性能分析工具的子工具。



JAVA性能分析

针对Java程序的性能分析和优化的工具，能图形化显示Java程序的堆、线程、锁、垃圾回收等信息，收集热点函数、定位程序瓶颈点，帮助用户采取针对性优化。

在线分析		采样分析	
总览： CPU使用率、Heap大小、GC活动、Thread数量、Class加载数量。		总览： CPU使用率、Heap使用、GC活动等。	
线程： 程序线程状态及锁分析、死锁检测。		线程： 线程状态及锁分析、阻塞对象和时间分析。	IO分析
内存： 程序所用堆积对象分析、堆内存分配/使用问题分析。	IO分析		
快照比对		内存： 程序所用堆积对象分析、应用潜在堆内存泄漏分析。	
上层应用Workload分析： 数据库连接池监控分析、HTTP请求分析、热点SQL分析。		方法分析： 热点函数CPU cycle占比及定位，火焰图查看热点函数及其调用栈。	

应用场景

- 客户Java应用软件在基于鲲鹏的服务器运行遇到性能问题时，可用Java性能分析来快速分析和定位。
- Java性能分析工具将采集如下数据：
 - Java进程运行环境信息，如：PID、JVM版本、JAVA版本、Main class、进程启动参数等。
 - Java进程的CPU活动、内存占用、类加载信息、线程运行状态信息、系统的存储容量等。
 - 可触发采集进程的堆转储信息，并获取堆转储中的类，实例，对象引用链等信息。
 - Java进程的文件IO、SocketIO操作、数据库操作、HTTP请求、SpringBoot运行信息等。
 - 对Java进程的方法、线程、内存、老年代对象、调用栈进行采样生成JFR采样文件。

功能特性

Java性能分析工具有如下两种方式：

- **在线分析**

在线分析包含对于目标JVM和Java程序的双重分析。包括Java虚拟机的内部状态，如Heap、GC活动、线程状态及上层Java程序的性能分析，如调用链分析、热点函数、锁分析、程序线程状态及对象生成分布等。通过agent的方式实时获取JVM运行数据，进行精确分析。主要功能包含：

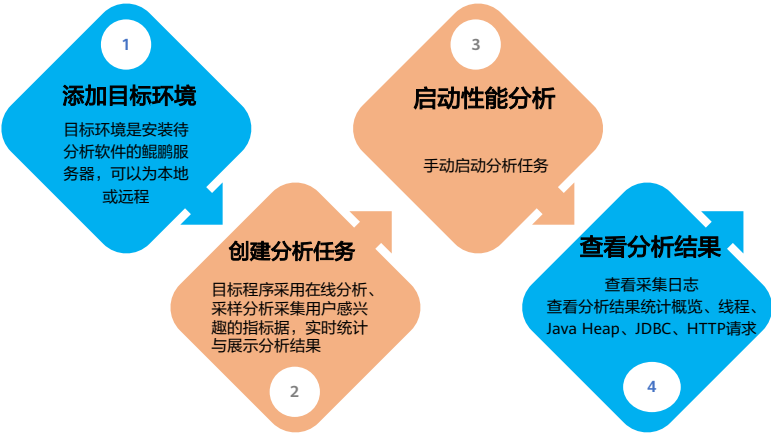
- 实时Java虚拟机系统状态显示。
- 热点分析。
- GC分析。

- **采样分析**

通过采样的方式，收集JVM的内部活动/性能事件，通过录制及回放的方式来进行离线分析。这种方式对系统的额外开销很小，对业务影响不大，适用于大型的Java程序。主要功能包括：

- Java虚拟机系统状态显示。
- Java进程/线程性能分析。
- 函数性能分析：函数性能分析主要通过收集系统性能事件，通过离线分析来定位热点函数、调用链、调用占比等相关功能。

Java性能分析工具使用流程



添加目标环境

- Step 1 登录Java性能分析工具Web界面，单击界面左侧“添加环境列表”；
- Step 2 打开添加“目标环境”窗口，并配置参数；
- Step 3 单机“确认”按钮，完成目标环境添加。

参数名称	参数说明
目标环境类型	可选“远程服务器”和“本地服务器”
服务器IP地址	输入待安装分析辅助软件的远程服务器的IP地址
端口	输入远程服务器的SSH服务端口号默认为22
用户名	输入登录远程服务器的用户名
密码	输入登录远程服务器的用户密码

添加目标环境

系统将通过SSH协议访问待部署分析辅助软件的服务器，请确认服务器信息正确无误。

目标环境类型

本地服务器

远程服务器

服务器IP地址:

端口:

22

用户名:

密码:

确认

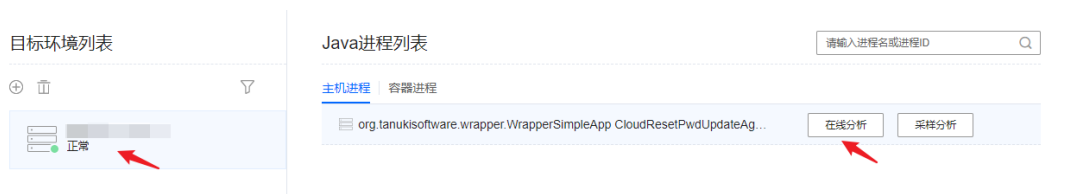
取消



- 目标环境：基于鲲鹏916的服务器、基于鲲鹏920的服务器。
- 服务端运行操作系统：openEuler 20.03（ LTS ）、CentOS 7.6、麒麟V10、Ubuntu 18.04.1等。

创建在线分析任务

- Step 1 在首页界面“目标环境列表”区域选择指定目标环境，在“Java进程列表”区域选择指定的Java进程；
- Step 2 单击“在线分析”；
- Step 3 进入实时采集分析界面。



- 在线分析包含对于目标JVM和Java程序的双重分析。包括Java虚拟机的内部状态如Heap，GC活动，线程状态及上层Java程序的性能分析，如调用链分析，热点函数，锁分析，程序线程状态及对象生成分布等。通过Agent的方式在线获取JVM运行数据，进行精确分析。
- 这种方式对系统有一定额外开销，对业务产生影响，适用于中小型的Java程序。

实时采集分析界面



- 在概览页签中：
- 在线显示Java虚拟机系统状态。
- 在线显示JVM的Heap大小、GC活动、Thread数量、Class加载数量和CPU使用率。
- 1. 实时采集分析界面，显示当前分析的进程PID，java环境版本号。
- 2. 满足分析要求后，可点击停止分析，回退到创建任务界面。
- 3. 实时分析过程中，软件根据预设项匹配，给出优化建议，提醒：非专业人士，不建议人为调整环境预置参数。
- 4. 分析数据可导出为离线结果，作为报告材料。
- 更多信息，详见：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpengfeat_06_0055.html。

查看线程信息

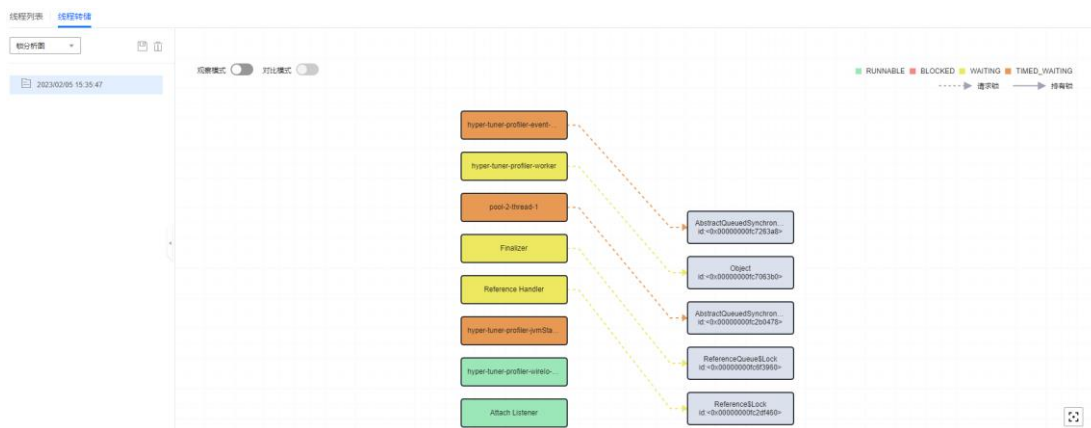
- 单击“CPU”页签，进入线程实时分析界面。



- Java程序通常是多线程运行，在CPU界面可查看当前程序的线程运行情况。
- “线程列表”列出当前分析的Java进程启动的线程名称和线程状态。线程可以是以下状态之一：Runnable、Waiting或Blocked。
- 可通过线程搜索框和“显示用法”快速筛选数据。
- “执行线程转储”按钮可转储当前线程状态，便于后续页签分析展示。

查看线程信息-线程转储

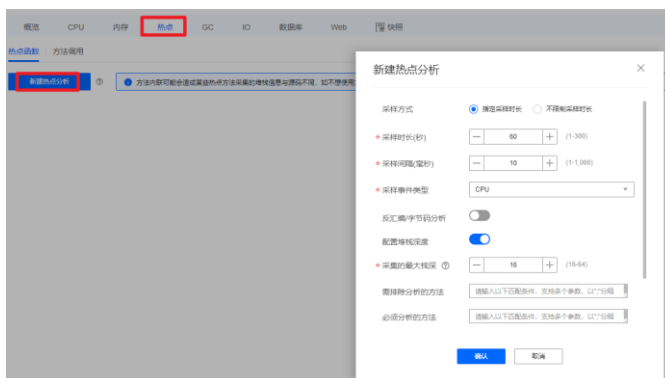
- 线程转储：JVM在某一个给定的时刻运行的所有线程的快照。默认是锁分析图。



- java的线程转储可以被定义为JVM中在某一个给定的时刻运行的所有线程的快照。
- 使用线程转储的常见场景：
 - 高CPU占用率。
 - 低CPU负载率和很长的响应时间。
 - 应用/服务宕机。
- 锁分析图通过连线表示线程对锁的占有情况。实线表示线程已占有锁，虚线表示线程阻塞在对应锁实例上，等待其他线程释放锁后占有。
- 观察模式：开启后，点击线程，将会高亮显示线程本身与持有锁。也可切换为以锁为观察视角，点击锁，会高亮显示锁本身和与请求的线程。
- 对比模式：开启后，需要选择要对比的线程转储，对比结果默认以线程为视角显示两个线程与持有的锁。建议对比模式配合观察模式，方便观察两个时间点线程与持有锁的状态变化。也可以切换为以锁为观察视角，观察两个时间点的某个锁和请求线程的状态变化。只有执行两次及以上线程转储之后才可以开启对比模式。
- 如果目标Java应用中存在死锁，会检测出死锁，可以结合线程转储，定位死锁产生原因。

热点分析

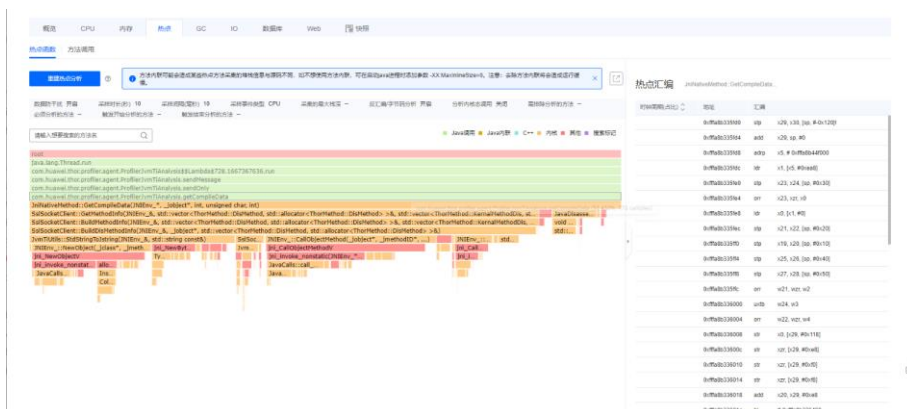
- 热点分析指的是通过perf性能分析工具采集某些时刻cpu栈顶信息，统计当前jvm中热点方法。以倒火焰图形式查询。
- 新建热点分析：点击“热点” - “新建热点分析”。



- 在线分析中，点击热点 -- 新建热点分析。
- 采样方式：配置是否限制采样时长。默认选择“指定采样时长”，可选：“指定采样时长”和“不限制采样时长”。
- 采样时长（秒）：显示采集数据的时长。采样方式选择“指定采样时长”时需配置。
- 采样间隔（毫秒）：显示采集数据的间隔时间。
- 采样事件类型：默认CPU即可。

热点分析-火焰图

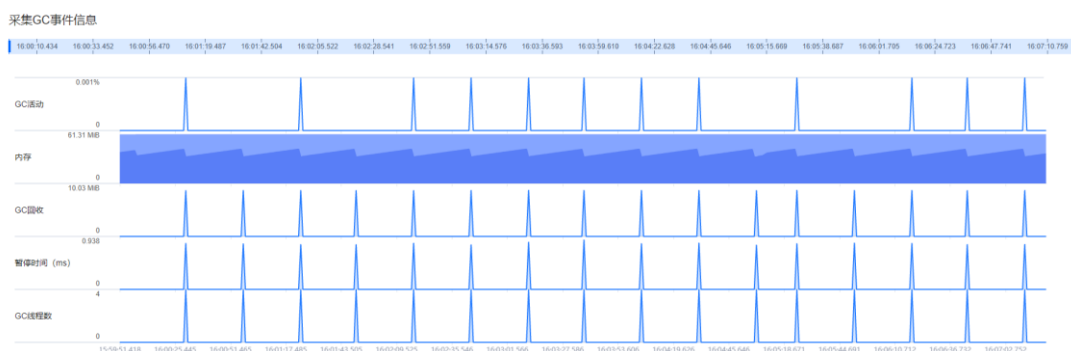
- 热点分析指的是通过perf性能分析工具采集某些时刻的cpu的栈顶信息，统计当前jvm中的热点方法。以倒火焰图的形式查询。



- 火焰图的基本构造：
 1. 每一列代表一个调用栈，每一格代表一个被调用函数；
 2. 方块上的字符标识调用方法，数字表示当前采样出现次数；
 3. Y 轴表示调用栈深度，X 轴将多个调用栈归并，并首字母排序展示；
 4. X 轴宽度表示采样数据中出现频次，即宽度越大，导致性能瓶颈的原因可能就越大（注意：是可能，不是确定）；
 5. 颜色没什么意义，随机分配。
- 关注火焰山顶部的"平顶山"（plateaus）出现的次数多，即没有子调用，抽样出现的频率高，说明执行方法的时间较长，或者执行频率太高（如长轮询），即CPU 大部分执行都分配给了“平顶山”，它才是性能瓶颈的根因。
- 总结方法论：火焰图看“平顶山”，山顶的函数可能存在性能问题！

GC分析

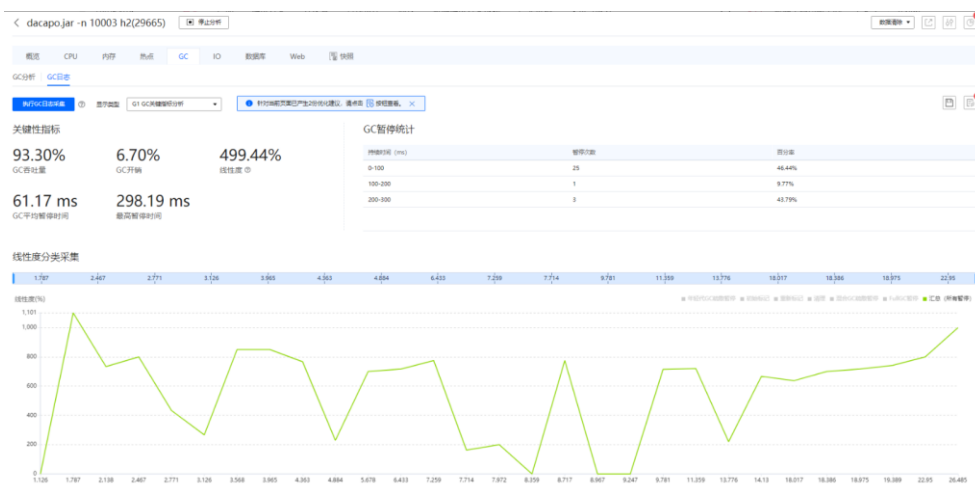
- Garbage Collection, Java进程的内存回收机制。
- 创建GC分析任务：页签栏中选择“GC”页签。



- GC活动：显示GC的暂停时间与单位时间（1秒）的比值的面积图。
- 内存：显示JVM申请的内存、使用中的内存和空闲的内存大小的堆叠面积图。
- GC回收：显示GC回收的内存的面积图。
- 暂停时间（ms）：显示GC引起的应用暂停执行的时间的面积图。
- GC线程数：显示GC进行过程中使用到的线程数的面积图。

GC日志分析

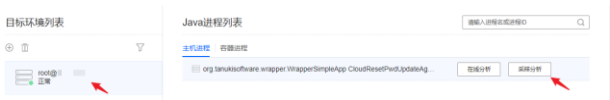
- 点击“执行GC日志采集”，查看GC关键性指标分析。



- GC吞吐量：显示除GC外的总耗时。
- GC开销：显示GC时占用的资源占比。
- 线性度：体现cpu多核利用率，计算公式为（用户耗时+系统耗时）/实际耗时。
- GC平均暂停时间：显示GC平均暂停时长。
- 最高暂停时间：显示进程在GC是最多暂停时长。
- GC暂停统计：显示GC暂停的总体数据。
- 线性度分类采集：显示采集时间段内线性度的实时变化。

创建采样分析任务

- Step 1 在首页界面“目标环境列表”区域选择指定目标环境，在“Java进程列表”区域选择指定的Java进程；
- Step 2 单击“采样分析”；
- Step 3 配置采集分析参数，进入采样阶段。



新建采样分析记录

记录方式 ☒ 指定记录时长 ☐ 不限制记录时长

* 采样时长 (s) (1 ~ 300)

方法采样 ☒

* Java方法采样间隔 (ms) (1 ~ 1000)

* Native方法采样间隔 (ms) (1 ~ 1000)

线程转储 ☒

* 转值间隔 (s) (1 ~ 300)

文件IO采样 ☐

Socket IO采样 ☐

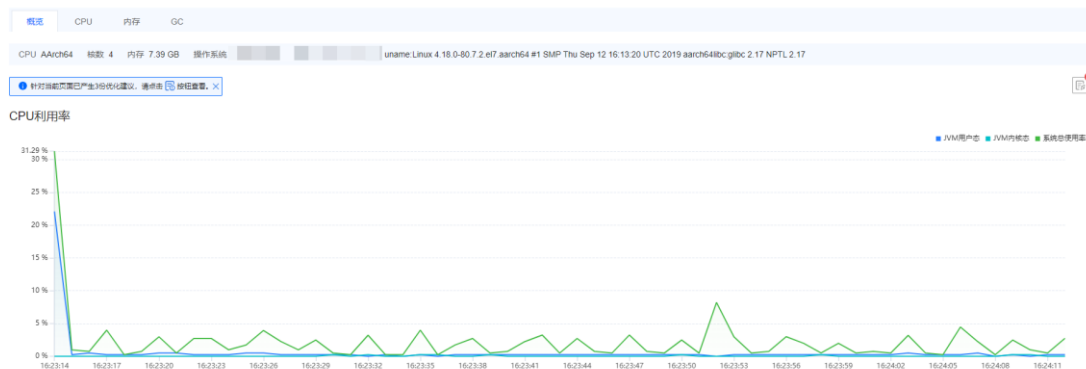
老年代对象采样 ☒



- 通过采样的方式，收集JVM的内部活动/性能事件，通过录制及回放的方式来进行离线分析。分析结果的维度与在线分析接近。
- 这种方式对系统的额外开销很小，对业务影响不大，适用于大型的Java程序。

结果分析-概览

- 界面显示java程序在系统中的资源占用情况，界面信息相对在线分析要少。



- JVM用户态：显示JVM用户态CPU利用率。
- JVM内核态：显示JVM内核态CPU利用率。
- 系统总使用率：显示计算机总的CPU利用率。

目录

1. 软件性能测试
2. 鲲鹏通用性能调优方法
- 3. 鲲鹏性能分析工具**
 - 工具介绍
 - 调优助手
 - 系统性能分析
 - Java性能分析
 - 系统诊断

系统诊断简介

- 系统诊断是针对基于鲲鹏服务器的性能分析工具，提供内存泄漏诊断（包括内存未释放和异常释放）、内存消耗信息分析展示、OOM诊断能力，帮助用户识别出源代码中内存使用的问题点，提升程序的可靠性。



- 系统诊断是针对基于鲲鹏的服务器的性能分析工具，提供内存泄漏诊断（包括内存未释放和异常释放）、内存越界诊断、内存消耗信息分析展示、OOM诊断能力、网络丢包等，帮助用户识别出源代码中内存使用的问题点，提升程序的可靠性，工具还支持压测系统，如：网络IO诊断，评估系统最大性能。

工具支持的功能特性 - 系统诊断

功能	描述
网络IO诊断	提供压测网络和诊断网络的能力。通过压测网络，获得网络最大能力，为网络IO性能优化提供基础参考数据；通过诊断网络，定位网络疑难问题，解决因网络配置和异常而导致的网络IO性能问题。
内存泄漏诊断	分析应用程序存在的内存泄漏点（包括内存未释放和异常释放），得出具体的泄漏信息，并支持关联出调用栈信息和源码。实时跟踪应用程序运行期间系统层、应用层、分配器层的内存消耗情况。分析发生OOM时的进程内存状态、系统内存状态和调用栈信息。
内存越界诊断	分析应用程序的内存异常访问点，给出异常访问类型和内存访问信息，并支持关联出调用栈和源码。
存储IO诊断	压测存储IO，获得存储设备最大能力，包括：吞吐量、IOPS、时延等，并以此评估存储能力，为存储IO性能优化提供基础参考数据。



- 四种功能的具体实操方法，详见鲲鹏官网，文档路径：文档首页>鲲鹏开发套件>鲲鹏性能分析工具>用户指南>特性指南>系统诊断。

应用场景 - 系统诊断工具

- 客户软件在基于鲲鹏的服务器上运行遇到性能问题时，可用系统诊断来快速分析和定位。
- 系统诊断工具将采集系统如下数据：
 - 内存泄漏次数\内存泄漏大小\内存异常释放次数\self内存泄漏\调用栈信息。
 - 系统的物理内存和虚拟内存大小\进程的内存MAP信息。
 - 应用的申请内存大小、申请次数、申请字节数、释放次数、释放字节数、泄漏次数以及泄漏字节数。
 - 分配器的分配内存、空闲内存、使用内存、Arena数、mmap区域数量以及mmap区域大小。
 - 系统软硬件配置和运行信息，例如：CPU类型、内存部署槽位、Kernel版本、内核参数、文件系统、系统运行日志参数等\系统的CPU、内存、存储IO、磁盘IO等性能指标\处理器PMU、SPE的性能数据。
 - 处理器访问Cache/内存的次数、带宽、吞吐率等\系统内核进行CPU资源调度、IO操作等数据。
 - 进程/线程的CPU、内存、存储IO、上下文切换、系统调用等数据；进程命令行信息，包括：进程名、进程参数。
 - 系统的热点函数及其调用栈；热点函数归属的程序/动态库（包含绝对路径）；热点函数的汇编指令和热点指令；热点函数所对应的源代码（需要用户自行提供）。

系统诊断工具 - Call Tree

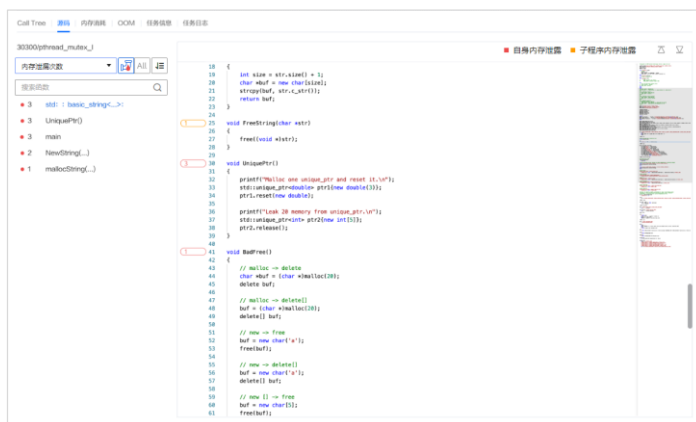
- 通过java程序调用树，可直观查看程序执行过程的函数调用关系，方便定位热点/故障函数。



- 对于大型项目，一些文件动辄几千行代码，上百个函数体，理解起来颇有些费劲。这个时候可以使用calltree工具对代码进行静态分析，然后产生调用关系树，使得可以对代码的构成有个初步的认识。这样可以站在高处，俯览全局，制定出一个切实可行的阅读理解方案。

系统诊断工具 - 源码

- 通过查看源码功能，直接在工具内查看故障源函数的代码，极速分析漏洞源。



- 如图，系统诊断工具在代码左侧给出了源码故障的位置，用不同颜色代表源码的故障程度，红色表示严重，需重点关注修改，不修改将导致程序报错；黄色为警告，影响程序运行，但不报错。

系统诊断工具 - 内存消耗

- 在线查看程序执行过程的内存消耗，直观呈现内存申请、释放、泄露等使用细节。



- 可视化界面：
- 进程 显示进程名。
- 可通过下拉菜单切换进程，也可以同搜索框按进程名进行搜索。
- 系统 显示物理内存和虚拟内存大小。
- 应用 显示申请内存大小、申请次数、申请字节数、释放次数、释放字节数、泄漏次数以及泄漏字节数。
- 分配器 显示分配器的分配内存、空闲内存、使用内存、Arena数、mmap区域数量以及mmap区域大小。
- 进程内存MAP信息 显示各个模块的物理内存和虚拟内存大小。

系统诊断工具 - OOM

Call Tree 源码 内存诊断 OOM 任务诊断 任务日志

2021-03-05 16:16:01.001

2021-03-05 16:16:01.001

2021-03-05 16:16:01.001

2021-03-05 16:16:01.001

基本信息

时间 2021-03-05 16:16:01.001
标签 CPU-0 PID-119 Core-mysql
宿主 PID-69948 Core-test_mysql i-virt 28025184380, avion-rsa 22081424040, flr-rsa 121840, shrim-rsa 0x40e-rsa 121840, shrim-rsa 0x40e

调用栈信息

1 |<ffff0000809e14c| dump_backtrace-0x07b23c
2 |<ffff0000808a074c| show_stack-0x24702c
3 |<ffff000020955da0| dump_stack-0x4c0a8
4 |<ffff0000202114b0| dump_header-0x40457c
5 |<ffff0000202114b0| oom_kill_process-0x20810a24
6 |<ffff000020211780| out_of_memory-0x0c0a4
7 |<ffff000020217644| _alloc_pages_node-0x7b0a0c0
8 |<ffff000020217644| _alloc_pages_node-0x7b0a0c0
9 |<ffff000020217644| do_anonymous_page-0x7b0a0c0
10 |<ffff00002021514c| _handle_mm_fault-0x40e0c0
11 |<ffff00002021514c| handle_mm_fault-0x40e0c0
12 |<ffff00008073a44| do_page_fault-0x7b0a0c0

系统内存信息

total pagecache pages 1450 pages RAM 4194237
pages in swap cache 0 pages HighMem movable chrt 0
Swap cache status acd 0, delete 0, find 0x0 pages reserved 12132
Free swap 0x0 pages hpoissoned 0
Total swap 0x0

进程内存信息

pid	uid	type	total_virt	rss	nr_pages	nr_privs	nr_pends
00000000040000	top	top	top	64	64	64	64
00000000050000	[anon]	[anon]	[anon]	1044	1044	1044	1044
00000000060000	Wmem_Rss-2.17.0	Wmem_Rss-2.17.0	Wmem_Rss-2.17.0	64	64	64	64
00000000070000	top	top	top	64	64	64	64
00040002000000	434242_24	434242_24	434242_24	4224	4224	4224	4224
22000000000000	434242_24	434242_24	434242_24	4224	4224	4224	4224

- 什么是OOM？OOM，全称“Out Of Memory”，翻译成中文就是“内存用完了”，来源于java.lang.OutOfMemoryError。
- 当JVM因为没有足够的内存来为对象分配空间并且垃圾回收器也已经没有空间可回收时，就会抛出这个error（注：非exception，因为这个问题已经严重到不足以被应用处理）。
- 产生OOM有两种可能：
 - 分配的少了：比如虚拟机本身可使用的内存（一般通过启动时的VM参数指定）太少。
 - 应用用的太多，并且用完没释放，浪费了。此时就会造成内存泄露或者内存溢出。
- 内存泄露：申请使用完的内存没有释放，导致虚拟机不能再次使用该内存，此时这段内存就泄露了，因为申请者不用了，而又不能被虚拟机分配给别人用。内存溢出：申请的内存超出了JVM能提供的内存大小，此时称之为溢出。

用户指南和工具软件包的获取方法

- 进入鲲鹏社区软件栈：<https://www.hikunpeng.com/developer/software>。
- 进入鲲鹏开发套件 Kunpeng Devikt >鲲鹏性能分析工具。
- 获取用户指南和工具的软件包。

鲲鹏性能分析工具

服务端运行平台

基于鲲鹏916的服务器、基于鲲鹏920的服务器

服务端运行操作系统

openEuler 20.03 (LTS)、CentOS 7.6、麒麟V10、Ubuntu 18.04.1等

* 详细的运行平台和操作系统对应关系请参见 [兼容性查询工具](#)

 浏览器工作模式

 Visual Studio Code

 IntelliJ

版本号

2.5.0

发布时间

软件包下载

数字签名下载

查看文档



- 工具更新较为频繁，详见文档发布记录：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpenghypertuner_06_0001.html。

思考题

1. （多选题）华为鲲鹏性能分析工具能够提供以下哪些方面的性能分析结果？
 - A. 分析Top热点函数
 - B. 分析函数火焰图
 - C. 分析热点函数代码映射
 - D. 分析不同函数对应top-down模型的各指标值

- 答案：ABCD

本章总结

- 本章主要介绍了应用性能测试的常见方法和策略，以及鲲鹏系统性能分析工具和鲲鹏Java性能分析工具的架构、功能特性、应用场景、部署方式和使用方法。

- 鲲鹏性能分析工具由四个子工具组成，分别为：系统性能分析、Java性能分析、系统诊断和调优助手。
- 系统性能分析是针对基于鲲鹏的服务器的性能分析工具，能收集服务器的处理器硬件、操作系统、进程/线程、函数等各层次的性能数据，分析系统性能指标，定位到系统瓶颈点及热点函数，并给出优化建议。该工具可以辅助用户快速定位和处理软件性能问题。
- Java性能分析是针对基于鲲鹏的服务器上运行的Java程序的性能分析和优化工具，能图形化显示Java程序的堆、线程、锁、垃圾回收等信息，收集热点函数、定位程序瓶颈点，帮助用户采取针对性优化。
- 系统诊断是针对基于鲲鹏的服务器的性能分析工具，提供内存泄漏诊断（包括内存未释放和异常释放）、内存越界诊断、内存消耗信息分析展示、OOM诊断能力、网络丢包等，帮助用户识别出源代码中内存使用的问题点，提升程序的可靠性，工具还支持压测系统，如：网络IO诊断，评估系统最大性能。
- 调优助手是针对基于鲲鹏的服务器的调优工具，能系统化组织性能指标，引导用户分析性能瓶颈，实现快速调优。
- 功能特性一览表：
https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpengintro_06_0002.html。

学习推荐

- 鲲鹏性能分析工具官网链接：

<https://www.hikunpeng.com/developer/devkit/hyper-tuner>

- 文档链接：

https://www.hikunpeng.com/document/detail/zh/kunpengdevps/hypertuner/usermanual/kunpengintro_06_0002.html

- 官网介绍分析工具功能、学习实践，大量在线课程、实验、最佳实践和代码样例。
- 文档细致介绍工具的使用方法，提供较官网首页更详细的介绍和操作指导。

缩略语

- PTGM: Performance Testing General Model, 经典的性能测试方法论之一, 旨在梳理性能测试的流程和相关环节使用到的测试工具, 是系统性的测试方法。
- NUMA: Non-Uniform Memory Access, 是一种分布式存储器访问方式。在多核处理器并行中广泛应用的今天, 当业务需要处理大量并发任务时, 可参考调优选项进行配置。
- SIMD: Single Instruction Multi Data, 单指令多数据流, 允许一条指令产生多个可以并行执行的操作, 在多核处理器中常作为程序执行优化选项。
- SMMU: System Memory Management Unit, 系统内存管理单元, 属于Arm架构处理器特有的概念。
- OOM: Out Of Memory, 内存不足, 当JVM因为没有足够的内存来为对象分配空间并且垃圾回收器也已经没有空间可回收时, 就会抛出这个error。

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。
Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

Copyright©2023 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



鲲鹏即时交流平台综合实验



前言

- 本章节是HCIA-Kunpeng Application Developer V2.0课程综合实验，主要介绍了鲲鹏即时交流平台从设计、部署、测试到调优的全流程。本实验包含四个子实验：基础环境搭建部署、代码迁移、平台部署、数据分析与优化。通过本实验，您将能够了解如何在鲲鹏平台上进行业务软件部署，巩固业务软件迁移、业务软件调优等学习成果。

目标

- 学完本课程后，您将能够：
 - 掌握在鲲鹏平台上部署软件的过程和方法；
 - 了解MySQL、Nginx、Hadoop、Hive等组件的作用及功能；
 - 了解应用代码迁移及前后端项目部署的方法；
 - 了解数据分析及调优的相关思路及方法。

目录

1. 鲲鹏即时交流平台介绍
 - 平台功能简介及实现方式
 - MySQL介绍
 - Nginx介绍
2. 基础实验环境部署
3. 鲲鹏代码迁移
4. 鲲鹏即时交流平台搭建
5. 基于Hive的数据处理

鲲鹏即时交流平台需求简介

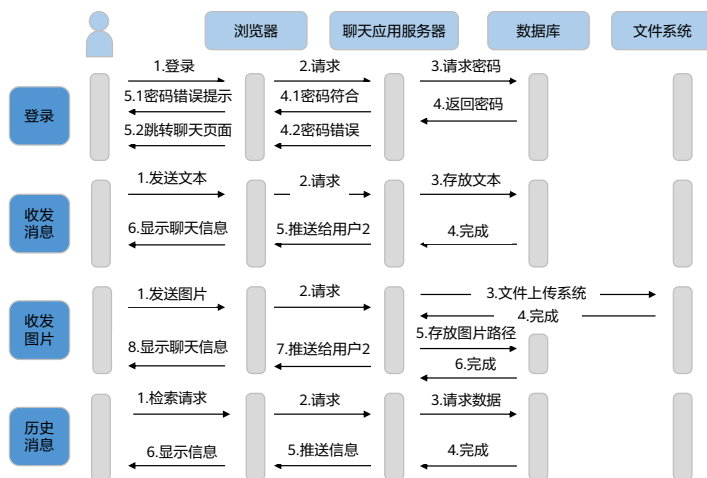
- 根据业务需求规划，鲲鹏即时交流平台是一个提供给用户进行即时交流的平台，用户可在平台上与其他用户创建对话并且进行即时交流，提供发送文字和图片的功能，并且可以查看最近的聊天记录，还可以在大数据中搜索最近的热门词汇。



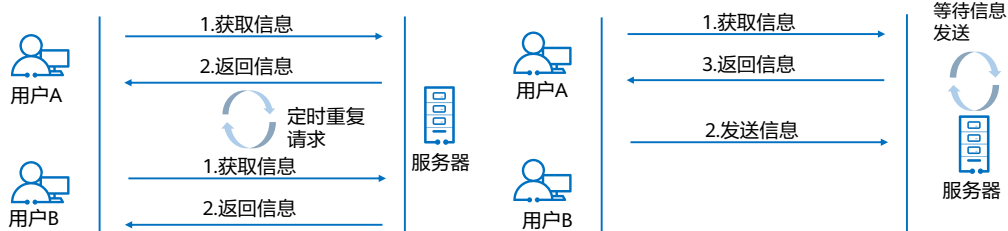
鲲鹏即时交流平台功能设计

• 设计基本功能：

- 登录
- 实时发送、接收消息
- 实时发送、接收图片
- 查询历史信息



实时聊天功能实现方案：http轮询



http短轮询:

- 通过后端http接口保存用户发送的聊天信息，并在客户端创建一个定时器，定时从后端接口获取用户A与用户B的聊天信息，并且重新渲染聊天界面。
- 优缺点：
 - 优点：后端程序编写比较容易。
 - 缺点：请求中有大半是无用，浪费带宽和服务器资源。

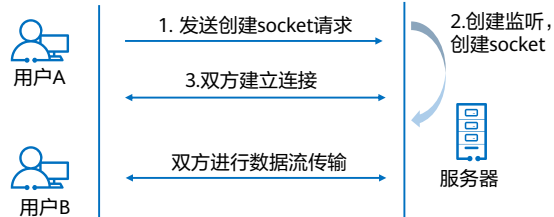
http长轮询:

- 客户端像传统轮询一样从服务器请求数据。然而，如果服务器没有可以立即返回给客户端的数据，则不会立刻返回一个空结果，而是保持这个请求等待数据到来，之后将数据作为结果返回给客户端。
- 优缺点：
 - 优点：在无消息的情况下不会频繁的请求，耗费资源小。
 - 缺点：服务器hold连接会消耗资源，返回数据顺序无保证，难以管理维护。

- 相同点：
 - 可以看出http长轮询和http短轮询都会hold一段时间。
- 不同点
 - 两者区别在于间隔发生在服务端还是浏览器端: http长轮询在服务端会hold一段时间，http短轮询在浏览器端hold一段时间。

实时聊天功能实现方案：Websocket

- Websocket协议是基于TCP的一种新的网络协议。它实现了浏览器与服务器全双工（full-duplex）通信，即允许服务器主动发送信息给客户端。
- 在实现Websocket连线过程中，需要通过浏览器发起Websocket连线请求，然后服务端发出回应，这个过程通常称为“握手”。在Websocket API，浏览器和服务器只需要做一个握手的动作，然后浏览器和服务器之间就形成了一条快速通道，浏览器和服务器可以通过该通道进行数据交互。



- HTTP协议有一个缺陷：通信只能由客户端发起，HTTP协议做不到服务器主动向客户端推送信息。如果服务器有连续的状态变化，客户端要获知就非常麻烦。
- 当服务器完成协议升级后（HTTP->Websocket），服务端就可以主动推送信息给客户端。

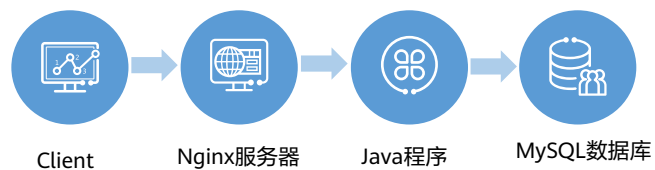
即时交流平台实时聊天功能实现

- 相较于http请求，Websocket是一个持久化的协议，并且客户端与服务端在实时通信方面的效率上有很大提升，故即时交流平台采用Websocket协议来实现实时聊天功能。

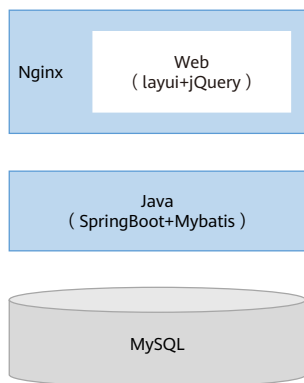
字段	Websocket	http
基础协议	TCP	TCP
TCP/IP模型层级	应用层	应用层
传输方向	双工通信	半双工通信
持久性	一次握手，长链接	有限制性，超最大请求数关闭
应用领域	实时通信	RESTful应用程序

鲲鹏即时交流平台逻辑架构设计

- 用户通过客户端访问Nginx Web服务器，Nginx对接了即时交流平台，该平台后端基于Java语言编写，平台的数据统一存储在MySQL数据库中。



鲲鹏即时交流平台架构



Java开发框架介绍：

Spring Boot：

Spring Boot是由Pivotal团队提供的全新框架，其设计目的是用来简化新Spring应用的初始搭建以及开发过程。该框架使用了特定的方式来进行配置，从而使开发人员不再需要定义样板化的配置。

Mybatis：

MyBatis是一个开源、轻量级的数据持久化框架，是JDBC和Hibernate的替代方案。内部封装了JDBC，简化了加载驱动、创建连接、创建Statement等繁杂的过程，开发者只需要关注SQL语句本身。

实验任务总览

- 实验开始前，需要在华为云服务器上搭建后续实验需要的环境，包括Nginx、MySQL，方便后续实验的进行。

- 鲲鹏即时交流平台的部署分为前端和后端项目的部署。前端采用layui+jQuery实现，后端使用JAVA语言进行编写，基于Springboot+Mybatis框架。

1、基础环境准备

3、即时交流平台部署

2、鲲鹏代码迁移

4、数据处理调优

- 使用鲲鹏代码迁移工具进行即时交流平台后端依赖库JNA的源码迁移，并对源码进行编译。

- 对平台的数据处理过程中，基于大数据解决方案，部署hadoop和hive，完成一系列数据分析任务，并针对大数据场景进行性能调优。

目录

1. 鲲鹏即时交流平台介绍
 - 平台功能简介及实现方式
 - MySQL介绍
 - Nginx介绍
2. 基础实验环境部署
3. 鲲鹏代码迁移
4. 鲲鹏即时交流平台搭建
5. 基于Hive的数据处理

什么是关系型数据库

- 关系型数据库：指采用了关系模型来组织数据的数据库。
关系模型指的就是二维表格模型，而一个关系型数据库就是由二维表及其之间的联系所组成的一个数据组织。
- 关系型数据库的优点：
 - 容易理解：二维表结构是非常贴近逻辑世界的一个概念，关系模型相对网状、层次等其他模型来说更容易理解。
 - 使用方便：通用的SQL语言使得操作关系型数据库非常方便。
 - 易于维护：丰富的完整性（实体完整性、参照完整性和用户定义的完整性）大大减低了数据冗余和数据不一致的概率。
- 常用关系型数据库：Oracle，MySQL，SQL Server，PostgreSQL，GaussDB，openGauss等。

MySQL简介

- MySQL是最流行的关系型数据库管理系统，在Web应用方面，MySQL是最好的RDBMS应用软件之一。
- 目前MySQL被广泛地应用在Internet上的中小型网站中。由于其体积小、速度快、总体拥有成本低，尤其是开放源码这一特点，使得很多公司都采用MySQL数据库以降低成本。



- Relational Database Management System: 关系数据库管理系统。

目录

1. 鲲鹏即时交流平台介绍

- 平台功能简介及实现方式
- MySQL介绍
- Nginx介绍

2. 基础实验环境部署

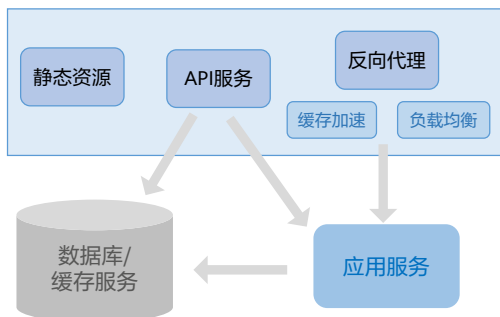
3. 鲲鹏代码迁移

4. 鲲鹏即时交流平台搭建

5. 基于Hive的数据处理

Nginx简介

- Nginx是一款轻量级的Web服务器、反向代理服务器，由于它的内存占用少，启动极快，高并发能力强，在互联网项目中广泛应用。

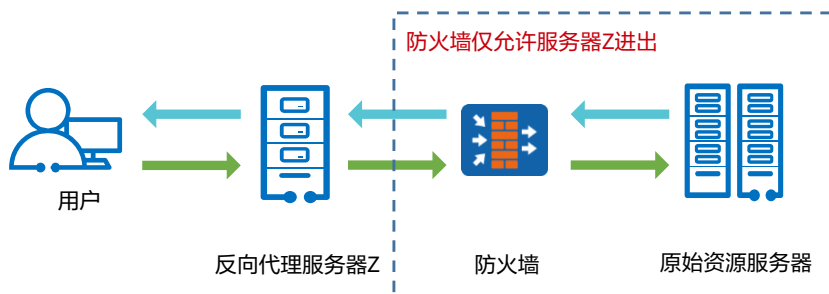


在本次实验项目中，Nginx主要用于放置静态前端Web资源，供客户端访问。



反向代理

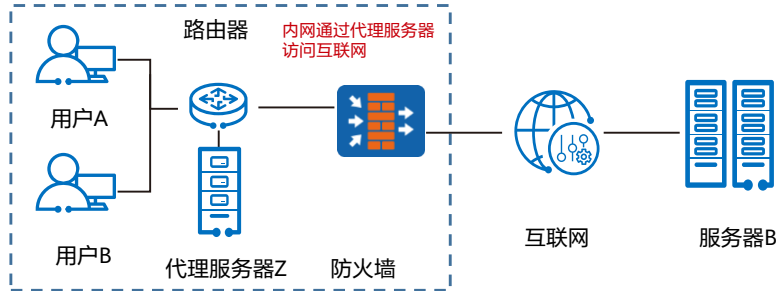
- 反向代理指以代理服务器来接受Internet上的连接请求，然后将请求转发给内部网络上的服务器，并将从服务器上得到的结果返回给Internet上请求连接的客户端，此时代理服务器对外就表现为一个反向代理服务器。



- 反向代理（Reverse proxy）在电脑网络中是代理服务器的一种。服务器根据客户端的请求，从其关联的一组或多组后端服务器（如Web服务器）上获取资源，然后再将这些资源返回给客户端，客户端只会得知反向代理的IP地址，而不知道在代理服务器后面的服务器集群的存在。
- 与正向代理不同，正向代理作为客户端的代理，将从互联网上获取的资源返回给一个或多个的客户端，服务端（如Web服务器）只知道代理的IP地址而不知道客户端的IP地址；而反向代理是作为服务器端（如Web服务器）的代理使用，而不是客户端。客户端借由正向代理可以间接访问很多不同互联网服务器（集群）的资源，而反向代理是供很多客户端都通过它间接访问不同后端服务器上的资源，而不需要知道这些后端服务器的存在，而以为所有资源都来自于这个反向代理服务器。

正向代理

- 正向代理是一个位于客户端和原始服务器（Origin Server）之间的服务器，为了从原始服务器获取内容，客户端向代理发送一个请求并指定目标（原始服务器），然后代理向原始服务器转交请求并将获得的内容返回给客户端。客户端必须要进行一些特别的设置才能使用正向代理。客户端才能使用正向代理。



- 客户端：在计算机网络中，在求来自另一节点（通常是服务器）的信息或服务的节点上的用户或进程。
- 原始服务器：存放资源或产生资源的服务器。
- 代理服务器：功能是代理网络用户去取得网络信息。形象地说，它是网络信息的中转站，是个人网络和Internet服务商之间的中间代理机构，负责转发合法的网络信息，对转发进行控制和登记。也可以执行路由策略控制。
- 正向代理：正向代理（forward）是一个位于客户端【用户A】和原始服务器（origin server）【服务器B】之间的服务器【代理服务器Z】，为了从原始服务器取得内容，用户A向代理服务器Z发送一个请求并指定目标（服务器B），然后代理服务器Z向服务器B转交请求并将获得的内容返回给客户端。客户端必须要进行一些特别的设置才能使用正向代理。

目录

1. 鲲鹏即时交流平台介绍
- 2. 基础实验环境部署**
3. 鲲鹏代码迁移
4. 鲲鹏即时交流平台搭建
5. 基于Hive的数据处理

主机环境选择

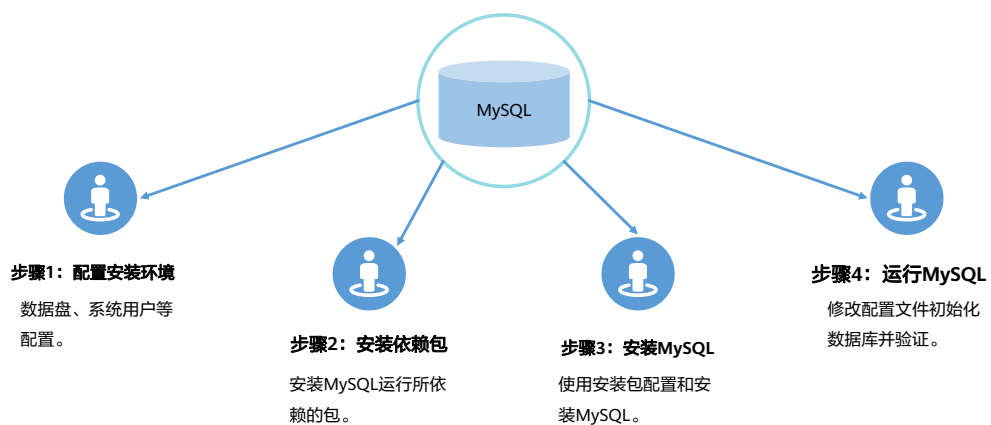
- 鲲鹏弹性云服务器（ECS）
 - 基础云服务之一，也是用户可以直接感知到鲲鹏的最重要的服务。用户通过购买鲲鹏弹性云服务器ECS，之后为云服务器添加磁盘、网络等资源，使其成为开发环境或者生产业务集群的一部分。

实验资源	云主机名称	配置	OS版本
鲲鹏弹性云服务器	ecs-kunpeng	kc1.xlarge.2 4vCPUs 8GiB	openEuler 22.03 64bit with ARM

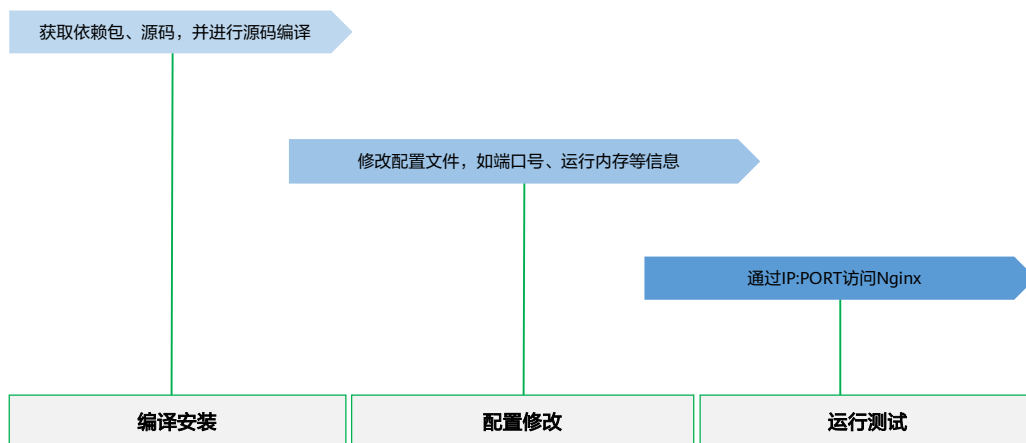
基础环境部署



MySQL部署步骤



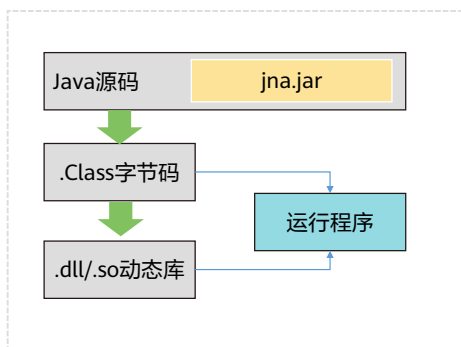
Nginx部署步骤



目录

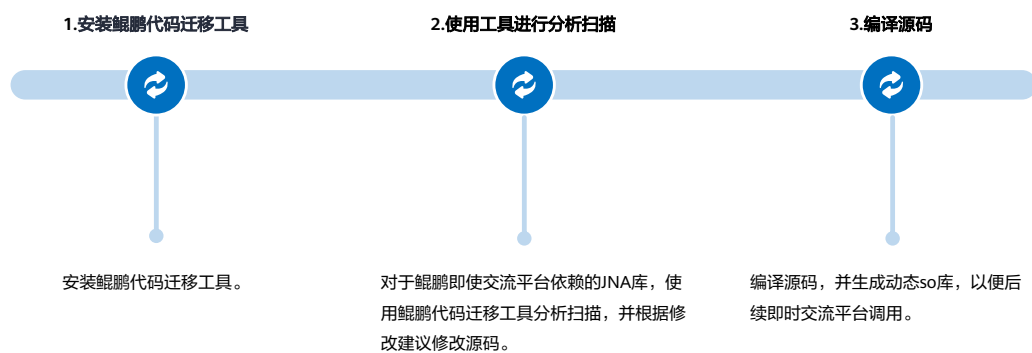
1. 鲲鹏即时交流平台介绍
2. 基础实验环境部署
- 3. 鲲鹏代码迁移**
4. 鲲鹏即时交流平台搭建
5. 基于Hive的数据处理

JNA简介



- 即时交流平台后端程序调用了使用c语言开发的so库函数，该函数需要重新编译才可在鲲鹏ECS服务器上运行。
- JNA (Java Native Access) 是一个开源的Java框架，是Sun公司推出的一种调用本地方法的技术，是建立在经典的JNI基础之上的一个框架。

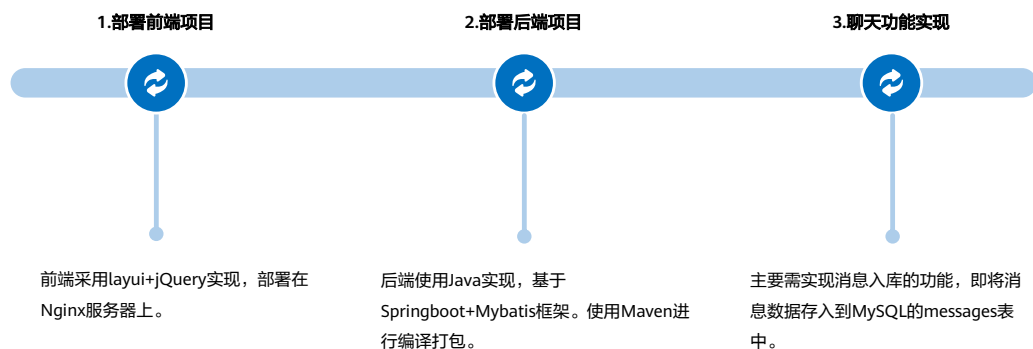
任务流程



目录

1. 鲲鹏即时交流平台介绍
2. 基础实验环境部署
3. 鲲鹏代码迁移
- 4. 鲲鹏即时交流平台搭建**
5. 基于Hive的数据处理

任务流程



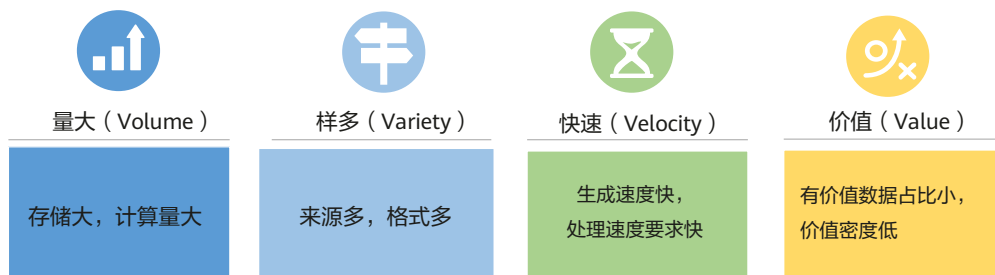
- 鲲鹏即时交流平台是一个提供给用户进行即时交流的平台，用户可在平台上与其他用户进行即时聊天交流，平台提供发送文字和图片的功能。
- 在本实验中，已经实现用户登陆、联系人、发送消息功能，发送图片的功能需要自行实现。

目录

1. 鲲鹏即时交流平台介绍
2. 基础实验环境部署
3. 鲲鹏代码迁移
4. 鲲鹏即时交流平台搭建
- 5. 基于Hive的数据处理**
 - 大数据简介
 - 性能调优思路
 - 实验流程介绍

大数据概述

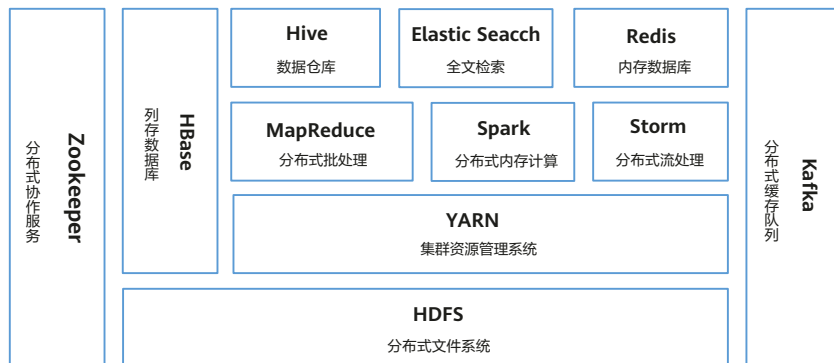
- 维基百科：“大数据是指无法在一定时间内用常规软件工具对其内容进行抓取、管理和处理的数据集合”。
- IDC：一般会涉及2种以上数据形式，数据量100T以上，且是高速、实时数据流；或者从小数据开始，但数据每年增长60%。
- Gartner：大数据的四个V：Volume、Variety、Velocity、Value。



- IDC (International Data Corporation) 在它编制的年度数字宇宙研究报告《从混沌中提取价值》(Extracting Value from Chaos) 中给大数据下了一个定义：大数据技术是新一代的技术与架构，它被设计用于在成本可承受 (Economically) 的条件下，通过非常快速 (Velocity) 的采集、发现和分析，从大体量 (Volumes)、多类别 (Variety) 的数据中提取价值 (Value)。

大数据介绍及组件关系分布

- 大数据是集收集、处理、存储为一体的技术总称。在海量数据处理的场景，大数据对计算及存储的要求较高，普遍以集群形式存在。不同的组件有不同的功能体现。

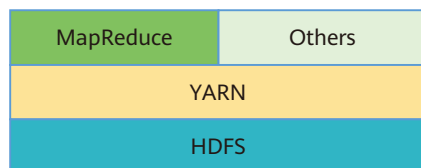


- HDFS Hadoop Distributed File System，简称HDFS，是一个分布式文件系统。HDFS是一个高度容错性的系统，适合部署在廉价的机器上。HDFS能提供高吞吐量的数据访问，非常适合大规模数据集上的应用。
- HBASE 是Hadoop的数据库，一个分布式、可扩展、大数据的存储。是为有数十亿行和数百万列的超大表设计的，是一种分布式数据库，可以对大数据进行随机性的实时读取/写入访问。提供类似谷歌Bigtable的存储能力，基于Hadoop和Hadoop分布式文件系统（HDFS）而建。
- Redis 是一个高性能的key-value存储系统，和Memcached类似，它支持存储的value类型相对更多，包括string（字符串）、list（链表）、set（集合）和zset（有序集合）。Redis的出现，很大程度补偿了memcached这类key/value存储的不足，在部分场合可以对关系数据库起到很好的补充作用。
- Kafka 是一种高吞吐量的分布式发布订阅消息系统，它可以处理消费者规模网站中的所有动作流数据，目前已成为大数据系统在异步和分布式消息之间的最佳选择。
- Spark 是一个高速、通用大数据计算处理引擎。拥有Hadoop MapReduce所具有的优点，但不同的是Job的中间输出结果可以保存在内存中，从而不再需要读写HDFS，因此Spark能更好地适用于数据挖掘与机器学习等需要迭代的MapReduce的算法。它可以与Hadoop和Apache Mesos一起使用，也可以独立使用。
- Storm是Twitter开源的一个类似于Hadoop的实时数据处理框架。编程模型简单，显著地降低了实时处理的难度，也是当下最流行的流计算框架之一。与其他计算框架相比，Storm最大的优点是毫秒级低延时。

- Hive 是基于Hadoop的一个数据仓库工具，可以将结构化的数据文件映射为一张数据库表，并提供简单的sql查询功能，可以将sql语句转换为MapReduce任务进行运行。其优点是学习成本低，可以通过类SQL语句快速实现简单的MapReduce统计，不必开发专门的MapReduce应用，十分适合数据仓库的统计分析。
- YARN 是一种新的Hadoop资源管理器，它是一个通用资源管理系统，可为上层应用提供统一的资源管理和调度，解决了旧MapReduce框架的性能瓶颈。它的基本思想是把资源管理和作业调度/监控的功能分割到单独的守护进程。
- ZooKeeper 是一个分布式的应用程序协调服务，是Hadoop和Hbase的重要组件。它是一个为分布式应用提供一致性服务的工具，让Hadoop集群里面的节点可以彼此协调。ZooKeeper现在已经成为了 Apache的顶级项目，为分布式系统提供了高效可靠且易于使用的协同服务。

Hadoop介绍

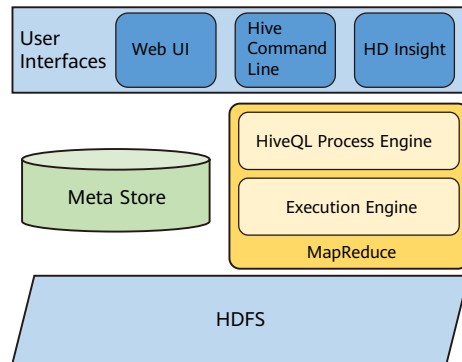
- Hadoop是一个分析和处理大数据的软件平台，Apache Hadoop是一种使用Java语言所实现的开源软件框架，利用多台计算机组成集群，以便更快地并行分析海量数据集。
- Hadoop的框架最核心的设计就是：HDFS和MapReduce。HDFS为海量的数据提供了存储，而MapReduce为海量的数据提供了计算。



- Hadoop 由四个主要模块组成：
- Hadoop 分布式文件系统 (HDFS)——一个在标准或低端硬件上运行的分布式文件系统。除了更高容错和原生支持大型数据集，HDFS 还提供比传统文件系统更出色的数据吞吐量。
- Yet Another Resource Negotiator (YARN)——管理与监控集群节点和资源使用情况。它会对作业和任务进行安排。
- MapReduce——一个帮助计划对数据运行并行计算的框架。该 Map 任务会提取输入数据，转换成能采用键值对形式对其进行计算的数据集。Reduce 任务会使用 Map 任务的输出来对输出进行汇总，并提供所需的结果。
- Hadoop Common——提供可在所有模块上使用的常见 Java 库。

Hive介绍

- Hive是一个数据仓库基础设施工具，用于处理Hadoop中的结构化数据。它位于Hadoop的顶部，用于汇总大数据，并使查询和分析变得轻松。
- 在Hadoop中，Hive是SQL解析引擎，它将SQL语句转译成MapReduce Job然后在Hadoop执行。Hive的表其实就是HDFS的目录/文件，按表名把文件夹分开。



- 用户接口/界面：Hive是一个数据仓库基础工具软件，可以创建用户和HDFS之间互动。用户界面，Hive的Web UI，Hive命令行，HiveHD洞察（在Windows服务器）。
- 元存储：Hive选择各自的数据库服务器，用以储存表，数据库，列模式或元数据表，它们的数据类型和HDFS映射。
- HiveQL
 - 处理引擎：HiveQL类似于SQL的查询上Metastore模式信息。这是传统的方式进行MapReduce程序的替代品之一。相反，使用Java编写的MapReduce程序，可以编写为MapReduce工作，并处理它的查询。
 - 执行引擎：HiveQL处理引擎和MapReduce的结合部分是由Hive执行引擎。执行引擎处理查询并产生结果和MapReduce的结果一样。它采用MapReduce方法。
- HDFS：Hadoop的分布式文件系统。

HiveQL简介

- HiveQL，简称HQL，是一种用于Hive的查询语言。Hive查询操作过程严格遵守Hadoop MapReduce的作业执行模型，Hive将用户的HiveQL语句通过解释器转换为MapReduce作业提交到Hadoop集群上，Hadoop监控作业执行过程，然后返回作业执行结果给用户。
- HiveQL用于分析和处理Metastore中的结构化数据。它与SQL非常相似，具有高度的可扩展性。它重用了关系数据库世界中熟悉的概念，如表、行、列和模式，以简化学习。
- HiveQL主要有如下特点：
 - HQL不支持行级别的增、改、删，所有数据在加载时就已经确定，不可更改。
 - 不支持事务。
 - 不支持等值连接，一般使用left join、right join或者inner join替代。

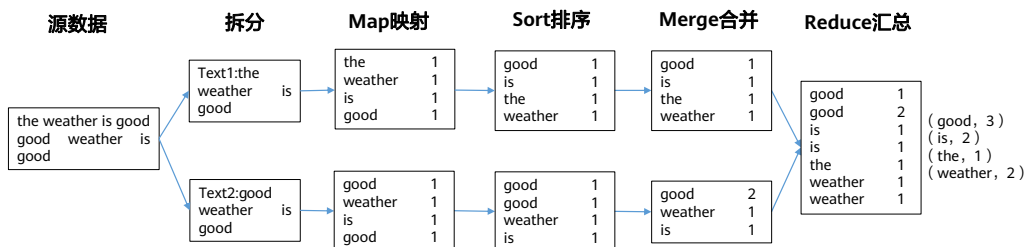
- HiveQL与SQL基本上一样，因为当初的设计目的，就是让会SQL不会编程MapReduce的也能使用Hadoop进行处理数据。

大数据计算模式及适用场景

大数据计算模式	解决问题	代表产品
批处理	针对大规模数据的批量处理	Hadoop、Hive、Spark等
即席分析查询	大规模数据的存储管理和查询分析	Dremel、Presto、Impala等
流处理	针对流数据的实时计算	Storm、S4、Flume、Flink
图计算	针对大规模图结构数据的处理	Pregel、GraphX、Giraph、PowerGraph、Hama、GoldenOrb等

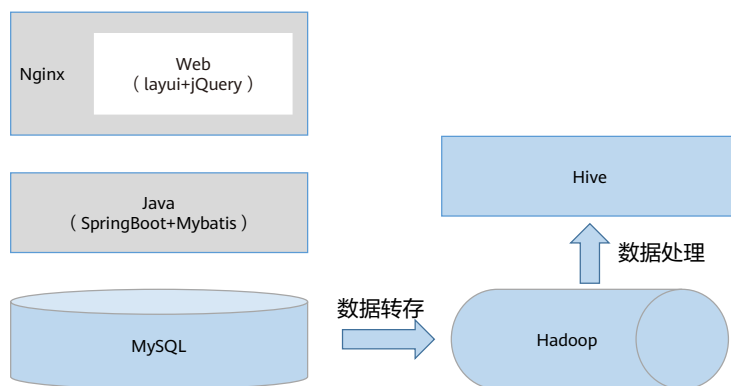
大数据并行计算特点天然匹配鲲鹏多核架构

- 海量数据需要更高的并发度来加速数据处理，鲲鹏多核计算的特点能够提升大数据任务的并发度，加速大数据的计算性能。此处以MapReduce模型为例：



但是，为了获得更好的性能，仍需根据硬件配置和应用程序特点，对软硬件系统做进一步的优化。

平台数据分析流向图



目录

1. 鲲鹏即时交流平台介绍
2. 基础实验环境部署
3. 鲲鹏代码迁移
4. 鲲鹏即时交流平台搭建
- 5. 基于Hive的数据处理**
 - 大数据简介
 - 性能调优思路
 - 实验流程介绍

调优原因

组件参数默认值保守

应用程序和操作系统为了兼容不同环境，涉及性能的参数默认值较小，不能发挥集群资源的最大性能。

合理配置上下游组件的资源分配

同一套大数据集群环境中会安装不同的组件，而不同组件对CPU、磁盘、网络等资源需求不同，需合理配置。

性能瓶颈因硬件配置而异

因硬件环境常无法统一，当某个硬件资源提前到达瓶颈，需根据实际硬件配置进行针对性的调优。

常用调优思路

保障测试压力：

- 当客户端压力不足以发挥大数据集群的性能时，需优先提高客户端压力。

分配物理资源：

- 根据组件特点，尽可能多地分配该组件依赖的物理资源（CPU、磁盘、内存、网络等）。

监控资源使用情况：

- 使用性能监控工具观察系统状态并进行记录，如CPU、磁盘、内存、网络、应用程序GC状况、热点函数等。

确定性能瓶颈：

- 基于组件、应用程序特点和监控数据识别性能瓶颈，瓶颈可能是物理资源、组件参数、测试工具、测试组网、JVM、锁等。

实施优化：

- 根据识别的瓶颈针对性地进行优化，其中，优化手段有时并不会生效，需进一步确定是否锁定瓶颈及优化手段是否正确。



基础调优操作，确保集群拥有较优的性能



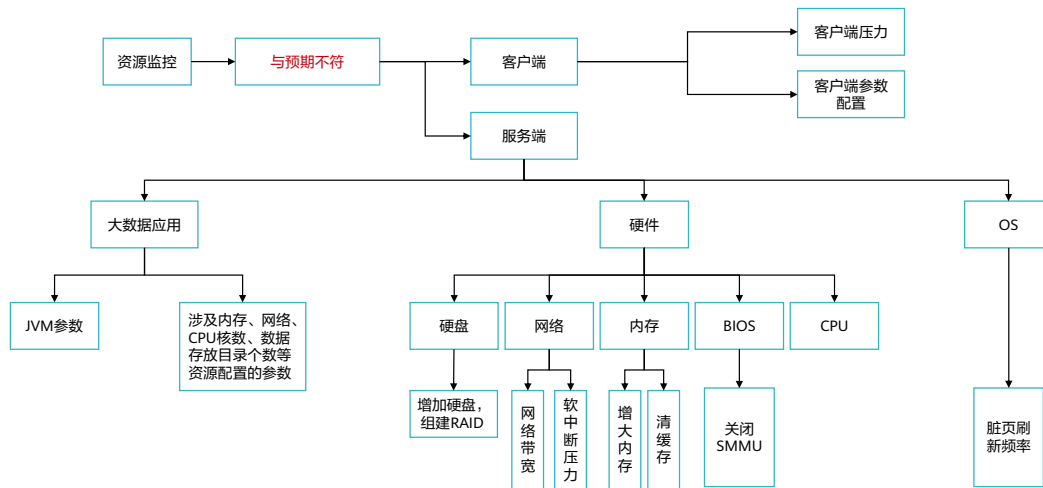
重复资源监控、确定瓶颈、优化动作
针对性解决问题，提升性能

- GC（Garbage Collection）垃圾回收，GC主要的职责就是分配内存；保证被引用的对象始终在内存中；把不被应用的对象从内存中释放。

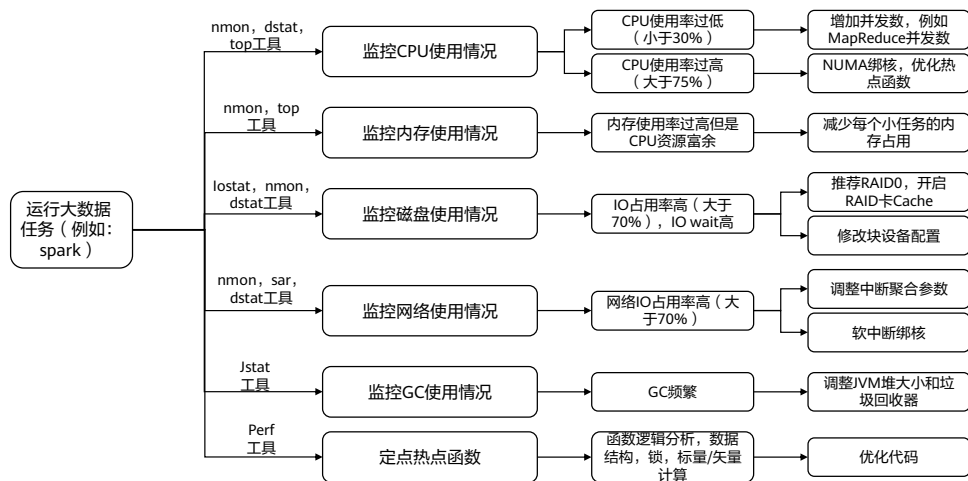
常见调优问题介绍

问题层面	常见问题	参考解决方法
应用层面	CPU占用率低	分析应用中与并发数、核数强相关的参数，适当增加；增加并发，但CPU占用率未提升，则分析其他瓶颈点
	CPU占用率高	Hadoop 3.X以上版本打开numa-aware特性，以下版本可考虑修改源码，利用numactl命令为真正占用CPU资源的进程分配CPU核，减少应用程序运行时，跨NUMA访问内存的情况 使用perf等性能监控工具抓取高CPU占用率进程（整机或单核）的热点函数，寻找是否有不合理的循环调用、锁占用、单线程的逻辑实现等，针对性优化源码
	内存消耗尽，但CPU等资源还有富余	大数据任务一般会开多个小任务同时执行业务逻辑，满足内存需要的前提下，适当降低每个任务的内存占用率，可增加任务数来提高整机的资源利用率，将鲲鹏CPU多核的特点发挥出来
	GC频繁	调整JVM堆、新生代、线程堆栈大小；选择合适的垃圾回收器
硬件层面	磁盘IO占用率高，CPU iowait高	开启raid卡cache；磁盘参数优化（脏页刷新频率、磁盘文件预读等）；增加磁盘数或更换性能更好的磁盘
	网络IO占用率高	网卡软中断绑核，将处理网卡中断的CPU核，设置在网卡所在的NUMA上，减少跨NUMA访问内存的情况 优化网卡参数（RingBuffer、LRO等）；增大网卡带宽
	内存占用多	适当调整内存页大小；关闭swap内存
客户端	组件参数已确保较优，但性能不好	检查客户端压力（如并发数、数据量设置）；检查客户端到服务端组件的网络带宽是否支持业务数据的流量

性能定位：问题定位流程



性能定位：调优流程



目录

1. 鲲鹏即时交流平台介绍
2. 基础实验环境部署
3. 鲲鹏代码迁移
4. 鲲鹏即时交流平台搭建
- 5. 基于Hive的数据处理**
 - 大数据简介
 - 性能调优思路
 - 实验流程介绍

任务流程

- Hadoop作为大数据分析工具，为海量的数据提供了存储和计算。
- Hive提供SQL查询功能，方便进行数据分析工作。

1、部署Hadoop、Hive

- 依照步骤安装鲲鹏性能分析工具。

3、部署鲲鹏性能分析工具

2、使用Hive进行数据分析

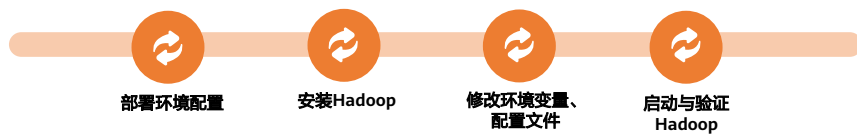
- 结合实验手册中设置的数据分析测试题，练习编写HQL完成数据分析任务。

4、使用鲲鹏性能分析工具优化

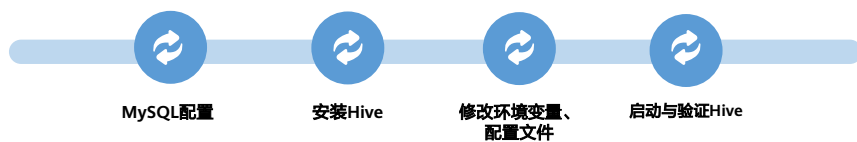
- 使用鲲鹏性能分析工具进行系统扫描，根据优化建议修改优化项并验证。

Hadoop、Hive部署步骤

- Hadoop部署



- Hive部署



思考题

1. （单选题）Nginx的功能不包括以下哪一项？
- A. 静态资源
 - B. API服务
 - C. 正向代理
 - D. 反向代理

- 答案：
 - C

思考题

2. （单选题）以下关于Websocket协议的描述，错误的是哪一项？

- A. Websocket协议是基于TCP协议实现的
- B. 传输方向为双工通信
- C. Websocket协议是一种持久化协议
- D. 服务器可以向客户端发送创建socket请求

• 答案：

- D

思考题

3. （单选题）以下关于HiveQL的描述，正确的是哪一项？
- A. 支持行级别的增删改查
 - B. 支持事务查询
 - C. Hive将用户的HiveQL语句通过解释器转换为MapReduce作业提交到Hadoop集群上
 - D. 查询效率较传统关系型数据库来说更快

- 答案：
 - C

本章总结

- 本章节主要介绍了鲲鹏即时交流平台从设计、部署、测试到调优的全流程，并结合实验帮助学员掌握鲲鹏代码迁移、数据调优思路与调优方法。实验内容包括前期基础环境搭建、基于鲲鹏代码迁移工具的源代码迁移、即时交流平台的安装部署以及数据的分析与调优。通过实验，帮助学员了解如何在鲲鹏平台上进行业务软件部署，巩固业务软件迁移、业务软件调优等学习成果。

缩略语

- JDBC: Java Database Connectivity, Java数据库连接, JDBC是Java语言中用来规范客户端程序如何来访问数据库的应用程序接口, 提供了诸如查询和更新数据库中数据的方法。
- HTTP: Hyper Text Transfer Protocol, 超文本传输协议, HTTP是一种用于分布式、协作式和超媒体信息系统的应用层协议, 是因特网上应用最为广泛的一种网络传输协议, 所有的WWW文件都必须遵守这个标准。
- SQL: Structured Query Language, 结构化查询语言, SQL是一种特殊目的的编程语言, 是一种数据库查询和程序设计语言, 用于存取数据以及查询、更新和管理关系数据库系统。
- ECS: Elastic Cloud Server, 弹性云服务器, 弹性云主机是由CPU、内存、镜像、云硬盘组成的一种可随时获取、弹性可扩展的计算服务器, 同时它结合VPC、虚拟防火墙、数据多副本保存等能力, 为您打造一个高效、可靠、安全的计算环境, 确保您的服务持久稳定运行。
- HDFS: Hadoop Distributed File System, Hadoop分布式文件系统, Hadoop分布式文件系统 (Hadoop Distributed File System) 能提供高吞吐量的数据访问, 适合大规模数据集方面的应用。
- IO: Input Output, 输入输出, 计算机系统主存和外围设备之间的数据移动过程。I/O是一个合称, 表示将数据读入计算机系统主存的操作, 以及将数据从计算机系统主存写到其它位置的操作。

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。
Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

Copyright©2023 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



华为鲲鹏解决方案



前言

- 本章节主要介绍华为鲲鹏解决方案的全景图和各项能力，加深学员对于鲲鹏计算平台的理解，使学员了解依托鲲鹏硬件所实现的解决方案。

目标

- 学完本课程后，您将能够了解到：
 - 华为鲲鹏解决方案全景
 - 华为鲲鹏通用解决方案

目录

1. 华为鲲鹏解决方案全景
2. 华为鲲鹏通用解决方案

华为鲲鹏解决方案全景

- 基于华为鲲鹏计算平台（云平台或物理平台）的解决方案分为通用解决方案及行业解决方案，通用解决方案包括高性能计算HPC、大数据基础设施、分布式存储、企业核心应用等；行业解决方案包括为运营商、政府、金融等行业提供的针对性解决方案。
- 部署方式可以全部采用鲲鹏架构，也可以采用鲲鹏架构和x86架构混合部署的方式。



华为鲲鹏云全栈解决方案竞争力



- 围绕数据库、容器、中间件，构建解决方案竞争力。
- ISV: Independent software vendor，独立软件供应商，补充业务提供商的一种，又叫做独立软件开发商。

TaiShan服务器覆盖主流应用场景

大数据



- 性能提升
- 加解密引擎
- 与X86融合部署

分布式存储



- IOPS性能提升
- 压缩解压缩引擎
- 与X86融合部署

数据库



- RoCE低时延网络
- 多核调度算法
- NUMA优化算法

原生应用



- 原生同构，无性能损耗
- 企业级可靠方案
- 百万级应用，成熟生态

HPC



- 八通道内存带宽
- 高密度服务器
- 液冷节能技术

高性能

多核高并发

高吞吐

原生算力同构

- TaiShan服务器高性能、多核高并发、高吞吐和原生算力同构的优势。适合为大数据、分布式存储、原生应用、高性能计算和数据库等应用高效加速，旨在满足数据中心多样性计算、绿色计算的需求。TaiShan服务器解决方案能够覆盖数据中心主流应用场景。
- 在过去的3年时间里，华为与各行业的ISV共同孵化解决方案，在优势场景进行了深度优化，为政府、金融、运营商、电力、互联网等广大行业客户提供了基于鲲鹏处理器的数据中心基础设施和服务。

目录

1. 华为鲲鹏解决方案全景
2. 华为鲲鹏通用解决方案
 - 高性能计算HPC解决方案
 - 大数据基础设施解决方案
 - 鲲鹏云手机解决方案

高性能计算HPC解决方案概述

- HPC（High Performance Computing，高性能计算）是一个计算机集群系统，它通过各种互联技术将多个计算机系统连接在一起，利用所有被连接到系统的综合计算能力来处理大型计算问题，所以又通常被称为高性能计算集群。
- 高性能计算云解决方案（HPC Cloud）是利用云部署方式来提供一种高效、可靠、灵活、安全的计算服务；能够为工业设计、海量数据处理等场景提供卓越的计算服务；帮助客户降低TCO，部署周期短，成本低，缩短产品上市周期，提升企业产品竞争力。
- 华为HPC解决方案主要分两层：
 - 华为主要提供相关的计算所需要的IAAS资源，包括计算、存储、网络等资源。
 - 集群调度层在不同的应用场景使用的软件略有不同，如工业仿真场景更多的使用的PBS，而针对基因测序使用的更多的是开源软件Sge以及Slurm。

- 解决方案部分：华为云主要是IaaS层的内容，第二层是指Altair PBS+商业应用模式，主要是软件调度+业务软件，一般是合作伙伴或ISV的内容。基因测序行业，教育科研的Sge/Slurm调度软件是开源的，由华为云提供。
- 解决方案核心价值与优势：
 - 部署周期短，成本低。
 - 部署灵活，不会造成资源的浪费或不足。
 - 公有云方式维护成本低。
 - 满足技术快速发展要求。
- 解决方案瞄准内存敏感型应用及整型计算（或者单精度计算）的应用，典型应用场景如下：
 - 科学计算、气象预测、环境预测、工业仿真、基因测序、能源勘探、动漫渲染和金融分析等等。

高性能计算HPC解决方案优势



高性能

基于鲲鹏计算平台，鲲鹏920处理器，最高64核，8内存通道，适合访存密集型及整型计算为主应用，华为智能网卡、SSD、Atlas加速卡，加速高性能计算应用，提升客户应用性能表现。



高效能

板级液冷，内存间走水，扰流强化换热，全液冷机柜散热，PUE低至1.05，降低客户管理运维成本。



安全可靠

自研芯片、操作系统、MPI、编译器、数学库等，掌握关键核心技术，提供产品可持续供应能力，遵从商业环境要求，信息安全主动防护，保障客户放心部署应用。



开放生态

开放合作，与ISV、合作伙伴、开发者、产业联盟共建全栈生态，兼容主流企业应用软件，支持丰富应用快速平滑移植，降低客户迁移成本。

- 性价比高：自服务能力、方便易用、生态丰富。

HPC解决方案性价比高

- 基因测序、流体力学、气象环保主要为内存敏感型应用以及整型计算（或者单精度计算）为主，适合鲲鹏云HPC应用场景。



- 性价比领先
 - 独家发布基于自研鲲鹏芯片的HPC解决方案，性价比优于传统方案30%。
 - 并行文件系统PFS，同时支持对象存储和POSIX接口，替代传统并行文件系统，数据免拷贝，TCO降低20%。
 - 独家支持100G InfiniBand高速网络，时延低至1.5微秒，支持RDMA，性能业界领先。
- 易部署、易使用、易维护
 - 提供云上HPC集群管理服务，支持集群一键式发放、快速部署，效率提升90%。
 - 提供集群管理、作业管理、节点管理等可视化HPC集群管理能力。
 - 持续集成HPC编译器、MPI等各种开源商用HPC中间件及业务软件。
- 丰富的HPC生态
 - 基因测序：GATK、bwa、Bowtie、Blast等近百款软件已成功移植。
 - 气象环保：WRF、ROMS、CAMX、CMAQ等近十款软件已成功移植。
 - 流体力学：OpenFORM、SU2等已经成功移植。

其他优势特性

多组织、区域租户共享

- 动态申请，动态共享，计算节点按需申请/释放
- 按需租用，避免过度投资，避免重复建设
- 用户隔离措施保障安全性
- 数据集中，跨组织和区域的合作共享

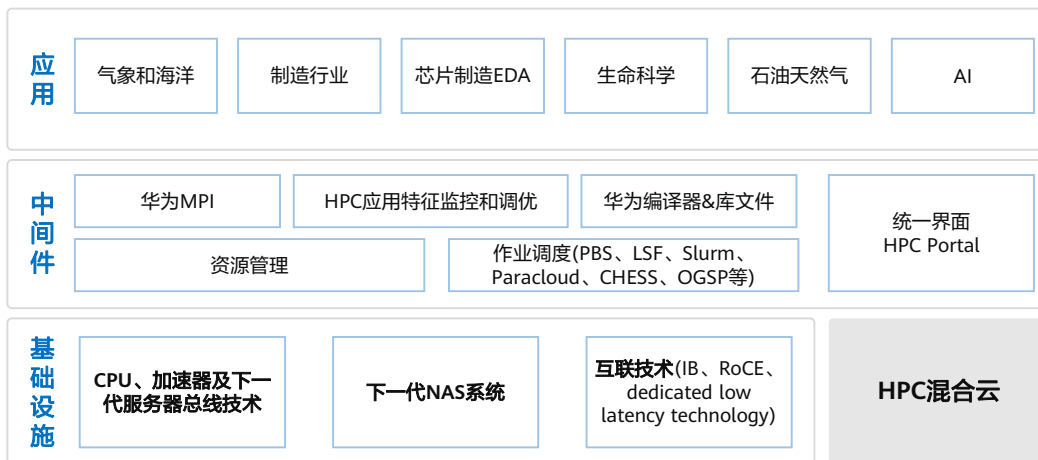
自服务能力

- 自动发放虚拟机、云化裸机，自动创建集群，长时间自动状态检测
- 将各种不同的HPC应用模板进行初始化导入，在VM模板中部署MPI库、编译库及优化配置等

部署方式灵活

- 节省建设周期，近乎无限的基础架构
- 虚机/云化裸机、各计算/存储实例灵活可选

华为鲲鹏面向HPC研发创新技术点



华为自研鲲鹏（ARM）HPC方案加速科研创新

自主可控

与合作伙伴构建应用生态



全面支持自主研发云平台

- 生态使能平台 支撑集成商行业服务
- 数据湖 支持分布式数据存储、大数据分析
- 异构计算云 多架构计算资源池、存储池、网络/安全池

构建自主研发硬件基础设施



关键芯片100%自主研发



- 高性能CPU
- 存储控制器
- 服务器管理

生态丰富

应用软件

制造 商业ISV 开源软件

生命科学 第一代测序 第三代测序

气象预报 气象模型 海洋模型 环境模型

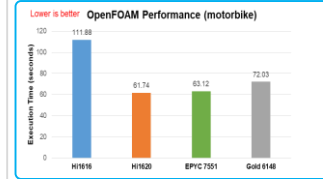
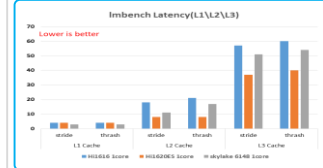
系统管理

集群管理、作业调度Portal

系统环境

操作系统 编译器工具链/数学库 MPI

性能领先



相比x86处理器，内存性能带宽高25%以上，更加适合Memory Bound类的应用。

气象HPC鲲鹏云解决方案

应用场景

天气、空气质量预报（3天）

海洋预测（10天）

多尺度、多物理场全球气候模式（长期）

解决方案

WRF模式特点

- 计算以单精度操作为主
- WRF等模式并行度高
- 对系统内存带宽要求高

鲲鹏云优势完美契合

- 64核/2.6GHz主频，超强整型计算
- 多核特征适合并行计算
- 8通道DDR4内存，带宽提升33%

方案价值

超高整型计算能力+软硬联合优化，气象业务无缝上云

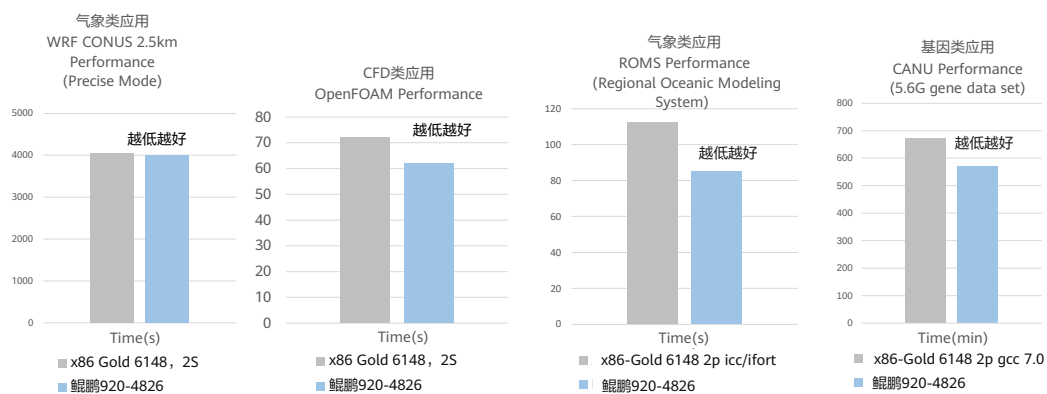
- 客户TCO降低40%
- 整体计算性能提升35%
- 性价比提升225%

CPU类型	节点	耗时（s）
E5-2640 v4@2.4GHz	14	3048.819
KunPeng920@2.6GHz	8	2273.314

- WRF（Weather Research and Forecasting）是新一代中尺度预报模式，被广泛应用的开源气象模拟软件。

鲲鹏加速CAE/CFD、气象和基因仿真应用性能

ARM高系统内存带宽匹配特定HPC应用需求



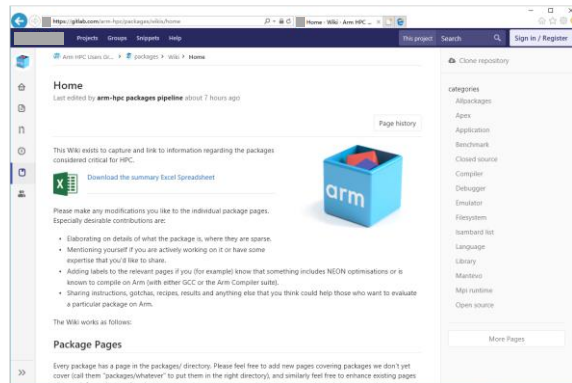
鲲鹏HPC开源生态

- OpenHPC是HPC的综合软件堆栈，是开源HPC软件组件的参考集合。OpenHPC 1.3.3版本已经通过华为鲲鹏（ARM）服务器的全面测试。



Available Software Overview (v1.3.3)

Functional Areas	Components
Base OS	CentOS 7.4, SLES12 SP3
Architecture	x86_64, aarch64
Administrative Tools	Conman, Ganglia, Lmod, LosF, Nagios, pdsh, pdsh-mod-slurm, prun, EasyBuild, ClusterShell, mrsh, Genders, Shine, Spack, test-suite
Provisioning	Warewulf, xCAT
Resource Mgmt.	SLURM, Munge, PBS Professional, PMix
Runtimes	OpenMP, OCR, Singularity
I/O Services	Lustre client, BeeGFS client
Numerical/Scientific Libraries	Boost, GSL, FFTW, Hypr, Metis, Mumps, OpenBLAS, PETSc, PLASMA, Scalapack, Scotch, SLEPc, SuperLU, SuperLU_Dist, Trilinos
I/O Libraries	HDPS (pHDFS), NetCDF/pNetCDF (including C++ and Fortran interfaces, Adios)
Compiler Families	GNU (gcc, g++, gfortran), Clang/LVM
MPI Families	MPICH2, OpenMPI, MPICH
Development Tools	Autotools, cmake, hwloc, mp-lay, R, SciPy/NumPy, Valgrind
Performance Tools	PAPI, IMB, mpir, pdttoolkit, TAU, Scalasca, ScoreP, SIONLib



鲲鹏HPC已验证平台及应用

平台软件

软件类型	软件型号&版本
OS	CentOS 7.6
编译器	GNU 4.9 (OS内嵌), 5, 6, 7, 8 LLVM
MPI	OpenMPI mpich3
调度软件	Slurm
集群管理软件	联科CHES warewulf Nagios

ARM公司已验证应用列表:

<https://gitlab.com/arm-hpc/packages/wikis/categories/allPackage>

OpenHPC软件集

<https://github.com/openhpc/ohpc/wiki/Component-List-v1.3.6>

应用场景



制造



基因测序



气象



分子动力学

理论上所有源码的HPC应用都可移植到ARM平台

- Pam-crash
- OpenFOAM
- SU2
- WRF
- ROMS
- CAMX
- CMAQ
- lammmps
- STAR 2.6.0c
- 0.9.6.3.1
- Diamond 0.9.22
- Spades 3.12.0
- Jellyfish 2.2.10
- Bzip2 1.0.6
- Rapsearch 2.24
- Canu 1.7.1
- Minimap 2.12

目录

1. 华为鲲鹏解决方案全景
2. 华为鲲鹏通用解决方案
 - 高性能计算HPC解决方案
 - 大数据基础设施解决方案
 - 鲲鹏云手机解决方案

大数据的技术种类和代表场景

大数据计算模式	解决问题	代表产品
批处理	针对大规模数据的批量处理	Hadoop、Spark、Hive等
即席分析查询	大规模数据的存储管理和查询分析	Dremel、Presto、Impala等
流处理	针对流数据的实时计算	Storm、S4、Flume、Flink
图计算	针对大规模图结构数据的处理	Pregel、GraphX、Giraph、PowerGraph、Hama、GoldenOrb等

- HDFS：分布式文件系统。
- MapReduce：分布式并行编程模型。
- YARN：资源管理和调度器。
- Tez：运行在YARN之上的下一代Hadoop查询处理框架。
- Hive：Hadoop上的数据仓库。
- Hbase：Hadoop上的非关系型的分布式数据库。
- Pig：一个基于Hadoop的大规模数据分析平台，提供类似SQL的查询语言Pig Latin。
- Sqoop：用于在Hadoop与传统数据库之间进行数据传递。
- Oozie：Hadoop上的工作流管理系统。
- Zookeeper：提供分布式协调一致性服务。
- Storm：流计算框架。
- Flume：一个高可用的，高可靠的，分布式的海量日志采集、聚合和传输的系统。
- Flink：流计算框架。
- Kafka：一种高吞吐量的分布式发布订阅消息系统，可以处理消费者规模的网站中的所有动作流数据。
- Spark：类似于Hadoop MapReduce的通用并行框架，引入了内存计算能力。

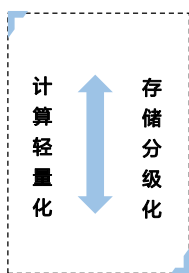
支持开源等多种大数据平台商用 - 生态开放



- 鲲鹏服务器支持开源大数据平台，对标Intel有性能提升。
- 鲲鹏服务器对于大数据的生态兼容较好，常见的应用都能迁移上来。

业界的技术趋势 - Hadoop存算分离之路

大数据架构变革



Hadoop 1.0

计算存储一体

大数据基本能力

- MapReduce
- 资源、数据管理合一
- 三副本

面向单一MR分析业务

第一代：融合型
2011

Hadoop 2.0

计算解耦、存储异构

解耦的大数据平台

- DataNode可扩4000节点
- 资源管理独立 (Yarn)
- 多计算引擎

面向复合型分析业务

第二代：扩展型
2013

Hadoop 3.0

多样化存储

开放的云生态

- 接入数据湖
- 支持EC类型存储
- K8s接管Yarn

面向数据湖，企业级大数据业务

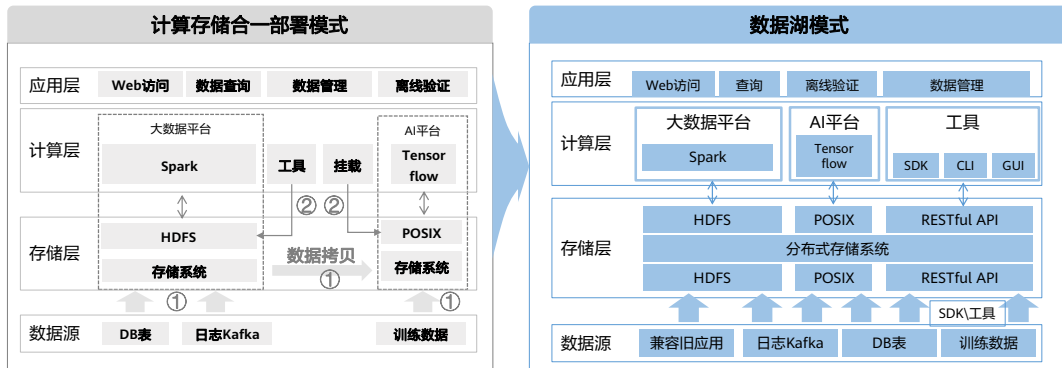
第三代：开放型
2017

1 Hadoop的计算层的逐渐轻量化，逐步与数据解耦

2 HDFS 存储层逐渐支持多种存储，与计算的绑定逐渐消除

3 云厂商已基于计算存储分离，提供成熟的数据湖架构

全分布式存储 – 一份数据支持多种协议，简化接入和使用流程



① HDFS无法提供多种协议支持，多应用场景数据流动较乱，数据难以共享

② 数据割裂造成管理成本增加，需要通过不同软件管理多套存储

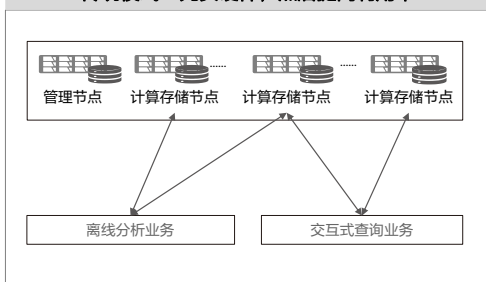
华为数据湖提供多协议访问和互通能力，数据只保存一份，避免数据多次拷贝，提高分析效率，且节约数据保存成本

问题背景:

- 计算存储合一部署模式，限制于HDFS单一接口，在应用访问数据多样化的背景下，逐渐难以应对。
- 不同生态如果使用了多套存储，会造成数据割裂，管理工具不统一造成运维工作量增加。

云原生时代 - 计算存储分离简化大数据开发

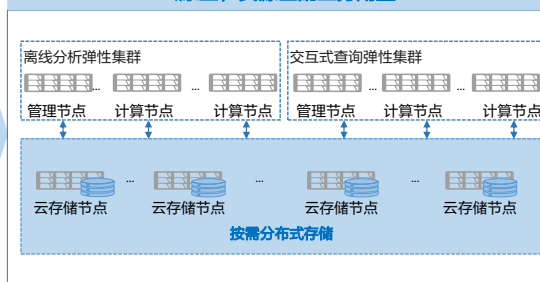
传统模式：先买硬件，然后提高利用率



计算与存储融合

- 关注计算能力水位时，还需要关注容量水位
- 不同业务共用一套资源，不利于管理
- 计算存储扩容比例难以协调

云原生，资源匹配业务用量



计算与存储分离

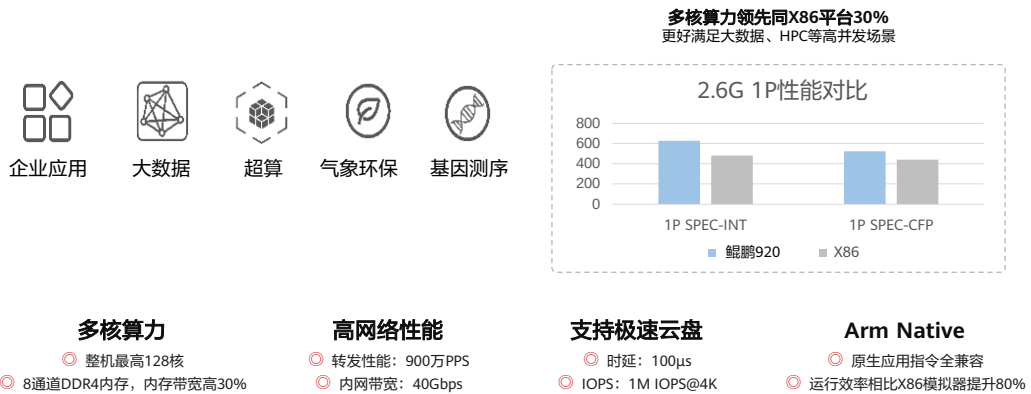
- 计算节点做到Cloud Native，可根据计算量自动化弹性伸缩
- 存储按照使用量无感扩容，业务不受任何影响
- 资源利用率更高，集群管理更容易

BigData Pro存算分离方案



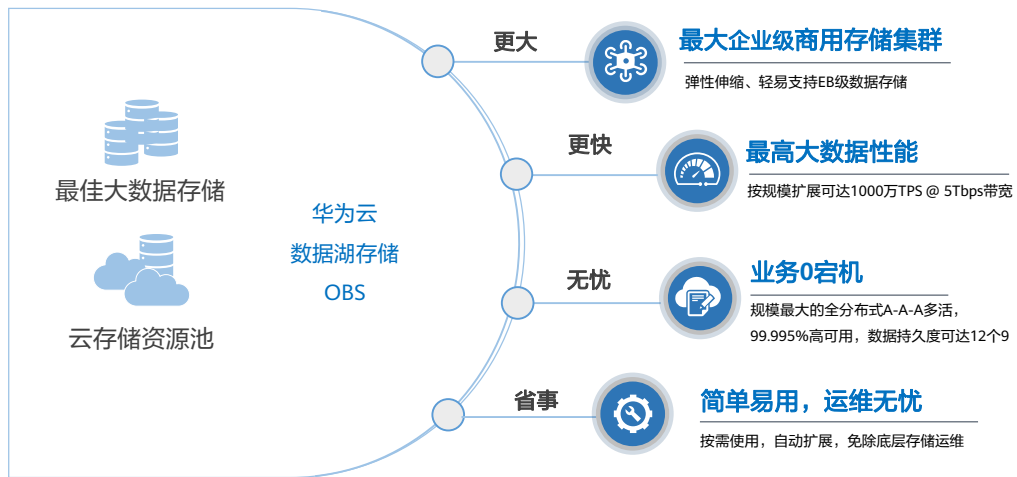
- BigData Pro方案是以华为多元化芯片技术为支撑：鲲鹏CPU提供了多核高性价比算力、昇腾910提供高性能AI训练能力、昇腾310提供高效AI推理能力。
- 大数据方案全面使用50GE网卡，华为自研SSD等技术，提供了超高的IO性能。
- 数据湖多协议互通使得一份数据能够被多种应用兼容读取，简化了应用开发。
- 方案兼容业界主流大数据生态、并被华为大数据生态无缝集成。
- 大数据场景在互联网行业非常成熟，通常使用于内容推荐、广告投送、运营分析、报表等场景；互联网之外，也有各行各业的大数据场景正在迅速孵化，如卫星遥感、自动驾驶等，这些都可以使用华为BigData Pro方案。

计算 - 鲲鹏助力在ETL场景借助多核算力提升性能30%



- ETL：Extract-Transform-Load，抽取-转换-加载，从来源端抽取数据、将抽取的数据转换以满足运作需求（可包含质量水平）、并加载至目的端（数据库或数据仓库）的过程。ETL较常用在数据仓储，但其对象并不限于此。

存储 - 单流300MB、多协议互通、超大总带宽、EB数据量

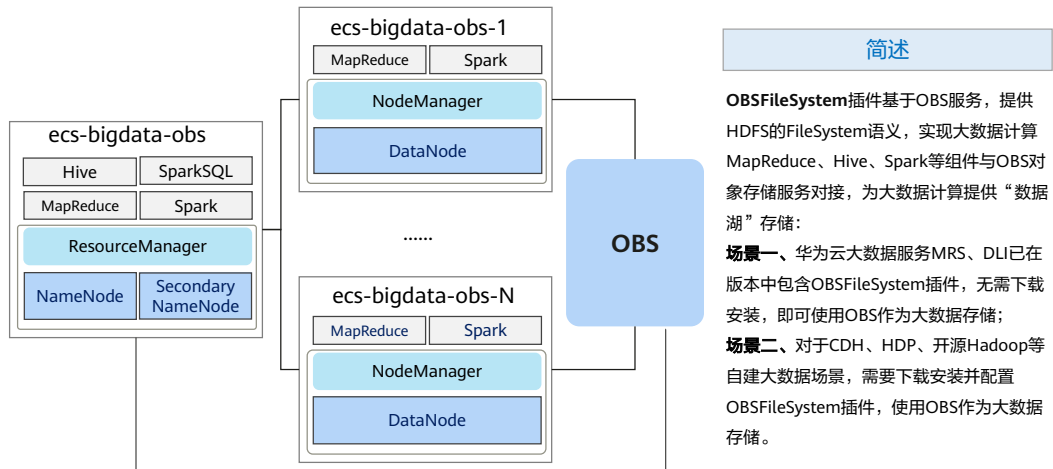


27 Huawei Confidential



- BigData Pro的数据湖存储使用了华为商用存储集群，规模轻易支持到EB级。
- 数据湖存储提升了大数据性能，适配计算集群的规模扩展。
- 数据持久度12个9，99.995%高可用性，多AZ多活。
- 按需使用，无感扩容，免除手动扩容麻烦。
- OBS 单流（单文件）上传和下载最大可达到300MB/S。

部署方案 - Hadoop/Spark对接OBS部署视图



- 浅灰色：MapReduce/Hive/Spark。
- 天蓝色：YARN。
- 蓝紫色：HDFS。

大数据典型客户案例

资源高效利用

计算资源利用率提升75%
存储资源利用率提升140%

性能极致

鲲鹏多核算力高并发性能
大数据、分布式场景性能对标X86
部分组件可提升10%-20%

安全可控

国产自研芯片
芯片级别安全
自主可控

云上数据中心

精细化运营

销售预测

故障诊断

语义云

收视率

个推系统

应用层架构不变，平迁至x86，降低迁移难度

Hadoop Hive HBase Spark 2.0.0 Kafka

开源和商用大数据组件迁移上**鲲鹏**

云容器 弹性云服务器 裸金属服务器



鲲鹏，高效算力底座

X86，承载业务系统

某大数据客户最佳实践

200+个大数据服务器（原IDC）

20+个Hadoop、HBase等大数据平台组件

800+TB数据迁移至鲲鹏平台

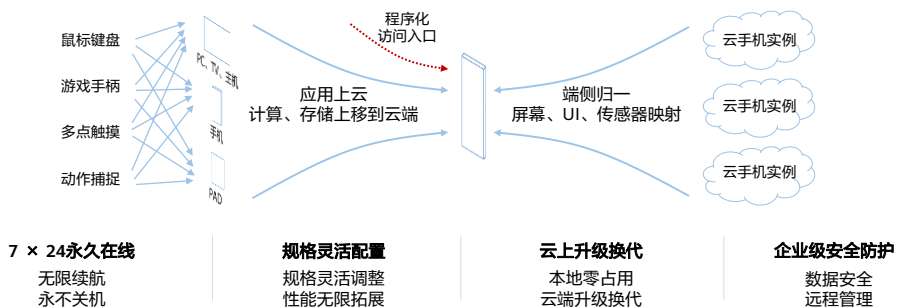
承载10大业务系统

目录

1. 华为鲲鹏解决方案全景
2. 华为鲲鹏通用解决方案
 - 高性能计算HPC解决方案
 - 大数据基础设施解决方案
 - 鲲鹏云手机解决方案

什么是云手机

- 华为云鲲鹏云手机，是基于华为云鲲鹏裸金属服务器，虚拟出带有原生安卓操作系统，同时具有虚拟手机功能的云服务器。
- 简单来说，云手机=云服务器 + Android OS，本质是将手机上的应用转移到云上的虚拟手机来运行。

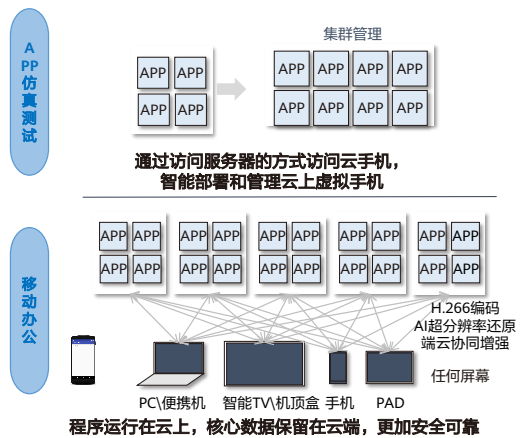


华为云鲲鹏云手机典型场景应用

ToC类场景



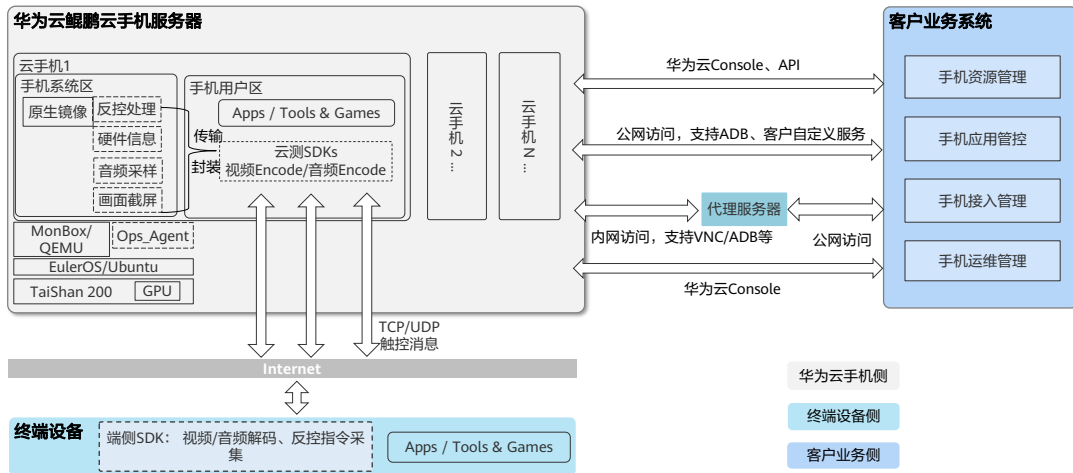
ToB类场景



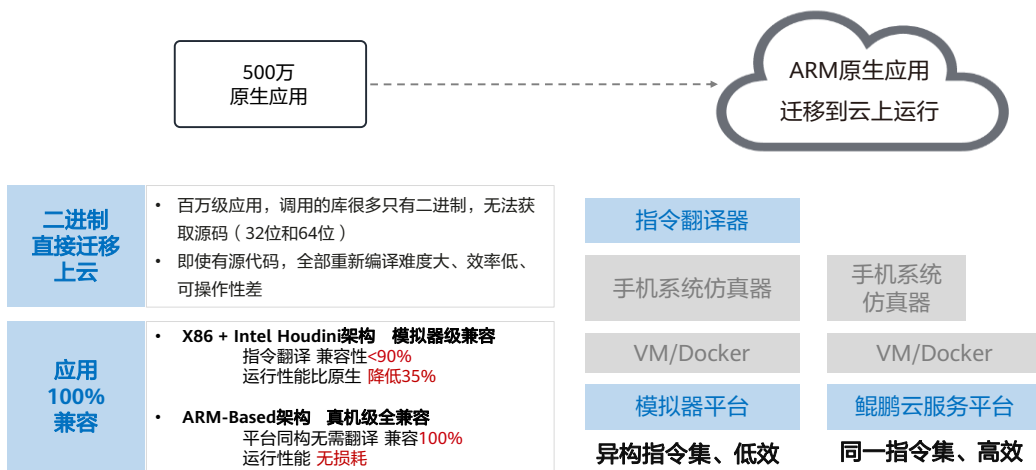
云手机市场常见解决方案对比分析

常见解决方案	x86模拟器	真机农场	手机开发板农场	华为TaiShan服务器
优势	- x86服务器易于获得，技术门槛低，资源可获得性高 - 业务系统架构设计灵活，可充分借鉴市场成熟的产品应用，构建独特竞争力 - 可灵活设定仿真手机规格，自由度高	- 与真机基本一致，仿真度高，应用兼容性好 - 技术应用门槛低，易于构建	- 与真机硬件架构基本一致，仿真度高，应用兼容性好 - 借助手机芯片高集成度设计，在游戏行业的应用场景下性价比高	- 能够灵活调整云手机规格，性能无上限 - 能够轻松应对APP对手机规格性能不断上升的需求 - 基于服务器的硬件设计，更匹配企业级可靠性、稳定性与集群部署能力 - ARM原生指令支持，兼容性好，性能高
痛点	- 需要额外指令转换，多达几倍的性能损失，集群规模性价比很低 - 仿真度和应用兼容性较差，应用兼容性问题难以解决	- 手工焊接与复杂接线难以保障产品质量，稳定性非常差 - 二手手机市场变化快，设计对应的手机在市场上的可获得性极差	- 不能获得高于单一手机芯片的性能，难以满足高性能场景应用，只能基于单台手机性能进行整数向下切割 - 供货周期长，供应链不稳定 - 基于消费类电子电路进行设计，难以满足公有云的服务可靠性及大规模集群管理需求	- 基于ARM服务器的软硬件设计复杂度高，自行构建技术门槛非常高 - 生态应用还比不上x86，GPU配套方面还不够完善，需要大厂推动 - 硬件发展初期迭代速度快，平均两年升级一代，迭代速度高于三年可用性

华为云鲲鹏云手机技术架构

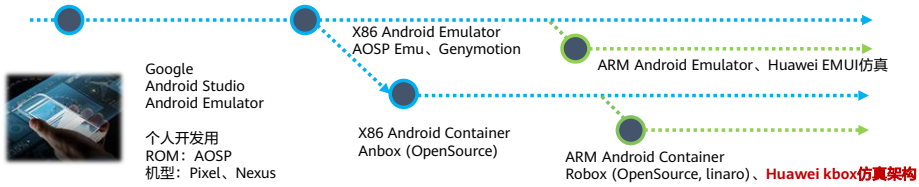


原生应用 - 端云同构优势，性能提升80%



自研kbox软件仿真架构强大性能提升

演进历程



强大优势

单服务器云手机密度提升**100%**

GPU设备直通到容器

- GPU设备直通到容器，移植GPU驱动直接对接Android渲染框架减少损耗

GPU内核驱动优化

- 优化内核驱动以适配鲲鹏架构环境

容器隔离优化

- 提升kbox手机容器间隔离性能，提升业务稳定性

实现单鲲鹏916 **60台**云手机并发

端到端云手机接入时延持续优化

- 实体手机端到端时延**100ms**

- 业界平均时延**150+ms**

- 云手机针对时延痛点，分解模块优化：

指令输入
上行网络
GPU渲染
视频编码
下行网络
视频解码
解码图像刷新到屏幕

实现端到端接入时延持续优化

云手机网络接入流量迭代降低

H.265压缩算法

- 压缩率比H.264高50%

SCC优化

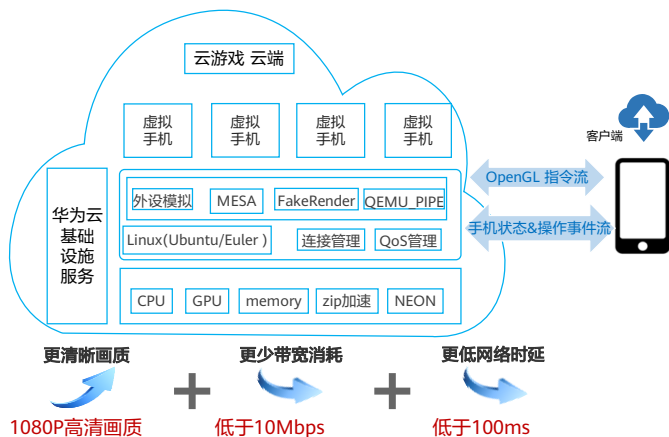
- App界面类型压缩率最大接近50%

AI+指令流渲染优化

- 基于CG、游戏引擎计算特征AI模型超分还原
- 分辨率越高，带宽节省优势越明显

实现云手机网络消耗流量不断降低

创新指令流分离渲染技术，为大屏带来高清画质



- **断线重连 —— 空渲染技术：**客户端和服务端维护相同的OpenGL状态机状态，当断线重连时，服务器将完整的状态机同步到客户端，重新建立连接。
- **快速加载 —— 顶点纹理缓存技术：**通过对顶点纹理的特征值进行提取，识别和剔除重复传输的顶点纹理数据。
- **网络防抖 —— 数据映射技术：**将服务器端分配的资源与客户端管理的资源建立Map映射，实现控制指令流的单向传输，避免网络时延增大时出现帧率大幅下降。
- **指令流动态压缩：**对控制指令流进行无损压缩，对纹理数据流进行有损压缩，对顶点数据流使用残差数据传输方式。
- **OpenGL ES 3.0：**完善OpenGL库API适配层，使得能支持OpenGL ES 3.x版本的游戏。

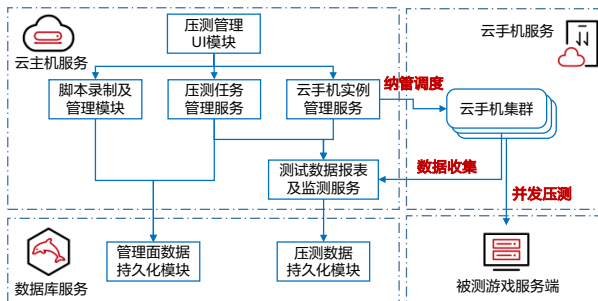
指令流传输，客户端渲染：更高的FPS，更清晰的画质，更少的带宽消耗，更低的网络时延

APP仿真测试场景典型客户案例

XX游戏作为一家专注于手机游戏研发、发行和运营服务的移动游戏公司，公司致力于通过能力、资源、资本联动打造一家世界领先的文创数字内容平台。

背景需求

- 传统服务器部署性能不足，管理复杂
- 手机农场成本高，升级困难
- 开发板稳定性差，成本高
- 需要低成本，易管理，可升级，稳定可靠的系统进行大规模游戏包测试



方案亮点

- 原生架构处理器，性能保证
- 服务器集群保障系统可靠稳定
- 全云化资源易管理
- 测试执行时间精确可控，计划管理有效，测试数据报告一键收集，效率高

客户价值

- 测试机故障率从60%降低至1%
- 测试可用时长从10小时/日提升至24小时/日
- 测试规模从百级节点并发提升至千级节点并发
- 测试数据导出时间从小时级降为分钟级

思考题

1. （多选题）以下哪些属于鲲鹏通用解决方案？
 - A. 大数据解决方案
 - B. HPC解决方案
 - C. 分布式存储

- 参考答案：
 - ABC

本章总结

- 本章主要介绍了基于华为鲲鹏相关的解决方案全景图，并对一些通用解决方案展开介绍，其中包括了高性能计算HPC解决方案、大数据基础设施解决方案、鲲鹏云手机解决方案。

缩略语

- HPC: High-performance computing, 高性能计算是一个计算机集群系统, 它通过各种互联技术将多个计算机系统连接在一起, 利用所有被连接系统的综合计算能力来处理大型计算问题, 所以又通常被称为高性能计算集群。
- SaaS: Software-as-a-service, 软件即服务, 是一种软件许可和交付模式, 在该模式中, 软件是基于订阅许可的, 并且是集中托管的, 包含提供该服务所必须的软件、硬件资源和管理服务。
- ISV: Independent software vendor, 独立软件供应商, 补充业务提供商的一种, 又叫做独立软件开发商。
- ECS: Elastic Cloud Server, 弹性云服务器, 弹性云主机是由CPU、内存、镜像、云硬盘组成的一种可随时获取、弹性可扩展的计算服务器, 同时它结合VPC、虚拟防火墙、数据多副本保存等能力, 为您打造一个高效、可靠、安全的计算环境, 确保您的服务持久稳定运行。
- BMS: Bare Metal Server, 裸金属服务器, 裸金属服务器提供单租户专属的物理服务器, 提供最卓越的性能, 满足核心应用场景对高性能及稳定性的需求, 同时可以和VPC等其他云产品灵活结合使用, 它结合了传统托管主机方式带来的稳定性能与云中资源高度弹性的优势。
- OS: Operating System, 操作系统, 管理计算机硬件与软件资源的计算机程序。
- CCI: Cloud Container Instance, 云容器实例, 服务提供 Serverless Container (无服务器容器) 引擎, 让您无需创建和管理服务器集群即可直接运行容器。
- OBS: Object Storage Service, 基于对象的云存储服务, 提供易扩展、高安全、高可靠性、低成本的数据存储能力, 用户可以通过基于HTTP协议的接口对对象进行管理和使用。对象存储服务一般应用于大规模的数据存储服务。
- TCO: total cost of ownership, 总体拥有成本, 包括产品采购到后期使用、维护的成本。

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。
Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

Copyright©2023 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



鲲鹏社区



前言

- 本章主要讲述鲲鹏社区是如何围绕鲲鹏体系打造技术支持、知识共享和产业互助平台的。

目标

- 学完本课程后，您将能够：
 - 熟悉鲲鹏社区的整体构架；
 - 了解鲲鹏社区各个板块的价值和功能；
 - 利用在鲲鹏社区中搜寻的资源实现自身鲲鹏相关业务的快速发展。

目录

1. 鲲鹏社区整体介绍
2. 鲲鹏开发者专栏
3. 学习发展
4. 鲲鹏产品及解决方案
5. 鲲鹏生态
6. 支持与交流
7. 资讯与活动

鲲鹏社区 - 鲲鹏生态综合型在线门户

- 鲲鹏社区是鲲鹏开发者技术支持和生态使能的一站式资源、服务平台，可提供完善的鲲鹏领域软件资源、专业技能、技术支持、生态政策和产品方案等内容，与社区参与者共建基于鲲鹏计算产业的综合性社区。
- 鲲鹏社区核心作用：
 - 为合作伙伴和开发者提供一站式的技术支撑平台；
 - 资料中心、资源与工具中心、技术支持中心、培训认证中心；
 - 鲲鹏计算产业蓬勃发展的对外展示窗口；
 - 鲲鹏的产业布局、生态伙伴汇聚、产业进展造势。
- 鲲鹏社区网址：<https://www.hikunpeng.com/>



- 鲲鹏社区是面向鲲鹏合作伙伴、开发者的全产业社区，集学习、实验、认证等功能为一体，是鲲鹏计算产业的官方社区。

鲲鹏社区 - 面向受众

- 面向受众：
 - 软/硬件开发者
 - 云服务伙伴、软件伙伴、整机伙伴
 - 行业客户
 - 产业链相关人：高校、媒体、分析师 ...



鲲鹏社区核心建设，需要匹配目标受众需求



- 鲲鹏社区是一个社区平台，提供了一系列内容，旨在增强客户与鲲鹏生态之间的联系。该平台包含许多不同的功能，包括活动（如鲲鹏开发者创享日）、应用程序（如鲲鹏俱乐部APP）和计划（如鲲鹏众智计划），帮助客户了解鲲鹏生态的最新信息，并与其他鲲鹏生态开发者和专家建立联系。

鲲鹏社区各板块简介

吸引	加入	开发	学习	分享
鲲鹏产品 鲲鹏解决方案	生态合作 兼容性查询	开发者专栏 技术支持	高校教学资源 在线学习实验	互动交流 活动资讯
- 了解整个鲲鹏系列化产品和云服务。 - 学习基于鲲鹏云的通用解决方案、行业解决方案。 - 了解鲲鹏的算力特点和优势场景。	- 了解华为鲲鹏合作伙伴计划：鲲鹏展翅、欧拉扬帆、鲲鹏智数。 - 最新的鲲鹏生态图谱和合作伙伴优秀实践分享。 - 查询合作伙伴鲲鹏认证情况，并下载认证证书。	- 获取开发工具和指导文档。 - 掌握应用迁移和调优的工程方法。 - 汇聚大量鲲鹏兼容软件版本，一站式下载。 “学-练-训-考”一站式成长计划，助力开发者领跑开发之路。	- 学习发展提供丰富的学习和实践资源。 - 参与鲲鹏云服务构建的软件迁移和调优的线上实验，进行开发实践。 - 了解并获取“智能基座”产教融合协同育人基地的学习资源。	- 可以发博客、写论坛，分享鲲鹏开发体验。 - 参加鲲鹏社区上丰富的活动，共赢计算时代。

- 鲲鹏社区的板块划分涵盖了鲲鹏生态的方方面面，开发者们可以在“产品”板块了解鲲鹏硬件的介绍、相关的云服务以及产品文档；“解决方案”板块则会提供一些鲲鹏相关成熟的行业案例，帮助开发者更好的了解鲲鹏提供的强大计算能力；“开发者”板块包含多种多样的技术主题、学习资料以及丰富的活动，零基础也可以学习前沿技术；如果有兴趣申请加入鲲鹏生态伙伴网络，则可以访问“合作伙伴”板块，获取第一手鲲鹏伙伴计划资讯；“高校”板块致力于联接校企主体、联接教学要素，面向高校教师、学生提供学习交流机会以及教学实践案例；“鲲鹏解决方案市场”板块是一站式软硬件产品、解决方案、伙伴和专家技术服务的展示和互动平台；为保证更多客户可以了解最新鲲鹏产业动态，“产业资讯”板块应运而生；如若需求一些鲲鹏技术上的帮助，可以访问、“支持与服务”板块。

目录

1. 鲲鹏社区整体介绍
- 2. 鲲鹏开发者专栏**
3. 学习发展
4. 鲲鹏产品及解决方案
5. 鲲鹏生态
6. 支持与交流
7. 资讯与活动

鲲鹏开发者 - 开发者主页



- 鲲鹏开发套件DevKit

- 鲲鹏帮助开发者加速应用迁移和算力升级，提供代码扫描、迁移、调优、毕昇编译器等，以插件的方式，开放集成到各种主流IDE，延续开发者现有的开发习惯。
- 支持TOP10常见语言迁移，编译性能提升 25 %，4大场景8大维度性能分析。
- <https://www.hikunpeng.com/zh/developer/devkit>。

- 鲲鹏应用使能套件BoostKit

- 围绕大数据、分布式存储、数据库、虚拟化、ARM原生、Web、CDN、HPC八大主流场景，开放高性能开源组件、加速软件包和工具/参考实现。
- 8大应用实现性能倍增。
- <https://www.hikunpeng.com/zh/developer/boostkit>。

- 鲲鹏HPC

- 提供基于鲲鹏的端到端高性能计算解决方案，覆盖鲲鹏服务器、欧拉开源操作系统、多瑙套件等HPC基础软硬件，实现全栈垂直优化，面向气象、制造、生命科学等行业应用提供超强算力。
- <https://www.hikunpeng.com/zh/developer/hpc>。
- 围绕开发者应用开发、迁移、调优场景，提供丰富的工具和软件资源。
- <https://www.hikunpeng.com/zh/developer/software>。

鲲鹏开发者 - 鲲鹏开发套件 DevKit



- 提供涵盖代码迁移、开发调试、编译、测试、调优及诊断等各环节的开发使能工具，方便开发者快速开发出鲲鹏亲和的高性能软件。
 - 在鲲鹏社区当中，可以下载到所有鲲鹏开发套件的工具：
 - <https://www.hikunpeng.com/developer/software>。

鲲鹏开发者 - 鲲鹏应用使能套件 BoostKit



12 Huawei Confidential



- 鲲鹏应用使能套件BoostKit，基于硬件、基础软件和应用软件的全栈优化，提供高性能开源组件、基础加速软件包和应用加速软件包，使能应用极致性能。
- 各BoostKit组件查看及下载链接：
- BoostKit大数据
 - <https://www.hikunpeng.com/developer/boostkit/big-data>。
- BoostKit分布式存储
 - <https://www.hikunpeng.com/developer/boostkit/sds>。
- BoostKit数据库
 - <https://www.hikunpeng.com/developer/boostkit/database>。
- BoostKit虚拟化
 - <https://www.hikunpeng.com/developer/boostkit/virtualization>。
- BoostKit ARM原生
 - <https://www.hikunpeng.com/developer/boostkit/arm-native>。
- BoostKit Web
 - <https://https://www.hikunpeng.com/developer/boostkit/cdn>。
- BoostKit HPC

- <https://www.hikunpeng.com/developer/hpc>。

鲲鹏开发者 - 开发者精选资源



鲲鹏初学者开始指南

鲲鹏开发者必备的基本知识



鲲鹏移植参考

针对不同场景，提供常见代码移植案例



鲲鹏技术疑难问题分析

汇总疑难问题的分析定位思路及案例



鲲鹏编程与调优指南

从多个维度介绍鲲鹏原生应用开发时，优化代码性能的方法



性能优化十板斧

鲲鹏处理器常用性能优化方法和分析工具



鲲鹏计算精度白皮书

提供数值计算精度误差产生的原因、改进措施和计算方法

- 鲲鹏初学者开始指南
 - <https://bbs.huaweicloud.com/forum/thread-30177-1-1.html>。
- 鲲鹏移植参考
 - https://www.hikunpeng.com/document/detail/zh/kunpenggrf/codeprtr/kunpengtaishanporting_12_0002.html。
- 鲲鹏技术疑难问题分析
 - https://www.hikunpeng.com/document/detail/zh/kunpenggrf/ref/kunpenggrfaq_10_0002.html。
- 鲲鹏编程与调优指南
 - https://www.hikunpeng.com/document/detail/zh/kunpenggrf/progtuneg/kunpengprogramming_05_0001.html。
- 性能优化十板斧
 - https://www.hikunpeng.com/document/detail/zh/kunpenggrf/tuningtip/kunpengtuning_12_0002.html。
- 鲲鹏计算精度白皮书
 - https://www.hikunpeng.com/document/detail/zh/kunpenggrf/computp-wp/kunpengcomputp_19_0002.html。

目录

1. 鲲鹏社区整体介绍
2. 鲲鹏开发者专栏
- 3. 学习发展**
4. 鲲鹏产品及解决方案
5. 鲲鹏生态
6. 支持与交流
7. 资讯与活动

鲲鹏开发者 - 学习专栏

- 学习专栏涵盖学习路径、在线课程、职业认证、微认证、在线实验，帮助学者快速提升技能，适应ICT时代技术发展。

学习路径

一站式成长计划，助力开发者从零开始构筑鲲鹏知识体系。

在线课程

体系化的培训课程，开放鲲鹏全栈能力，与开发者共同成长。

职业认证

官方专业认证，培养与认证能够在鲲鹏计算平台上进行业务部署以及常见问题处理的工程师。

微认证

一站式在线学习、实验与考试，零基础也可学习前沿技术知识，快速获得场景化的技能提升。

在线实验

一键创建实验环境，开发者通过实验手册指导，在云端实现云服务的实践、调测和验证。

- 学习专栏

- <https://www.hikunpeng.com/learn/growth>。

鲲鹏在线课程

- 鲲鹏在线课程是由鲲鹏社区设计的教学课程集合，主要面向鲲鹏软件开发人员。课程的目的是帮助开发人员了解鲲鹏体系的基础知识、软硬件产品、云服务，并学习如何将x86应用迁移到鲲鹏平台上等。目前主要提供以下课程：
 - 鲲鹏软件迁移实践
 - DevKit 2.0: 加速应用迁移，使能极简开发
 - 手把手带你使用代码迁移工具实现源码迁移
 - 鲲鹏软件性能调优实践
 - 揭秘鲲鹏处理器
 - 基于BoostKit的MySQL性能优化
 - 鲲鹏开发套件-如何获取插件、安装插件
 - 基于鲲鹏HPC解决方案的应用实践

- 鲲鹏在线课程
 - <https://www.hikunpeng.com/learn/courses>。

鲲鹏在线实验

- 鲲鹏在线实验：使用虚拟华为云账号，根据详细的实验手册，一步步指导操作，模拟真实场景，完善的虚拟环境配置搭建，可快速体验华为云服务，在云端实现云服务的实践、调测和验证。



随时随地在线进行鲲鹏计算平台的调测和验证实践，一键申请实验资料，量级轻、上手快、体验好：

- 通过鲲鹏开发套件实现源码迁移
- 基于鲲鹏亲和开发框架进行原生开发
- 大数据OmniData算子下推特性实践
- 基于鲲鹏多瑙调度器的作业调度实践
- 通过鲲鹏开发套件实现软件包扫描和重构
- 基于鲲鹏应用使能套件进行MySQL性能调优
- 基于鲲鹏应用使能套件的加速库数学库应用指导
- 通过鲲鹏开发套件实现鲲鹏GCC功能支持实践

- 鲲鹏在线实验会提供完整的实验手册，且用户在使用过程中可以实时感知到自己的实验当前进度，提供轻量级、上手快、体验好的在线实验。
 - <https://www.hikunpeng.com/learn/experiments>。

鲲鹏认证体系

- 鲲鹏认证体系是针对不同用户、产品类别，精心打造层次化的培训认证，提供职业认证和微认证。职业认证可以帮助个人快速掌握鲲鹏相关理论知识和实践动手能力。微认证则提供一站式在线学习、实验与考试，助力提升场景化技能。

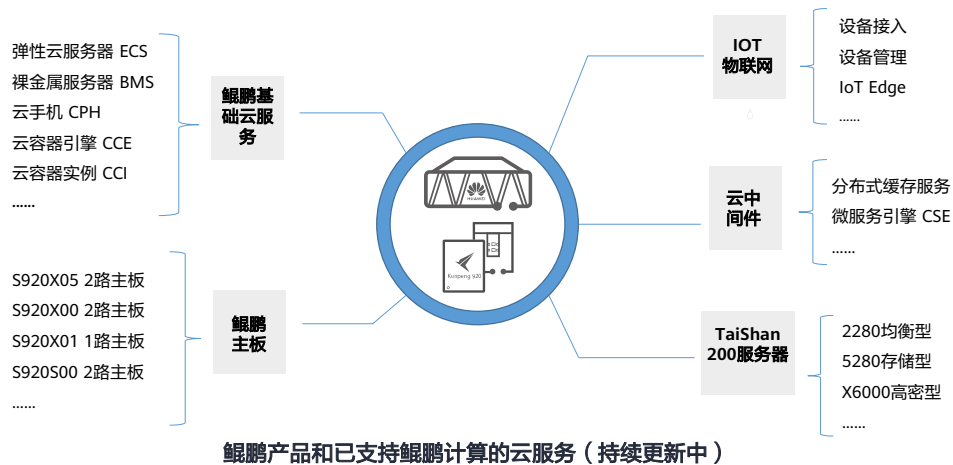


- 鲲鹏微认证
 - <https://www.hikunpeng.com/learn/micro-certification>。
- 鲲鹏职业认证
 - <https://www.hikunpeng.com/learn/career-certification>。

目录

1. 鲲鹏社区整体介绍
2. 鲲鹏开发者专栏
3. 学习发展
- 4. 鲲鹏产品及解决方案**
5. 鲲鹏生态
6. 支持与交流
7. 资讯与活动

鲲鹏产品与云服务



- 鲲鹏处理器产品
 - <https://www.hikunpeng.com/compute/kunpeng920>。
- 鲲鹏服务器主板及整机产品
 - <https://www.hikunpeng.com/compute/server-motherboard?data=S920X05>。

鲲鹏产品与云服务优势

- 相比于其他同类型的产品和云服务，鲲鹏具有以下优势：

多核算力

- 多核整形算力领先15%
- 整机最高128核
- 整机并发性能提升30%

高性价比

- 计算性能领先8%
- 内存带宽领先46%
- 综合性价比领先30%

端云协同

- 原生应用场景，端云同构，性能提升80%
- 100%原生指令兼容，算法优化，无需改造

生态丰富

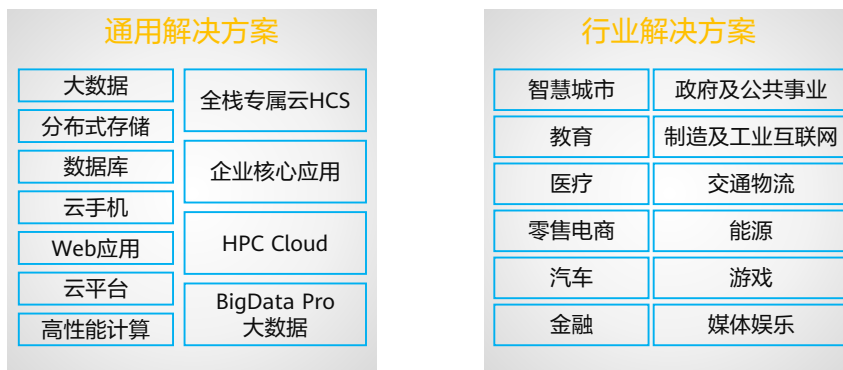
- 已兼容CentOS、Ubuntu、openEuler等20+款主流操作系统
- 100+企业核心业务应用，联合广大ISV，支持应用范围持续扩充



- 鲲鹏云服务开启多元计算新架构，加速企业创新升级
 - <https://www.hikunpeng.com/compute/cloud-service>。

鲲鹏解决方案

- 多样化的解决方案，匹配客户与合作伙伴应用场景。

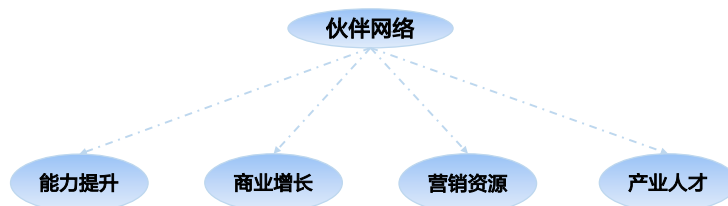


目录

1. 鲲鹏社区整体介绍
2. 鲲鹏开发者专栏
3. 学习发展
4. 鲲鹏产品及解决方案
- 5. 鲲鹏生态**
6. 支持与交流
7. 资讯与活动

鲲鹏生态伙伴网络

- 鲲鹏生态伙伴网络是华为围绕鲲鹏产业技术路线推出的系列生态伙伴计划，它通过提供学习、构建、上市、销售等资源支持，致力于帮助鲲鹏生态伙伴构建产业竞争力、联接客户创造商机，在一个开放、创新、协作、互惠的生态系统里，共同成长、共创未来。



- 鲲鹏生态伙伴网络全景图
 - <https://www.hikunpeng.com/ecosystem/kepn>。
- 能力提升：获得生态伙伴专属的技术培训、管理培训，快速提升生态伙伴围绕鲲鹏全栈技术的产品与解决方案构建与创新能力，以及围绕企业成长的运营管理能力。
- 商业增长：获得联接华为客户与伙伴的机会，共享自主创新技术带来的行业数字化商业增长机会。
- 营销资源：获得参加华为与鲲鹏生态伙伴主办的系列市场营销活动、产业峰会、开发者技术沙龙、专题研讨会等。
- 产业人才：获得由全国优质合作高校精心培养的鲲鹏全栈产业技术人才，加速构建企业产品与解决方案竞争力。

伙伴专区：三大伙伴权益计划



- 欧拉扬帆伙伴计划
 - https://www.hikunpeng.com/zh/ecosystem/openEuler_partner-program。
- 鲲鹏展翅伙伴计划
 - <https://www.hikunpeng.com/ecosystem/server-partner-program>。
- 鲲鹏科研创新使能计划
 - https://www.hikunpeng.com/ecosystem/kunpeng_sci-res-innov_program。

欧拉扬帆伙伴计划（华为）

- 欧拉扬帆伙伴计划（华为）是华为围绕openEuler推出的一项生态伙伴计划，旨在保障openEuler生态健康有序发展，加速生态伙伴基于openEuler系列产品和部件构建产品与解决方案，使能伙伴生态发展与商业成功。
- 加入我们，您可以获得以下权益：
 - openEuler职业认证券
 - NRE年度可申请额度
 - 高级技术支持（FAE/PAE）
 - 产品规划协同
 - 专属合作运营支持团队（PR/BD）
 - 进入华为行业解决方案预集成与华为云市场优选清单
 -

- 伙伴认证要求包括但不限于资质要求、贡献要求、能力要求和综合要求，从分类而言，分为应用软件伙伴、基础软件伙伴、生态运营伙伴。

鲲鹏展翅伙伴计划

- 鲲鹏展翅伙伴计划是华为围绕鲲鹏计算平台推出的一项生态伙伴计划，旨在帮助伙伴整合鲲鹏计算产业的全栈技术、品牌资源，构建产品与解决方案竞争力，联接客户与鲲鹏生态系统，创造无限商机。
- 加入我们，您可以获得以下权益：
 - 鲲鹏生态创新中心学习资源
 - 数字化领导力培训服务
 - 伙伴开发人员帐号
 - Powered by Kunpeng授权
 - 鲲鹏技术兼容性认证证书
 - 营销资源及商业机会支持
 -

- 鲲鹏生态伙伴认证要求包括但不限于资质要求、贡献要求、能力要求和综合要求，从分类而言，分为整机硬件伙伴、应用软件伙伴、基础软件伙伴、一体机解决方案伙伴、生态运营伙伴。

鲲鹏科研创新使能计划

- 鲲鹏科研创新使能计划是华为围绕鲲鹏计算基础设施推出的一项生态伙伴计划，旨在使能国内高校和科研院所依托鲲鹏计算基础设施开展科学研究和自主可信软件研发与技术攻关工作。
- 加入我们，您可以获得以下权益：



- 鲲鹏科研创新使能计划是主要针对高校科研团队、科研院所机构以及科研老师推出的伙伴计划，提供包含学习、构建、研究资源以及影响力四个方面的权益。

目录

1. 鲲鹏社区整体介绍
2. 鲲鹏开发者专栏
3. 学习发展
4. 鲲鹏产品及解决方案
5. 鲲鹏生态
- 6. 支持与交流**
7. 资讯与活动

鲲鹏论坛

- 鲲鹏论坛是技术大拿、行业专家、热心开发者的交流圣地，各位名家高手坐而论道，共享实践经验、技术观点，目前拥有12大频道，涵盖鲲鹏生态覆盖的技术知识、活动等，帮助开发者快速拓展技术视野。



- 鲲鹏论坛
 - <https://www.hikunpeng.com/forum/>。

鲲鹏博客

- 汇聚华为云社区“鲲鹏”领域博客、问答、视频等精彩内容，形成开发者和技术爱好者交流与分享主阵地。



- 鲲鹏博客
 - <https://bbs.huaweicloud.com/tags/200021>。

目录

1. 鲲鹏社区整体介绍
2. 鲲鹏开发者专栏
3. 学习发展
4. 鲲鹏产品及解决方案
5. 鲲鹏生态
6. 支持与交流
- 7. 资讯与活动**

产业资讯 - 鲲鹏活动

- 鲲鹏活动专区致力于与开发者们通过各类精彩纷呈的活动来实现技术交流，与开发者一起成长，共赢计算时代。



- 大赛、话题、沙龙、峰会... 异彩纷呈，学玩结合，使能开发者快速晋级达人行列。

产业资讯 - 鲲鹏资讯

- 鲲鹏资讯专区是鲲鹏官方发布的最新动态或消息的平台，为开发者们提供关于鲲鹏的第一手资讯。



- 鲲鹏资讯
 - <https://www.hikunpeng.com/activities/news>。

思考题

1. （多选题）鲲鹏开发套件（DevKit）提供了哪些工具？

- A. 鲲鹏代码迁移工具
- B. 鲲鹏性能分析工具
- C. 鲲鹏分布式存储使能套件
- D. 二进制指令翻译工具

- 答案：
 - A、B、D

本章总结

- 本章简要介绍鲲鹏社区各个模块所承载的价值与信息所在，以及用户如何访问和使用社区工具的方式。帮助学员了解鲲鹏社区作为一个综合型在线门户，可以提供完善的软件资源、技术知识、产品方案、生态政策、交易平台等，汇聚全栈的资源 and 经验，使能鲲鹏开发者、合作伙伴技能成长，轻松完成应用移植和价值变现。

更多信息

- 鲲鹏社区首页: <https://www.hikunpeng.com/zh/>
- 鲲鹏软件栈: <https://www.hikunpeng.com/developer/software>
- 鲲鹏生态伙伴网络: <https://www.hikunpeng.com/ecosystem/kepn>
- 鲲鹏论坛: <https://www.hikunpeng.com/forum/>
- 鲲鹏计算产业: <https://www.hikunpeng.com/compute/industry>

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。
Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

Copyright©2023 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

